

VTX-1 ISA 1.0 Draft 1.0-draft

参考手册

SGPR + VGPR · 统一 64 位编码 · 66 指令家族 · 379 指令形式

目录

VTX-1 ISA 1.0 Draft	10
SGPR + VGPR 架构	10
VTX-1 ISA 1.0 Draft: 文档状态	11
1. 这份草案是什么	11
2. 哪些文件说了算	11
3. 要求用词	11
4. 架构固定点	12
5. 符合性底线	12
VTX-1 ISA 1.0 Draft: 术语与记号	14
1. 从设备到 lane	14
2. 调度和占用率	14
3. 寄存器术语	14
4. lane 掩码	15
5. 执行域和机器 class	15
6. PC、静态指令和动态指令	16
7. 分歧和重汇聚	16
8. 数值和位写法	17
9. 伪代码约定	17
10. 就绪、阻塞和完成	18
VTX-1 ISA 1.0 Draft: 编程模型	19
1. 内核描述符	19
2. 启动前校验	20
3. 参数 ABI	21

4. 架构寄存器	22
5. 物理寄存器和 occupancy	23
6. CTA 驻留和调度	24
7. 启动初态	24
8. 特殊只读信息	25
9. 精确故障	25
VTX-1 ISA 1.0 Draft: 执行模型	28
1. 一句话理解执行方式	28
2. 每条动态指令的共同步骤	28
3. execution_domain=scalar	29
4. execution_domain=vector	30
5. execution_domain=warp_control	31
6. 隐藏重汇聚栈	33
7. 为什么分歧区里不能跑 scalar 执行域	35
8. 结构化控制流要求	36
9. Warp 集合操作	36
10. CTA 屏障	36
11. 调度、前进和 occupancy	38
12. 完成和死锁	39
13. 七个执行域的快速对照	39
VTX-1 ISA 1.0 Draft: 内存模型	41
1. 先记住七条直观规则	41
2. 两类访存指令	41
3. 地址空间和地址形成	42
4. 事件和普通访问粒度	45

5. 先用大白话理解顺序	45
6. 形式关系	45
7. U32/U64 原子	48
8. FENCE 和 CTA 屏障	49
9. 主机所有权	50
10. 数据竞争和 DRF 保证	50
11. 最小判例	51
VTX-1 ISA 1.0 Draft: 数值环境	52
1. 先记住六条直观规则	52
2. S 与 V 的执行规则	52
3. 位型与寄存器表示	53
4. NaN、无穷和有符号零	54
5. 舍入和 subnormal	55
6. 精确 F32 运算	55
7. 比较	55
8. FMIN 和 FMAX	56
9. 转换	56
10. 近似函数	57
11. FFMA 的单次舍入	58
12. MMA.M16N8K16.F16.F16.F32	58
13. 一致性测试要求	59
6. 编码与汇编	61
6.1 总体原则	61
6.2 class 与 format 分配	61

6.3 执行域、格式、family 与 form	63
6.4 guard	64
6.5 payload 基本格式	66
6.6 寄存器编码与资源边界	73
6.7 立即数	73
6.8 PC-relative 控制目标	74
6.9 must-zero 与唯一编码	74
6.10 规范汇编文本	75
6.11 译码顺序与错误分类	76
6.12 机器可读清单要求	77
7. 指令分类与语义	79
7.1 一眼看懂分类树	79
7.2 双寄存器执行模型	80
7.3 S-only、V-only 和 S/V 双版本	81
7.4 move 与跨域操作	82
7.5 整数与位运算	83
7.6 compare、select 与转换	84
7.7 浮点	85
7.8 普通访存与 mixed addressing	85
7.9 原子 load、store 和 RMW	86
7.10 W 控制流与隐藏重汇聚	87
7.11 SYNC	88
7.12 X 跨 lane	89
7.13 MMA	89
7.14 system、故障和边界	90

7.15 逐 form 生成要求	91
8. 合规性	92
8.1 结果和测试记录	92
8.2 测试向量格式	92
8.3 固定 64 位编码	93
8.4 通用执行与精确提交	95
8.5 双寄存器和 scalar-ready	95
8.6 跨域与 GETREG	97
8.7 整数、位运算和 pack	98
8.8 compare、select、转换和 FP	99
8.9 普通内存和 mixed addressing	100
8.10 S_ATOM/V_ATOM	102
8.11 W 控制流	103
8.12 SYNC 与 X	105
8.13 MMA	107
8.14 all-form 覆盖门禁	108
8.15 差分、形式属性和发布报告	110
附录A 指令形式参考 (自动生成)	112
NOP	112
TRAP	113
S_GETREG	115
V_GETREG	118
S_SETREG	124
V_SETREG	126

S_MOV	127
S_ADD	133
S_SUB	138
S_MUL	143
S_MAD	149
S_MUL.WIDE	150
S_MAD.WIDE	153
S_INT_MISC	156
S_DIV_REM	163
S_BITWISE	168
S_SHIFT_ROTATE	178
S_BITCOUNT	185
S_PACK	188
S_COMPARE	192
S_SELECT	215
S_CVT	218
S_F32_ARITH	226
S_F32_DIV	237
V_MOV	240
V_ADD	245
V_SUB	252
V_MUL	258
V_MAD	264
V_MUL.WIDE	266
V_MAD.WIDE	269

V_INT_MISC	273
V_DIV_REM	283
V_BITWISE	290
V_SHIFT_ROTATE	301
V_BITCOUNT	317
V_PACK	321
V_COMPARE	326
V_SELECT	357
V_CVT	361
V_F32_ARITH	370
V_F32_DIV	385
V_F16_ARITH	388
V_F16_CVT	405
V_PRED_LOGIC	408
S_LD	414
S_ST	424
V_LD	430
V_ST	472
S_ATOM	495
V_ATOM	584
SSY	711
BRA	712
BRA.P	714
JOIN	716

EXIT	717
CALL	719
RET	722
JUMP.IND	724
FENCE	725
BAR.SYNC	729
X_BROADCAST	731
S_READFIRST	734
V_VOTE	737
V_SHUFFLE	741
MMA	749

VTX-1 ISA 1.0 Draft

状态：Draft

SGPR + VGPR 架构

本规范同时定义每 warp 归属的标量寄存器（SGPR）与逐 lane 切片的向量寄存器（VGPR）。

- 指令家族：66
- 指令形式：379
- 指令字宽：64 位

VTX-1 ISA 1.0 Draft: 文档状态

1. 这份草案是什么

本文定义 VTX-1 ISA 1.0 Draft。ISA（指令集架构）是软件和处理器共同遵守的规则：软件按这些规则生成指令，处理器按这些规则执行指令。

“Draft（草案）”表示内容还在评审，字段和语义在正式冻结前仍可修改。一个实现只有完整满足本草案写明的强制要求，才可以声明“符合 VTX-1 ISA 1.0 Draft”。

本草案直接定义一套新的 1.0 架构。本文不定义任何其他版本的装载、转换或混用规则。

2. 哪些文件说了算

本草案的架构核心由下面四个文件组成：

- docs/00-status.md：说明文档状态、规范边界和要求用词；
- docs/01-terms-and-notation.md：统一术语、掩码和伪代码写法；
- docs/02-programming-model.md：定义内核描述符、参数、寄存器、资源和故障；
- docs/03-execution-model.md：定义 scalar、vector、warp-control 和分歧重汇聚的执行规则。

这四个文件对同一件事只能有一个定义。分工如下：

- 名词是什么意思，以 01-terms-and-notation.md 为准；
- 软件能看到哪些状态、启动时怎样分配资源，以 02-programming-model.md 为准；
- 一条指令在什么状态下能执行、执行后怎样改状态，以 03-execution-model.md 为准。

isa/vtx1/isa.yaml 是逐条指令元数据的权威位置：faults 字段定义故障码和名称，fault_priority 字段单独定义故障优先级，form 中的 execution_domain 和 required_state 定义执行域及额外入口状态。

docs/02-programming-model.md 必须逐字抄写 fault_priority，docs/03-execution-model.md 只能引用，不能另排顺序。七个合法 execution_domain 值也以 YAML 为权威。

这四个文件是架构状态和执行规则的权威位置。YAML 不得另写一套状态机。若 YAML 和文档冲突，规范发布必须停止，先把两边改一致，工具和实现不能自行挑一边继续。

未列入本节的材料不构成本 Draft 的架构核心要求，也不能用来填补这里没有写清的行为。

3. 要求用词

下列词具有固定含义：

- 必须（MUST）：所有符合实现都要这样做；
- 禁止（MUST NOT）：所有符合实现都不能这样做；
- 应当（SHOULD）：通常要这样做；不这样做时，实现者必须有明确理由；
- 不应当（SHOULD NOT）：通常不要这样做；这样做时，实现者必须有明确理由；
- 可以（MAY）：实现可以自行选择。

没有使用这些词的说明文字用于帮助理解；它不能推翻明确的强制规则。

4. 架构固定点

VTX-1 ISA 1.0 Draft 固定以下基本决定:

1. 一个 warp (线程束) 有 32 个 lane (通道), 共用一个程序计数器和一套隐藏的重汇聚栈。
2. 架构上, 每个 warp 有 s0..s255、vp0..vp15、SCC、只读 EXEC 和只读 LIVE; 每个 lane 有自己的 v0..v255。
3. SGPR (每 warp 一份的标量寄存器) 和 VGPR (每 lane 一份的向量寄存器) 的物理存储位于 SM/CU。SM/CU 是实际接收并执行 CTA 的计算单元。每个驻留 warp 从这些物理寄存器中分到自己的切片。
4. 每个 form (具体编码形式) 都必须标明 execution_domain (执行域), 取值只能是 system (系统操作)、scalar (每 warp 一次)、vector (每 lane 执行)、warp_control (warp 控制)、warp_collective (warp 集合操作)、cta_sync (CTA 同步)、warp_matrix (warp 矩阵操作)。
5. 机器码中的 8 个 class 只是编码分类: SYS、SALU、VALU、MEMORY、CONTROL、SYNC、CROSSLANE、MATRIX。class 不直接决定怎样执行; 例如 MEMORY class 里面既有 scalar form, 也有 vector form。
6. 所有 execution_domain=scalar 的指令每个 warp 只执行一次, 而且必须先满足 scalar-ready (标量就绪)。scalar-ready 要求 active_mask == live_mask、live_mask 非空, 并且没有处于 FIRST 或 SECOND 的未完成分歧帧。active_mask 是当前执行路径的 lane, live_mask 是尚未退出的 lane。
7. S_ALU (普通标量整数和逻辑运算)、S_FP (标量浮点)、S_GETREG (读取统一特殊寄存器)、SMEM (标量访存)、SATOM (标量原子操作)、S_READFIRST (读取最低编号 active lane) 都属于 execution_domain=scalar, 没有例外。
8. execution_domain=vector 的指令只对当前 active lane 执行, 不要求 scalar-ready。V1、V2、V3、VCMP 这四种向量编码格式各带一个 scalar-source selector 字段: V1 是 1 bit 的 ssrc, 其余三个是 2 bit 的 ssrc_sel。selector 为 0 时所有源寄存器号都在 VGPR 文件中解释; selector 非 0 时恰好一个源寄存器号改为在 SGPR 文件中解释, 同一个标量值对所有参与 lane 相同。一条 vector 指令最多只能有一个 SGPR 源, 架构中没有独立的广播指令。
9. CALL (直接调用)、CALL.IND (从 SGPR 取目标的间接调用)、JUMP.IND (从 SGPR 取目标的间接跳转)、RET (返回) 都是 execution_domain=warp_control, 但因为它们使用每 warp 一套的统一调用栈或统一间接目标, 所以必须额外满足 scalar-ready。直接 BRA 和 BRA.P 不要求 scalar-ready。
10. 软件不能写 EXEC。分歧、重汇聚和退出只能由 SSY、BRA.P、JOIN、EXIT 及其隐藏状态机处理。
11. 一个 CTA (线程块) 必须整体驻留在同一个 SM/CU 上; CTA 内各 warp 可以独立调度。
12. 每个 CTA 固定有 8 个屏障槽 0..7。owner 的唯一身份是 CTA 内 linear_tid = warp_id*32 + lane_id。live_owner_set 启动时是 CTA 内全部真实线程, 不含尾部不存在的 lane, 并且只在 EXIT 时收缩。
13. 唯一的规范屏障指令是 BAR.SYNC.CTA id。架构不提供 split (arrive/wait 分离) 屏障、屏障 token 或 generation 计数。屏障在槽的 arrived_set 等于 CTA 的 live_owner_set 时立即完成, 槽随即原子清回 idle; idle 的定义就是 arrived_set 为空且没有 waiter。
14. BAR.SYNC.CTA 阻塞整个 warp 的当前动态路径; 一个 waiter 固定保存 {warp_id, owner_snapshot, resume_pc}, 每 warp 同时至多一条 blocked record。阻塞和恢复都不改 active/live 掩码、重汇聚栈或调用栈, 挂起路径不能趁机切入。它要求 scalar-ready, 因此分歧 warp 在记录任何 arrival 之前就报故障。

这些固定点的完整定义分别见 02-programming-model.md 和 03-execution-model.md。

5. 符合性底线

符合实现必须做到：

- 在 CTA 开始执行前完成描述符、文本、参数和资源校验；
- 保持 s、v、vp、SCC、EXEC、LIVE 的可观察行为与本草案一致；
- 对任何不满足 scalar-ready 的 execution_domain: scalar form 准确报告 DIVERGENCE_FAULT；
- CALL、CALL.IND、JUMP.IND、RET、BAR.SYNC.CTA 的 required_state: scalar_ready 不满足时，同样报告 DIVERGENCE_FAULT，同时保持它们各自的执行域分类；
- 只通过规定的 warp-control 状态机改变 active lane 集和 live lane 集；
- 在一条指令故障时，不提交该指令的任何部分效果；
- 对每条向量指令最多解码出一个 SGPR 源；selector 落在保留编码上时报告 ILLEGAL_INSTRUCTION；
- 对屏障执行固定 owner 身份和 arrived_set == live_owner_set 的完成条件；EXIT 把退出线程从 live_owner_set 移除，但本身不产生 shared release；
- 只有全部 warp 完成且 8 个槽都处于 idle 状态时才完成 CTA；
- 只把成功 BAR.SYNC.CTA 的 release/acquire 用于 CTA shared memory；global、local、param、const 和 host 不因屏障自动得到顺序；
- 不把调度顺序、物理寄存器编号、缓存行为或实际执行周期暴露成未定义的架构承诺。

实现可以采用不同的流水线、寄存器文件组织和调度算法，只要软件看到的结果与本草案完全一致。

VTX-1 ISA 1.0 Draft: 术语与记号

1. 从设备到 lane

- device (设备) : 能接收并执行 VTX-1 内核的设备。
- kernel (内核) : 一段要在设备上并行执行的程序。
- launch (启动) : 用一组参数和网格尺寸运行一次内核。
- CTA (线程块) : 一组能共享 shared memory (共享内存) 并使用 CTA 屏障互相同步的线程。
- grid (网格) : 一次启动里的全部 CTA。
- SM/CU (计算单元) : 设备中真正接收 CTA、保存其驻留状态并发射指令的硬件单元。不同产品可以把它叫 SM 或 CU; 本文把两种叫法写成 SM/CU。
- warp (线程束) : 32 个 lane 组成的执行组。一个 warp 共用一个 PC (程序计数器) 和一套控制状态。
- lane (通道) : warp 中编号为 0..31 的单个执行位置, 对应一个软件线程。
- resident (驻留) : CTA 及其 warp 的寄存器、共享内存和控制状态已经分配在某个 SM/CU 上, 可以被该 SM/CU 调度。

CTA 的三维线程号按 X 最快的顺序变成一维编号:

```
linear_tid = tid_x + ntid_x * (tid_y + ntid_y * tid_z)
warp_id   = floor(linear_tid / 32)
lane_id   = linear_tid mod 32
```

最后一个 warp 可能没有装满。若 linear_tid 不小于 CTA 的真实线程数, 该位置叫 不存在 lane。不存在 lane 永远不进入 LIVE (存活掩码) 或 EXEC (当前执行掩码), 也不读写寄存器、内存和屏障状态。

2. 调度和占用率

- schedule (调度) : SM/CU 从当前可以运行的 warp 中挑一个发射下一条指令。
- occupancy (占用率) : 一个 SM/CU 能同时驻留多少 CTA 或 warp。它受后文定义的 SGPR (每 warp 一份的寄存器)、VGPR (每 lane 一份的寄存器)、共享内存、重汇聚栈和实现上限共同约束。
- warp 独立调度: 同一 CTA 的 warp 不必同一周期执行, 也不保证按 warp_id 顺序执行。一个 warp 阻塞时, 另一个就绪 warp 可以继续。

一个 CTA 必须整体驻留在一个 SM/CU 上, 禁止把同一 CTA 的不同 warp 拆到不同 SM/CU。这样 CTA 的 shared memory 和 CTA 屏障始终由同一个 SM/CU 管理。

3. 寄存器术语

- SGPR (标量通用寄存器) : 每个 warp 只有一份的 32 位寄存器, 名字是 s0..s255。同一 warp 的所有 lane 看到同一个 sN 值。
- VGPR (向量通用寄存器) : 每个 lane 各有一份的 32 位寄存器, 名字是 v0..v255。vN 在 32 个 lane 上可以保存 32 个不同值。
- vp0..vp15 (lane 掩码寄存器) : 每个 warp 有 16 个, 每个都是 32 位; 位 i 控制 lane i。
- SCC (标量条件码) : 每个 warp 一个 1 位状态。只有把 SCC 明确列为源操作数的指令才读取它; SCC 不会自动变成所有 scalar 指令的开关。

- EXEC: 只读的 32 位当前执行掩码, 数值等于 active_mask。
- LIVE: 只读的 32 位存活掩码, 数值等于 live_mask。
- scalar source (标量源): 一条 vector 指令中被 scalar-source selector 指定为读 SGPR 的那个源操作数。它的 32 位值对所有参与 lane 都相同, 因此天然具有广播效果。一条 vector 指令最多有一个标量源。
- scalar-source selector (标量源选择器): V1、V2、V3、VCMP 格式中的一个编码字段, 决定哪个源寄存器号在 SGPR 文件中解释。V1 用 1 bit 的 ssrc, 其余三个用 2 bit 的 ssrc_sel; 值 0 表示没有标量源。
- register slice (寄存器切片): SM/CU 从物理 SGPR/VGPR 容量中划给一个驻留 warp 的那一部分。

s0 表示整个 warp 共用的一个 32 位值, 不是 32 份值。v0 表示每个 lane 各有一个 32 位值, 因此一个完整 warp 的 v0 一共有 32 份值。

SGPR 和 VGPR 的物理寄存器文件位于 SM/CU。架构寄存器名不等于固定物理编号; 实现可以改名、分 bank (分存储组) 或在不被软件发现的情况下搬移数据。

寄存器只保存位模式。SGPR、VGPR、vp 和 SCC 都不携带隐藏的影子状态, 也没有任何架构可见的标签跟着寄存器值传播。

4. lane 掩码

本文把 lane 集合写成 32 位掩码。位 i 为 1 表示集合里有 lane i。

- live_mask: 属于该 warp、尚未执行 EXIT 的真实 lane。架构寄存器 LIVE 是它的只读视图。
- active_mask: 当前 PC 上实际走这条控制路径的 lane。架构寄存器 EXEC 是它的只读视图。
- entry_mask: 一条动态指令开始时对 active_mask 拍下的快照。
- guard_mask: vector 指令由 vpN 或 !vpN 算出的 lane 掩码。
- participating_mask: vector 指令真正产生效果的 lane, 等于 entry_mask & guard_mask。
- suspended lane (挂起 lane): 仍在 live_mask 中, 但当前不在 active_mask 中; 它正在等待另一条分支路径执行完。

任何时候都必须满足:

```
active_mask & ~live_mask == 0
```

也就是 active lane 一定还是 live lane。写 EXEC 或 LIVE 不是普通寄存器操作; 任何试图把它们作为目标操作数的编码都非法。

5. 执行域和机器 class

每个指令 form (具体编码形式) 都有一个 execution_domain (执行域) 字段。它只允许七个值:

- system: 系统控制、陷阱或杂项操作;
- scalar: 每个 warp 执行一次;
- vector: 对 participating lane 分别执行;
- warp_control: 管理 warp 的 PC、当前路径、重汇聚栈或调用栈;
- warp_collective: 让一个 warp 的多个 lane 一起完成投票、shuffle 等集合操作;
- cta_sync: 让一个 CTA 内的线程做屏障或内存同步;
- warp_matrix: 让一个 warp 合作完成矩阵操作。

机器码另有 8 个 class (编码类) : SYS、SALU、VALU、MEMORY、CONTROL、SYNC、CROSSLANE、MATRIX。class 只负责把 64 位机器字分区解码, 不直接规定执行方式。最容易混淆的是 MEMORY: 同一个 class 里, 标量加载、存储和原子 form 的执行域是 scalar, 逐 lane 访存 form 的执行域是 vector。

scalar-ready (标量就绪) 表示 warp 同时满足三件事: live_mask 非空、active_mask == live_mask, 并且重汇聚栈里没有 FIRST 或 SECOND 帧。

execution_domain 说明“这条 form 怎样执行”, required_state 说明“执行前还要满足什么状态”。所有 execution_domain: scalar form 都必须写 required_state: scalar_ready。这个规则不看机器 class, 也不看它属于算术、浮点、特殊寄存器、访存还是原子操作。

execution_domain: vector 不检查 scalar-ready。普通 warp_control 也不检查; 直接 BRA 和 BRA.P 在分歧路径中仍可运行。CALL、CALL.IND、JUMP.IND、RET 是明确例外: 它们写有 required_state: scalar_ready, 但执行域仍是 warp_control。BAR.SYNC.CTA 同样写有 required_state: scalar_ready, 执行域是 cta_sync。

SSY、BRA、BRA.P、JOIN、EXIT 都是 warp_control。它们不属于 scalar ALU。即使当前 warp 处于分歧路径, 它们仍按 03-execution-model.md 的控制规则执行。

6. PC、静态指令和动态指令

- PC (程序计数器) : warp 下一条要执行的指令地址, 用相对当前内核文本起点的字节偏移表示。
- 静态指令: 内核文本中某个 PC 上保存的指令。
- 动态指令: 某个 warp 实际执行某条静态指令的一次事件。两个 warp 执行同一个 PC, 算两个不同动态指令。

本 Draft 中每条指令是 8 字节, 因此:

```
next_pc(PC) = PC + 8
```

合法 PC 必须 8 字节对齐, 并且整条指令都位于文本范围内:

```
PC mod 8 == 0
0 <= PC
PC + 8 <= text_size
```

7. 分歧和重汇聚

- divergence (分歧) : 同一 warp 中, 一部分 active lane 走分支目标, 另一部分走顺序下一条。
- reconvergence (重汇聚) : 两条分支路径都处理完后, 把仍存活的 lane 合回同一个 active_mask。
- reconvergence stack (重汇聚栈) : 每个 warp 隐藏的一组后进先出帧, 保存汇合 PC、待执行路径和已到达 lane。
- ARMED: SSY 已经预约汇合点, 但还没有出现需要拆成两路的分歧。
- FIRST: 第一条路径正在执行, 第二条路径已保存在栈帧里。
- SECOND: 第二条路径正在执行, 第一条路径的 lane 正在汇合点等待。

FIRST 或 SECOND 帧叫 未完成分歧帧。即使某个时刻 active_mask 恰好又等于 live_mask, 只要栈里还有这类帧, scalar 指令仍然不合法。

CTA 屏障使用这些固定术语:

- slot (槽) : 每 CTA 的 8 个编号槽之一, id 为 0..7;
- owner identity (owner 身份) : 只使用 CTA 内 linear_tid = warp_id*32 + lane_id; 二元组 (warp_id, lane_id) 只是等价表示, 所有架构集合和比较都以 linear_tid 为元素;

- live owner set (存活 owner 集)：仍需在屏障处被等待的 linear_tid 集合。CTA 启动时它是全部真实线程，不含不存在的尾 lane；EXIT 把退出线程从其中移除；
- arrived set (已到达集合)：当前尚未完成的这次屏障中已经成功到达的 linear_tid 集合；同一 linear_tid 只能进入一次；
- idle slot (空闲槽)：arrived_set 为空且没有 waiter 的槽。

“active owner”集合 A 是把某条 BAR.SYNC.CTA 动态指令入口 active_mask 的每个置位 lane 转成 linear_tid 后得到的集合。因为 BAR.SYNC.CTA 要求 scalar-ready，A 恰好是该 warp 当前全部 live lane，不存在挂起路径或已退出 lane 混入的情况。

屏障阻塞记录固定写成：

```
BarrierWaitRecord {
    warp_id: U32
    owner_snapshot: set<linear_tid>
    resume_pc: U64
}
```

owner_snapshot 是入口 A 的冻结副本，resume_pc=old_PC+8。BAR.SYNC.CTA 阻塞的是整个 warp 当前动态路径。阻塞时 warp 的 PC 留在屏障指令上，active_mask/live_mask、重汇聚栈和调用栈保持不变；挂起路径不能切入。每个 warp 同一时刻至多有一条 blocked record。恢复只清除该记录、写 PC=resume_pc 并把 warp 置为 ready，其他状态不变。

8. 数值和位写法

- U8/U16/U32/U64：对应位宽的无符号整数。
- S8/S16/S32/S64：对应位宽的二补数有符号整数。
- F16/F32：IEEE 754 binary16 和 binary32 位型。
- x[n:m]：从位 n 到位 m，两端都包含；位 0 是最低位。
- bit(i)：只有位 i 为 1 的 32 位掩码。
- popcount(x)：掩码 x 中 1 的个数。
- lowest(x)：非零掩码中编号最小的置位。
- [base, base+size)：从 base 开始、到 base+size 之前结束的半开区间。
- align_up(x,a)：把 x 向上补到 a 的倍数；a 必须是 2 的幂。
- UNSPEC：位宽确定，但初始值不保证是什么。程序必须先写后读，不能依赖它恰好为零。

所有指令、参数和内存中的多字节数都使用小端格式，也就是低有效字节放在低地址。

地址和大小检查先用不会溢出的数学整数计算，再判断是否越界。禁止先按 U32 或 U64 回绕，再把回绕后的地址当成合法地址。

9. 伪代码约定

本文伪代码中的常用操作如下：

```
snapshot(x)          # 为当前动态指令保存不再变化的副本
fault(code, mask, aux) # 本动态指令不提交，并报告故障
block(reason)         # 保存当前状态，等待条件满足后继续
commit(effects)       # 把整条指令的效果一次性变为可见
clear(slot)           # 清空 arrived_set 和 waiter 列表，槽回到 idle
```

除屏障等明确写成两阶段的指令外，一条动态指令必须“先检查，后整体提交”：

```
I = decode_and_validate(fetch64(PC))
E = snapshot(active_mask)
sources = snapshot_required_sources(I, E)
effects = evaluate_and_validate(I, E, sources)
if effects has fault:
    fault(...)
else:
    commit(effects)
```

提交前，目标寄存器、内存、PC、lane 掩码和重汇聚栈都不能出现部分更新。

10. 就绪、阻塞和完成

- ready（就绪）：warp 有非空 active_mask，也没有在等待屏障、内存或其他事件。
- blocked（阻塞）：warp 还没完成，但眼下不能发射下一条指令；屏障 blocked record 存在时整个 warp 都不能切换到挂起路径。
- complete（完成）：warp 的 live_mask 已经为空，而且没有未清理的重汇聚或同步状态。
- deadlock（死锁）：内核还没完成，却没有任何 warp 能继续，也没有已在途事件能让 warp 重新就绪。
- livelock（活锁）：指令一直在执行，但程序永远达不到完成状态。

具体完成和死锁条件见 03-execution-model.md。

VTX-1 ISA 1.0 Draft: 编程模型

1. 内核描述符

每个可启动内核必须带一个 kernel descriptor（内核描述符）。它是一张 80 字节的小表，告诉运行时入口在哪里、内核会用多少寄存器和每个 CTA（线程块）要多少存储。

描述符必须按 8 字节对齐，字段使用小端格式：

下表中的 SGPR 是每 warp 一份的 32 位寄存器，VGPR 是每 lane 一份的 32 位寄存器，vp0..vp15 是每 warp 一份的 32 位 lane 掩码寄存器。shared memory 是 CTA 共用的存储，local memory 是每个 lane 私有的存储。

偏移	大小	字段	1.0 Draft 要求
0	4	magic	字节串 VTXK
4	2	descriptor_size	必须为 80
6	2	descriptor_version	必须为 1
8	2	isa_major	必须为 1
10	2	isa_minor	必须为 0
12	4	flags	保留，必须为 0
16	8	entry_pc	PC（程序计数器）的入口值，相对内核文本起点的字节偏移
24	8	text_size	内核文本总字节数
32	2	sgpr_count	每 warp 分配的 SGPR 数，范围 16..256
34	2	vgpr_count	每 lane 分配的 VGPR 数，范围 0..256
36	2	vp_count	每 warp 分配的 vp 寄存器数，范围 0..16
38	2	reconv_stack_depth	每 warp 所需重汇聚栈深度，范围 0..16
40	4	static_shared_size	每 CTA 的静态 shared memory 字节数
44	4	dynamic_shared_max	每 CTA 允许请求的动态 shared memory 上限
48	4	local_size_per_lane	每个真实 lane 的 local memory 字节数
52	4	param_logical_size	参数中最后一个有效字节的独占末端
56	8	param_layout_offset	从描述符起点到参数布局表的字节偏移

偏移	大小	字段	1.0 Draft 要求
64	4	param_count	参数布局记录数
68	4	reserved0	必须为 0
72	2	call_stack_depth	每 warp 的统一调用栈最大深度，范围 0..16
74	6	reserved1	必须全部为 0

text_size 必须非零且是 8 的倍数。entry_pc 必须满足：

```
entry_pc mod 8 == 0
entry_pc + 8 <= text_size
```

描述符中的资源数字是硬要求，不是性能提示。运行时不能用更小的值启动内核，也不能等执行到一半才发现资源不够。

1.1 三个寄存器计数字段

sgpr_count、vgpr_count、vp_count 分别给出该内核可以引用的 s、v、vp 编号上界：

```
sN 合法，当且仅当 0 <= N < sgpr_count
vN 合法，当且仅当 0 <= N < vgpr_count
vpN 合法，当且仅当 0 <= N < vp_count
```

参数窗口固定使用 s0..s15，所以 sgpr_count 不能小于 16。64 位值占两个连续寄存器时，低 32 位放在较小编号，起始编号必须为偶数，并且两个寄存器都要低于相应 count。

SCC、EXEC 和 LIVE 不计入 sgpr_count。它们是单独的架构状态。

reconv_stack_depth 是静态控制流分析得到的最大同时嵌套帧数。文本需要的深度超过声明值是无效描述符；声明值超过 16 也无效。

call_stack_depth 是每个 warp 最多能同时保存多少个返回地址。它必须满足 0..16；16 是 YAML architectural_limits.call_stack_depth 给出的架构最大值。值 0 表示没有可用调用帧；此时执行任何 CALL 或 CALL.IND 都会因为栈已满而故障。每个调用帧只保存一个 return_pc，也就是调用结束后要回去的 PC。

2. 启动前校验

运行时必须在任何 CTA 开始执行前完成下面的检查：

1. 检查描述符大小、版本、保留字段和所有范围；
2. 检查完整文本，每个 form 的 execution_domain 是七个权威值之一；所有 execution_domain: scalar form 以及 CALL/CALL.IND/JUMP.IND/RET 都必须写 required_state: scalar_ready；
3. 检查每个 sN、vN、vpN 和连续寄存器组没有超过 descriptor count；
4. 检查任何目标操作数都不是只读 EXEC 或 LIVE；
5. 检查 SSY、BRA.P、JOIN 的结构化控制流和最大重汇聚栈深度；调用栈是否超深在每次 CALL 执行时精确检查；
6. 检查 grid 和 CTA 三维尺寸都非零，CTA 线程总数不超过实现上限；
7. 检查参数布局、参数值和参数存储大小；
8. 检查 requested_dynamic_shared <= dynamic_shared_max；
9. 检查 static_shared_size + requested_dynamic_shared 没有溢出且不超过单 CTA 上限；

10. 检查至少有一个 SM/CU 能同时容纳整个 CTA 的寄存器、shared memory、local memory 和控制状态。

任何一步失败都是 launch error（启动错误）：内核一条指令也不执行，不得留下半个已启动 CTA。运行时应当区分：

- INVALID_DESCRIPTOR：描述符字段错误；
- INVALID_TEXT：指令或静态操作数错误；
- INVALID_CONTROL_FLOW：控制流结构或栈深错误；
- INVALID_ARGUMENT：参数布局或参数值错误；
- INVALID_RESOURCE：单个 CTA 无法放进任何 SM/CU。

3. 参数 ABI

ABI（应用二进制接口）是调用者把参数交给内核的固定摆放规则。

3.1 参数布局记录

参数布局表含 param_count 条记录，每条 16 字节：

记录偏移	大小	字段	含义
0	4	byte_offset	参数在参数块里的起始字节
4	2	type	参数类型码
6	2	element_count	数组元素数
8	4	byte_size	参数总字节数
12	4	flags	保留，必须为 0

1.0 Draft 定义这些类型：

类型码	类型	单元素大小	自然对齐
0x0001	U8	1	1
0x0002	S8	1	1
0x0003	U16	2	2
0x0004	S16	2	2
0x0005	U32	4	4
0x0006	S32	4	4
0x0007	U64	8	8
0x0008	S64	8	8
0x0009	F16	2	2
0x000A	F32	4	4
0x000B	GLOBAL_PTR	8	8
0x000C	CONST_PTR	8	8

类型码	类型	单元素大小	自然对齐
0x000D	BYTES	1	1

记录必须按 `byte_offset` 严格递增，字段不能重叠，起点必须满足自然对齐。固定大小类型必须满足：

```
element_count >= 1
byte_size == element_count * element_size
```

BYTES 的 `element_count` 必须为 1，`byte_size` 必须非零。最后一个字段的末端必须正好等于 `param_logical_size`。字段之间可以留空；空出来的字节由运行时清零。

没有参数时，必须同时满足：

```
param_count == 0
param_logical_size == 0
param_layout_offset == 0
```

有参数时，`param_layout_offset` 必须 8 字节对齐，布局表必须完整位于模块内，且不能与描述符或文本重叠。

3.2 参数存储和 SGPR 窗口

运行时实际建立的参数存储大小为：

```
param_storage_size = max(64, align_up(param_logical_size, 16))
```

运行时先把整块参数存储清零，再按布局记录写入调用者提供的参数。参数存储在启动后只读。

参数的前 64 字节固定复制到 `s0..s15`：

```
s0 = parameter bytes [0, 4)
s1 = parameter bytes [4, 8)
...
s15 = parameter bytes [60, 64)
```

每个 `sN` 按小端方式得到一个 32 位值。参数不足 64 字节的尾部已经清零，因此对应 SGPR 也为零。

这份复制对 CTA 中每个 warp 都做一次，所以每个 warp 启动时的 `s0..s15` 相同。超过 64 字节的参数通过只读 `param` 地址空间和参数加载指令读取。

`GLOBAL_PTR` 和 `CONST_PTR` 只是参数布局记录上的静态声明，供启动时校验参数块并确定该指针指向哪个窗口。它们不给运行期的寄存器值附加任何身份：写进 SGPR 之后就是普通的 64 位数值。一次访存落在哪个地址空间完全由指令 `opcode` 决定，与地址值本身无关。

4. 架构寄存器

4.1 每 warp 状态

每个 warp 有：

- `s0..s255`：256 个架构可命名的 32 位 SGPR，实际可用上界由 `sgpr_count` 给出；
- `vp0..vp15`：16 个架构可命名的 32 位 lane 掩码寄存器，实际可用上界由 `vp_count` 给出；
- SCC：1 位标量条件码；只有明确列出 SCC 源操作数的指令才读取它；
- EXEC：32 位只读 active lane 掩码；
- LIVE：32 位只读 live lane 掩码；
- 一个 PC；
- 一套隐藏重汇聚栈；

- 一套隐藏的 warp 统一调用栈，每帧只有 return_pc;
- 阻塞和故障状态。

4.2 每 lane 状态

每个真实 lane 有 v0..v255，每个都是 32 位；实际可用上界由 vgpr_count 给出。

例如，v3 不是整个 warp 共用的值。lane 0 的 v3 和 lane 1 的 v3 是两个独立的 32 位值。

VGPR 只保存 32 位模式。架构不给寄存器附加任何影子状态：没有 barrier token 标签，也没有地址 provenance 标签。任何一条写 VGPR 的 form 只改写这 32 位数值，实现不得让软件观察到额外的隐藏 per-register 状态。

4.3 vp 写入规则

vector 比较等产生 vpN 结果时，只改参与 lane 对应的位，其他位保持原值：

```
new_vp = (old_vp & ~participating_mask)
        | (computed_bits & participating_mask)
```

读取 vpN 不会自动把它与 LIVE 或 EXEC 合并。vector 指令形成参与集时，才按 03-execution-model.md 显式计算。

4.4 EXEC 和 LIVE 只读

软件可以读取 EXEC 和 LIVE，但禁止写它们：

- EXEC 随当前控制路径变化；
- LIVE 只会在 lane 成功执行 EXIT 后清位；
- MOV、逻辑运算、寄存器恢复或任何普通目标写入都不能修改它们；
- 把 EXEC 或 LIVE 编成目标操作数，启动校验报 INVALID_TEXT；若非法编码绕过启动校验，执行时报 ILLEGAL_OPERAND。

软件若要按条件执行 vector 指令，应写 vpN 并把它用作 vector guard，不能改写 EXEC。

5. 物理寄存器和 occupancy

架构寄存器说明“软件能看到什么”，物理寄存器说明“SM/CU 实际拿什么存”。

SGPR 和 VGPR 的物理存储都位于 SM/CU。CTA 驻留时，SM/CU 为其中每个 warp 分配寄存器切片：

```
每 warp SGPR 需求 = sgpr_count 个 32 位槽
每 warp VGPR 需求 = 32 * vgpr_count 个 32 位槽
每 warp vp 需求 = vp_count 个 32 位掩码槽
```

不存在 lane 在架构上没有值，但实现可以为了规则整齐，仍按完整 32-lane 切片预留 VGPR 物理空间。资源准入必须按实现公开的计算方法执行，不能少分后让两个驻留 warp 互相覆盖。

一个 CTA 的寄存器需求等于其全部 warp 需求之和。再加上：

- static_shared_size + requested_dynamic_shared 字节 shared memory；
- 每个真实 lane 的 local_size_per_lane 字节 local memory 配额；
- 每 warp 的重汇聚栈、call_stack_depth 个返回地址槽和其他控制状态；
- 实现规定的 CTA/warp 数量上限。

这些资源一起决定 occupancy。sgpr_count 或 vgpr_count 越大，同一 SM/CU 能同时驻留的 warp 通常越少；shared memory 越大，同一 SM/CU 能同时驻留的 CTA 通常也越少。

资源不足以再放一个 CTA 时，该 CTA 等待，不得只把它的一部分 warp 放进去。已经驻留的 CTA 释放足够资源后，等待 CTA 才能整体进入。

6. CTA 驻留和调度

一个 CTA 的所有 warp 必须驻留在同一个 SM/CU，直到 CTA 完成或内核故障。原因很直接：它们共用同一块 shared memory，并通过 CTA 屏障同步。

“整体驻留”不等于“同时执行”。CTA 内每个 warp 有自己的 PC、EXEC、LIVE、寄存器切片、重汇聚栈和调用栈。SM/CU 可以逐周期独立挑选就绪 warp：

- 不保证低编号 warp 先运行；
- 不保证同一 CTA 的 warp 轮流运行；
- 一个 warp 等内存或屏障时，其他 warp 可以运行；
- 软件不能把调度先后当成同步手段。

7. 启动初态

每个 warp 启动时：

```
PC          = entry_pc
live_mask    = 本 warp 真实 lane 的位图
active_mask  = live_mask
reconv_stack = empty
call_stack   = empty
blocked      = false
blocked_record = none

s0..s15     = 参数前 64 字节
s16..s255   = UNSPEC
vp0..vp15   = 0
SCC         = 0
v0..v255    = UNSPEC (对每个真实 lane 分别成立)
EXEC        = active_mask 的只读视图
LIVE        = live_mask 的只读视图
```

超过 descriptor count 的寄存器虽然有架构名字，但该内核不能引用。UNSPEC 不表示随机数生成器；它只表示规范不保证初值，程序必须先写后读。

每个 CTA 启动时得到：

- 一块 static_shared_size + requested_dynamic_shared 字节的 shared memory；
- 每个真实 lane 一块 local_size_per_lane 字节的 local memory；
- 8 个 CTA 屏障槽 0..7，每槽初始化为 arrived_set=empty、waiters=empty，也就是 idle；
- 一个 CTA 级 live_owner_set，初值是 CTA 启动时全部真实线程的 linear_tid；
- CTA 和 grid 的坐标及尺寸。

linear_tid = warp_id*32+lane_id 是 owner 的唯一集合元素；不存在的尾 lane 从一开始就不在 live_owner_set 中。EXIT 把退出线程的 linear_tid 从 live_owner_set 移除。shared memory 和 local memory 的初始数据为 UNSPEC，除非其他章节对某段存储明确规定清零。

8. 特殊只读信息

实现必须让指令能读取下列只读信息。具体编码由指令清单定义：

名称	位宽	内容
LANE_ID	32	warp 内 lane 号 0..31
WARP_ID	32	CTA 内 warp 号
TID_X/Y/Z	各 32	CTA 内线程坐标
CTA_ID_X/Y/Z	各 32	grid 内 CTA 坐标
NTID_X/Y/Z	各 32	CTA 三维尺寸
NCTA_X/Y/Z	各 32	grid 三维尺寸
SM_ID	32	当前驻留 SM/CU 的实现定义编号
WARP_SIZE	32	恒为 32
DYNAMIC_SHARED_SIZE	32	本 CTA 的动态 shared memory 字节数
PARAM_BASE	64	只读参数存储的字节 0 地址
EXEC	32	当前动态指令入口的 active lane 掩码
LIVE	32	当前动态指令入口的 live lane 掩码

vector 读取 lane 相关信息时，每个参与 lane 得到自己的值。execution_domain=scalar 的指令通常只能读取 warp 或 CTA 一致的信息；尝试用普通 scalar 指令读取 LANE_ID 或线程坐标这类逐 lane 值，必须报 ILLEGAL_OPERAND。S_READFIRST 是明确例外：它先通过 scalar-ready 检查，再按指令定义读取最低编号 active lane 的 VGPR。

9. 精确故障

精确故障表示故障指令本身没有留下任何部分效果。PC、寄存器、内存、lane 掩码、重汇聚栈和调用栈都停在该指令入口状态。

isa/vtx1/isa.yaml 的 faults 字段是故障码和名称的权威位置；fault_priority 是优先级的独立权威字段。下面先按故障码列出名称，随后逐字抄写 fault_priority。两处不一致时必须阻断发布。

码	名称	直接含义
0x0001	ILLEGAL_INSTRUCTION	指令字、操作码或保留位非法
0x0002	ILLEGAL_OPERAND	寄存器类别、编号、目标或动态操作数组合非法

码	名称	直接含义
0x0003	DIVERGENCE_FAULT	要求 warp 完全重汇聚的 form 在非 scalar-ready 状态下执行；覆盖全部 execution_domain: scalar form，以及 CALL、CALL.IND、JUMP.IND、RET、BAR.SYNC.CTA 上写明的 required_state: scalar_ready
0x0004	RECONVERGENCE_FAULT	重汇聚栈、帧、目标或 JOIN 顺序错误，调用栈上溢或下溢，以及 RET 时仍有未关闭的 callee 帧
0x0005	MISALIGNED_ACCESS	内存访问没有满足对齐要求
0x0006	MEMORY_BOUNDS	地址越过对应地址空间
0x0007	INTEGER_FAULT	除零等已定义整数错误
0x0008	COLLECTIVE_FAULT	warp 集合指令或矩阵指令的参与规则错误
0x0009	SOFTWARE_TRAP	程序主动执行 TRAP
0x000A	DEADLOCK	满足架构死锁条件

故障记录至少包含：

```
FaultRecord {
  code: U16,
  pc: U64,
  cta_id: (U32, U32, U32),
  warp_id: U32,
  lane_mask: U32,
  address_or_aux: U64
}
```

DIVERGENCE_FAULT 是 warp 状态错误，lane_mask 必须记录指令入口的 active_mask，address_or_aux 必须为 0。它覆盖所有 execution_domain: scalar form，也覆盖 warp_control 类 CALL、CALL.IND、JUMP.IND、RET 以及 cta_sync 类 BAR.SYNC.CTA 上明确写出的 required_state: scalar_ready。完整触发条件见 03-execution-model.md。

DIVERGENCE_FAULT 和 RECONVERGENCE_FAULT 的分界是固定的：前者只表示“这条 form 需要一个完全重汇聚的 warp，而当前 warp 不是”，它在指令执行前由入口状态检查产生，不改变任何控制状态；后者表示重汇聚状态机或调用栈本身被用错，例如 JOIN 与栈顶帧阶段不符、SSY 目标非法、调用栈上溢或下溢。同一条动态指令若两者都成立，按 fault_priority 先报 DIVERGENCE_FAULT。

同一动态指令同时发现多个问题时，必须逐字采用 YAML 的 fault_priority：

```
fault_priority:
- ILLEGAL_INSTRUCTION
- ILLEGAL_OPERAND
- DIVERGENCE_FAULT
- RECONVERGENCE_FAULT
- COLLECTIVE_FAULT
- INTEGER_FAULT
- MISALIGNED_ACCESS
- MEMORY_BOUNDS
- SOFTWARE_TRAP
```

上面的先后次序就是 YAML `fault_priority`，排在前面的故障优先。DEADLOCK 不在该字段中；它是没有动态指令可继续时才判定的全局状态，不参加同一动态指令的竞争。其他章节只能引用 `fault_priority`，不能从 `faults` 的编号或排列自行推导优先级。

启动时能发现的静态错误应当在启动前拒绝。运行时故障发生后，整个 kernel 进入失败态，任何 CTA 都不能再提交新指令。

VTX-1 ISA 1.0 Draft: 执行模型

1. 一句话理解执行方式

VTX-1 使用 SIMT (单条指令管理多个线程) 方式执行。一个 warp (线程束) 只有一个 PC (程序计数器) ; EXEC (当前执行掩码) 中为 1 的 lane (通道) 在这个 PC 上一起执行同一条指令。

SGPR 是每 warp 一份的 32 位寄存器, VGPR 是每 lane 一份的 32 位寄存器, vp0..vp15 是每 warp 一份的 lane 掩码寄存器, SCC 是每 warp 一份的 1 位条件码。CTA (线程块) 是一组能共享内存并使用屏障同步的线程。

不同 warp 独立调度。实现可以让多个 warp 同时前进, 也可以交错发射, 但每个 warp 自己看到的指令顺序必须符合本文。

每个指令 form 都带 execution_domain (执行域), 取值只能是:

- system: 系统控制、陷阱和杂项;
- scalar: 每个 warp 执行一次;
- vector: 每个参与 lane 各执行一次;
- warp_control: 管理 PC、当前路径、重汇聚栈或调用栈;
- warp_collective: 一个 warp 的多个 lane 合作;
- cta_sync: 一个 CTA 内的线程同步;
- warp_matrix: 一个 warp 合作做矩阵运算。

机器码的 SYS/SALU/VALU/MEMORY/CONTROL/SYNC/CROSSLANE/MATRIX 是 8 个编码 class, 不是执行域。尤其是 MEMORY class: 标量访存 form 仍按 scalar 执行, 逐 lane 访存 form 仍按 vector 执行。

2. 每条动态指令的共同步骤

除明确写成阻塞式两阶段操作的指令外, 一条动态指令按下面的逻辑执行:

```
word = fetch64(PC)
I = decode_and_validate_static(word)
E = snapshot(active_mask)
L = snapshot(live_mask)

validate_execution_domain_state(I, E, L)
sources = snapshot_required_sources(I, E)
effects = evaluate_and_validate(I, E, L, sources)

if any check failed:
    fault(select_fault_by_priority)
else:
    commit(effects)
```

顺序含义如下:

1. 先检查机器码和静态操作数, 所以非法编码不能被 guard 掩盖;
2. 再保存入口 EXEC 和 LIVE;
3. 再按 execution_domain 检查当前 warp 状态是否合法;
4. 所有需要的源值先读完;
5. 所有结果和故障先算完;

6. 最后一次性提交。

若指令故障，该指令不得留下部分 SGPR、VGPR、vp0..vp15、SCC、内存、PC、EXEC、LIVE 或栈更新。

YAML 必须给每个 execution_domain: scalar form 写 required_state: scalar_ready，也必须给 CALL、CALL.IND、JUMP.IND、RET 和 BAR.SYNC.CTA 写同一个 required_state。执行时按这个字段检查：

```
if l.required_state == scalar_ready:
    require scalar_ready()
else:
    no scalar-ready check
```

这里的额外检查不会改变执行域：CALL、CALL.IND、JUMP.IND、RET 仍是 warp_control，BAR.SYNC.CTA 仍是 cta_sync。直接 BRA 和 BRA.P 不做 scalar-ready 检查。

同一动态指令发现多个故障时，只使用 YAML 的 fault_priority；02-programming-model.md 第 9 节逐字抄写了该字段。本文件不从 faults 列表或故障码另排第二套顺序。

普通非控制指令成功后，若该指令没有另行规定 PC 效果：

```
PC = old_PC + 8
```

3. execution_domain=scalar

3.1 所有 scalar 执行域都一样检查

execution_domain=scalar 的指令每个 warp 只执行一次，不会因为 warp 有 32 个 active lane 就执行 32 次。

下面这些常见家族全部必须标成 execution_domain=scalar，并在读取动态源之前检查 scalar-ready：

- S_ALU：SGPR 整数、位运算、比较等普通标量运算；
- S_FP：SGPR 浮点运算；
- S_GETREG：读取 warp 或 CTA 一致的特殊寄存器；
- SMEM：用统一地址做一次标量内存访问；
- SATOM：用统一地址做一次标量原子操作；
- S_READFIRST：从当前最低编号 active lane 的 VGPR 取一个值，写入 SGPR。

这张表不是举例，而是硬规则：这些家族的所有 form（具体编码形式）都要先满足 scalar-ready。不能因为 SMEM 或 SATOM 属于访存，也不能因为 S_READFIRST 会读取一个 VGPR，就跳过检查。

除 S_READFIRST 这类明确写出取 lane 规则的指令外，execution_domain=scalar 的指令只能读取该指令允许的 SGPR、SCC 和 warp/CTA 一致的只读信息。它不能随便读取某个 vN 来猜一个代表 lane，也不能读取 LANE_ID、逐线程坐标等每 lane 不同的值。

3.2 Scalar 合法条件

每条 execution_domain=scalar 指令在读取动态源之前，必须检查下面三个条件：

```
live_mask != 0
active_mask == live_mask
reconv_stack 中不存在 phase 为 FIRST 或 SECOND 的帧
```

三个条件必须同时成立。

大白话解释：

- warp 里至少还有一个活线程；

- 所有活线程此刻都在同一条路径上;
- 栈里没有一条分支路径还没跑完。

这三个条件合起来就叫 `scalar_ready()`。只看 `active_mask == live_mask` 不够。某些控制流在中间时刻可能碰巧让两个掩码相等，但只要栈里仍有 FIRST 或 SECOND 帧，`scalar` 就不安全。

任一条件不满足时：

```
fault(
    code = DIVERGENCE_FAULT,
    lane_mask = active_mask,
    aux = 0)
```

故障指令不读动态源，也不改任何状态。

3.3 ARMED 帧不妨碍 scalar

ARMED 帧只表示 SSY 已经预约未来的汇合点，还没有真正拆成两条路径。只要另外两个条件也满足，只有 ARMED 帧时 `execution_domain=scalar` 的指令合法。

3.4 SCC 不是通用开关

SCC 只有 1 位，但它不会自动控制所有 `execution_domain=scalar` 指令。只有某个 form 明确把 SCC 列为源操作数时，该指令才读取 SCC，并按自己的逐条语义使用这个值。

因此，规范不能泛化出“只要 SCC 为假，任意 S 指令就不执行”这条规则。没有显式 SCC 源的指令完全不看 SCC；有显式 SCC 源的指令也必须先通过 `scalar-ready` 和静态操作数检查。

`execution_domain=scalar` 的指令不使用 `vpN` 作为 lane guard，因为它本来就不是逐 lane 执行。

4. execution_domain=vector

4.1 参与 lane

`execution_domain=vector` 的指令不检查 `scalar-ready`。它开始时先保存：

```
E = snapshot(active_mask)
```

没有 guard 的 vector 指令：

```
P = E
```

带 @vpN guard 的 vector 指令：

```
P = E & snapshot(vpN)
```

带 @!vpN guard 的 vector 指令：

```
P = E & ~snapshot(vpN)
```

P 是 participating mask，也就是实际执行数据操作的 lane。不存在 lane、已经退出的 lane、挂起分支中的 lane 都不会进入 P。

4.2 混合源：一个 SGPR 源

vector 指令的源寄存器号在哪个寄存器文件里解释，由编码里的 `scalar-source selector` 决定。V1 格式用 1 bit 的 `ssrc`，V2、V3、VCMP 用 2 bit 的 `ssrc_sel`。selector 为 0 时全部源都读 VGPR；非 0 时它恰好指定一个源位置改读 SGPR：

```
selector == 0    所有源都是 VGPR
V1: ssrc == 1    va 读 SGPR
V2: ssrc_sel == 1 va 读 SGPR
```

```

    ssrc_sel == 2    vb 读 SGPR
    ssrc_sel == 3    保留
VCMP: 与 V2 相同
V3: ssrc_sel == 1    va 读 SGPR
    ssrc_sel == 2    vb 读 SGPR
    ssrc_sel == 3    vc 读 SGPR

```

被选中的源在该动态指令里对所有参与 lane 给出同一个 32 位值：

```

for each lane i in P:
    vector_source[i] = vN[i]    # 每 lane 自己的值
    scalar_source[i] = sM      # 所有 lane 得到同一个值

```

也就是说广播效果内建在读操作里，不需要先把值搬进 VGPR，架构里也没有独立的广播指令。selector 落在保留编码上时报告 ILLEGAL_INSTRUCTION；这条检查在读任何源之前完成。

一条 vector 指令最多只能有一个 SGPR 源。要同时用到两个 uniform 值时，软件必须先用一条 V_MOV 之类的混合源指令把其中一个搬进 VGPR。

读 SGPR 不改变执行域：这类指令仍是 vector，仍不要求 scalar-ready，SGPR 只是只读的统一输入。selector 也不影响写回：目标始终是 VGPR 或 vpN。

4.3 Vector 写回

写 VGPR 时，只改 P 中的 lane：

```

for each lane i in P:
    vD[i] = result[i]
for each lane i not in P:
    vD[i] remains unchanged

```

写 vpN 时，只改 P 对应的位：

```
vpD = (old_vpD & ~P) | (result_bits & P)
```

普通 vector ALU 指令不能写 SGPR 或 SCC。需要产生每 warp 单一结果的归约或投票指令属于明确列出的 warp collective（线程束集合操作），必须按该指令自己的会合规则执行。

4.4 Vector 访存和故障

vector 访存只为 P 中的 lane 形成地址。E-P 中的 lane：

- 不读取 VGPR 地址源；
- 不检查对齐或范围；
- 不访问内存；
- 不产生 lane 局部访存故障；
- 不写目标。

若 P 中任何 lane 产生会让整条指令故障的错误，整条 warp 动态指令回滚。故障记录的 lane_mask 标出检测到该错误的 lane。

P 为空时，合法 vector 指令不读动态源、不写数据，并正常前进到下一条。机器码和静态操作数仍然必须合法。

5. execution_domain=warp_control

warp_control 直接管理 warp，不属于 scalar ALU。SSY、BRA、BRA.P、JOIN、EXIT 不检查 scalar-ready；它们必须能在分歧路径中把控制流走完。

CALL、CALL.IND、JUMP.IND、RET 也属于 warp_control，并明确写有 required_state: scalar_ready。检查失败就报告 DIVERGENCE_FAULT，不读取间接目标或调用栈，也不改 PC。

直接 BRA 和 BRA.P 明确不套这条额外规则。不能因为它们也会改 PC，就把它误当成调用指令。

5.1 Warp 统一调用栈

每个 warp 有一套隐藏调用栈，实际深度限制是 descriptor 的 call_stack_depth，并且该字段不能超过架构最大值 16。每个调用帧只保存：

```
CallFrame {  
    return_pc: U64  
}
```

调用帧不保存 EXEC、LIVE、SCC、SGPR、VGPR、vpN 或重汇聚状态。之所以可以这么简单，是因为调用前必须 scalar-ready：所有 live lane 在同一路径上，而且没有 FIRST 或 SECOND 未完成分歧帧。

scalar-ready 不禁止 ARMED 帧，所以 CALL 允许调用者留下尚未发生分歧的 ARMED 帧。它们仍在重汇聚栈中，归调用者所有；CALL 不把它们复制进调用帧，也不弹出它们。

5.2 CALL 和 CALL.IND

CALL target 使用指令里的直接目标，CALL.IND source 从明确编码的 SGPR 源取得统一目标。两者都按下面的规则执行：

```
require scalar_ready()  
require target 是文本内对齐的合法 PC  
require old_PC + 8 是文本内合法返回 PC  
require call_stack.depth < descriptor.call_stack_depth  
  
push CallFrame(return_pc = old_PC + 8)  
PC = target
```

目标或返回 PC 非法时报告 ILLEGAL_OPERAND。调用栈已经达到 call_stack_depth 时报告 RECONVERGENCE_FAULT，整条 CALL 不压入半个帧，也不跳转。

5.3 JUMP.IND

JUMP.IND source 从明确编码的 SGPR 源读取统一目标，不压入也不弹出调用栈：

```
require scalar_ready()  
require target 是文本内对齐的合法 PC  
PC = target
```

目标非法报告 ILLEGAL_OPERAND。JUMP.IND 仍是 warp_control，scalar-ready 只是它的额外入口条件。

5.4 RET

RET 按下面的规则返回：

```
require scalar_ready()  
require call_stack is not empty  
require reconv_stack 中不存在 owner_call_depth == call_stack.depth 的帧  
  
R = top.return_pc  
pop call_stack  
PC = R
```

callee（被调用代码）可以在内部使用 SSSY/BRA.P/JOIN，但必须在执行 RET 前闭合自己建立的所有 SSSY 帧。调用者在 CALL 前已经存在的 ARMED 帧可以保留，RET 不弹它们。

空调用栈执行 RET、CALL 超过 call_stack_depth，或 RET 时 callee 仍留有自己的重汇聚帧，都报告 RECONVERGENCE_FAULT。调用栈和 PC 保持原样。

普通 MOV、逻辑运算或恢复指令都不能写 EXEC 或 LIVE。只有本文件规定的 warp-control 状态机能改变它们对应的内部掩码。

6. 隐藏重汇聚栈

6.1 帧内容

每个 warp 有一套软件不能直接读写的后进先出栈。最大硬上限是 16 帧，实际允许深度由 descriptor 的 reconv_stack_depth 声明。

每帧包含：

```
Frame {
    reconv_pc: U64,    # JOIN 所在 PC
    entry_mask: U32,   # 进入该控制区域的仍存活 lane
    pending_pc: U64,   # 尚未执行路径的起点
    pending_mask: U32, # 尚未执行路径的 lane
    arrived_mask: U32, # 已在 JOIN 等待的 lane
    owner_call_depth: U8, # SSY 执行时的调用栈深度
    phase: ARMED | FIRST | SECOND
}
```

owner_call_depth 只用来确认 RET 前 callee 自己建立的帧都已闭合。它属于重汇聚帧，不属于调用帧；调用帧仍然只保存 return_pc。

始终必须满足：

```
active_mask & ~live_mask == 0
frame.entry_mask & ~live_mask == 0
frame.pending_mask & ~live_mask == 0
frame.arrived_mask & ~live_mask == 0
frame.pending_mask & frame.arrived_mask == 0
```

EXIT 会从 live_mask 以及所有相关栈掩码中一起删除退出 lane，所以退出 lane 永远不会被重汇聚“复活”。

6.2 SSY

执行 SSY R 时：

```
require R 是合法且对齐的指令地址
require R 上的静态指令是 JOIN
require stack.depth < descriptor.reconv_stack_depth

push Frame(
    reconv_pc = R,
    entry_mask = active_mask,
    pending_pc = 0,
    pending_mask = 0,
    arrived_mask = 0,
    owner_call_depth = call_stack.depth,
    phase = ARMED)

PC = old_PC + 8
```

目标非法时报告 ILLEGAL_OPERAND；超过声明深度时报告 RECONVERGENCE_FAULT。SSY 本身不改变 EXEC。

6.3 BRA

BRA target 让整个当前 active_mask 跳到同一目标：

```
require target 是合法且对齐的指令地址
PC = target
```

它不改变 active_mask、live_mask 或栈。静态控制流验证必须保证它不会非法跳出未完成的 SSY.JOIN 区域。

6.4 BRA.P

BRA.P condition, target 的 condition 是逐 lane 条件，可以来自 vpN、!vpN 或指令清单定义的等价 lane 条件。它不是 scalar guard。

先计算：

```
E = active_mask
T = E & evaluate_lane_condition(condition)
F = E & ~evaluate_lane_condition(condition)
```

若所有 active lane 走顺序路径：

```
if T == 0:
    active_mask = F
    PC = old_PC + 8
```

若所有 active lane 都跳转：

```
if F == 0:
    active_mask = T
    PC = target
```

这两种都叫 uniform branch（统一分支），不会消耗 ARMED 帧。

若 T 和 F 都非空，发生分歧：

```
require stack is not empty
require top.phase == ARMED
require top.entry_mask == E

top.pending_pc = old_PC + 8
top.pending_mask = F
top.arrived_mask = 0
top.phase = FIRST

active_mask = T
PC = target
```

VTX-1 固定先执行条件为真的跳转路径，再执行条件为假的顺序路径。软件不能假设两条路径并行前进。

分歧时没有匹配的 ARMED 栈顶，或 entry_mask 不等于当前 E，报告 RECONVERGENCE_FAULT。

6.5 JOIN

JOIN 必须在栈顶帧的 reconv_pc 上执行。

若帧仍是 ARMED，说明控制区域内没有真正发生分歧：

```
require active_mask == top.entry_mask
pop top
PC = old_PC + 8
```

若帧是 FIRST，第一条路径已经到达汇合点：

```
top.arrived_mask = active_mask
top.phase = SECOND
active_mask = top.pending_mask & live_mask
PC = top.pending_pc
top.pending_mask = 0
```

此时第一条路径的 lane 暂停，开始执行第二条路径。

若帧是 SECOND，第二条路径也到了：

```
A = top.arrived_mask | active_mask
require A == top.entry_mask
pop top
active_mask = A & live_mask
PC = old_PC + 8
```

空栈、PC 不匹配、phase 不合法或掩码对不上，都报告 RECONVERGENCE_FAULT。

6.6 EXIT

令 X 为这次 EXIT 要退出的 lane：

- 无条件 EXIT：X = active_mask；
- 带显式 lane 条件的 EXIT：X = active_mask & condition_mask。

EXIT 没有任何屏障前置条件；它不会因为槽状态失败。提交时先把每个 lane 属于 X 转成 linear_tid=warp_id*32+lane_id，把这些身份从 CTA 的 live_owner_set 中移除，然后更新掩码：

```
live_owner_set -= exiting_linear_tids

live_mask  = live_mask & ~X
active_mask = active_mask & ~X

for each frame f:
    f.entry_mask  &= ~X
    f.pending_mask &= ~X
    f.arrived_mask &= ~X
```

缩小 live_owner_set 可能让某个正在等待的槽立刻满足完成条件，那些 waiter 因此被唤醒。但 EXIT 本身不产生 shared release；退出线程之前写的 shared 数据只在它自己执行过成功屏障时才被同步。

若还有 active lane，它们从 old_PC + 8 继续。若当前路径已经空了，必须执行下面的正规化，直到找到另一条非空路径或 warp 完成：

```
while active_mask == 0:
    if stack is empty:
        require live_mask == 0
        complete warp
        stop

    f = top

    if f.phase == ARMED:
        require f.entry_mask == 0
        pop top
        continue

    if f.phase == FIRST:
        f.phase = SECOND
        active_mask = f.pending_mask & live_mask
        PC = f.pending_pc
        f.pending_mask = 0
        if active_mask != 0:
            stop
            continue

    if f.phase == SECOND:
        active_mask = f.arrived_mask & live_mask
        PC = f.reconv_pc + 8
        pop top
        if active_mask != 0:
            stop
            continue
```

因此，第一条路径全部 EXIT 后，第二条路径仍会执行；第二条路径全部 EXIT 后，已经到达 JOIN 的 lane 仍会恢复。已经退出的 lane 永远不会回来。

7. 为什么分歧区里不能跑 scalar 执行域

进入 FIRST 后，只有第一条路径的 lane active；进入 SECOND 后，只有第二条路径的 lane active。若这时运行 scalar，两个路径可能对同一个 SGPR 给出互相覆盖的结果，也可能让后执行路径看到前一路留下的临时值。

VTX-1 不让软件猜这种行为。只要有 FIRST 或 SECOND 未完成帧，任何 execution_domain=scalar 的指令都直接报告 DIVERGENCE_FAULT。这包括 S_ALU、S_FP、S_GETREG、SMEM、SATOM、S_READFIRST，不能只限制普通算术。编译器必须把这些指令放在分歧前或完成 JOIN 后；若确实需要逐 lane 计算，就使用 vector 指令。

普通 warp_control 指令是例外，因为没有它就无法走完分支并回到 JOIN。CALL、CALL.IND、JUMP.IND、RET 仍要检查 scalar-ready。vector 也是合法的，因为它只改当前 participating lane 的 VGPR 或 vpN 状态。

8. 结构化控制流要求

内核发布前，验证器必须检查每个 SSY R：

1. R 指向该 SSY 之后的一条 JOIN；
2. 从 SSY 后进入的每条有限路径，要么执行 EXIT，要么到达该 JOIN；
3. 区域内不能跳到 R 之后、另一个不匹配的 JOIN 或文本外；
4. 嵌套 SSY..JOIN 区间只能完整包含，不能交叉；
5. 同时嵌套的最大帧数不超过 reconv_stack_depth。

对直接可见的调用边，验证器还必须检查 callee 内建立的每个 SSY 都在该 callee 的 RET 前由匹配 JOIN 闭合，不能把 callee 的重汇聚帧带回调用者。间接调用无法完全静态确定目标，所以 RET 仍要用 owner_call_depth 做运行时检查。

不满足时，启动前报 INVALID_CONTROL_FLOW。静态检查不能替代运行时检查；实际执行 BRA.P、JOIN、CALL 或 RET 时，仍要核对应栈状态。

9. Warp 集合操作

warp collective（线程束集合操作）会读取多个 lane 的值，并产生一个或多个 lane 的结果，例如投票、ballot（把真假收成位图）或 shuffle（跨 lane 取值）。

每条 collective 必须明确：

- 候选 lane 是哪些；
- 哪些 lane 提供输入；
- 哪些 lane 接收结果；
- member mask（成员掩码）是否必须完全包含在当前 active_mask；
- 输入不一致时报告什么故障。

除具体指令另有更严格规则外，基本合同是：

```
E = snapshot(active_mask)
M = 所有 E 中 lane 必须一致提供的 member_mask
require M & ~E == 0
contributors = M
receivers = E
```

成员掩码包含非 active lane，或 active lane 提供的 M 不一致，报告 COLLECTIVE_FAULT。这类指令的执行域是 warp_collective，不套用 scalar-ready，但必须满足自己的会合条件。

10. CTA 屏障

execution_domain=cta_sync 只有一条屏障指令：

```
BAR.SYNC.CTA id
```

架构不提供把到达和等待分开的 split 屏障，也不提供屏障 token、generation 计数或子集屏障。需要“先到达、后等待”的软件必须自己用 shared memory 上的原子操作和 FENCE 构造，那些结构完全落在第 4 章的内存模型里，不需要额外的屏障状态。

每个 CTA 固定有 8 个槽 id=0..7，另有一个 CTA 级的 live_owner_set：

```
BarrierSlot {
  arrived_set: set<linear_tid>
  waiters: map<warp_id, BarrierWaitRecord>
}

BarrierWaitRecord {
  warp_id: U32
  owner_snapshot: set<linear_tid>
  resume_pc: U64
}

live_owner_set: set<linear_tid>    # 每 CTA 一个，8 个槽共用
```

owner 的唯一身份是 CTA 内 linear_tid=warp_id*32+lane_id；(warp_id, lane_id) 只是等价表示，所有集合和比较都以 linear_tid 为元素。启动时 8 个槽的 arrived_set 和 waiters 都为空，也就是全部 idle；

live_owner_set 是 CTA 启动时全部真实线程的 linear_tid，不含不存在的尾 lane。EXIT 把退出线程从 live_owner_set 移除，这是它唯一会变小的方式。没有 expected 字段。

BAR.SYNC.CTA 写有 required_state: scalar_ready。因此它先取

```
A = {warp_id*32 + lane_id | lane_id 位于 snapshot(active_mask)}
```

时，active_mask 必然等于 live_mask，A 恰好是该 warp 当前全部 live lane。分歧 warp 在记录任何 arrival 之前就报告 DIVERGENCE_FAULT，不留下部分 arrival、blocked record 或 PC 效果。这条规则替代了旧模型里的模式隔离、重复到达和 wrong-owner 检查：一个 warp 要么整体到达，要么根本没到达。

10.1 阻塞记录

屏障阻塞整个 warp 的当前动态路径。每个 warp 同一时刻至多有一条 barrier blocked record；ready warp 发射屏障时该记录必须为空。统一操作为：

```
block_barrier(S, A, old_PC):
  require warp.blocked_record == none
  R = BarrierWaitRecord(
    warp_id = current_warp_id,
    owner_snapshot = snapshot(A),
    resume_pc = old_PC + 8)
  require current_warp_id not in S.waiters
  S.waiters[current_warp_id] = R
  warp.blocked_record = (slot_id, R)
  warp.ready = false
  # PC 留在屏障上；active_mask/live_mask/reconv_stack/call_stack 全部不变

resume_barrier(S, R):
  require warp[R.warp_id].blocked_record == (slot_id, R)
  shared_acquire(R.owner_snapshot)
  delete S.waiters[R.warp_id]
  warp[R.warp_id].blocked_record = none
  warp[R.warp_id].PC = R.resume_pc
  warp[R.warp_id].ready = true
  # active_mask/live_mask/reconv_stack/call_stack 全部不变
```

blocked record 存在期间，调度器不能发射该 warp，也不能切入其重汇聚栈中的挂起路径。恢复只执行上面列出的 PC、ready 和记录清理，不重新读源，也不改任何控制掩码或栈。

10.2 BAR.SYNC.CTA id

```

require scalar_ready          # 否则 DIVERGENCE_FAULT
A = {warp_id*32 + lane_id | lane_id 位于 snapshot(active_mask)}
S = slot[id]

atomically:
    S.arrived_set |= A
    shared_release(tid in A)
    block_barrier(S, A, old_PC)

if S.arrived_set == live_owner_set:
    records = snapshot(all S.waiters)
    for every R in records together:
        resume_barrier(S, R)
    clear(S)                  # arrived_set = {}, waiters = {}

```

“together” 表示所有等待者在同一个完成动作中恢复，不允许先让一部分跨过屏障。清空之后槽立即回到 idle，可以马上被下一次屏障复用；槽里不留任何跨屏障的残余状态，所以也没有“旧代”可以被误认。

完成条件是 `arrived_set == live_owner_set`，而不是与某个启动时固定集合比较。因此 EXIT 缩小 `live_owner_set` 时也要重新检查每个非 idle 槽：

```

after live_owner_set shrinks:
    for each slot S with S.arrived_set != {}:
        if S.arrived_set == live_owner_set:
            resume all S.waiters together
        clear(S)

```

EXIT 本身不做 shared release，所以被它放行的 waiter 只获得其他真正到达者贡献的 release。

10.3 EXIT、分歧和死锁

EXIT 没有屏障前置条件，也不会报屏障相关故障。退出线程只是从 `live_owner_set` 中消失，剩下的 owner 因此少等一个人。

若一个 warp 在屏障上阻塞，而同 CTA 另一些 owner 既不到达该槽也不退出，`arrived_set` 永远追不上 `live_owner_set`，程序按第 12 节报告 DEADLOCK。同一个 warp 的不同 warp 内路径不会造成这种情况：屏障要求 scalar-ready，warp 只能整体到达。典型死锁来自不同 warp 走了不同的控制流，例如只有一部分 warp 执行了循环体里的 `BAR.SYNC.CTA`。

10.4 内存边

每个成功 `BAR.SYNC.CTA` arrival 都是 shared、CTA scope 的 release，恢复是 shared、CTA scope 的 acquire。它们不自动排序 global、local、param、const 或 host；global 通信仍要使用合法原子和需要的 FENCE。

11. 调度、前进和 occupancy

同一 CTA 的 warp 独立调度。只要一个 warp ready，SM/CU 就可以发射它，不需要等待 CTA 中其他 warp 到同一 PC。

实现必须对持续 ready 的驻留 warp 提供弱公平性：不能只因为调度器偏爱别的 warp，就让它永久饿死。这里的“弱公平”不承诺具体周期数，也不承诺固定轮转顺序。

occupancy 会改变“同一时刻有哪些 CTA 已经驻留”，但不能改变程序的合法结果。软件禁止依赖：

- 某两个 CTA 一定同时驻留；
- 某个 warp_id 一定先执行；
- 大寄存器内核仍能达到某个固定 occupancy；
- 自旋等待一个尚未驻留的 CTA 一定前进。

sgpr_count、vgpr_count、vp_count、重汇聚栈深度、call_stack_depth、shared memory 和 local memory 都可能降低 occupancy。资源不够放入整个 CTA 时，CTA 必须等待，不能拆开放到多个 SM/CU。

12. 完成和死锁

warp 完成条件：

```
live_mask == 0
reconv_stack is empty
call_stack is empty
blocked_record == none
没有该 warp 尚未清理的集合状态
```

若 live_mask==0 时调用栈仍非空，实现必须报告 RECONVERGENCE_FAULT，不能把带着未返回调用帧的 warp 宣告完成。

CTA 完成条件：

```
全部 warp 完成
并且 8 个 slot 都满足：
    arrived_set == {}
    waiters == {}
```

上面这组槽条件就是 idle。全部 warp 完成时 live_owner_set 必然为空，所以任何还有到达者的槽都会先被完成并清空；若实现观察到 warp 全部完成而某个槽仍非 idle，说明它没有正确执行 10.2 的重新检查。

kernel 完成条件：

```
全部 CTA 完成
没有故障
```

kernel 尚未完成时，若同时满足：

1. 没有 ready warp；
2. 没有正在执行、尚未提交的架构指令；
3. 没有已接受的内存、屏障或运行时事件能在无需再发射指令的情况下让 warp ready；
4. 至少还有一个未完成 warp、lane 或同步状态；

实现必须报告 DEADLOCK。

墙钟超时不等于架构死锁。一个仍在不断执行指令的无限循环属于不终止或活锁，不满足“没有 ready warp”这一条。

13. 七个执行域的快速对照

execution_domain	大白话含义	scalar-ready 规则
system	系统控制、陷阱和杂项	不因执行域自动检查
scalar	每 warp 做一次，可读写 SGPR；只有明确操作数才能读 SCC	每个 form 都必须检查
vector	每个参与 lane 做一次，可读 VGPR、vpN，并可由 selector 把其中一个源改成 SGPR	不检查
warp_control	改 PC、路径、重汇聚栈或调用栈	普通控制不检查；CALL/CALL.IND/JUMP.IND/RET 必须检查

execution_domain	大白话含义	scalar-ready 规则
warp_collective	一个 warp 的多个 lane 合作投票或交换数据	不检查，但要满足集合会合合同
cta_sync	CTA 线程做屏障或内存同步	BAR.SYNC.CTA 必须检查；FENCE 不检查
warp_matrix	一个 warp 合作完成矩阵运算	不检查，但要满足矩阵参与合同

机器 class 不出现在这张表里，因为它只决定编码。MEMORY class 内部仍要看 form 的执行域，不能把所有访问一概当成 vector 或 scalar。

这张表只是索引。遇到细节时，以本文件前面各节的完整规则为准。

VTX-1 ISA 1.0 Draft: 内存模型

本章规定程序能看到什么值、哪些先后关系必须成立。缓存、访存合并、shared bank 和互连实现都不是架构语义。实现可以在内部重排请求，但最后得到的执行必须满足本章。

文中的“必须”“禁止”“可以”分别表示 MUST、MUST NOT、MAY。

1. 先记住七条直观规则

1. SGPR 是每 warp 一份，VGPR 是每 lane 一份。scalar 访存读写 SGPR，整条 warp 只做一次；vector 访存读写 VGPR，每个参与 lane 各做一次。
2. 所有 scalar 访存和 scalar 原子都要求头部 guard 固定为 PT，并且 warp scalar-ready。静态 guard 不是 PT 时按非法编码处理；不满足 scalar-ready 时固定报告 DIVERGENCE_FAULT。两种失败都不读动态源、不形成地址、不产生事件。
3. vector 访存按参与 lane 展开。vp 条件筛出几个参与 lane，就有几个彼此独立的事件。即使硬件把它们合成一个总线请求，内存模型里仍是多个事件。
4. 地址可以“统一基址 + 各 lane 索引”。global、param、const 的向量访问可以用一个 SGPR64 基址，再加每 lane 的 VGPR32 无符号索引。SV-mix 固定先做 zero_extend，不能把最高位当符号位。
5. 地址空间由 opcode 决定，不由地址值决定。每条访存 form 的助记符里写明 GLOBAL、SHARED、LOCAL、PARAM 或 CONST；地址寄存器只保存位模式，不携带空间身份，也没有 generic 空间和运行期空间推断。
6. 空间有明确限制。local 只能向量访问；param 和 const 只读；param、const、global 可以标量读，global 可以标量写；shared 同时支持标量和向量访问。
7. BAR.SYNC.CTA 只管整个 CTA 的 shared。它不是任意线程子集屏障，也不会替代 global 的原子通信或主机所有权转移。

含数据竞争、原子与普通访问非法混用、或违反主机所有权的程序是未定义程序。未定义不是一种可捕获的设备故障。地址越界、未对齐和错误空间仍按各自精确故障处理；scalar-ready 失败固定为 DIVERGENCE_FAULT。

2. 两类访存指令

2.1 标量访存

scalar 访存使用 SGPR 操作数。成功的 scalar load 对整个 warp 产生恰好一个 R 事件，成功的 scalar store 产生恰好一个 W 事件。该事件的执行代理是 warp，记作 agent=warp，没有 lane 编号。成功必有一个，失败才是零个。

每条 scalar load 和 scalar store 都必须在读取动态源之前检查执行模型定义的 scalar-ready(warp)。大白话说，warp 必须至少还有一个活 lane，全部活 lane 必须在同一路径上，而且重汇聚栈中不能有未完成的 FIRST 或 SECOND 帧。只要有一条分歧路径还没走完，就不是 scalar-ready；不存在“这一条 scalar 指令碰巧安全，所以可以执行”的例外。

scalar-ready 检查失败固定报告 DIVERGENCE_FAULT，不读取 SGPR 地址或数据源，不形成地址，不产生 R/W/A_*、ppo 或其他内存关系，也不写任何 SGPR。

scalar load 把一次读取结果写入 SGPR；scalar store 从 SGPR 取得一次写入值。无论 warp 有多少个参与 lane，都不能把一次 scalar store 解释为多次相同写入，也不能把一次 scalar 原子解释为多次 RMW。

允许的空间如下：

空间	标量 load	标量 store/atomic
param	允许	禁止
const	允许	禁止
global	允许	允许
shared	允许	允许
local	禁止	禁止

2.2 向量访存

vector 访存使用 VGPR 数据操作数。令

```
E = 当前路径上的候选 lane  
无 vp 条件: P = E  
@vpN:      P = E & snapshot(vpN)  
@!vpN:     P = E & ~snapshot(vpN)
```

对 P 中每个 lane 恰好产生一个事件，事件携带自己的 lane。不在 P 中的 lane 不形成地址、不访问内存、不写 VGPR，也不因坏地址而故障。

同一条向量指令的不同 lane 即使得到相同地址，也仍是不同事件：

- 多个 lane 读同址是多个读；
- 多个 lane 普通写同址且没有 hb 排序，是数据竞争；
- 多个 lane 对同址做同宽原子，会分别占据该位置 mo 中的一个位置。

实现可以合并物理请求，但不得合并事件身份、原子次数、故障检查或 rf/co/mo 关系。

向量访问可用于 global、shared、local、param 和 const；param、const 仍然只读。local 只能走本类访问，每个 lane 只能访问自己的 local allocation。

2.3 一条指令的精确提交

scalar 访存先检查静态操作数和固定 PT，再检查 scalar-ready，随后才读取 SGPR 并检查唯一地址；全部成功后恰好提交一个事件。vector 访存先确定 P，再检查全部参与地址、范围和对齐；全部成功后对 P 中每个 lane 恰好提交一个事件。任一参与地址失败时，整条动态指令不产生任何内存事件，也不部分写回 SGPR/VGPR。非参与 lane 不进入检查。

标量和向量只决定“产生几个架构事件”，不规定硬件发出几个缓存请求。

3. 地址空间和地址形成

3.1 空间和窗口

全部空间按字节寻址，数据采用小端序。

空间	可见范围	地址宽度和来源
global	设备内 CTA；主机须遵守所有权	SGPR64 或 VGPR64 基址
shared	同一 CTA	CTA 内 U32 字节偏移

空间	可见范围	地址宽度和来源
local	单 lane	该 lane 私有窗口内的 U32 字节偏移
param	本次 kernel 启动，只读	SGPR64 或 VGPR64 基址
const	设备只读	SGPR64 或 VGPR64 基址

一次访问落在哪个空间只由 form 决定：每条访存 form 的操作数类型固定写明空间，助记符里也带同一个空间后缀。架构没有 generic 空间，寄存器里的地址值不带空间身份，实现禁止在运行期根据数值猜测空间。

global、param、const allocation 都有空间身份、基址、长度和生命周期。一次访问必须完整落在同一个活跃 allocation 中，不能跨到相邻 allocation。这项检查在实现内部按 allocation 表完成，与寄存器内容无关。

shared 每 CTA 一份，local 每真实 lane 一份；不同 lane 的 local 永不别名。

3.2 三种地址合同

地址 form 只能明确选择 uniform、lane、SV-mix 三种合同之一。地址先用无界数学整数计算，再检查窗口、allocation 范围和对齐；任何中间步骤都不能按 32 位或 64 位回绕。

uniform

整个 warp 使用同一个 SGPR 地址。global、param、const 使用 SGPR64 base；shared 使用 CTA 内 SGPR32 offset。基本模板是：

```
EA = unsigned(SGPR_base) + sign_extend(immediate)
```

SMEMX 仍属于 uniform，只是多一个 SGPR32 index：

```
SMEMX:
EA = unsigned(SGPR_base)
  + zero_extend(SGPR32_index)
  + sign_extend(immediate)
```

base 和 index 都只是字节数值。uniform 只说明地址统一，不决定事件数：scalar memory 成功后整个 warp 恰好一个事件；若某个 vector form 使用 uniform 地址，仍对 P 中每个 lane 各产生一个事件。

lane

每个参与 lane 从自己的 VGPR 形成地址：

```
EA[lane] = unsigned(VGPR_base[lane])
  + sign_extend(immediate)
```

global、param、const 的 lane base 为 VGPR64；shared 和 local 使用各 lane 的 VGPR32 窗口 offset。local 只能使用 lane 合同，不能使用 uniform、SMEMX 或 SV-mix。同一个数值 offset 在不同 lane 的 local 中仍指向不同私有 allocation，因为 local form 的窗口按 lane 选取。

SV-mix

SGPR 给出统一 base，VGPR 给出逐 lane 无符号字节 index：

```
EA[lane] = unsigned(SGPR_base)
  + zero_extend(VGPR32_index[lane]) * scale
  + sign_extend(immediate)
```

scale 只能取具体 form 明写的值。所有 SV-mix form 都必须对 VGPR32 index 做 zero_extend；使用 sign_extend 或二补数负 offset 是错误实现。SV-mix 可用于 global、param、const 和 shared，但不能用于 local。

VATOMX 是 global vector atomic 的 SV-mix 固定模板：

```
VATOMX:
EA[lane] = unsigned(SGPR64_base)
          + zero_extend(VGPR32_index[lane])
```

VATOMX 的 scale 固定为 1，且没有额外缩放变体。它先按 vp 得到 P，再为 P 中每个 lane 形成地址；成功后每 lane 恰好一个原子事件。多个 lane 算出同一地址时仍是多个事件，并分别进入该位置的 mo。

3.3 越界与对齐

令 $end = start + size$ 。出现以下任一情况均为 MEMORY_BOUNDS：

- $start < 0$ ；
- global、param、const 的 start 或 end 超出 U64 地址范围；
- shared/local 的结果超出对应窗口；
- 访问没有完整落在同一个活跃 allocation。

自然对齐要求如下：

类型	对齐
U8	1
U16、F16	2
U32、F32	4
U64	8
V2.U32、V4.U32	8、16
U32 原子	4
U64 原子	8

未对齐报 MISALIGNED_ACCESS。同一参与地址同时未对齐和越界时，按精确故障优先级报告。

3.4 64 位地址在两个寄存器文件之间搬运

地址就是 64 位数值，没有影子状态，所以在寄存器之间搬运它只是搬 64 个位。两条真实机器 form 覆盖两个方向，它们都不是伪指令，也不能拆成两条 .B32 来替代：

```
V_MOV.B64 vE:v(E+1), sA:s(A+1)    # ssrc=1, SGPR64 -> 每 lane VGPR64
S_READFIRST.B64 sE:s(E+1), vA:v(A+1)
```

V_MOV.B64 是 V1 格式的混合源 form。ssrc=0 时它是普通的 VGPR64 到 VGPR64 复制；ssrc=1 时源在 SGPR 文件中解释，于入口冻结一次那个偶数对齐的 SGPR 对，再对 P 中每个 lane 整体写入对应 VGPR 对。这就是把一个统一的 64 位地址送进各 lane 的规范做法。它是 vector form，不要求 scalar-ready；非参与 lane 保持原值。

S_READFIRST.B64 的头部 guard 固定为 PT，并且先检查 scalar-ready。通过后选择编号最小的 live lane，冻结该 lane 的 VGPR 偶数连续寄存器对，再整体写入 SGPR 偶数连续寄存器对。它不检查其他 lane 是否同值。

两条指令都只复制 64 个数值位。目标地址属于哪个空间由后续访存指令的 opcode 决定，与这次搬运无关；把一个 shared offset 搬进 SGPR64 再交给 S_LD.GLOBAL 不是“指针伪造”，而是一个越界或越窗口的地址，按 3.3 节的精确故障处理。

超出窗口、落在已释放 allocation，或没有完整落在同一个活跃 allocation 的地址报 MEMORY_BOUNDS；寄存器类别、编号或对齐不符合 form 要求时报 ILLEGAL_OPERAND。

4. 事件和普通访问粒度

一个候选执行由事件集合 E 和本章后续关系组成。每个事件至少记录：

```
id, kind, agent, warp, lane-or-none, cta, instruction-instance,
space, allocation, byte-range, value, scope, order
```

事件种类为：

- I: allocation 或所有权纪元开始时的概念初始写；
- R、W: 普通读写；
- A_R、A_W、A_RMW: 原子读、写、读改写；
- F(scope): FENCE；
- B(slot,phase): BAR.SYNC.CTA 的到达和恢复；
- H: allocation、启动、完成和所有权转移。

原子事件的 order 和 scope 来自该动态指令机器字中的 modifier 位，不由地址、寄存器值或实现猜测。译码出的 modifier 会原样成为事件属性，并参与后面的 ppo/sw/hb。

自然对齐 U8、U16/F16、U32/F32 普通访问是一个单拷贝元素。普通 U64 由低地址和高地址两个 U32 单拷贝元素组成；V2/V4.U32 由 2/4 个 U32 元素组成。U64 和多元素向量的不同 U32 元素可以分别取值，不保证整条数据的单拷贝原子性。

普通读的 rf 按字节定义，但同一个单拷贝元素不能从两个同宽并发写中各取一部分。已经由 hb 排序的窄写可以按小端顺序拼出较宽读。并发混宽冲突通常构成数据竞争，程序不能依赖某种撕裂结果。

5. 先用大白话理解顺序

- 同一个地址上的写有先后队列：普通字节用 co，原子位置用 mo。
- 每个读必须说清楚“读的是哪次写”，这就是 rf。
- 一个读选了旧写，那么它自然位于后续写之前，这就是 fr。
- 单个 warp/lane 里不是所有程序顺序都强制保留；真正必须保留的部分叫 ppo。
- release/acquire、CTA 屏障和运行时所有权可以在不同代理之间搭桥，这些桥叫 sw。
- ppo 和 sw 反复串起来得到 hb。如果 A hb B，意思是 A 必须先于 B 对程序生效。

下面给出完整关系，不能只凭上面的比喻实现。

6. 形式关系

以下关系均为严格关系。r+ 表示传递闭包， r^{-1} 表示逆关系。

6.1 po 和 ppo

po 按动态指令顺序连接同一执行代理的事件：

- 同一 lane 的向量事件按该 lane 经历的动态指令顺序排列；
- warp 标量事件按 warp 的动态指令顺序排列；
- 一个标量事件与同 warp 中更早或更晚的向量事件按对应动态指令顺序排列；

- 同一条向量指令的不同 lane 事件之间没有 po。

ppo 是必须保留的 po 子集，由以下边的并集组成：

1. 同一代理且字节区间重叠的 po-loc；
2. SGPR 或 VGPR 的真数据依赖、地址依赖和控制依赖；
3. 混合源 V_MOV.B32/B64 和 S_READFIRST.B32/B64 建立的寄存器值依赖；
4. 任意事件到其后 FENCE，以及 FENCE 到其后任意事件；
5. 任意事件到其后 RELEASE/ACQ_REL 原子，以及 ACQUIRE/ACQ_REL 原子到其后任意事件；
6. BAR.SYNC.CTA 到达前的 shared 事件到该 owner 的 release 到达事件，以及屏障恢复的 acquire 到之后的 shared 事件；
7. 同一 RMW 的读部到写部；
8. 运行时启动、完成和所有权转移要求的边。

锁步执行本身不自动让两个 lane 的普通访问互相排序。只有上面列出的边会进入 ppo。

6.2 rf: 读自哪次写

每个普通读字节必须恰好从同空间、同 allocation、同所有权纪元、同地址的一个 I 或 W 取值，记作 rf_b。值必须相等，并满足第 4 节的单拷贝约束。

每个原子读或 RMW 的读部必须从相同自然对齐起点、相同宽度的原子初值或 mo 修改取得完整 U32/U64，记作 rf_a，禁止撕裂。

rf = 所有 rf_b 的并集，再并上 rf_a

跨 lane、跨 warp 或跨主机/设备代理的 rf 边记为 rfe。普通读不得从其 hb 未来的写取值，也不得越过同代理 po-loc 中更晚的覆盖写去读旧值。

6.3 co: 普通字节写序

对每个普通字节位置 b，co_b 是该所有权纪元中 I_b 和全部普通写该字节事件之间的严格全序，且 I_b 最小。

co = 所有 co_b 的并集

一个多字节普通写分别出现在每个所写字节的 co_b 中。属于同一个单拷贝元素的覆盖字节必须保持一致的写顺序；普通 U64 或 V2/V4.U32 的不同 U32 元素不要求耦合。

6.4 mo: 原子修改序

原子位置键为：

x = (allocation-id, start, width)

其中 width 只能为 4 或 8，且 start 自然对齐。对每个 x，mo_x 是初始原子值、全部 A_W(x) 和 A_RMW(x) 的唯一严格全序，初始值最小。纯 A_R 不占 mo 位置。

每个 RMW 从它在 mo_x 中的紧邻前驱读取，然后不可分割地插入一个新值。失败 CAS 也把旧值原样写回，并占一个 mo_x 位置。

mo = 所有 mo_x 的并集

一个位置的 mo 不排序另一个位置。co 和 mo 都必须扩展同址的 ppo 与 hb 写写顺序。

6.5 fr: 读到后续写

若普通读 r 从 w 读取, 且 $w \text{ co_b } w'$, 则有 $r \text{ fr_b } w'$ 。若原子读或 RMW 的读部从 a 读取, 且 $a \text{ mo_x } a'$, 则有 $r \text{ fr_a } a'$ 。

$\text{fr} = \text{所有 fr_b 的并集, 再并上 fr_a}$

6.6 scope、sw 和 hb

scope 只允许三个规范名称 CTA、DEVICE、SYSTEM, 覆盖范围逐级增大:

- CTA: 同一 CTA 的 lane 和 warp;
- DEVICE: 同一设备上的全部 CTA 和设备代理;
- SYSTEM: 设备与运行时声明的一致主机域。

其他拼写不是第四种 scope, 也不能代替这三个规范名称。

shared 的可见范围固定为 CTA, 因此 shared 原子事件只允许 CTA scope。DEVICE 或 SYSTEM 不能把 shared 扩大到 CTA 外; 这种 modifier 与 shared 空间的组合法非法。

两个带 scope 的事件只有在双方 scope 都覆盖对方代理时才相容。CTA 不能同步另一个 CTA, DEVICE 不能同步主机。

sw 由以下情况组成:

1. RELEASE/ACQ_REL 修改原子 X 与 ACQUIRE/ACQ_REL 原子读 Y scope 相容, 且 Y 读自 X 或以 X 开始的 release sequence, 则 $X \text{ sw } Y$ 。
2. release fence Fr 在 po 中先于原子修改 X , acquire 原子 Y 读取 X 的 release sequence, 且相关 scope 相容, 则 $\text{Fr sw } Y$ 。
3. release 原子 X 的 release sequence 被原子 Y 读取, 且 $Y \text{ po } \text{Fa}$ 、 Fa 是 acquire fence, 相关 scope 相容, 则 $X \text{ sw } \text{Fa}$ 。
4. 同时满足 $\text{Fr po } X$ 、 $Y \text{ po } \text{Fa}$, 且 Y 读取 X 的 release sequence 时, 在全部相关 scope 相容后有 $\text{Fr sw } \text{Fa}$ 。
5. 一次成功的 BAR.SYNC.CTA, 把每个到达 owner 的 shared release 侧连接到每个恢复 waiter 的 shared acquire 侧。阻塞记录冻结 owner_snapshot: $\text{set} \langle \text{linear_tid} \rangle$; 恢复时 acquire 正好应用于该快照。EXIT 只缩小 live_owner_set, 不贡献 release 侧, 因此不建立这类 sw 边。
6. 主机 release 所有权并启动 kernel, 启动事件同步到设备入口; 设备全部完成后, 完成事件同步到重新取得所有权的主机访问。

release sequence 是 mo_x 中从一个 release 修改开始、随后只包含连续 RMW 的最大序列; 遇到普通原子写就结束。

$\text{hb} = (\text{ppo 并上 sw}) \text{ 的传递闭包}$

hb 必须无环。

6.7 闭合公理

一个候选执行只有同时满足以下条件才允许:

1. 良构。每个普通读字节恰有一个合法 rf_b ; 每个原子读部恰有一个同址同宽 rf_a ; 每个普通字节有唯一 co_b ; 每个原子位置有唯一 mo_x 。
2. 原子性。每个 RMW 从 mo 紧邻前驱读取并作为一个不可分割修改提交。
3. HB 一致。hb 无环; 读不得从 hb 未来取值; co/mo 不得逆转同址 hb 写写顺序。
4. 每位置一致。po-loc、rf、fr、co、mo 的并集必须无环。

5. 全局可见性。ppo、sw、rfe、fr、co、mo 的并集必须无环。
6. 无凭空值。真依赖与 rfe 的并集必须无环；任何读值都必须来自实际写或 I。
7. 精确提交。故障、被 vp 条件排除或回滚的指令效果不在 E 中；成功 vector 指令的全部 lane 效果在架构指令边界共同提交。

这些公理共同闭合了 rf/co/mo/fr/ppo/sw/hb，不是让实现任选其中一部分。

7. U32/U64 原子

U32 和 U64 原子只用于自然对齐的 global 或 shared 位置。scalar 原子的头部 guard 固定为 PT，并且必须先通过 scalar-ready；成功后每条动态指令恰好一个原子事件。失败时不读 SGPR、不形成地址、不读取旧值，也不占 mo 位置。vector 原子按 P 中每个 lane 恰好一个事件，不套用 scalar-ready。

两种宽度均支持原子 LOAD、STORE，以及 ADD、MIN、MAX、AND、OR、XOR、XCHG、CAS。MIN/MAX 按无符号 U32/U64 比较；ADD 按 2^{32} 或 2^{64} 取模；CAS 比较完整位型并返回旧值。

它们进入内存关系的方式固定如下：

指令类别	事件	rf_a	mo	fr_a
atomic LOAD	一个 A_R	从同址同宽初始值或某个 mo 修改读取完整值	不新增位置	指向其读取修改之后的全部 mo 修改
atomic STORE	一个 A_W	无读部	追加一个新值	无读部，因此自身不产生 fr_a
atomic RMW	一个 A_RMW	读取自己在 mo 中的紧邻前驱	不可分割地追加一个新值	由读部指向后续修改
atomic CAS	一个 A_RMW	读取自己在 mo 中的紧邻前驱并返回旧值	成功写 replacement；失败把旧值原样追加，二者都占一个位置	由读部指向后续修改

一个 A_RMW 虽有读部和写部，架构事件数仍是一个。atomic LOAD/STORE 是独立 form，不能用“ADD 0”或“XCHG 后丢结果”替代，否则 rf_a、mo、order 合法集合都会变掉。

7.1 动态 order/scope modifier

每条原子机器字动态携带 2 位 order 和 2 位 scope：

modifier 位型	order
0	RELAXED
1	ACQUIRE
2	RELEASE
3	ACQ_REL

modifier 位型	scope
0	CTA

modifier 位型	scope
1	DEVICE
2	SYSTEM
3	保留；译码分类为 ILLEGAL_INSTRUCTION

order 决定本地先后，scope 决定能和哪些代理建立 sw。它们是每次动态原子事件自己的属性，不是 kernel 全局模式。scope=3 是保留编码，固定分类为 ILLEGAL_INSTRUCTION；它不是第四种 scope，也不能按 SYSTEM 或 CTA 处理。scope 为 0/1/2、但已知 modifier 组成不合法的操作/空间组合时，分类为 ILLEGAL_OPERAND。两类失败都不产生原子事件或其他架构效果。

canonical 原子语法必须显式带出 space、order 和 scope，字段顺序固定为：

```
(S_ATOM|V_ATOM).<op>.<type>.<space>.<order>.<scope>
```

也就是 space 紧跟 type，之后才是 order 和 scope。例如：

```
S_ATOM.LOAD.U32.GLOBAL.ACQUIRE.DEVICE s0, [s2:s3 + 0]
V_ATOM.XCHG.U32.GLOBAL.ACQ_REL.DEVICE v0, [s2:s3 + v4], v5
S_ATOM.STORE.U64.SHARED.RELEASE.CTA [s2 + 0], s4:s5
```

文本中的 order/scope 后缀装入同一 operation form 的 modifier 位，不产生另一套 opcode。省略 space、交换 space/order/scope 顺序，或给 shared 写非 CTA scope，都不是 canonical 合法语法。

合法组合只有下表：

原子类别	合法 order	global 合法 scope	shared 合法 scope
LOAD	RELAXED、ACQUIRE	CTA、DEVICE、SYSTEM	CTA
STORE	RELAXED、RELEASE	CTA、DEVICE、SYSTEM	CTA
ADD/MIN/MAX/AND/OR/XOR/XCHG	四种全部	CTA、DEVICE、SYSTEM	CTA
CAS	四种全部	CTA、DEVICE、SYSTEM	CTA

param、const 和 local 不支持原子。没有 SC order，也没有跨所有原子地址的总序。把 scope 写大不会凭空制造一次通信，更不能让 shared 离开 CTA。

同一个所有权纪元中，一旦某个 4/8 字节区间被原子访问，该精确区间就进入原子纪元：

- 后续重叠访问必须使用相同起点、相同宽度的原子；
- 普通访问与它有任何重叠，程序未定义；
- 不同起点或宽度的重叠原子，程序未定义；
- U32 和 U64 原子永不共享同一个 mo。

原子操作只搬运和计算位模式。U32 和 U64 原子都不携带任何影子状态，也不需要区分“保留 tag”和“清除 tag”的情况；一次 U64 原子读写的就是那 8 个字节。

8. FENCE 和 CTA 屏障

FENCE.CTA/DEVICE/SYSTEM 对执行代理做 acquire-release 排序。它对 warp 的 scalar 事件和相关 vector 事件建立第 6 节规定的 ppo；它不是会合点，也不等待其他 warp。只有配合同址原子通信并满足 scope 相容时，它才建立跨代理 sw。

BAR.SYNC.CTA id 是唯一的屏障指令，只有全 CTA 语义。每 CTA 有 8 个槽 0..7；owner 唯一身份为 CTA 内 $\text{linear_tid} = \text{warp_id} * 32 + \text{lane_id}$ 。完成条件是槽的 `arrived_set` 等于 CTA 当前的 `live_owner_set`；不存在的尾 lane 从不计入，EXIT 会把退出线程移出 `live_owner_set`，也没有 `expected` 或子集参数。

- 每次到达是一次 shared CTA release，覆盖该 warp 当前全部 live lane：屏障要求 scalar-ready，所以 warp 只能整体到达。
- `arrived_set` 追上 `live_owner_set` 后，所有等待者一起恢复并各自取得 shared CTA acquire，槽随即清回 idle。
- EXIT 可以通过缩小 `live_owner_set` 让屏障完成，但它自己不是 release，因此不给任何 waiter 建立 sw 边。

屏障阻塞记录保存 {warp_id, owner_snapshot, resume_pc}。恢复只把该 warp 的 PC 写成 `resume_pc` 并置 ready；active/live 掩码、重汇聚栈、调用栈和 shared 事件历史都不改。槽清空不是内存事件；因为槽里不保留任何跨屏障状态，同一个槽的两次屏障之间也没有需要区分的“代”。

这些边只覆盖 shared 事件。屏障不排序 global、local、param、const，也不涉及 host。用屏障交换 global 数据仍需合法原子发布/获取；需要的顺序仍由 atomic order/scope 和 FENCE 建立。

需要 arrive/wait 分离的软件用 shared memory 上的原子计数器加 FENCE.CTA 自行实现：release 侧用 RELEASE 原子递增，acquire 侧用 ACQUIRE 原子自旋读。这样得到的顺序由第 6 节的原子 sw 规则给出，不依赖任何屏障槽状态。

9. 主机所有权

global allocation 的所有权为 HOST、DEVICE 或运行时明确创建的 SYSTEM_SHARED。

通常采用独占转移：

1. 主机在 HOST 状态创建并初始化 allocation。
2. 启动前，运行时以 release 操作转为 DEVICE；设备入口通过 acquire 看到初始化。
3. kernel 执行期间，主机不得访问 DEVICE allocation，设备也不得访问 HOST allocation。
4. kernel 全部设备事件结束后，运行时以 SYSTEM release/acquire 转回 HOST。
5. 主机重新取得所有权后才能读取结果或释放 allocation。

FENCE.SYSTEM 只排序事件，不改变所有者，不能让主机在 kernel 执行时轮询一个 DEVICE allocation。

只有实现和运行时都声明支持时，allocation 才可进入 SYSTEM_SHARED。此时主机自然对齐、lock-free 的 U32/U64 原子映射到 SYSTEM scope 的同宽 A_R/A_W/A_RMW，并与设备共享 mo。主机只映射 RELAXED、ACQUIRE、RELEASE、ACQ_REL 四种 order。普通 payload 仍须通过 SYSTEM scope 的发布/获取形成 hb。

10. 数据竞争和 DRF 保证

两个事件满足以下条件时冲突：

1. 来自不同 lane、warp 或主机/设备代理；
2. 位于同一空间、同一 allocation 和同一所有权纪元；
3. 字节区间重叠；
4. 至少一个是写。

若冲突访问至少一方是普通访问，且双方互不 hb，程序存在数据竞争并整体未定义。param/const 的只读、每 lane 私有 local、合法同址同宽原子不构成数据竞争。

DRF 保证：程序若对所有输入都无数据竞争、遵守原子纪元和主机所有权，并用本章 sw 完成跨代理通信，那么普通访问等价于某个尊重 hb 与每位置 co 的交错执行；每个原子位置按自己的 mo 执行。不同地址仍没有架构总序。

11. 最小判例

11.1 标量和向量事件数

```
S_LD.GLOBAL.U32 s0, [s2:s3]    // scalar-ready 时，整个 warp 只有一个 R
V_LD.GLOBAL.U32 v0, [s2:s3 + v4] // P 中每个 lane 各有一个 R
```

第一条的头部 guard 必须是 PT，且必须满足 scalar-ready；成功恰好一个事件，失败零个事件。第二条对 P 中每个 lane 恰好一个事件，即使所有 v4 都相等，也不能退化成一个 scalar 事件。

11.2 64 位地址跨寄存器文件

```
V_MOV.B64 v2:v3, s4:s5    # ssrc=1
S_READFIRST.B64 s6:s7, v2:v3
```

若 s4:s5 保存一个合法 global 地址，则每个参与 lane 的 v2:v3 都得到同一个 64 位值；S_READFIRST.B64 再从编号最小的 live lane 把这 64 位复制回 s6:s7。两步都只搬位，空间仍由随后的访存 opcode 决定。

11.3 shared 屏障

```
warp0: V_ST.SHARED.U32 [x], 1; BAR.SYNC.CTA 0
warp1: BAR.SYNC.CTA 0; v5 = V_LD.SHARED.U32 [x]
```

成功的全 CTA 屏障后，v5 必须看到 1。把 x 换成 global，仅有屏障不足，未排序的普通冲突是数据竞争。

11.4 主机完成边

```
host: W x=1; launch
device: R x; W y=2; complete
host: R y
```

合法所有权转移保证设备读到 x=1，完成后主机读到 y=2。kernel 执行期间主机访问 DEVICE allocation 属于所有权违规。

VTX-1 ISA 1.0 Draft: 数值环境

本章规定浮点结果必须得到什么位型。除明确带 .APPROX 的指令外，结果必须逐位一致，不能把“宿主 CPU 大概算得一样”当作实现。

1. 先记住六条直观规则

1. S.F32 和 V.F32 算是同一种 binary32。区别只在寄存器和执行次数：S 形式读写每 warp 一份的 SGPR，算一次；V 形式读写每 lane 一份的 VGPR，对每个参与 lane 独立计算。
2. F16 是向量数值格式。V.F16 使用 VGPR 低 16 位；普通标量浮点算术没有 S.F16 形式。MMA 的 A/B 片段可以保存 F16。
3. 默认只有一种舍入。浮点结果固定使用“最近值，正中间取偶数”（RNE）。没有动态舍入寄存器，也没有每 lane 不同的舍入模式。
4. 小数不会偷偷冲成零。normal 以下仍保留 subnormal；输入也不做 DAZ，结果也不做 FTZ。
5. NaN 结果有统一答案。只要一个数值运算应得到 NaN，就返回目标格式的正号 canonical qNaN；load/store/MOV 这种纯位搬运则原样保留 payload。
6. 所有 S 浮点和 S 转换都要求 scalar-ready。算术、比较、FMIN/FMAX、FABS/FNEG、近似函数和整数/浮点转换没有例外。失败固定报告 DIVERGENCE_FAULT，不读 SGPR、不写 SGPR 或 SCC。

VTX-1 不提供浮点异常标志、trap enable、动态舍入状态、DAZ 或 FTZ。invalid、除零、上溢、下溢和 inexact 都不产生设备故障。

本章只关心 32 位和 16 位数值位型。寄存器上没有任何隐藏影子状态，浮点算术、数值转换和 MMA 输出都只写位型，不需要额外说明标签如何传播。

2. S 与 V 的执行规则

2.1 S.F32

S.F32 指令使用 SGPR。一次动态 S.F32 指令对整个 warp 只计算一次并写一份 SGPR 结果。它不能按参与 lane 重复计算，也不能因为某些 VGPR 中碰巧有相同值而冒充 V.F32。

本章所有 S.F32 指令 and 所有 S 数值转换都必须在读取任何动态源之前检查执行模型定义的 scalar-ready。warp 必须至少还有一个活 lane，全部活 lane 必须在同一路径上，而且重汇聚栈中不能有未完成的 FIRST 或 SECOND 帧。

只要还有一条分歧路径没走完，任何 S 浮点或 S 转换都固定报告 DIVERGENCE_FAULT。不存在“比较可以跑”“只读 SGPR 可以跑”“结果碰巧相同可以跑”或“这一条不会写结果所以可以跑”的例外。

失败指令不读 SGPR，不做 NaN 分类或舍入，不写 SGPR 或 SCC，也不留下部分结果。

2.2 V.F32 和 V.F16

V 指令对

E = 当前路径上的候选 lane
无 vp 条件: P = E
@vpN: P = E & snapshot(vpN)
@!vpN: P = E & ~snapshot(vpN)

中的每个 lane 独立读取该 lane 的 VGPR、独立计算、独立写回。不同 lane 之间没有隐含归约、进位或 NaN 共享。不在 P 中的 lane 不读取源，目标保持不变。

除 warp collective 和 MMA 外，一个 lane 的特殊值不会影响另一个 lane。

V 浮点和 V 转换不套用 scalar-ready；它们只更新 P 中 lane 的 VGPR 或 vp。

V 浮点 form 的 scalar-source selector 可以把其中一个源改成 SGPR，例如 V_FMUL.F32 vd, va, sB。这只改变那个源从哪个寄存器文件读取，不改变数值语义：舍入、NaN 传播和 subnormal 处理与两个源都来自 VGPR 时完全一致。执行类别仍是 V，也不会因此变成 S 指令。一条 V 浮点指令最多只能有一个 SGPR 源。

2.3 同一算法，不同寄存器

如果 S.F32 与 V.F32 具有相同操作名和相同输入位型，它们必须应用完全相同的 NaN、无穷、零、subnormal 和舍入规则。S/V 前缀不是“快精度”和“准精度”的区别；它只决定 SGPR/VGPR、执行次数以及是否必须通过 scalar-ready。

3. 位型与寄存器表示

3.1 F16 / IEEE 754 binary16

bit 15	bit 14..10	bit 9..0
sign	exponent	fraction
1	5	10

指数偏置为 15：

- $e=0, f=0$: +0 或 -0;
- $e=0, f!=0$: $(-1)^s * 2^{-14} * (f / 2^{10})$ ，即 subnormal;
- $1 \leq e \leq 30$: $(-1)^s * 2^{(e-15)} * (1 + f/2^{10})$;
- $e=31, f=0$: +Inf 或 -Inf;
- $e=31, f!=0$: NaN。

最小正 subnormal 是 2^{-24} ，最小正 normal 是 2^{-14} ，最大有限值是 65504。canonical qNaN 为：

F16 canonical qNaN = 0x7e00

V.F16 放在一个 VGPR 的低 16 位。任何产生 V.F16 寄存器结果的数值指令必须把高 16 位写成 0；消费 V.F16 的数值指令只解释低 16 位。纯 MOV.U32 仍复制全部 32 位，不检查它是不是合法 F16 数值。

普通 S.F16 算术和 S.F16 转换不是本数值环境的一部分；把 F16 当作 U16/U32 位型做标量搬运不等于支持 S.F16 算术。

3.2 F32 / IEEE 754 binary32

bit 31	bit 30..23	bit 22..0
sign	exponent	fraction
1	8	23

指数偏置为 127：

- $e=0, f=0$: +0 或 -0;
- $e=0, f!=0$: $(-1)^s * 2^{-126} * (f / 2^{23})$;
- $1 \leq e \leq 254$: $(-1)^s * 2^{(e-127)} * (1 + f/2^{23})$;
- $e=255, f=0$: +Inf 或 -Inf;

- $e=255, f!=0$: NaN。

最小正 subnormal 是 2^{-149} ，最小正 normal 是 2^{-126} ，最大有限值是 $(2-2^{-23}) \cdot 2^{127}$ 。canonical qNaN 为：

F32 canonical qNaN = 0x7fc00000

F32 占一个 SGPR 或 VGPR。load/store 只搬运位型，不做数值转换。

4. NaN、无穷和有符号零

4.1 NaN

NaN 的 fraction 最高位为 1 时是 quiet NaN，为 0 时是 signaling NaN。架构不暴露 signaling 异常。

以下规则对 S.F32、V.F32、V.F16 和 MMA 都适用：

- FADD、FSUB、FMUL、FDIV、FSQRT、FFMA、数值转换、近似函数和 MMA 只要语义要求 NaN，就返回目标格式的 canonical qNaN；
- 输入 NaN 的符号和 payload 不传播；
- sNaN 先按 NaN 处理，但不设置异常标志；
- 两个 NaN 也不选择其中一个 payload；
- FABS 只清 sign，FNEG 只翻转 sign；二者是纯位操作，保留 payload 和 signaling/quiet 位；
- MOV、load、store 是纯位搬运，不 canonicalize NaN；
- FMIN/FMAX 使用第 8 节的 number 选择规则。

常见无效形式：

```
(+Inf) + (-Inf)
0 * Inf
Inf / Inf
0 / 0
sqrt(负有限数)
sqrt(-Inf)
FMA 中无穷乘积再加相反符号无穷
```

这些形式均返回目标格式 canonical qNaN。

4.2 无穷

除无效形式外，无穷按 IEEE 754 扩展实数规则参与计算，例如：

- 有限非零数除以 $+0/-0$ 得到带正确符号的 Inf；
- 有限数加同号 Inf 得同号 Inf；
- $\text{sqrt}(+\text{Inf}) = +\text{Inf}$ ；
- 有限结果上溢时按 RNE 得到最大有限数或 Inf，取决于精确值落在哪一侧。

4.3 有符号零

$+0$ 和 -0 数值比较相等，但位型不同：

- 相反符号的精确抵消在 RNE 下得到 $+0$ ， $-0 + -0$ 得 -0 ；
- 乘法和除法结果为零时，符号为操作数符号异或；

- $\text{sqrt}(-0) = -0$;
- $\text{FABS}(-0) = +0$, $\text{FNEG}(+0) = -0$;
- FFMA 按精确表达式 $a*b+c$ 一次舍入后的 IEEE 零符号规则;
- $\text{FMIN}(-0,+0)$ 返回 -0 , $\text{FMAX}(-0,+0)$ 返回 $+0$ 。

5. 舍入和 subnormal

记 $\text{RN16}(x)$ 、 $\text{RN32}(x)$ 为把精确实数 x 舍入到 F16、F32。固定舍入模式为 round to nearest, ties to even (RNE) :

1. 选择离精确值最近的可表示值;
2. 如果精确值正好在两个值中点, 选择最低有效保留位为 0 的那个;
3. 下溢继续使用同一规则并保留 subnormal;
4. 上溢也使用同一规则, 不是简单地“只要超出最大有限数就立刻变 Inf”。

F16 正向上溢的 RNE 分界为 65520: 小于分界的相应值可舍入到 65504, 达到分界时舍入到 $+\text{Inf}$ 。负值对称处理。F32 的对应正分界为 $2^{128} - 2^{103}$ 。

所有输入 subnormal 按完整数学值参与计算, 所有 subnormal 结果按 RNE 保留。实现内部可以使用更宽精度, 但必须在每个架构规定的舍入点写出相同位型。

除浮点转整数明确使用 RTZ 外, 指令编码不能改变舍入模式。

6. 精确 F32 运算

对有限输入, 先在数学上的无限精度中求表达式, 再在规定位置舍入:

```
FADD.F32(a,b) = RN32(a+b)
FSUB.F32(a,b) = RN32(a-b)
FMUL.F32(a,b) = RN32(a*b)
FDIV.F32(a,b) = RN32(a/b)
FSQRT.F32(a)  = RN32(sqrt(a))
FFMA.F32(a,b,c) = RN32(a*b+c)
```

FADD、FSUB、FMUL、FDIV、FSQRT 各只在最终结果处舍入一次。FFMA 的乘积不单独舍入。

S 和 V 形式都按这些公式执行。V 形式对每个参与 lane 单独应用公式; S 形式先通过 scalar-ready, 再对 SGPR 输入应用一次。

实现不得把:

```
FMUL t,a,b
FADD d,t,c
```

自动改成 FFMA, 因为前者有两个舍入点。也不得把 FFMA 拆成乘法和加法。允许改变数值结果的快速数学优化不属于 ISA 语义。

7. 比较

S/V FSETP.{EQ,NE,LT,LE,GT,GE,ORD,UNO}.F32 使用相同数值规则:

- 任一输入是 NaN: NE 和 UNO 为真; 其他比较为假;
- 两个输入都不是 NaN: ORD 为真, UNO 为假, 其余按数值顺序;

- $+0 == -0$ ，二者之间 LT 和 GT 都为假；
- Inf 按扩展实数顺序比较。

比较不修改输入，也不产生浮点异常状态。S 比较必须先通过 scalar-ready，然后把 warp 共享结果写入 SCC；V 比较把每个参与 lane 的结果写入 vp。S 比较不能在未完成的分歧中执行。

8. FMIN 和 FMAX

S/V FMIN/FMAX 采用 minimumNumber / maximumNumber 风格。S 形式必须先通过 scalar-ready，V 形式逐参与 lane 执行：

1. 只有一个输入是 NaN：返回另一个输入的原始位型；
2. 两个输入都是 NaN：返回 F32 canonical qNaN；
3. 数值不等：返回较小值或较大值的原始位型；
4. 输入为 $-0, +0$ ：FMIN 返回 -0 ，FMAX 返回 $+0$ ；
5. 数值相等且位型相同：返回该位型。

因此，单个 NaN 不会盖住一个普通数；但双 NaN 仍得到统一 canonical qNaN。

9. 转换

9.1 整数与 F32

S/V 整数转 F32：

```
CVT.F32.S32 = RN32(int32(src))
CVT.F32.U32 = RN32(uint32(src))
```

源先变成精确数学整数，再做 RNE。绝对值不超过 2^{24} 的可表示整数必须精确。

F32 转整数先使用 round toward zero (RTZ) 截断，再饱和：

源	CVT.S32.F32	CVT.U32.F32
NaN	0	0
+Inf 或大于上界	0x7fffffff	0xffffffff
-Inf 或小于下界	0x80000000	0
范围内有限值	trunc(x) 的二补数	trunc(x)

-0 转为整数 0。转换不产生浮点或整数故障。

这里所有 S 转换都读写 SGPR，并且必须先通过 scalar-ready；失败是 DIVERGENCE_FAULT，不是数值饱和，也不会写目标 SGPR。所有 V 转换逐参与 lane 读写 VGPR，不套用 scalar-ready。

9.2 V.F16 与 V.F32

F16/F32 数值转换只有 V 形式：

V_CVT.F32.F16：

- normal、subnormal、零和 Inf 精确扩展；
- 符号保留；

- 任意 F16 NaN 变为 0x7fc00000。

V_CVT.F16.F32:

- 有限值结果为 RN16(x);
- +0/-0、+Inf/-Inf 保留符号;
- 任意 F32 NaN 变为低 16 位 0x7e00;
- 目标 VGPR 高 16 位写 0。

V load F16 把两个内存字节零扩展到 VGPR, V store F16 只写源 VGPR 低 16 位; 二者不做转换或 NaN canonicalization。

10. 近似函数

.APPROX 只表示允许一个明确受限的误差, 不表示任意答案。近似函数返回 S.F32 或逐 lane V.F32; 两种形式使用同一误差合同。S 形式仍必须先通过 scalar-ready, 不能因为结果本来就是近似值而放宽执行状态。

对非 NaN F32 位型 u 定义保持数值顺序的整数键:

```
ordered(u) = (~u) & 0xffffffff, sign(u)=1
             u | 0x80000000,  sign(u)=0

ulp_distance(a,b) =
    abs(ordered(bits(a)) - ordered(bits(b)))
```

参考值 ref 是无限精度实函数结果经 RN32 后的位型。若 ref 有限且非零, 实现结果必须同号且与 ref 相差不超过 2 ULP。若 ref 是零或 Inf, 结果必须逐位等于 ref。NaN 必须为 F32 canonical qNaN。同一实现对相同输入必须确定。

10.1 FRCP.APPROX.F32

参考函数为 1/x:

- +0/-0 -> +Inf/-Inf;
- +Inf/-Inf -> +0/-0;
- NaN -> canonical qNaN;
- 其余输入相对 RN32(1/x) 不超过 2 ULP。

10.2 FRSQRT.APPROX.F32

参考函数为 1/sqrt(x):

- +0 -> +Inf, -0 -> -Inf;
- +Inf -> +0;
- 负有限数和 -Inf -> canonical qNaN;
- NaN -> canonical qNaN;
- 正有限数相对 RN32(1/sqrt(x)) 不超过 2 ULP, 结果不得为负。

10.3 FEXP2.APPROX.F32

参考函数为 2^x:

- $-\text{Inf} \rightarrow +0$, $+\text{Inf} \rightarrow +\text{Inf}$;
- NaN $\rightarrow$ canonical qNaN;
- 有限输入相对 $\text{RN32}(2^x)$ 不超过 2 ULP;
- 结果非负;
- 对任意非 NaN F32 输入 $a < b$, 结果必须满足 $\text{FEXP2}(a) \leq \text{FEXP2}(b)$ 。

11. FFMA 的单次舍入

S/V FFMA.F32 a,b,c 都计算精确表达式 $a*b+c$, 只在最终写回执行一次 RN32。S 形式必须先通过 scalar-ready; V 形式逐参与 lane 计算:

- 乘积不先舍入;
- subnormal 中间积不提前清零;
- 任一输入 NaN 返回 canonical qNaN;
- $0*\text{Inf}$ 或 $\text{Inf}*0$ 返回 canonical qNaN;
- 无穷乘积加相反符号无穷返回 canonical qNaN;
- 其他有限、无穷和零按 IEEE 融合操作处理。

所以 FFMA 与 FMUL; FADD 不一定得到相同末位。

12. MMA.M16N8K16.F16.F16.F32

MMA 只定义这一种 M16N8K16、 $F16 \times F16$ 加 F32 的 form。它属于 MATRIX 执行域, 是整 warp 协作指令, 不是普通 V.FP, 也不是 S 指令。头部 guard 固定为 PT, 不使用 vp 删减参与者, 也不检查 scalar-ready。

12.1 参与和寄存器组

全部 32 个 lane 都必须仍存活且 active, 也就是执行集合等于 live 集合并且恰有 32 个 lane。缺 lane、分歧中只到一部分 lane、不同动态 PC 或会合失败, 都报告 COLLECTIVE_FAULT, 所有 D 保持原值。

令四个编码基址分别为 vd、va、vb、vc。每个 lane 使用:

矩阵	每 lane VGPR	基址对齐
A	va..va+3, 4 个	4
B	vb..vb+1, 2 个	2
C	vc..vc+3, 4 个	4
D	vd..vd+3, 4 个	4

完整组必须落在可用 VGPR 范围内。A、B、C 三个源组必须两两不重叠; D 必须与 A、B 不重叠。唯一允许的别名是 D 与 C 完整相同, 即 $vd=vc$; D/C 部分重叠或任何其他组间重叠都为 ILLEGAL_OPERAND。

通过检查后, 先冻结全部 32 个 lane 的 A、B、C, 再开始任何 D 写回。因此 $D=C$ 是安全的原地累加。

12.2 A/B/C/D 元素映射

令 lane 编号 l 在 $0..31$ 。F16 半字编号 $h=0$ 表示 VGPR 位 $[15:0]$, $h=1$ 表示位 $[31:16]$; 两个半字都按小端 F16 位型解释。

A 片段中, $ra=0..3$:

```
qA = 8*l + 2*ra + h
m = qA div 16
k = qA mod 16
A[m,k] = F16_half(VGPR[vA+ra, lane=l], h)
```

B 片段中, $rb=0..1$:

```
qB = 4*l + 2*rb + h
k = qB div 8
n = qB mod 8
B[k,n] = F16_half(VGPR[vB+rb, lane=l], h)
```

C 和 D 每个元素占一个完整 VGPR。对 $r=0..3$:

```
q = 4*l + r
m = q div 8
n = q mod 8

C[m,n] = F32_bits(VGPR[vC+r, lane=l])
VGPR[vD+r, lane=l] = bits(D[m,n])
```

这些公式把 A 的 256 个 F16、B 的 128 个 F16、C/D 的 128 个 F32 各映射一次且不重复。实现不能换一种 lane 布局。

12.3 固定数值步骤

每个 A/B F16 先按 V_CVT.F32.F16 扩展。有限 F16 到 F32 是精确的; 任意 F16 NaN 变成 F32 canonical qNaN。

每个输出 (m,n) 独立令 $acc=C[m,n]$, 然后严格按 $k=0,1,...,15$ 递增执行:

```
acc = RN32(F32(A[m,k]) * F32(B[k,n]) + acc)
```

每一步就是一次完整的 F32 FFMA: 乘积不先舍入, 只在该步末执行一次 RNE; 下一步读取已经舍入的 F32 acc。禁止重排 k、树形归约、跨 k 保留额外精度, 或用 FP64 累加后只舍入一次。

特殊值逐步处理:

- 任一乘数或 acc 为 NaN 时, 本步得到 F32 canonical qNaN; sNaN 被安静化且 payload 不传播;
- $0*\text{Inf}$ 、 $\text{Inf}*0$, 或无穷乘积再加相反符号无穷, 得到 canonical qNaN;
- 其他 Inf 按 F32 FFMA 规则传播;
- subnormal 输入和中间结果不做 DAZ/FTZ;
- 有符号零按 F32 FFMA 的一次舍入规则决定;
- 一旦某一步得到 canonical qNaN, 后续步骤仍为 canonical qNaN。

全部 128 个 D 元素算完并通过检查后, 所有 D VGPR 一次性提交。MMA 不产生内存事件, 也不隐含内存栅栏。

13. 一致性测试要求

符合实现至少必须逐位测试:

- S.F32 与 V.F32 对相同输入得到相同数值位型;
- 每一种 S 浮点、S 比较和 S 转换在非 scalar-ready 状态都报告 DIVERGENCE_FAULT, 不读动态源、不写 SGPR 或 SCC;

- 未完成的 FIRST 或 SECOND 分歧中不存在任何可执行的 S 浮点或 S 转换例外;
- F16/F32 的正负零、最小/最大 subnormal、最小 normal、最大有限值、正负 Inf、qNaN 和 sNaN;
- RNE 中点取偶, 以及 subnormal/normal、最大有限值/Inf 两个边界;
- 无 DAZ、无 FTZ;
- NaN canonicalization 与 MOV/load/store 的 payload 原样搬运;
- FADD/FSUB 完全抵消的零符号, FMUL 的 $0*\text{Inf}$;
- FFMA 与拆分乘加不同的测试向量;
- FMIN/FMAX 的单 NaN、双 NaN 和 $-0/+0$;
- 整数转换的 NaN、Inf、边界和饱和;
- V.F16 写回高 16 位清零;
- .APPROX 特殊值、2 ULP 上界和 FEXP2 单调性;
- MMA 的 32-lane 完整参与、A/B/C/D 映射公式、组对齐、D=C 唯一别名、源冻结和整体提交;
- MMA 的 $k=0..15$ 递增 FFMA、每步 RNE, 以及 NaN、Inf、subnormal 和有符号零。

如果宿主平台会扩展精度、自动合约 FMA、冲掉 subnormal 或传播不同 NaN payload, 模拟器必须显式屏蔽这些行为。

6. 编码与汇编

本章定义 VTX-1 的全新固定 64 位指令编码。它与任何旧编码均不兼容；实现不得根据旧机器字、旧 opcode 或旧字段位置进行猜测、回退或双重解码。

本章中的“必须”“禁止”“可以”分别对应 MUST、MUST NOT、MAY。

6.1 总体原则

每条指令恰好占 64 位（8 字节），正常顺序执行时：

```
next_pc = pc + 8
```

指令字 W 的位 0 是最低有效位。统一头部如下：

```
bit 63          19 18   13 12   7 6   4 3   0
+-----+-----+-----+-----+-----+
|          payload[44:0]          | guard[5:0] | opcode[5:0] | format | class |
+-----+-----+-----+-----+-----+

class = W[3:0]
format = W[6:4]
opcode = W[12:7]
guard = W[18:13]
payload = W[63:19]
```

头部字段的含义是：

- class：编码大类，决定接下来如何解释 format。
- format：操作数装箱格式，决定 payload 的基本字段位置。
- opcode：在 (class, format) 内局部编号的操作码。
- guard：向量执行的可选 lane 谓词。它不是普通数据操作数。
- payload：寄存器号、立即数及 opcode 专用字段，共 45 位。

opcode 不是全局编号。同一个 6 位数值可以在不同 (class, format) 下分配给不同指令。真正唯一的译码叶子是 form：每个 form 自己声明一个 (class, format, opcode) 三元组，该三元组在整个 ISA 清单中只能映射到这一个 form。译码器先由三元组找到 form，再按该 form 的 payload 约束完成解码。

6.1.1 字节序和取指

指令在文本段中按小端字节序保存：

```
B[i] = W[8*i +: 8], i = 0..7
W =  $\sum (\text{uint64}(B[i]) \ll (8*i))$ 
```

例如 W = 0x1122334455667788 在内存中从低地址到高地址为：

```
88 77 66 55 44 33 22 11
```

PC 必须 8 字节对齐，且 [pc, pc+8) 必须完整位于当前内核文本内。未对齐取指、越界取指或不足 8 字节的尾部均为 ILLEGAL_INSTRUCTION；实现不得读取相邻对象来补齐指令。

6.2 class 与 format 分配

6.2.1 class

class	名称	用途
0	SYS	系统、特殊寄存器、陷阱及杂项
1	SALU	标量算术与逻辑
2	VALU	向量算术与逻辑
3	MEMORY	标量/向量访存与原子
4	CONTROL	分支、重汇聚和控制流
5	SYNC	屏障与内存同步
6	CROSSLANE	warp 内跨 lane 集合操作
7	MATRIX	矩阵乘加
8..15	保留	必须拒绝

class=8..15 产生 ILLEGAL_INSTRUCTION，不得解释为 NOP、提示指令或私有扩展。

6.2.2 每个 class 的 format

format 只在所属 class 内有意义：

class	format 值	format 名称
SYS	0	SYS
SALU	0	S1
SALU	1	S2
SALU	2	S3
SALU	3	SCMP
SALU	4	SIMM
VALU	0	V1
VALU	1	V2
VALU	2	V3
VALU	3	VCMP
VALU	4	VIMM
MEMORY	0	SMEM
MEMORY	1	VMEM
MEMORY	2	VSHMEM
MEMORY	3	VLMEM
MEMORY	4	SATOM
MEMORY	5	VATOM
MEMORY	6	SMEMX
MEMORY	7	VATOMX

class	format 值	format 名称
CONTROL	0	CTRL
SYNC	0	SYNC
CROSSLANE	0	COLL
MATRIX	0	MMA

表中未列出的 class/format 组合全部保留，并产生 ILLEGAL_INSTRUCTION。

在一个合法 (class, format) 内，opcode=0..63 中只有 form 清单明确分配的值合法。未分配 opcode 产生 ILLEGAL_INSTRUCTION。每个 form 必须同时给出 family、助记符、数据类型、执行域、操作数角色、payload 扩展字段和全部静态约束。

6.3 执行域、格式、family 与 form

6.3.1 执行域

execution_domain 只能取以下七个值：

值	动态执行含义
system	系统、特殊状态或陷阱操作；具体参与者由 form 定义
scalar	每个 warp 执行一次，主要读写 SGPR/SCC
vector	在参与 lane 上逐 lane 执行，主要读写 VGPR/VP
warp_control	每个 warp 执行一次并修改 PC、掩码或控制栈
warp_collective	warp 内多个 lane 共同完成一次集合操作
cta_sync	CTA 范围屏障或同步操作
warp_matrix	warp 协作矩阵操作

不得使用 control、synchronization、collective、matrix 等近义值。

执行域影响参与者、状态副作用和 guard 合法性，但不直接规定 payload 的位布局。

execution_domain 与机器头部的 class 不是同一概念。class 是编码和 opcode 分配的命名空间，execution_domain 才规定动态执行方式。尤其是 MEMORY class 同时包含 scalar 和 vector form：

- SMEM、SMEMX、SATOM 的 execution_domain: scalar；
- VMEM、VSHMEM、VLMEM、VATOM、VATOMX 的 execution_domain: vector。

因此不能因为 class=MEMORY 就跳过 scalar-ready 检查，也不能因为 form 位于 MEMORY class 就推断它一定逐 lane 执行。

6.3.2 编码格式

编码格式回答“操作数放在哪些位”。例如 V2 表示一个向量目标和两个向量源使用固定的三个 8 位槽。V2 本身不表示整数、浮点、加法或乘法。

6.3.3 family: 语义分组

family 是语义分组，回答“一组 form 共同表达什么操作以及采用哪些数值规则”，例如：

- 整数算术;
- 浮点算术;
- 位运算;
- 比较;
- 数据移动;
- 内存访问;
- 原子;
- 控制流;
- 同步或集合。

family 不是编码字段，不参与唯一译码，也不是 format 的别名。一个 family 可以包含多个 form，并跨越不同格式。例如标量整数加法 family 可以同时包含：

```
form=iadd_s2_u32 class=SALU format=S2 opcode=...
form=iadd_simm_u32 class=SALU format=SIMM opcode=...
```

两个 form 属于同一个 IADD family：前者从两个 SGPR 取源，后者从一个 SGPR 和一个立即数取源。译码器不能先找 family 再猜格式；它必须直接用每个 form 声明的 (class, format, opcode) 找到唯一叶子。

6.3.4 form：唯一译码叶子

form 是完整、可编码、可执行的叶子定义。它必须固定：

- (class, format, opcode);
- family 和 canonical 助记符;
- execution_domain 和 required_state;
- guard_policy;
- 每个 payload 位的用途;
- 操作数类型、立即数解释、must-zero 和静态约束;
- 精确语义与故障。

family 可以有任意多个 form，但两个 form 禁止声明相同的 (class, format, opcode)。字段值也禁止把一个 form 二次分派成另一个 form。payload modifier 可以在同一个 operation form 内取多个明确列出的合法值；modifier 值不是新的 form，也不得为每个 modifier 组合额外消耗 opcode。

最直白的例子是：

```
IADD.U32 v1, v2, v3
FADD.F32 v1, v2, v3
```

两条指令都使用 VALU/V2，所以 vd=v1、va=v2、vb=v3 的位位置完全相同。但是它们属于不同 family，form 和 opcode 也不同：IADD family 执行 32 位整数加法，FADD family 执行 IEEE 754 binary32 加法。译码器绝不能因为看到 V2 就认定它是整数指令，也不能因为寄存器号相同就猜测数据类型。

反过来，同一个 family 可以使用多个格式。例如整数加法的寄存器 form 可使用 S2，带立即数 form 可使用 SIMM；向量版本也可分别使用 V2 和 VIMM。family 相同并不意味着编码格式相同。

6.4 guard

6.4.1 编码

guard[5:0] 的合法值为：

```
0    PT
1    !PT
2..17 vp0..vp15    (编码 = 2 + n)
18..33 !vp0..!vp15 (编码 = 18 + n)
34..63 reserved
```

PT 恒真，!PT 恒假。vp0..vp15 是逐 lane 向量谓词，取反只影响本次条件读取，不修改谓词寄存器。保留 guard 编码产生 ILLEGAL_INSTRUCTION。

canonical 汇编省略 @PT，其他形式写作：

```
@!PT
@vp3
@!vp3
```

6.4.2 按 form 判断 guard

guard 合法性只能在译码出 form 后，根据该 form 的 execution_domain 和 guard_policy 判断。machine class 和 format 都不能单独决定 guard：

guard_policy	header guard 规则
optional	允许 PT、!PT、vpN、!vpN；该 policy 只允许用于 execution_domain: vector
required_pt	必须精确编码为 PT(0)
explicit_condition	header guard 必须为 PT(0)；实际 lane 条件来自 payload 的显式数据条件字段

典型反例说明为什么不能按 class 判断：

- V_GETREG 位于 SYS class，但其 form 是 execution_domain: vector、guard_policy: optional，所以允许非 PT；
- V_SHUFFLE.DOWN.B32 位于 CROSSLANE class、使用 COLL format，是真正的 warp_collective form，必须 required_pt；
- S_READFIRST 也位于 CROSSLANE/COLL，但它是 scalar form，必须 required_pt 并检查 scalar-ready。

所有 execution_domain: cta_sync、warp_collective、warp_matrix 的 form 都必须 guard_policy: required_pt。因此 SYNC form、全部 COLL 集合 form 和唯一 MMA form 都只能使用 PT。

每个 execution_domain: scalar 的 form 都必须声明 guard_policy: required_pt 和 required_state: scalar_ready，并在读取任何动态源前检查：

```
live_mask != 0
active_mask == live_mask
reconv_stack 中不存在 phase 为 FIRST 或 SECOND 的帧
```

三个条件必须同时成立。条件不满足产生 DIVERGENCE_FAULT，且不得读取 SGPR、SCC 或内存源。ARMED 帧本身不破坏 scalar-ready。

CONTROL form 的状态要求逐 form 声明，不能由 machine class 推导：

form	guard_policy	required_state
CALL direct	required_pt	scalar_ready

form	guard_policy	required_state
CALL.IND	required_pt	scalar_ready
JUMP.IND	required_pt	scalar_ready
RET	required_pt	scalar_ready
BRA	required_pt	none
BRA.P	explicit_condition	none

因此 BRA/BRA.P 在分歧状态下仍可按控制流语义执行，禁止对它们附加 scalar-ready 条件。

CTRL 中作为数据操作数的分支条件不属于头部 guard。warp_collective 和 warp_matrix form 不允许用 lane guard 改变集合参与者。

guard 为假只抑制该 lane 的架构效果，不抑制静态译码。未知 opcode、保留字段、must-zero 非零、非法寄存器组等错误，即使 guard 为 !PT 也必须被检测。

6.5 payload 基本格式

以下各表使用 payload 局部编号 $P[44:0] = W[63:19]$ 。x 表示 opcode 专用扩展区。每个 form 必须进一步把 x 的每一位定义为具名字段或 must-zero；不存在“实现忽略”的扩展位。

6.5.1 SYS

P 位	字段
[7:0]	a
[15:8]	b
[23:16]	c
[39:24]	imm16
[44:40]	x5

a/b/c 的寄存器类别由 opcode 固定。未使用的槽必须为零。系统指令不得通过字段值猜测操作数形式。

6.5.2 SALU

format	P 位布局
S1	[7:0] sd, [15:8] sa, [44:16] x29
S2	[7:0] sd, [15:8] sa, [23:16] sb, [44:24] x21
S3	[7:0] sd, [15:8] sa, [23:16] sb, [31:24] sc, [44:32] x13
SCMP	[7:0] zero8, [15:8] sa, [23:16] sb, [44:24] x21
SIMM	[7:0] sd, [15:8] sa, [39:16] imm24, [44:40] x5

S1/S2/S3 分别提供一、二、三个标量源槽。SCMP 没有显式目标寄存器：zero8 必须为零，比较 sa 与 sb 后隐式写每 warp 一份的 1 位条件码 SCC，false 写 0，true 写 1。SIMM 提供一个 SGPR 源和一个最多 24 位的立即数容器。全部 SALU form 都声明 guard_policy: required_pt 和 required_state: scalar_ready。

6.5.3 VALU

format	P 位布局
V1	[7:0] vd, [15:8] va, [16] ssrc, [44:17] x28
V2	[7:0] vd, [15:8] va, [23:16] vb, [25:24] ssrc_sel, [44:26] x19
V3	[7:0] vd, [15:8] va, [23:16] vb, [31:24] vc, [33:32] ssrc_sel, [44:34] x11
VCMP	[3:0] vpd, [7:4] zero4, [15:8] va, [23:16] vb, [25:24] ssrc_sel, [44:26] x19
VIMM	[7:0] vd, [15:8] va, [39:16] imm24, [44:40] x5

VCMP 通过 vpd 显式选择并写 vp0..vp15，且 zero4 必须为零；它不写 SCC。VIMM 的目标是 VGPR；需要 “比较寄存器与立即数” 的程序必须使用明确分配的比较立即数 opcode/格式，若清单未分配则先物化常量，汇编器不得擅自把 VIMM.vd 解释为谓词目标。

这四种 VALU 格式与 SALU 的对应格式的唯一结构差别，就是它们从扩展字段里切出一个 scalar-source selector。V1 用 1 位的 ssrc，V2、V3、VCMP 用 2 位的 ssrc_sel。selector 不改变源槽的位置和宽度，只改变那 8 位寄存器号在哪个寄存器文件里解释：

format	selector 字段	合法值	含义
V1	ssrc	0	va 读 VGPR
V1	ssrc	1	va 读 SGPR
V2、VCMP	ssrc_sel	0 / 1 / 2	无标量源 / va 读 SGPR / vb 读 SGPR
V2、VCMP	ssrc_sel	3	保留，ILLEGAL_INSTRUCTION
V3	ssrc_sel	0 / 1 / 2 / 3	无标量源 / va / vb / vc 读 SGPR

因此一条 VALU 指令最多只有一个 SGPR 源。清单中把可以这样切换的源操作数写成 vsrc32 或 vsrc64 类型；只有 execution_domain: vector 且格式属于 V1/V2/V3/VCMP 的 form 才允许出现这两种类型。目标操作数永远是 VGPR 或 vpN，不受 selector 影响。

不含 vsrc* 操作数的 VALU form（例如 VIMM 系列，或所有源都固定为 VGPR 的 form）必须把 selector 字段编码为零；它在这些 form 里是 must-zero 洞，非零就是 ILLEGAL_INSTRUCTION。

vsrc64 的两个寄存器文件都要求偶数对齐的完整寄存器对，语法上必须写全，例如：

```
V_MOV.B64 v2:v3, v4:v5    # ssrc=0
V_MOV.B64 v2:v3, s4:s5    # ssrc=1
```

汇编器由源操作数的前缀唯一确定 selector 值：写 sN 就置对应的 selector 码，写 vN 就保持该位置为 VGPR。同一条指令里出现两个 sN 源没有可用编码，必须报错，而不是自行插入搬运指令。

6.5.4 MEMORY

SMEM:

P 位	字段
[7:0]	sdata
[15:8]	sbase
[39:16]	simm24
[44:40]	x5

SMEMX:

P 位	字段
[7:0]	sdata
[15:8]	sbase
[23:16]	sindex
[39:24]	imm16
[44:40]	mods

SMEMX 的地址形式是 “SGPR base + SGPR index + immediate”。sbase 是统一地址的 SGPR 基址或基址组，sindex 是 SGPR 索引，imm16 是有符号字节偏移。三者的扩展、索引缩放和基址组宽由具体 form 固定；未定义的 mods 位必须为零。译码器不得根据某个值为零来省略或改换地址模式。SMEMX 是 scalar form，必须声明 guard_policy: required_pt 和 required_state: scalar_ready。

VMEM/VSHMEM/VLMEM 共享字段位置，但地址空间和地址形成规则不同：

P 位	字段
[7:0]	vdata
[15:8]	vaddr
[23:16]	sbase
[39:24]	simm16
[44:40]	x5

VMEM 有且只有以下三种地址 form；syntax 和 must-zero 规则都是 form 的一部分：

地址 form	canonical 地址 syntax	sbase	vaddr
uniform base	[sB:s(B+1) + imm]	64 位 SGPR base，必须显式寄存器对	must-zero
uniform base + lane index	[sB:s(B+1) + vl + imm]	64 位 SGPR base，必须显式寄存器对	32 位 VGPR index
lane base	[vB:v(B+1) + imm]	must-zero	64 位 VGPR base，必须显式寄存器对

例如：

```
V_LD.GLOBALU32 v0, [s2:s3 + 16]
V_LD.GLOBALU32 v0, [s2:s3 + v4 + 16]
V_LD.GLOBALU32 v0, [v6:v7 + 16]
```

地址模式属于 form 定义，不是运行时 selector。uniform-base form 即使实际 base 数值为零，vaddr 仍必须为零；lane-base form 的 sbase 必须为零；indexed form 的 sbase/vaddr 都是有效操作数，不是 must-zero。SV-mix (uniform base + lane index) 地址形成固定为：

```
effective_address[lane] =
    SGPR64(sbase:sbase+1)
    + zero_extend_64(VGPR32(vaddr)[lane])
    + sign_extend_64(simm16)
```

VGPR32 index 必须零扩展，禁止符号扩展或先按 32 位回绕。

LOCAL 只允许单个 32 位 vaddr 加 simm16：

```
V_LD.LOCAL.U32 v0, [v2 + 16]
V_ST.LOCAL.U32 [v2 + 16], v0
```

所有 LOCAL form 的 sbase 必须为零；LOCAL 禁止 SGPR base、SGPR+VGPR indexed 地址和 64 位 VGPR base。VSHMEM 的合法地址 form 由 shared-memory 清单单独固定，不得借用 VMEM 的 global 64 位 base 规则。

SATOM/VATOM：

P 位	SATOM	VATOM
[7:0]	sdst	vdst
[15:8]	sbase	vaddr
[23:16]	sdata0	vdata0
[31:24]	sdata1	vdata1
[39:32]	simm8	simm8
[41:40]	order	order
[43:42]	scope	scope
[44]	x1	x1

VATOMX：

P 位	字段
[7:0]	vdst
[15:8]	sbase
[23:16]	vindex
[31:24]	vdata0
[39:32]	vdata1
[41:40]	order
[43:42]	scope
[44]	x

VATOMX 的地址形式固定为：

```
effective_address[lane] =
    SGPR64(sbase:sbase+1) + zero_extend(VGPR32(vindex))
```

索引 scale 固定为 1 字节，payload 中没有 scale 字段；该格式也没有位移字段，canonical syntax 不得追加 + 0：

```
V_ATOM.ADD.U32.GLOBAL.ACQ_REL.DEVICE v0, [s2:s3 + v4], v5
```

x 必须为零。VATOMX 是 vector form，使用和 VATOM 相同的 order/scope modifier，并允许 guard_policy: optional。

原子顺序编码固定为：

```
0 RELAXED
1 ACQUIRE
2 RELEASE
3 ACQ_REL
```

原子 scope 编码固定为：

```
0 CTA
1 DEVICE
2 SYSTEM
3 reserved
```

scope 名称统一使用 DEVICE，不得输出或接受 GPU 作为 canonical 名称。scope=3 产生 ILLEGAL_INSTRUCTION。

order 和 scope 是 payload modifier。opcode 只选择原子 operation form（例如 LOAD、STORE、ADD、XCHG、CAS）及该 form 固定的宽度/地址模式；同一个 (class, format, opcode) form 可以接受多个合法 order/scope 值。禁止为 ADD.RELAXED.DEVICE、ADD.ACQUIRE.DEVICE 等组合另分配 opcode 或另建 form。

合法矩阵为：

operation 类别	合法 order	global 合法 scope	shared 合法 scope
LOAD	RELAXED, ACQUIRE	CTA, DEVICE, SYSTEM	CTA
STORE	RELAXED, RELEASE	CTA, DEVICE, SYSTEM	CTA
RMW（含 XCHG、算术/位运算、CAS）	RELAXED, ACQUIRE, RELEASE, ACQ_REL	CTA, DEVICE, SYSTEM	CTA

不在矩阵中的已定义 modifier 组合产生 ILLEGAL_OPERAND。shared 原子若 scope != CTA 必须拒绝；不能把更大的 scope 静默收窄为 CTA。

canonical 原子助记符的字段顺序固定为：

```
S_ATOM.<op>.<type>.<space>.<order>.<scope>
V_ATOM.<op>.<type>.<space>.<order>.<scope>
```

文本必须同时显式写出 space、order 和 scope，不存在省略或换序：

```
S_ATOM.LOAD.U32.GLOBAL.ACQUIRE.DEVICE s0, [s2:s3 + 0]
V_ATOM.STORE.U32.GLOBAL.RELEASE.SYSTEM [v2:v3 + 0], v4
V_ATOM.ADD.U32.GLOBAL.ACQ_REL.CTA v0, [s2:s3 + v4], v5
```

交换 operation 的唯一 canonical 名称是 XCHG。汇编器和反汇编器不得输出 EXCH、EXCHANGE 或其他拼写。

非 CAS 原子的 data1 必须为零。地址空间、访问宽度、load/store/atomic operation 和数据类型由 opcode 明确指定，禁止从某个寄存器槽是否为零来猜测。

6.5.5 CTRL

P 位	字段
[29:0]	disp30

P 位	字段
[35:30]	cond6
[43:36]	aux8
[44]	x1

disp30 是唯一允许的直接控制目标表示，是相对 next_pc 的有符号指令字位移，见 6.8。cond6 作为数据条件时使用与 guard 相同的 PT/!PT/vp/!vp 编码，但它不是 header guard。aux8 的角色由 opcode 固定。无条件控制指令未使用的 cond6/aux8 必须为零；不带直接目标的控制指令未使用的 disp30 必须为零。

CALL direct 的目的地只能编码在 disp30 中。CALL.IND/JUMP.IND 使用 aux8 编码 SGPR 目标基址，其 disp30/cond6/x1 必须为零；它们不引入另一种直接目标编码。RET 从架构调用栈取得目的地，因此 disp30/cond6/aux8/x1 必须全部为零。

CALL direct、CALL.IND、JUMP.IND 和 RET 都必须声明 guard_policy: required_pt、required_state: scalar_ready。BRA 必须声明 guard_policy: required_pt、required_state: none；BRA.P 必须声明 guard_policy: explicit_condition、required_state: none。

SSY 成功压入重汇聚帧时，必须把当前调用栈深度快照到隐藏字段：

```
frame.owner_call_depth = call_stack.depth
```

owner_call_depth 不在 CTRL payload 中，也没有软件可见编码；它是 SSY 动态效果的一部分。RET 在弹出调用帧前必须拒绝任何 owner_call_depth == call_stack.depth 的未闭合重汇聚帧，报告 RECONVERGENCE_FAULT；较小 owner depth 的调用者帧保持不变。

6.5.6 SYNC

P 位	字段
[7:0]	a
[15:8]	b
[31:16]	imm16
[34:32]	slot3
[36:35]	scope2
[38:37]	order2
[44:39]	x6

a/b 是 opcode 指定类别的寄存器槽。屏障槽 slot3 可表达 0..7。唯一使用它的指令是 BAR.SYNC.CTA，其中 slot3 是显式 barrier_id；非屏障同步指令必须把 slot3 置零。scope/order 只在相应 opcode 明确定义时有效，否则必须为零。

屏障的规范编码只有一条：

family	(class,format,opcode)	canonical 汇编	payload 非零字段	示例机器字
bar-sync	(5,0,3)	BAR.SYNC.CTA id	slot3=id	BAR.SYNC.CTA 3 → 0x001800000000185

a/b/imm16/scope2/order2/x6 都必须为零。屏障不写寄存器，也不读寄存器源，所以 SYNC payload 里没有屏障寄存器槽；(5,0,4) 和 (5,0,5) 在 1.0 Draft 中未分配，译码为 ILLEGAL_INSTRUCTION。

BarrierWaitRecord {warp_id,owner_snapshot,resume_pc}、warp blocked record、槽内 waiter 映射和 CTA 的 live_owner_set 都是执行状态，不占 SYNC payload。resume_pc 固定为该动态屏障指令的 old_PC+8，不能由汇编显式提供。

6.5.7 COLL

P 位	字段
[7:0]	vd
[15:8]	va
[23:16]	vb
[31:24]	smask
[39:32]	imm8
[44:40]	x5

smask 是保存 lane mask、标量源或标量目标的 SGPR 号，不是 8 位 lane mask 本身。COLL 格式只承载 warp_collective 和 scalar form，它们都要求 header guard 为 PT。COLL 没有 scalar-source selector：需要把统一值送进各 lane 的场合用 V1 的混合源 V_MOV，不用跨 lane 格式。

S_READFIRST.B64 的 64 位两端都必须显式写完整、偶数对齐且不越界的寄存器对：

```
S_READFIRST.B64 s6:s7, v8:v9
```

它的 smask 编码 SGPR 目标对基址，va 编码 VGPR 源对基址，vd/vb/imm8/x5 必须为零；它是 execution_domain: scalar、guard_policy: required_pt、required_state: scalar_ready。它必须从同一个最低编号 active lane 原子快照两个 32 位半部，禁止两个半部分别选择 lane。

反方向的 SGPR64 到各 lane VGPR64 搬运由 V1 格式的 V_MOV.B64 vE:v(E+1), sA:s(A+1) (ssrc=1) 完成，见 6.5.3。

V_SHUFFLE.DOWN.B32 有两个保留相同 width 编码的 form：

```
V_SHUFFLE.DOWN.B32 vd, vs, vdelta, width # (CROSSLANE,0,11)
V_SHUFFLE.DOWN.B32 vd, vs, delta, width # (CROSSLANE,0,13)
```

寄存器 form 的 vb 是 VGPR delta 编号；立即数 form 的 vb 直接编码 0..31 的无符号 delta。两者的 imm8 都按面值编码 width 属于 {2,4,8,16,32}，smask/x5 必须为零。form 只能由 opcode 区分，不得根据 vb 的数值或操作数恰好为零猜测。

6.5.8 MMA

P 位	字段
[7:0]	vd_base
[15:8]	va_base
[23:16]	vb_base
[31:24]	vc_base
[39:32]	attr8
[44:40]	x5

MATRIX class 只定义一个完整 form:

```
family: MMA
form: m16n8k16_f16_f16_f32
mnemonic: MMA.M16N8K16.F16.F16.F32
class: MATRIX
format: MMA
opcode: 0
execution_domain: warp_matrix
guard_policy: required_pt
required_state: none
```

四个寄存器号都是该 form 固定映射的片段组基址；形状、A/B/C/D 元素类型、累加顺序、组长度、对齐和别名规则全部由这一个 form 完整规定。attr8 和 x5 必须全零。其他 MMA opcode、形状、类型、饱和模式或 modifier 均保留，不得从 attr8/x5 猜测扩展。MMA 是 warp 集合操作，header guard 必须为 PT。

6.6 寄存器编码与资源边界

SGPR 和 VGPR 使用彼此独立的 8 位编号空间：

```
s0..s255 SGPR
v0..v255 VGPR
vp0..vp15 向量谓词
SCC      每 warp 一份的隐式 1 位标量条件码
```

8 位字段能表达 0..255，不等于每个内核都能使用 256 个寄存器。模块资源描述必须分别给出 sgpr_count 和 vgpr_count；合法引用必须满足：

```
0 <= sN < sgpr_count
0 <= vN < vgpr_count
```

若实现或模块还声明 vp_count，则必须满足 $0 \leq \text{vpN} < \text{vp_count} \leq 16$ ；否则架构可见的默认上限为 16。

寄存器组由其低编号基址编码。组的每个成员都必须低于相应资源计数，并满足该操作数声明的对齐。例如两个 VGPR 的组要求偶数基址时，v7 非法，v254:v255 只有在 vgpr_count=256 时才合法。MMA 片段组使用其 opcode 明确规定的长度和对齐，不能套用普通双寄存器规则。

任何 64 位操作数在汇编文本中都必须显式写成寄存器对：

```
s2:s3
v6:v7
```

该规则适用于 64 位源、目标和地址 base。机器字段仍只编码低编号基址，但汇编器禁止接受单独的 s2 或 v6 作为 64 位操作数，反汇编器也禁止省略第二个寄存器。

字段类别是静态的。要求 SGPR 的槽不能写 vN，要求 VGPR 的槽不能写 sN。编码中不存在通用寄存器号，也不存在通过最高位区分 SGPR/VGPR 的规则。

某 opcode 未使用的寄存器槽编码为零时，零只是 canonical 填充值，不表示读取 s0 或 v0。

SCC 没有寄存器编号，也不占任何 8 位寄存器槽。SCMP 隐式写 SCC。本编码没有 SCC 源字段，也不定义泛化的“SCC 条件执行”；CTRL.cond6 只能编码 PT/!PT/vpN/!vpN，不能编码 SCC。

6.7 立即数

6.7.1 类型

每个立即数操作数必须声明以下一种解释：

- uimmN：范围 $0 \dots 2^N - 1$ ，执行时零扩展。
- simmN：范围 $-2^{N-1} \dots 2^{N-1} - 1$ ，低 N 位使用二补数，执行时符号扩展。

- bitsN: 恰好 N 位原始位型，不赋予有符号数值含义。
- 枚举: 只接受清单列出的名称和值。

汇编器禁止静默截断、取模或饱和超范围字面量。移位执行可能只读取源的低若干位，不代表立即数字面量可以超出其声明范围。

6.7.2 小于容器宽度的立即数

当语义立即数宽度 N 小于格式容器宽度 M 时，只使用容器低 N 位，容器高 M-N 位必须为零。该规则对有符号立即数同样适用。

例如 simm12(-1) 放入 imm24 时:

```
合法: imm24 = 0x000fff
非法: imm24 = 0xfffffff
```

执行时先取低 12 位，再从 12 位符号扩展。这样同一个立即数只有一种机器编码。

SIMM/VIMM 的容器宽度是 24 位; SYS 的容器宽度是 16 位; SMEMX 和向量 memory 的位移容器是 16 位; SATOM/VATOM 的位移容器是 8 位; VATOMX 没有立即数容器。某 opcode 可以使用更窄的立即数，但不能使用比所在容器更宽的立即数。

一条指令最多有一个普通立即数容器。若某个运算需要不能直接编码的常量，汇编器必须报错或由显式宏展开为多条指令; 不得悄悄改变 opcode、交换非交换操作数或借用未定义 payload 位。

浮点立即数只有在 opcode 明确声明 bitsN 格式时才可直接编码。不能装入 24 位容器的 binary32 常量必须通过常量物化序列或内存加载获得。

6.8 PC-relative 控制目标

所有直接控制目标都使用 CTRL.disp30，并相对于当前指令的 next_pc 计算:

```
D      = sign_extend_30(disp30)
target_pc = next_pc + (D << 3)
next_pc  = pc + 8
```

位移单位是 8 字节指令字，不是字节。汇编或链接标签 target 时:

```
delta = target - (pc + 8)
require delta % 8 == 0
D = delta / 8
require -2^29 <= D <= 2^29 - 1
disp30 = D & (2^30 - 1)
```

目标必须由 disp30 相对 next_pc 得出，结果必须 8 字节对齐，并指向当前内核文本中的一条完整指令。任何其他目标字段或基准都不是合法编码。

例如:

```
pc=0x40, target=0x80:
next_pc=0x48, D=(0x80-0x48)/8=7, disp30=0x00000007

pc=0x80, target=0x40:
next_pc=0x88, D=(0x40-0x88)/8=-9, disp30=0x3ffffff7
```

链接器只能对 disp30 应用 PC-relative 重定位。重定位溢出必须报错，不得截断，也不得自动插入跳板，除非调用者明确启用了会改变代码布局的链接器变换。

6.9 must-zero 与唯一编码

每个 form 必须把自己的 45 个 payload 位逐位归入且只归入以下一种:

1. 操数字段;
2. 有定义的枚举或修饰字段;
3. must-zero。

任何没有当前 opcode 语义的位都是 must-zero, 包括:

- 基本格式中的未使用寄存器槽;
- opcode 未定义的 x 位;
- 小立即数容器中未使用的高位;
- SCMP.zero8;
- VMEM uniform-base form 的 vaddr、lane-base form 的 sbase, 以及全部 LOCAL form 的 sbase;
- VATOMX.x;
- 非 CAS 原子的 data1;
- 不使用 scope/order 的格式中的对应字段;
- VCMP.zero4;
- 不带目标的控制指令中的 disp30。

任一 must-zero 位为 1 都产生 ILLEGAL_INSTRUCTION。实现禁止忽略垃圾位后继续执行。

唯一编码要求是:

- 一个合法机器字只能匹配一个 form;
- 一个 canonical 汇编指令只能生成一个机器字;
- 不得根据寄存器号是否为零、立即数值大小或保留位模式选择另一种解释;
- 语法别名必须在编码前归一化;
- canonical 反汇编后重新汇编必须逐位得到原机器字。

```
assemble(disassemble(W, canonical=true)) == W
```

“唯一编码”指抽象指令及其显式操作数、类型、guard 和修饰符具有唯一表示, 不表示所有可观察效果相同的指令必须共用机器字。例如不同 opcode 的 @!PT 指令都不提交 lane 效果, 但仍是不同的抽象指令和不同编码。

6.10 规范汇编文本

canonical 文本遵守以下规则:

- 助记符、类型、地址空间、scope 和 order 使用大写;
- SGPR 写作 sN, VGPR 写作 vN, 向量谓词写作 vpN;
- @PT 省略, 其他合法 guard 显式写出;
- 数值立即数默认十进制, 原始位型默认十六进制;
- 负数必须带 -, 不得依赖超宽十六进制字面量猜测符号;
- 直接控制目标优先显示符号; 无符号时显示相对 next_pc 的有符号位移, 不显示其他目标表示;
- 64 位操作数必须显示完整寄存器对, 其他寄存器组必须显示完整范围或使用其专用片段语法;
- 原子助记符必须严格按 <op>.<type>.<space>.<order>.<scope> 排列, 交换操作只写 XCHG;

- 屏障只显示为 BAR.SYNC.CTA id, id 必须显式出现;
- 混合源操作数按实际寄存器文件显示为 sN/sE:s(E+1) 或 vN/vE:v(E+1), 反汇编不得把 SGPR 源印成 VGPR 号;
- 不允许根据助记符拼写、寄存器前缀或字面量大小模糊选择多个候选形式。

汇编器可以接受大小写、显式 @PT、零偏移省略等无损语法别名, 但必须先归一化到唯一形式。BARRIER、BARRIER_ARRIVE、BARRIER_WAIT、V_BCAST、MEMBAR 都不是任何指令的兼容名称或 canonical 别名, 必须按未知助记符拒绝。内存排序指令的唯一助记符是 FENCE.CTA/DEVICE/SYSTEM。需要多条机器指令的伪操作属于宏, 不是编码别名; listing 和调试信息必须显示实际展开。

若源文本不能唯一确定 (class, format, opcode)、数据类型、寄存器类别或立即数解释, 汇编器必须报错并列出冲突候选, 不得按声明顺序或“最接近”原则选择。

非法机器字的反汇编必须输出:

```
.word 0x.....
```

并附带首个静态拒绝原因, 不得发明可执行助记符。

6.11 译码顺序与错误分类

实现可以并行完成检查, 但架构结果必须等价于:

```
W = load_u64_le(text, pc)

require aligned_and_complete(pc)           else ILLEGAL_INSTRUCTION
require class_is_assigned(W[3:0])          else ILLEGAL_INSTRUCTION
require format_is_assigned(W[3:0], W[6:4])  else ILLEGAL_INSTRUCTION
require opcode_is_assigned(W[3:0], W[6:4], W[12:7]) else ILLEGAL_INSTRUCTION
form = lookup_exact_form(class, format, opcode)

require guard_code_is_defined(W[18:13])     else ILLEGAL_INSTRUCTION
require guard_allowed_for_form(form, W[18:13]) else ILLEGAL_INSTRUCTION
require all_must_zero_bits_are_zero(form, W[63:19]) else ILLEGAL_INSTRUCTION
require all_encoded_enums_are_defined(form, W[63:19]) else ILLEGAL_INSTRUCTION

require register_banks_match(form)          else ILLEGAL_OPERAND
require register_ids_within_resource_counts(form) else ILLEGAL_OPERAND
require register_groups_aligned_and_in_range(form) else ILLEGAL_OPERAND
require static_operand_combinations_are_legal(form) else ILLEGAL_OPERAND
require direct_target_is_legal_if_checked_now(form) else ILLEGAL_OPERAND

if form.required_state == scalar_ready:
    require scalar_ready(warp_state)         else DIVERGENCE_FAULT
```

错误分类固定如下:

ILLEGAL_INSTRUCTION

表示机器字结构本身不是已定义的 canonical 编码, 包括:

- 未分配 class、format 或 opcode;
- guard 编码保留, 或 form 的 guard_policy 不接受该 header guard;
- must-zero 位非零;
- 保留枚举值;
- 原子 scope=3;
- opcode 专用字段出现未分配组合;

- 取指未对齐、不完整或越界。

ILLEGAL_OPERAND

表示已经选出唯一合法形式，但静态操作数不满足该形式约束，包括：

- SGPR/VGPR 类别错误；
- 寄存器号超出模块资源计数；
- 寄存器组越界、基址未对齐或禁止的部分重叠；
- 已定义字段值之间形成禁止组合，包括已知 order/scope 值不满足 LOAD/STORE/RMW 或地址空间合法矩阵；
- PC-relative 目标不对齐、越出当前内核文本或不满足控制流约束。

汇编/链接错误

源级超范围立即数、未知助记符、歧义形式、错误寄存器前缀和重定位溢出必须在生成机器码前报告。工具不得故意生成非法机器字，再把问题推迟到运行时。

动态地址越界、实际访存未对齐、除零、屏障协议不一致和集合参与者不一致不属于静态编码错误；它们由相应执行语义产生运行时故障。

DIVERGENCE_FAULT 是已成功静态译码后对当前 warp 动态状态的检查结果。它适用于所有 required_state: scalar_ready 的 form，包括全部 scalar form，以及 CALL direct、CALL.IND、JUMP.IND 和 RET；它不适用于 BRA/BRA.P，也不属于非法机器编码。

静态错误检查对整个 warp 只做一次，先于 guard 和 scalar-ready 求值。发生静态错误时不得提交 SGPR、VGPR、VP、SCC、内存、PC、同步或重汇聚状态。

6.12 机器可读清单要求

清单把物理布局和操作数绑定分开保存，各有唯一归属：

- 根 format_registry 拥有物理布局。每个编码格式在这里给出一次自己的 class、payload 位范围和完整字段表（字段名、lsb、width、kind、描述）。
- 每个 form 拥有操作数绑定。它声明自己属于哪个 encoding_format、自己的 opcode，以及每个操作数绑定到哪个字段。

因此单个 form 至少必须给出：

```
family: v-add          # 语义分组，语义化 slug
form: u32              # family 内唯一
mnemonic: V_ADD.U32
syntax: V_ADD.U32 v0, v1, v2
encoding_format: V2
opcode: 0x00
execution_domain: vector
required_state: none
guard_policy: optional
operands: [...]        # 每项含 name/type/access/field
semantics: ...
constraints: [...]
faults: [...]
example: {assembly: ..., machine_word: ...}
```

form 里不得重复 class、format 或 fields：它们由 encoding_format 加 format_registry 唯一决定，工具必须现场推导。任何绑定不到操作数的 payload 字段自动成为 must-zero 洞，不需要另写 must_zero 列表。

只有两类信息无法从 registry 推导，因此允许逐 form 覆盖：

- field_values：把某个字段固定成一个常量。例如 FENCE 三个 form 用它把 scope2/order2 钉死成各自的组合。
- field_notes：给某个字段一个 form 专属的描述，用于同一个物理槽在不同 form 中承载不同含义的情况，例如 V_SHUFFLE.DOWN.B32 的立即数 delta form。

family ID 是语义化 slug ($^{\wedge}[a-z0-9]+(-[a-z0-9]+)^*\$$ ，如 v-add、bar-sync)，不是不透明编号。

生成器必须拒绝：

- 两个 form 重复声明同一 (encoding_format, opcode) 或同一译码三元组；
- form 直接书写 class、format 或 fields；
- form 引用 format_registry 中不存在的 encoding_format，或绑定到该格式没有的字段名；
- format_registry 中位段重叠、越出 64 位或遗漏 payload 位；
- 同一机器字匹配多个形式；
- 未定义的 x 位；
- opcode 的字段类别与操作数类别不一致；
- vsrc32/vsrc64 操作数出现在非 V1/V2/V3/VCMP 格式或非 vector 执行域的 form 上；
- 含 vsrc* 操作数的 form 把 selector 字段当成 must-zero 洞，或不含 vsrc* 的 form 让 selector 变成可变字段；
- execution_domain 不属于本章规定的七值集合；
- guard_policy、required_state 或 form 级 guard 矩阵与本章规则不一致；
- 原子 order/scope 被错误拆成额外 opcode/form，或合法矩阵不一致；
- 原子 canonical 名称未按 <op>.<type>.<space>.<order>.<scope> 排列，或使用 EXCH/EXCHANGE 而不是 XCHG；
- MATRIX class 出现第二个 MMA form，或唯一 MMA form 的 attr8/x5 非零；
- 立即数宽度大于其格式容器；
- 示例不能 canonical 汇编，或 round-trip 改变机器字。

执行域、machine class、编码格式、family 和 form 必须作为不同概念保存。family 只做语义分组，form 才是唯一译码叶子；每个 form 的 (class, format, opcode) 三元组必须全局唯一。生成器不得从 V2 自动推导 v-add family，不得从 MEMORY class 推导 execution domain，也不得从 family 名反推其 payload 布局。完整指令表、汇编器、反汇编器、验证器和 RTL/CModel 解码表必须由同一份清单生成。

7. 指令分类与语义

本章定义 VTX-1 ISA 1.0 Draft 的指令分类和执行语义。这里讲“指令做什么”；每个 family/form 的位段、selector、操作数宽度、合法 modifier 和机器字示例由 isa/vtx1/isa.yaml 生成。实现不得根据助记符猜编码，也不得在本章之外补出隐藏语义。

7.1 一眼看懂分类树

VTX-1 每条指令固定为 64 位。机器码里的 class 只有下面 8 种合法值：

```
SYS    系统、特殊寄存器、陷阱和杂项
SALU   标量算术与逻辑
VALU   向量算术与逻辑
MEMORY 标量/向量访存与原子
CONTROL 分支、调用、返回和重汇聚
SYNC   barrier 与内存同步
CROSSLANE warp 内跨 lane 操作
MATRIX 矩阵乘加
```

每个 YAML form 必须恰好属于这 8 个 class 之一。保留 class、一个 form 同时落入两个 class，或靠操作数猜 class，都属于非法编码。

execution_domain 是另一件事。它回答“这条 form 动态执行几次、按什么状态规则执行”，合法值恰好是：

```
system / scalar / vector / warp_control /
warp_collective / cta_sync / warp_matrix
```

8 个 machine class 和 7 个 execution domain 不能混成一张表。常见关系是：

```
SYS    -> system, 也可承载 scalar/vector 的 GETREG/SETREG form
SALU   -> scalar
VALU   -> vector
MEMORY -> scalar 或 vector
CONTROL -> warp_control
SYNC   -> cta_sync
CROSSLANE -> warp_collective; 跨域搬运 form 可按 YAML 标为 scalar/vector
MATRIX -> warp_matrix
```

最终关系以每个 YAML form 自己的 class 和 execution_domain 为准，不能只看助记符前缀反推。

为了让人读起来省事，本章还使用 S/V/W/SYNC/X/MMA 这些用户可读简称。简称只是告诉人“主要在干什么”，不是 YAML 字段，更不是第二套编码 class：

```
S  标量方向，常见于 SALU、scalar MEMORY、scalar SYS
V  向量方向，常见于 VALU、vector MEMORY、vector SYS
W  控制流，对应 CONTROL
SYNC 同步，对应 SYNC
X  跨 lane/跨域，对应 CROSSLANE
MMA 矩阵，对应 MATRIX
```

SYS class 中纯系统 form 仍直接称为 SYS。X_BROADCAST、S_READFIRST 的名字说明数据方向或集合行为，但机器 class 仍由 YAML 决定。所以“这一节从 V 方向解释它”和“它编码在 CROSSLANE class”并不冲突；机器 class 永远以 8 类之一为准。

命名规则如下：

- 标量数据指令使用 S_ 前缀，例如 S_ADD、S_LD、S_ATOM.ADD.U32.GLOBAL.RELAXED.DEVICE。
- 向量数据指令使用 V_ 前缀，例如 V_ADD、V_LD、V_ATOM.ADD.U32.GLOBAL.RELAXED.DEVICE。
- 跨寄存器域和跨 lane 的规范名称固定为 X_BROADCAST、S_READFIRST、S_GETREG、V_GETREG。
- 控制流清单固定为 BRA、BRA.P、SSY、JOIN、EXIT、CALL、CALL.IND、JUMP.IND、RET。

- SYNC、X 和 MMA 的完整规范名称由 YAML 给出；文本别名不能产生第二个机器编码。

S_BROADCAST 和 V_BCAST 都不是规范名称，也不是任何指令的别名。把一个标量值送到各 lane 不需要专门的指令：任何 V1/V2/V3/VCMP 向量 form 都可以用 scalar-source selector 直接读一个 SGPR 源，需要独立副本时写 V_MOV.B32 vd, sN。

7.2 双寄存器执行模型

7.2.1 S 域和 V 域

每个 warp 有两套彼此分开的寄存器：

- S 寄存器：每个 warp 一份。一个 sN 是一个 32 位标量槽；64 位操作数规范写作 sE:s(E+1)。
- V 寄存器：每个 lane 一份。vN[lane] 是该 lane 的 32 位槽；64 位操作数规范写作 vE:v(E+1)。
- SCC：每个 warp 一份的 1 位标量条件，只表示 false 或 true。
- vpN：每个 warp 一份的 32 位 lane 掩码；第 i 位对应 lane i。vp0..vp15 可由向量比较和跨 lane 指令读写。

寄存器本身没有整数或浮点标签。类型由当前 form 决定；B32、U32、S32 和 F32 都可以占同一个 32 位槽。

64 位寄存器对的基址 E 必须为偶数，两个编号必须相邻并完整写出，例如 s0:s1、v6:v7。前一个槽保存低 32 位，后一个槽保存高 32 位；编码字段只保存偶数基址。s1:s2、v4:v6、只写 s0/v0 来冒充 64 位值，或任一成员越过 descriptor 的寄存器计数，都是 ILLEGAL_OPERAND。

S 指令只执行一次。V 指令对参与集合中的每个 lane 独立执行一次。一个 V 指令的不同 lane 可以读到不同值、形成不同地址、得到不同结果。

7.2.2 active、participating 和 scalar-ready

对 V 数据指令，入口先冻结：

```
E = active_mask
G = 对 E 中每个 lane 求头部 guard
P = E & G
```

E 是入口 active 集合，P 是实际参与集合。guard 为假的 lane 不读动态源、不形成地址、不产生 lane 局部故障，也不写任何目标。scalar form 的 guard_policy 是 required_pt，先检查 scalar-ready，然后执行一次；SCC 不是通用 scalar guard。只有 S_SELECT 或 YAML 明确列出的 CONTROL form 才把 SCC 当数据条件读取。CONTROL、SYNC、CROSSLANE 和 MATRIX 按各自章节决定参与集合。

warp 在某条指令入口满足以下全部条件时称为 scalar-ready：

```
live_mask != 0
active_mask == live_mask
reconv_stack 中不存在 phase 为 FIRST 或 SECOND 的帧
```

大白话说，就是 warp 里还有活 lane、所有活 lane 此刻都在一起，而且没有另一条分支路径等着执行。只有 ARMED 帧不妨碍 scalar-ready，因为它只是预约 JOIN，还没有真正发生分歧。尾 warp 中物理不存在的 lane 不属于 live_mask。

所有 S 指令都要求 scalar-ready。这包括 SALU、scalar MEMORY、scalar SYS 和 S_READFIRST，没有“普通 S 可以例外”的规则。检查发生在读取任何动态源之前；任一条件不满足时：

```
fault(
  code = DIVERGENCE_FAULT,
  lane_mask = active_mask,
  aux = 0)
```


故障指令不读取动态源，不写 SGPR、VGPR、vpN、SCC 或内存，也不推进 PC。scalar form 不从 SCC 派生参与集合；SCC 只由 S_SELECT 或 YAML 明确列出的 CONTROL form 当作数据条件读取。

7.2.3 通用事务

除阻塞型 SYNC 外，每条动态指令在架构上按一个事务完成：

1. fetch 从 PC 读取一个 64 位小端机器字
2. decode 按 YAML 校验 class、family、form、字段和 must-zero
3. classcheck 按执行域检查 scalar-ready 或集合/控制状态
4. freeze 冻结 E、P、全部源和所需隐藏状态
5. evaluate 计算结果、地址、目标 PC 和动态约束
6. validate 收集整条指令的故障
7. commit 无故障才一次提交全部效果

静态译码不受 active mask 或 guard 抑制。具体先后只引用 docs/02-programming-model.md 的权威表；按该表，ILLEGAL_INSTRUCTION 和静态 ILLEGAL_OPERAND 位于 DIVERGENCE_FAULT 之前。对静态合法的 S form，非 scalar-ready 的结果固定是 DIVERGENCE_FAULT。任一参与 lane 失败，整条指令都不提交；不能出现“低 lane 已写、高 lane 才报错”。

所有源逻辑上先读后写，所以目标可以与源完全重合。多槽寄存器组只能完全重合或完全不相交，除非对应 form 明确允许其他关系。

普通非控制指令成功后 $PC = PC + 8$ 。合法且 P 为空的 V 指令不产生数据效果，只推进 PC。S 指令若 $live_mask = 0$ ，应在更早的 scalar-ready 检查中报告 DIVERGENCE_FAULT，不能走空参与集合捷径。

7.3 S-only、V-only 和 S/V 双版本

7.3.1 S/V 双版本

下面这些 category 原则上同时提供 S_ 和 V_ 版本；实际 form、宽度和 modifier 以 YAML 为准：

```
move  MOV、位型复制、立即数物化
int   ADD、SUB、MUL、MAD、DIV、REM、MIN、MAX、ABS、NEG
bit   AND、OR、XOR、NOT、移位、rotate、计数、位域、pack
cmp   整数和浮点比较
select S_SELECT 按 SCC、V_SELECT 按 vpN 的对应位二选一
cvt   整数、浮点和宽度转换
fp    ADD、SUB、MUL、FMA、DIV、SQRT、MIN、MAX、ABS、NEG、近似函数
mem   普通 load/store
atom  atomic load/store/RMW/CAS
sys   读取对该域合法的特殊寄存器
```

同一个 operation 的 S/V 版本使用相同数学规则，但执行次数、寄存器域和内存事件数不同。S_ADD 只产生一个标量结果；V_ADD 给 P 中每个 lane 产生一个结果。

所有列在 S 侧的版本，不论编码 class 是 SALU、MEMORY 还是 SYS，都先执行 7.2.2 的 scalar-ready 检查。

7.3.2 S-only

下列能力只属于 S 域：

- S_READFIRST：把一个 V 源的 first-lane 值送入 S 域，只能在 scalar-ready 时执行。
- 读取只具有 warp 单值语义的 S_GETREG form。
- 为 JUMP.IND、CALL.IND 提供间接目标 SGPR，并为标量访存提供地址基址。控制指令本身仍属于 W/CONTROL，不因此变成 S。

YAML 可以声明更多 S-only form，但必须写出不能存在 V 版本的理由和对应测试。

7.3.3 V-only

下列能力只属于 V 域：

- 用 scalar-source selector 把一个 SGPR 当统一源读入逐 lane 运算。
- 读取 lane id、lane-local 状态等逐 lane 特殊寄存器的 V_GETREG form。
- 以 V 地址项形成每 lane 不同地址的访存。
- 产生 vpN lane 掩码，并可作为 BRA.P 的逐 lane 条件。

X 和 MMA 也会读写 V 寄存器，但它们是独立顶层类别，不归入 V category。

7.3.4 不允许偷偷跨域

除 vsrc* 混合源、S_READFIRST 和 form 明写的混合地址外：

- S 指令不能读 V 寄存器；
- V 指令不能把 V 结果直接写进 S 寄存器；
- 汇编器不能靠同号寄存器名自动插入搬运或 read-first；
- 实现不能因为“所有 lane 的值碰巧相同”把 V 源当成 S 源。

混合源也不是无限制的跨域：一条 V 指令最多一个 SGPR 源，且必须由 selector 显式编码。写出两个 sN 源的汇编是错误，汇编器必须报错而不是自行插入一条搬运指令。

7.4 move 与跨域操作

7.4.1 同域 move

S_MOV 和 V_MOV 按 form 宽度逐位复制，不做数值转换，不改变 NaN payload，不做符号扩展。窄值扩展只能由明确的 load 或 cvt form 完成。

V_MOV 只搬位。寄存器上没有隐藏影子状态，因此 move 不需要额外说明标签如何创建、复制或清除。

7.4.2 混合源 V_MOV：SGPR 到各 lane

```
V_MOV.B32 vd, vs      # ssrc=0
V_MOV.B32 vd, ss      # ssrc=1
V_MOV.B64 v0:v1, v2:v3 # ssrc=0
V_MOV.B64 v0:v1, s0:s1 # ssrc=1
```

V_MOV 是 V1 格式的 form，源操作数类型是 vsrc32 或 vsrc64。ssrc=1 时那 8 位寄存器号在 SGPR 文件中解释，这就是把标量值送进各 lane 的规范做法。它的执行域是 vector、guard_policy: optional，按普通 vector 规则令 P = E & guard，不要求 scalar-ready。

在入口冻结一次源；对每个 lane 属于 P：

```
vd[lane] = frozen(src)
```

guard 为假的 lane 不读取源、不写目标；非参与 lane 的 vd 保持不变。

.B64 使用完整偶数连续寄存器对，一次复制 64 位，两个 32 位半部来自同一次冻结的快照。

很多情况下连这条 move 都不需要：既然 V_ADD.U32、V_CMP.LT.U32、V_FFMA.F32 这类 form 本身就能读一个 SGPR 源，直接写 V_ADD.U32 v1, v0, s6 比先搬后算更短。只有一条指令需要两个 uniform 值，或者要把 uniform 值多次复用而寄存器压力允许时，才值得先物化成 VGPR。

7.4.3 X_BROADCAST

```
X_BROADCAST vd, vs, lane
```

X_BROADCAST 才是 lane 到 lane 的广播。它先冻结选中 lane 的 vs，再把这一份值写到所有规定的接收 lane。lane 是立即数还是统一 SGPR、源 lane 必须属于哪个集合、接收集合是什么，都由对应 YAML form 的结构化操作数和约束给出；实现不能自行改成“当前第一个 lane”。

这是 execution_domain: warp_collective 的集合操作。所有规定参与者必须在同一动态实例上取得一致的 lane 选择，源 lane 必须可用，否则产生 COLLECTIVE_FAULT，并且所有目标保持不变。它与混合源读 SGPR 是两件不同的事：X_BROADCAST 是 lane 到 lane，混合源是 SGPR 文件到各 lane。

7.4.4 S_READFIRST

```
S_READFIRST.B32 sd, vs  
S_READFIRST.B64 s0:s1, v0:v1
```

S_READFIRST 仍是 execution_domain: scalar、guard_policy: required_pt，不是 optional-guard 例外。先要求 scalar-ready，再令：

```
first = 编号最小的 live lane  
sd = frozen(vs[first])
```

.B64 从同一个 first lane 一次读取完整 VGPR 对；两个 32 位半部必须来自同一个 lane 的同一次快照，不能分别选择 lane。

若 live_mask 为空，warp 已完成，不会取到该指令。S_READFIRST 不做 vote，也不检查其他 lane 是否同值；需要验证同值时必须先用 X 类指令。

7.4.5 S_GETREG 与 V_GETREG

S_GETREG 先检查 scalar-ready，再读取规范标为 uniform 的特殊寄存器，例如 warp id、CTA 尺寸、时钟快照或 scalar-ready 状态。V_GETREG 可以读取 uniform 或 per-lane 项；读取 uniform 项时，各参与 lane 得到同值。

V_GETREG 虽编码在 SYS class，执行域仍是 vector，也是明确允许 guard_policy: optional 的例外。它按 P = E & guard 执行，不要求 scalar-ready；guard 为假的 lane 不读取特殊寄存器项，目标保持原值。机器 class 不能覆盖这条 form 自己的执行域和 guard policy。

两者都在 freeze 阶段取快照。未知寄存器号、宽度不匹配、用 S_GETREG 读取 per-lane 项，均为 ILLEGAL_OPERAND。

7.5 整数与位运算

7.5.1 基本整数

对宽度 N：

```
ADD = (a + b) mod 2^N  
SUB = (a - b) mod 2^N  
NEG = (-a) mod 2^N  
MULLO = low_N(a * b)  
MAD = low_N(a * b + c)
```

MIN.S/MAX.S 按 N 位二补码比较，MIN.U/MAX.U 按无符号比较；相等时返回位型相同的任一输入都得到同一结果。ABS.S 对负数做 N 位 NEG，对非负数原样返回，所以最小负数的结果仍是同一位型，不额外饱和。NEG、ABS、MIN、MAX 和普通 MAD 都必须按 YAML 中实际存在的 S/V form 分别实现，不能拿 wide 或浮点 form 代替。

DIV 和 REM 的有符号版本向零截断；除数为零产生 INTEGER_FAULT。有符号最小值除以 -1 的结果或故障行为必须由该 form 的 YAML 语义参数明确给出。

只有比较、除法、右算术移位、MIN/MAX、扩展和转换等真正依赖符号解释的 form 使用 .S 或 .U。单纯搬位的 form 使用 .B。

7.5.2 wide multiply 和 wide MAD

MUL.WIDE.U32/S32 产生完整 64 位乘积：

```
U32: result64 = zero_extend(a,32) * zero_extend(b,32)
S32: result64 = two_complement_64(s32(a) * s32(b))
```

MAD.WIDE.U32/S32 使用 64 位加数：

```
result64 = (wide_product(a,b) + addend64) mod 2^64
```

S/V 版本都遵守同一规则。64 位目标组整体提交，不能先写低半再写高半。

7.5.3 位型 form 合并规则

以下操作不区分 signed/unsigned，规范只保留一个 B32 或 B64 位型 form：

```
MOV, AND, OR, XOR, NOT
SHL, SHR (逻辑), ROL, ROR
POPC, CLZ, CTZ, BREV
BFE, BFI, PACK, UNPACK
```

工具链不得为同一机器行为另造 .S32 和 .U32 两个 form。SAR.S32/S64 因复制符号位而单独存在。

7.5.4 shift、rotate、CLZ/CTZ

寄存器给出的移位或旋转量按 amount mod N 使用。立即数必须在其字段范围内，汇编器不能靠截断接受越界值。

```
ROL_N(x,k) = ((x << k) | (x >> (N-k))) mod 2^N
ROR_N(x,k) = ((x >> k) | (x << (N-k))) mod 2^N
```

当 k=0 时直接返回 x，不得在宿主语言中计算移位 N。

CLZ_N(0)=N，CTZ_N(0)=N。非零输入分别返回最高端和最低端连续零的个数。

7.5.5 pack、unpack 和位域

PACK 按操作数顺序把窄位型放进目标的低位到高位，未被覆盖的目标位由 form 明确为清零或来自 base；不能保留未说明的旧值。UNPACK 是反操作，并按 form 选择零扩展、符号扩展或原样位型。

对 N 位 BFE/BFI：

```
o = min(unsigned(offset), N)
n = min(unsigned(width), N - o)

BFE: n==0 ? 0 : (src >> o) & low_mask(n)
BFI: n==0 ? base : (base & ~(low_mask(n)<<o))
      | ((insert & low_mask(n))<<o)
```

width=0 和 offset>=N 都不是故障。low_mask(N) 必须直接构造全 1，不能依赖宿主语言的 $1 < N$ 。

BREV.BN 精确反转 N 个有效位：结果位 i 等于输入位 N-1-i。它不做逐字节反转。BFE/BFI 的 offset、width、base 和 insert 取自 YAML 指定的操作数槽；实现不得因为某个槽为零就切换到另一种 form。

7.6 compare、select 与转换

compare 的 S/V 能力必须对称。对 EQ/NE/LT/LE/GT/GE, YAML 中每个 S 类型 form 都必须有同类型、同关系的 V form, 反过来也一样; 差别只有源/目标寄存器域、执行次数和 scalar-ready。S 比较写 1 位 SCC; V 比较写一个 32 位 vpN, 其中只更新参与 lane 对应的位, 其他位保持原值。

整数比较只在 form 明确要求时解释为 .S32 或 .U32; 逐位相等/不等不另造有符号和无符号副本。F32 比较按助记符声明的关系执行: EQ/LT/LE/GT/GE 遇到任一 NaN 都为假, NE 遇到任一 NaN 为真。比较不改写源浮点位型, 也不产生浮点异常标志。

S_SELECT 根据 SCC 选择完整标量值。V_SELECT 对每个参与 lane, 根据指定 vpN 的对应 lane 位选择完整向量值。多槽值必须整组取自同一个输入, 不能低半取 a、高半取 b。

CVT 也要求 S/V 对称: 相同的 dst-type.src-type 必须同时提供 S_CVT 和 V_CVT, 或两边都不存在。CVT 是数值转换, MOV 是位复制。所有舍入、饱和、NaN、F16 高位清零和 subnormal 规则以第 5 章为准; S/V 版本只改变执行域, 不改变数值结果。任何使用 64 位源或目标的 CVT 都必须使用前述完整偶数连续寄存器对语法。

7.7 浮点

S_FP 和 V_FP 共享同一个数值环境:

- binary16/binary32;
- 精确 form 使用固定 RNE;
- 支持渐进下溢, 不使用隐式 FTZ/DAZ;
- FMA 是一次融合运算;
- 没有可见浮点异常标志;
- NaN、无穷和有符号零按第 5 章逐位处理;
- .APPROX form 必须满足 YAML 给出的特殊值和 ULP 上限。

V F16 form 对每个参与 lane 独立读取输入槽低 16 位, 按 binary16 规则计算, 并把结果写到目标槽低 16 位; 目标高 16 位清零。F16 FMA 只舍入一次, 非 FMA 运算在该 form 的结果点舍入。NaN、subnormal、Inf 和 -0/+0 仍按第 5 章处理, 不能先悄悄提升到 F32、做多次运算后再截断来改变规定结果。

实现可以用不同内部数据通路, 但 S/V 同输入位型必须得到同结果位型。

7.8 普通访存与 mixed addressing

7.8.1 地址模板

地址先用无界数学整数计算, 再做范围和对齐检查。规范只使用下面三类名字:

```
uniform-base:
    EA = unsigned(SGPR_base) + sign_extend(imm)

lane-address:
    EA[lane] = unsigned(VGPR_address[lane]) + sign_extend(imm)

SV-mix:
    EA[lane] = unsigned(SGPR64_base)
               + zero_extend(VGPR32_index[lane]) * scale
               + sign_extend(imm16)
```

uniform-base 是每 warp 一个地址, scalar memory 成功后恰好一个事件。SMEMX 是它的 indexed 变体:

```
EA = unsigned(SGPR64_base)
     + zero_extend(SGPR32_index)
     + sign_extend(imm16)
```

SMEMX 的 index 单位固定为字节，也就是 scale=1；未定义的 mods 位必须为零。它不把 SMEMX 变成 vector，也不允许读取 VGPR。

lane-address 和 SV-mix 使用 VMEM 系列格式。字段角色固定为：

```
vdata  向量 load 目标或 store 源
vaddr  lane-address 时为 VGPR64 地址对；
        SV-mix 时为 VGPR32 无符号 index
sbase  SV-mix 时为 SGPR64 uniform base；lane-address 时必须为零
simm16 有符号字节 immediate
x5     form 未定义的位必须为零
```

SV-mix 的 VGPR32 index 固定零扩展；最高位为 1 也仍是大正数，绝不能按有符号数解释。scale 只能取具体 form 明写的值；未声明缩放时固定为 1。global、param、const 可以使用 lane-address 或 SV-mix；shared 使用 32 位 lane-address，也可使用 form 明确给出的 SV-mix。local 只能使用 lane-address，只能由 vector memory 访问，每个 lane 的数值偏移落在自己的 local allocation；local 禁止 scalar、uniform-base 和 SV-mix。

SMEMX/VMEM/VATOMX 只能使用各自 form 结构化列出的地址项。地址空间由 opcode 决定，寄存器里的地址值不带空间身份。错误寄存器类别或非法操作数组合为 ILLEGAL_OPERAND，保留 scale/modifier 或非零 must-zero 位为 ILLEGAL_INSTRUCTION，数学地址越界为 MEMORY_BOUNDS，自然对齐失败为 MISALIGNED_ACCESS；多故障仍按第 2 章优先级。vector mixed address 中任一参与 lane 失败，整条指令零事件回滚。

S_MEM 先检查 scalar-ready。一次成功的 S load、S store 或其他非原子 scalar memory form 必须恰好产生一个内存事件，不能是零个，也不能按 lane 复制。V_MEM 对 P 中每个 lane 产生一个事件；即使多个 lane 得到同一 EA，也仍是不同的 lane 事件。

7.8.2 load/store

所有空间均按小端字节序。自然对齐由访问总宽度决定。地址回绕、跨 allocation、越界和错空间按第 4 章处理。

窄 load 必须明确扩展：

```
LD.U8/U16 -> 零扩展
LD.S8/S16 -> 符号扩展
LD.B8/B16 -> 零填充到目标槽，其位型不带数值符号
LD.F16     -> 低 16 位装载，高位清零
```

store 只写 form 指定的低位。ST.S8 和 ST.U8 若机器行为完全相同，只保留一个位型 store form；S/U 不得制造重复编码。

向量宽访存按 YAML 指定的元素顺序映射到连续寄存器组。任一元素或任一 lane 校验失败，整条指令都没有内存事件。

7.9 原子 load、store 和 RMW

原子 family 的规范前缀只有 S_ATOM 和 V_ATOM。S_ATOMIC_LOAD、V_ATOMIC_STORE、S_ATOMIC_CAS 之类不是规范名称。S_ATOM 先检查 scalar-ready，成功后每条动态指令恰好产生一个原子事件。V_ATOM 为 P 中每个 lane 产生一个事件；同一条 V_ATOM 的多个 lane 命中同址时，它们在该位置的 modification order 中各占一个位置，先后次序不由 lane 编号规定。

原子类别包括：

```
LOAD          只读，不改 modification order
STORE         只写，在 modification order 中追加新值
ADD/MIN/MAX/AND/OR/XOR/XCHG 读改写，返回旧值并追加新值
CAS           比较并交换，始终返回旧值
```

交换操作的规范名称只有 XCHG。

canonical 语法固定为：

```
S_ATOM.<op>.<type>.<space>.<order>.<scope> ...  
V_ATOM.<op>.<type>.<space>.<order>.<scope> ...
```

space 必须显式写成 GLOBAL 或 SHARED，位置固定在 type 后；它由 operation form/opcode 选择，不是靠地址寄存器猜出来。order 和 scope 是编码 modifier，不是为每个组合复制一套新的操作语义或 family。canonical 文本必须把三者都写出，不能省略默认值、交换顺序或改用小写。合法矩阵是：

op	order	global scope	shared scope
LOAD	RELAXED, ACQUIRE	CTA/DEVICE/SYSTEM	CTA
STORE	RELAXED, RELEASE	CTA/DEVICE/SYSTEM	CTA
ADD/MIN/MAX/ AND/OR/XOR/XCHG	RELAXED/ACQUIRE/RELEASE/ACQ_REL	CTA/DEVICE/SYSTEM	CTA
CAS	RELAXED/ACQUIRE/RELEASE/ACQ_REL	CTA/DEVICE/SYSTEM	CTA

例如 S_ATOM.LOAD.U32.GLOBAL.ACQUIRE.DEVICE、V_ATOM.CAS.U64.GLOBAL.ACQ_REL.SYSTEM 和 S_ATOM.XCHG.U32.SHARED.RELAXED.CTA。param、const、local 不支持原子。

原始机器字中的 scope=3 是保留编码，固定产生 ILLEGAL_INSTRUCTION。它不能先被解释成未知名称，也不能降级为某个合法 scope。

已知 modifier 组成了表外 order/space/scope 组合才产生 ILLEGAL_OPERAND。两类错误都在读取地址或数据源之前发现，原子事件数为零。

LOAD 没有写数据源，STORE 没有旧值目标，CAS 必须同时带 expected、replacement 和 old-value 目标。CAS 原子地读取 old；old == expected 时写 replacement，否则把 old 原样写回该位置；两种情况都把 old 返回目标，并按第 4 章进入 modification order。atomic load/store 不能伪装成“加零”或“交换后丢弃结果”。

VATOMX 是 V_ATOM 的 SV-mix 编码格式，字段是 vdst/sbase/vindex/vdata0/vdata1/order/scope/x，没有 immediate 字段。它的地址为：

```
EA[lane] = unsigned(SGPR64_base)  
+ zero_extend(VGPR32_index[lane])
```

VATOMX 的 scale 固定为 1，不存在额外缩放变体，x 必须为零。额外 scale、非法操作数、越界、未对齐和任一 lane 失败分别按 7.8.1 与第 2 章处理，并整条零事件回滚。

7.10 W 控制流与隐藏重汇聚

7.10.1 一个 PC，隐藏状态

一个 warp 只有一个架构 PC。实现保存路径集合、待执行路径、调用返回点和重汇聚信息，但这些都是隐藏状态：程序不能读栈深、修改 pending mask，或依赖实现内部先存哪一项。

隐藏不等于随意。相同入口状态、SCC 和 vpN 必须得到相同的 PC、active mask 和可见提交顺序。

7.10.2 BRA、BRA.P、SSY、JOIN、EXIT

- BRA target: 所有 active lane 一起跳到直接目标。
- BRA.P condition,target: 按 vpN 或 !vpN 把入口 active 集合分成 taken 和 fall-through。若两边都非空，硬件在隐藏重汇聚状态中保存另一条路径，并按 YAML 规定的固定路径次序执行。
- SSY join_target: 声明后续分歧的结构化汇合点，并建立一个隐藏重汇聚区域。新帧必须记录 owner_call_depth = call_stack.depth，也就是 SSY 执行这一时刻的调用深度；该字段留在重汇聚帧内。
- JOIN: 到达声明的汇合点。若另一条路径尚未执行，切换到它；全部仍 live 的路径到达后恢复为一个集合并继续。

- EXIT: 永久删除参与 lane; 隐藏状态中的所有相关集合同时删除这些 lane, lane 不得在 JOIN 后复活。

SSY、BRA.P、JOIN、EXIT 的机器指令保留, 但软件只描述结构, 不能直接管理重汇聚栈。嵌套区域必须正确包含, 不能交叉。非法目标、错误 JOIN、区域越界或隐藏状态不满足产生 RECONVERGENCE_FAULT, 并按整条指令回滚。

```
SSY 成功提交:
push ReconvFrame(
    reconv_pc = join_target,
    owner_call_depth = call_stack.depth,
    ...其余结构化重汇聚字段...)
```

后续 CALL 不得改写已有帧的 owner_call_depth。JOIN 弹出匹配帧; RET 只拒绝仍存在且 owner_call_depth == 当前 call_stack.depth 的 callee 帧, 较小深度的 caller 帧可以保留。

7.10.3 CALL、RET 和间接跳转

```
CALL target    只保存 PC+8, 再跳到直接目标
CALL.IND sTarget 只保存 PC+8, 再跳到 SGPR64 目标
JUMP.IND sTarget 不保存返回点, 跳到 SGPR 目标
RET            恢复最近一次 CALL 保存的 return_pc
```

直接和间接目标都必须 8 字节对齐、完整落在当前文本内并指向合法指令。间接目标只能来自 S 域, 不能逐 lane 不同。

CALL、CALL.IND、JUMP.IND 和 RET 的机器 class 都是 CONTROL, 用户可读分类都是 W, 不得改标成 SALU。它们的 required_state 都是 scalar_ready, 并且必须在读取间接目标 SGPR 或调用栈之前检查; 不满足时产生 DIVERGENCE_FAULT。

调用栈是每 warp 一份、程序不可见的 LIFO 状态, 不占普通 S/V 寄存器。每个调用帧只保存一个 return_pc=PC+8, 不保存 active mask、重汇聚深度或其他控制上下文。最大深度只取 descriptor 的 call_stack_depth:

1. CALL/CALL.IND 先完成静态检查和 scalar-ready 检查, 再验证目标、PC+8 和 call_stack.depth < descriptor.call_stack_depth;
2. 全部成功后, 原子地压入一个调用项并把 PC 改成目标;
3. SSY 新建的每个重汇聚帧记录当时的 owner_call_depth=call_stack.depth; 这个字段属于重汇聚帧, 不属于调用帧;
4. RET 先检查 scalar-ready、非空调用栈, 并要求重汇聚栈中不存在 owner_call_depth == call_stack.depth 的帧, 再原子地弹出栈顶并恢复 return_pc;
5. JUMP.IND 只改 PC, 绝不压栈或弹栈。

空栈 RET、CALL 达到 descriptor 深度, 以及 RET 时仍有当前 callee 所有的重汇聚帧都产生 RECONVERGENCE_FAULT。任何失败都不留下半压栈、半弹栈或已改变的 PC。

7.10.4 精确控制提交

控制目标和隐藏状态必须一起提交。目标非法时不能先压入调用项; RET 失败时不能先弹出; JOIN 失败时不能先切换 active mask。

7.11 SYNC

SYNC 类包括内存栅栏和唯一一条 CTA 屏障。内存栅栏只建立第 4 章定义的顺序, 不是 lane 会合。屏障的规范指令名和操作数次序固定为:

```
BAR.SYNC.CTA id
```


架构没有 split 屏障、屏障 token 和 generation 计数。每 CTA 有 8 个槽 0..7，每槽只保存 arrived_set 和 waiter 映射；另有一个 8 槽共用的 CTA 级 live_owner_set。owner 唯一身份为 CTA 内 linear_tid=warp_id*32+lane_id。启动时全部槽为 idle（两者都空），live_owner_set 是 CTA 启动时全部真实线程的 linear_tid；尾部不存在 lane 从不计入，EXIT 把退出线程移除。

BAR.SYNC.CTA 的 required_state 是 scalar_ready，guard_policy 是 required_pt，执行域是 cta_sync：

- 先检查 scalar-ready；不满足时报告 DIVERGENCE_FAULT，不登记任何 arrival，也不改 PC。因为通过检查后 active_mask == live_mask，一个 warp 只能整体到达，不存在部分到达、重复到达或 wrong-owner 的情形。
- 把入口 active lane 转成 owner_snapshot: set<linear_tid>，做一次原子 arrival commit 和 shared CTA release，然后用 BarrierWaitRecord {warp_id,owner_snapshot,resume_pc=old_PC+8} 阻塞整个 warp。
- arrived_set 等于当前 live_owner_set 时，全部记录一起 acquire，并只写各自 PC=resume_pc、清 blocked record、置 ready，槽随即清空回 idle。
- EXIT 缩小 live_owner_set 后必须重新检查每个非 idle 槽，因为完成条件可能刚刚被满足。EXIT 自身不是 shared release。

每 warp 同时至多一条 blocked record。阻塞期间 PC 留在屏障指令上，active/live 掩码、重汇聚栈和调用栈保持不变，挂起路径不能切入；恢复也不改这些状态。

因为槽在完成时被清空，同一个槽的两次屏障之间不留任何状态，也就没有需要区分的“代”，实现不需要防止计数器回绕。

如果 CTA 内一部分 warp 到达某个槽，另一部分既不到达也不退出，arrived_set 永远追不上 live_owner_set，程序按第 3 章第 12 节报告 DEADLOCK。屏障的 release/acquire 只排序 shared；global、local、param、const 和 host 不因此有序，global 通信仍需原子和需要的 FENCE。完整状态转移伪代码见第 3 章第 10 节。

CTA 只有在全部 warp 完成且 8 个槽都 idle 时完成。idle 固定表示 arrived_set 和 waiters 都为空。

7.12 X 跨 lane

X 指令读取多个 lane 的冻结 V 源，并按 form 写 VGPR、SGPR、vpN 或 SCC。每个 form 必须明确：

C 候选 lane
M 显式成员 mask（若有）
P 实际贡献者
R 结果接收者

vote/ballot 对 P 做布尔归约；shuffle 从 P 中选择源 lane；match 比较 P 中的键。member mask、mode 和 width 等必须在 C 中一致，逐 lane control 可以不一致。找不到合法源时是回退、写零还是故障，必须由具体 form 写明，不能由实现选择。

X 指令在读取任何目标前冻结全部源，因此原地 shuffle 合法。参与协议不一致产生 COLLECTIVE_FAULT，所有 lane 目标保持不变。

V_SHUFFLE.DOWN.B32 同时提供 VGPR delta 和 0..31 立即数 delta form。两者都在 width 属于 {2,4,8,16,32} 的子组内选择 source_lane = lane_id + delta；源 lane 不 active 时结果为零。立即数 form 只是消除固定归约树中的 delta 装载指令，不改变参与、会合、冻结源或故障语义。

7.13 MMA

M16N8K16 形状只定义下面一个 form：

MMA.M16N8K16.F16.F16.F32 dbase, abase, bbase, cbase

该 form 的 YAML matrix_contract 必须结构化表达下面同一份寄存器数量、对齐、lane 映射、元素映射、别名、数值和参与合同；YAML 与正文任一坐标不一致都必须阻断生成和发布。

它的 execution_domain 是 warp_matrix，固定 PT，要求 32 个 lane 全部参与，即 active_mask == live_mask == 0xffffffff。A、B 是 F16，C、D 是 F32。每 lane 的 A/B/C/D 片段分别占 4/2/4/4 个 VGPR，基址分别按 4/2/4/4 对齐。只允许 D=C 完整别名；D 与 A/B 重叠、D/C 部分重叠都非法。

下面是完整映射。令 lane l=0..31；r 是相对片段基址的寄存器偏移；h=0 取 VGPR 低 16 位，h=1 取高 16 位：

```
# A: 每 lane 4 VGPR, 共 8 个 F16
m = l // 2
k = 8*(l % 2) + 2*r + h      # r=0..3, h=0..1
A[m,k] = F16(VGPR[abase+r][16*h+:16])

# B: 每 lane 2 VGPR, 共 4 个 F16
k = l // 2
n = 4*(l % 2) + 2*r + h      # r=0..1, h=0..1
B[k,n] = F16(VGPR[bbase+r][16*h+:16])

# C 和 D: 每 lane 各 4 VGPR, 每槽一个 F32
m = l // 2
n = 4*(l % 2) + r            # r=0..3
C[m,n] = F32(VGPR[cbase+r])
D[m,n] <-> VGPR[dbase+r]
```

所有 lane 的 A/B/C 必须先冻结。对每个 m=0..15, n=0..7:

```
acc = C[m,n]
for k = 0,1,...,15:          # 次序固定, 不能树形重排
    acc = FFMA.F32(
        V_CVT.F32.F16(A[m,k]),
        V_CVT.F32.F16(B[k,n]),
        acc)
D[m,n] = acc
```

每一步 FFMA 都在步末按 RNE 舍入到 F32，下一步读取这个已舍入值；不能跨 k 保留额外精度或改用 FP64 累加。F16 转 F32、NaN、Inf、subnormal 和有符号零完全按第 5 章。全部 D 片段只在所有结构和数值检查成功后一次提交。

缺 lane、不同动态 PC、基址不一致、组越界、错误对齐、非法别名或会合失败产生规定故障，所有 D 保持不变。MMA 不产生内存事件，也不隐含内存栅栏。其他 M16N8K16 拼写、类型或 modifier 都不是合法 form；其他形状若存在，只能来自 YAML 中另行给出的 form 和结构化合同。

7.14 system、故障和边界

SYS class 用于 NOP、YIELD、TRAP、GETREG、实现查询和 YAML 明确列出的系统操作。SYS 不是“全部都按 S 执行”的同义词；每个 SYS form 仍要声明用户可读分类和执行域。S_GETREG 属于 S，因此要求 scalar-ready；V_GETREG 属于 V，不套用该检查。

- NOP 只推进 PC。
- YIELD 是调度提示，不建立内存顺序。
- TRAP reason 产生规范软件故障；reason 只作为附加数据。
- 未声明为可写的系统状态不能通过普通指令修改。

同一动态指令的故障优先级、fault record 和全回滚规则只以 docs/02-programming-model.md 的故障优先级表为权威。本章不另排顺序；所有动态检查都必须先收集、再按第 2 章选一个故障，不能因实现内部的 lane 顺序改变结果。

7.15 逐 form 生成要求

构建系统必须从 isa/vtx1/isa.yaml 为每个启用 form 生成语义条目，至少包含：

```
form_id
canonical mnemonic and syntax
machine class: SYS / SALU / VALU / MEMORY / CONTROL / SYNC / CROSSLANE / MATRIX
reader-facing shorthand, if useful: S / V / W / SYNC / X / MMA
semantic category
fixed 64-bit encoding fields
operand domains: SGPR / VGPR / SCC / vpN / memory / hidden state
guard_policy
required_state
address template
numeric and bit-width parameters
source snapshot and destination commit set
faults and ordering
normal, boundary and illegal examples
canonical machine word
```

family 数和 form 数不得写死在本章、测试代码或报告模板中。发布时实际数量只能由当前 YAML 去重生成，并与生成附录和 all-form 测试清单逐项核对。

8. 合规性

本章定义 VTX-1 ISA 1.0 Draft 的强制测试。适用对象包括汇编器、反汇编器、译码器、模拟器、RTL、设备、装载器和参考模型。只跑几个示例不能判定通过；每个启用 form 都必须进入 all-form 覆盖。

8.1 结果和测试记录

测试结果只有三种：

- PASS：适用于该对象的强制测试全部通过，并且启用 form 覆盖率为 100%。
- FAIL：任一强制测试失败、缺少 oracle、缺少 coverage tag，或发现未解释差异。
- NOT_APPLICABLE：测试对象不承担该角色。基础 ISA 中已启用的 form 不能标成这一项。

每次测试运行至少记录：

```
{
  "suite_version": "1.0-draft",
  "isa_version": "1.0-draft",
  "yaml_digest": "...",
  "enabled_features": [],
  "dut_name": "...",
  "dut_revision": "...",
  "seed": "0x0000000000000000",
  "families_total": 0,
  "forms_total": 0,
  "forms_passed": 0,
  "failures": []
}
```

families_total 和 forms_total 必须在运行时从当前 YAML 去重计算。测试源码、文档和报告模板都不得写死数量。

所有随机测试必须保存 64 位 seed 和最小化后的失败输入。允许多种调度或 modification-order 结果时，oracle 必须给出允许集合或可执行判定器，不能只拿某一次参考运行作标准答案。

8.2 测试向量格式

规范仓库中的向量必须是机器可读的 YAML 或 JSON。每个用例至少包含：

```
id
requirement
form_id
required_features
dut_roles
initial_state
program_or_word
schedule_constraints
expected_state or allowed_outcomes
forbidden_outcomes
expected_fault
state_must_remain
coverage_tags
```

初始状态必须明确：

- PC、live mask、active mask 和隐藏控制状态；
- SGPR、VGPR、1 位 SCC、32 位 vp0..vp15 和特殊寄存器快照；
- scalar-ready 是否成立；

- 内存空间、allocation、初值和原子 modification order;
- 每槽 arrived_set/waiter 映射、CTA 的 live_owner_set、warp blocked record、MMA/collective 会合状态;
- 文本范围、模块字段和启用 feature。

测试不能依赖未初始化值，除非目标就是检查 UNSPEC 分类；这类测试不能要求某个具体位型。

8.3 固定 64 位编码

8.3.1 每个 form 的正向向量

生成器必须从 YAML 为每个启用 form 至少生成：

1. 字段最小合法值；
2. 字段最大合法值；
3. 每个合法 selector 和 modifier；
4. 每种 SGPR、VGPR、vpN 的最低、最高和组边界，以及对声明 SCC 源的 form 测试 SCC 两个值；
5. 每个立即数槽的最小值、最大值、零和负边界（若为有符号）；
6. 每种合法 guard；
7. 允许的完整源/目标别名；
8. 直接目标、间接目标和调用目标的边界；
9. 汇编、机器字、反汇编、再次汇编的闭环。

每条机器指令必须恰好 8 字节。测试要直接检查：

```
instruction_bits == 64
next_sequential_pc == pc + 8
encoded_bytes == little_endian(machine_word)
```

机器字期望值必须由独立编码 oracle 按 YAML 位段构造，不能调用被测汇编器生成。

8.3.2 分类树

对 YAML 中每个 form 检查：

```
class 属于 {SYS,SALU,VALU,MEMORY,CONTROL,SYNC,CROSSLANE,MATRIX}
execution_domain 属于 {system,scalar,vector,warp_control,warp_collective,cta_sync,warp_matrix}
一个机器字只选出一个 form
一个 form 只落入一个机器 class
```

测试必须把 8 个 machine class 和 7 个 execution_domain 分开取数、分开报告。至少验证 SYS 可以承载 system/scalar/vector form、MEMORY 可以承载 scalar/vector form，以及 class 不能从 execution_domain 机械推导。每个 form 的实际组合必须与 YAML 两个字段逐字一致。

文档可以用 S/V/W/SYNC/X/MMA 帮读者找指令，但测试不得把这些简称当成编码字段。显示简称时必须遵守：

```
SALU 通常显示在 S
VALU 通常显示在 V
scalar/vector MEMORY 分别显示在 S/V
CONTROL 显示在 W
SYNC/MATRIX 分别显示在 SYNC/MMA
CROSSLANE 中 collective form 通常显示在 X;
S_READFIRST 可按数据方向显示在 S，但不改变 machine class
SYS 中的 scalar/vector form 可分别放进 S/V 说明，纯系统 form 保持 SYS
```

还必须逐项检查：

- S form 的普通数据操作数不引用 V 域；
- V form 的目标不直接写 S 域；
- CONTROL form 在用户可读说明中只能按 W 控制流解释；
- CALL、CALL.IND、JUMP.IND、RET 必须保持 class=CONTROL，不能放入 SALU；
- SYNC、CROSSLANE、MATRIX 不能伪装成 SALU/VALU；
- X_BROADCAST、S_READFIRST 的 machine class、execution_domain 分别与 YAML 一致；
- X_BROADCAST 固定为 lane→lane，S_READFIRST 固定为 V→S，混合源固定为 SGPR 文件→各 lane；
- V_GETREG 即使编码在 SYS class，也必须保持 execution_domain=vector、guard_policy=optional；
- S_READFIRST 必须保持 execution_domain=scalar、guard_policy=required_pt、required_state=scalar_ready；
- X_BROADCAST、S_READFIRST、S_GETREG、V_GETREG 的名称、方向和操作数域固定；
- 文本 S_BROADCAST 和 V_BCAST 都被汇编器按未知助记符拒绝；
- BRA、BRA.P、SSY、JOIN、EXIT、CALL、CALL.IND、JUMP.IND、RET 的 canonical 文本保持这些名称。

8.3.3 译码负例

每个 form 都要自动做单比特和组合变异，至少覆盖：

- 未分配 class、family、form 或 selector；
- 每个 must-zero 位单独置 1，以及全部置 1；
- 保留 modifier、非法立即数位置和多个立即数源；
- S/V 寄存器域错配；
- 寄存器组未对齐、越过上限和禁止的部分重叠；
- 把 SCC、vpN、SGPR、VGPR 放进错误的源/目标槽；
- 64 位操作数省略第二个寄存器、使用奇数基址、不连续编号或越过 descriptor count；
- guard 不满足 form policy；
- 固定 64 位文本被截短、PC 未按 8 字节对齐；
- 直接目标字段溢出、未对齐、越出文本；
- scope/order 的保留值或非法组合；
- mixed-addressing 使用未列出的 S/V 项组合；
- atomic load 带写数据、atomic store 带读目标、非 CAS 带 replacement；
- MMA 片段字段垃圾、对齐错误和组越界。

静态错误要在以下环境重复：

```
active mask 全开
active mask 非空但 guard 全假
active mask 为空
```

三种环境必须得到同一静态故障，证明译码不会被 guard 或 active mask 关掉。

8.3.4 canonical 文本

对每个合法机器字：

```
T = disassemble(W, canonical=true)
W2 = assemble(T)
assert W2 == W
assert disassemble(W2, canonical=true) == T
```

非法机器字不能反汇编成可重新组装的合法指令。超范围字面量、错误寄存器域、重复 signed/unsigned 位型拼写和歧义后缀必须在汇编阶段报错，不能静默截断或换 form。

8.4 通用执行与精确提交

每个语义用例在目标指令前后逐位比较：

```
PC
live/active mask
隐藏重汇聚与调用状态
全部 SGPR、VGPR、SCC 和 `vpN`
目标内存字节
原子 modification order
每槽 arrived_set/waiters 和 CTA live_owner_set
collective/MMA 状态
事件日志
fault record
```

发生故障时，除新增规范 fault record 和 kernel 失败状态外，入口快照必须保持。特别检查：

- PC 不推进；
- S 目标、所有 lane 的 V 目标都不写；
- 成功 lane 的 store 也不提交；
- 原子不占 modification-order 位置；
- CALL 不留下返回项，RET 不提前弹项；
- JOIN 不提前切 active mask；
- SYNC 不留下半次到达；
- X/MMA 不写部分结果。

对每个可能产生 lane 局部故障的 V form，至少测试最低 lane 失败、最高 lane 失败、多个 lane 不同原因失败、失败 lane 被 guard 抑制，以及其他 lane 本来可以成功的情况。

8.5 双寄存器和 scalar-ready

8.5.1 S 指令只执行一次

每个 S 算术、位、比较、转换、FP、访存和原子 form 至少验证：

- S 源只有一份，不随 lane 号改变；
- scalar-ready 时每个 warp 只产生一次结果；
- S load/store 只产生一个内存事件；
- S atomic 只在 modification order 中占一个位置；
- 目标与源完全别名时按入口快照计算；

- 非 scalar-ready 时固定产生 DIVERGENCE_FAULT，不读取动态源，也不产生任何数据或内存效果；
- guard_policy 必须是 required_pt；SCC 只在 S_SELECT 或 YAML 明确列出的 CONTROL 条件中读取。

这里的“每个 S form”必须覆盖 SALU、scalar MEMORY 和 scalar SYS。不能只测 S ALU 后就声称所有 S 指令通过。

状态优先级还要用毒值验证：静态编码和静态操作数都合法，但动态源会除零、地址会越界或动态特殊寄存器值会非法时，只要入口非 scalar-ready，就只能得到 DIVERGENCE_FAULT，证明实现根本没有读取动态源。所有多故障组合的唯一权威顺序来自 docs/02-programming-model.md；本章测试从该顺序生成期望值，不另写第二套优先级。

所有 sgpr64/vgpr64 操作数还必须检查 canonical 对语法：

- 只接受 sE:s(E+1) 或 vE:v(E+1)，且 E 为偶数；
- 前槽是低 32 位，后槽是高 32 位；
- 编码、反汇编和再汇编保留完整范围；
- 奇数基址、跳号、反序、缺半和末端越界都报 ILLEGAL_OPERAND；
- 多对操作数别名时按完整 64 位入口快照执行，不能出现半写回。

8.5.2 V 指令逐 lane 执行

每个 V form 使用可识别的 lane 数据，检查：

- P 中每个 lane 独立读源和写目标；
- guard-false 与 inactive lane 的目标保持；
- 不同 lane 可以得到不同结果和地址；
- 任一 lane 动态故障使整个 warp 指令回滚；
- 同名 V 寄存器在不同 lane 之间不别名。

8.5.3 scalar-ready 状态矩阵

必须构造以下状态：

```
SR1 live_mask 非空, active_mask == live_mask, 无重汇聚帧
SR2 只有一个 live lane, active_mask == live_mask
SR3 live_mask 非空, active_mask == live_mask, 只有 ARMED 帧
NS1 live_mask == 0
NS2 active_mask 是 live_mask 真子集
NS3 active_mask == live_mask, 但栈中有 FIRST 帧
NS4 active_mask == live_mask, 但栈中有 SECOND 帧
```

SR1..SR3 必须判为 scalar-ready；NS1..NS4 必须判为非 scalar-ready。这个矩阵必须套到每个 S form；所有 NS 用例都期望：

```
code = DIVERGENCE_FAULT
lane_mask = active_mask
aux = 0
```

S_READFIRST 专项检查：

- SR1 中读取编号最小 live lane；
- 尾 warp 中跳过物理不存在 lane；
- SR2 正常读取唯一 lane；

- SR3 在只有 ARMED 帧时正常读取；
- 其他 V lane 值不同不构成故障；
- NS1..NS4 产生 DIVERGENCE_FAULT；
- 状态故障时不读取 V 源、不写 SGPR、不改 PC、不改隐藏状态。

8.6 跨域与 GETREG

8.6.1 混合源 selector

selector 覆盖必须由 YAML 自动生成，不允许手写代表列表：

```
mixed_source_forms = 所有含 vsrc32 或 vsrc64 操作数的 form
for form in mixed_source_forms:
    assert form.encoding_format in {V1, V2, V3, VCMP}
    assert form.execution_domain == vector
    for code in legal_selector_codes(form.encoding_format):
        assert encode_decode_round_trip(form, code)
        assert semantic_oracle_passes(form, code)
for form in 所有其他 form:
    assert selector 字段（若存在）是 must-zero 洞
```

合法 selector 码取自格式定义：V1 是 {0,1}，V2 和 VCMP 是 {0,1,2}，V3 是 {0,1,2,3}。每个 form 的每个合法码都必须有独立正例，缺任一个即 FAIL。

对每个混合源 form 和每个非零 selector 码检查：

- 被选中的源位置从 SGPR 文件读取，其余源位置仍从 VGPR 文件读取；
- 让 sN 和 vN 取同一个编号但不同内容，证明实现真的换了寄存器文件，而不是照旧读 VGPR；
- P 中每个 lane 得到同一个冻结标量值；把该 SGPR 的值设成能与逐 lane VGPR 值区分的图样；
- 非参与 lane 的目标保持旧值；
- 实现使用入口快照：并行更新 SGPR 不影响本条指令的结果；
- 不要求 scalar-ready；在 NS2..NS4 中仍按入口 E 和 guard 形成 P，不能误报 DIVERGENCE_FAULT；
- 目标始终是 VGPR 或 vpN，不因 selector 变成 SGPR；
- P = E & guard；PT、vpN、!vpN 分别覆盖全体、真子集、空集。

selector 负例至少覆盖：

- V2/VCMP 的 ssrc_sel == 3: ILLEGAL_INSTRUCTION，不读任何源；
- 不含 vsrc* 操作数的 form 把 selector 位置 1: ILLEGAL_INSTRUCTION；
- 汇编文本在一条指令里写两个 sN 源：汇编阶段报错，且不得自动插入搬运指令；
- 汇编文本把 SGPR 写在 form 不允许的源位置（例如 V2 只允许 va/vb，不存在第三个位置）：汇编阶段报错；
- 反汇编把 SGPR 源印成 vN：FAIL。

vsr64 专项：两个寄存器文件都必须使用完整偶数连续寄存器对。

- V_MOV.B64 vd_pair, s_pair (ssrc=1) 逐 lane 比较完整 64 位，两个半部来自同一次冻结快照；
- V_MOV.B64 vd_pair, v_pair (ssrc=0) 仍是逐 lane VGPR 复制；
- guard-false lane 的旧 VGPR64 保持；

- 奇数基址、缺半、越界和部分寄存器对写回都必须拒绝或整条回滚；
- `V_MOV.B64` → `S_READFIRST.B64` 往返必须逐位保持 64 位值。

8.6.2 S_READFIRST

启用 form 清单必须至少包含 .B32 和 .B64；缺少任一项即 FAIL。除 scalar-ready 矩阵外，还要检查 form 固定为 `guard_policy: required_pt`，`first-lane` 选择不受物理调度、lane 执行先后或值大小影响。`first` 永远是编号最小的 live lane。

`S_READFIRST.B64` 必须让不同 lane 持有不同的 VGPR64 值，结果的完整 64 位只能来自同一个 first lane，不能一半来自一个 lane、另一半来自另一个 lane。B64 目标必须整体提交。

8.6.3 X_BROADCAST

对 YAML 中每个 `X_BROADCAST` form 检查：

- 源来自指定 lane 的 VGPR，接收者得到冻结的同一源值；
- 立即数 lane 和 SGPR lane selector（若对应 form 存在）都按各自字段解释；
- 不同 lane 值能证明实现没有误读 SGPR；
- 不存在源 lane、选择不一致或参与协议错误时产生 `COLLECTIVE_FAULT`；
- 故障时所有接收者保持旧目标；
- 汇编器不能把 `X_BROADCAST` 与混合源 `V_MOV` 互当别名。

8.6.4 S_GETREG / V_GETREG

对特殊寄存器表中的每一项自动生成：

- 规范宽度；
- 入口快照稳定性；
- `uniform` 项由 `S_GETREG` 和 `V_GETREG` 读取时数值一致；
- `per-lane` 项由 `V_GETREG` 得到逐 lane 值；
- `S_GETREG` 读取 `per-lane` 项时报 `ILLEGAL_OPERAND`；
- 未知编号、错误目标宽度和非法寄存器组。

每个合法 `S_GETREG` form 都必须跑 scalar-ready 矩阵；`NS1..NS4` 只能得到 `DIVERGENCE_FAULT`。`V_GETREG` 不套用这项检查。

每个 `V_GETREG` form 必须固定验证 `execution_domain: vector`、`guard_policy: optional`，即使 machine class 是 SYS。PT、vpN、!vpN 都要覆盖；只允许 P 中 lane 读取快照并写回，`guard-false` lane 保持原目标。非 scalar-ready 状态下仍按 vector 规则执行，不能误报 `DIVERGENCE_FAULT`。

不得用被测 GETREG 实现生成 oracle 的特殊寄存器期望值。

8.7 整数、位运算和 pack

每个 S/V 双版本使用相同输入位型，并比较除寄存器域外完全相同的数学结果。基础向量包括：

```
0
1
全 1
```

```
最高位单独为 1
最低位单独为 1
0xaaaaaaaa / 0x55555555
signed 最小值、最大值
unsigned 最大值
```

wide multiply/MAD 必须覆盖:

- U32 最大值乘最大值;
- S32 最小值、-1、0、1、最大值交叉;
- 乘积低 32 位为零但高 32 位非零;
- 64 位 addend 产生跨低半进位;
- 最终 64 位回绕;
- 目标组整体别名和部分重叠负例;
- S/V 版本逐位一致。

普通 MAD、MIN、MAX、ABS、NEG 逐 form 覆盖:

- MAD 的乘法溢出、加法进位和最终 N 位回绕;
- .S 与 .U 的 MIN/MAX 在 0x80000000 对 1 时得到不同次序;
- MIN/MAX 两输入相等;
- NEG 的 0、1、全 1 和最小负数;
- ABS 的正数、负数、0 和最小负数;
- S/V 对应 form 使用同一输入位型并得到同一数学位型。

CTZ/rotate 必须覆盖:

```
CTZ(0) == width
CTZ(1) == 0
CTZ(high_bit) == width-1
rotate amount = 0, 1, width-1, width, width+1
register amount 的模 width 行为
立即数越界在汇编阶段被拒绝
```

PACK/UNPACK 必须做位置基向量: 每次只把一个源子字段置 1, 确认它只落到规定目标位。还要检查源顺序、高位清零或 base 保留、符号/零扩展, 以及原地别名。

BREV 必须用单 bit 从最低位移动到最高位、从最高位移动到最低位, 以及非对称字节图样, 防止实现误做 byte-swap。BFE/BFI 必须交叉覆盖 offset 为 0、N-1、N、N+1, width 为 0、1、剩余宽度和超出剩余宽度; 同时检查 BFI 未覆盖位来自 base、覆盖位来自 insert, 并覆盖完整别名。

位型 form 检查器必须拒绝无意义的 S/U 重复项:

```
MOV/logic/logical-shift/rotate/count/bitfield/pack
同一宽度、同一操作数布局 and 同一语义只能有一个 B form
```

8.8 compare、select、转换和 FP

compare 清单必须先做 S/V 对称性检查: EQ/NE/LT/LE/GT/GE 的每个 B32、S32、U32、F32 S form 都要有同关系、同类型的 V form, 反过来也一样。整数比较对 signed/unsigned 分别使用能改变次序的样本, 例如 0x80000000 与 1。S 比较写 1 位 SCC; V 比较写 32 位 vpN, 只更新参与 lane 对应位。

select 必须覆盖 SCC=true/false、vpN 对应位为 1/0、vector guard=false、多槽整组选取和源/目标别名。禁止出现低半来自一个源、高半来自另一个源。

每个 F32 compare form 必须覆盖普通有序值、相等值、-0/+0、+Inf/-Inf、左 NaN、右 NaN和双 NaN。EQ/LT/LE/GT/GE 遇到任一 NaN 必须为假，NE 必须为真。S form 只写 SCC，V form 只更新参与 lane 对应的 vpN 位。

CVT 清单同样要求 S/V 对称：每个 dst-type.src-type 要么两边都有，要么两边都没有。每对 form 共享同一逐位 oracle和边界向量；另外分别检查 scalar-ready、参与 lane 写回和寄存器域。64 位源或目标必须再跑完整偶数连续寄存器对语法矩阵。

转换和 FP 结果按第 5 章逐位比较。每个适用 form 至少覆盖：

```
+0, -0, +1, -1
最大有限数、最小 normal
最大 subnormal、最小 subnormal
+Inf, -Inf
多个 qNaN/sNaN payload 和符号
halfway 的下方、正中、上方
```

强制专项：

- FMA 的融合结果与先乘后加不同的样本；
- subnormal 输入、输出和逐级下溢；
- FMIN/FMAX 的单 NaN、双 NaN和 -0/+0；
- FABS/FNEG 的 payload 保留；
- F16 到 F32 的全部 F16 位型；
- 每个 V F16 算术 form 的目标高 16 位清零、每 lane 独立结果和 guard-false 目标保持；
- V F16 的全部 65536 个单操作数位型；二/三操作数 form 至少覆盖分类笛卡尔积、halfway 和能区分融合/非融合的定向样本；
- F32 到整数的 NaN、Inf、上下限邻点和饱和；
- .APPROX form 的特殊值逐位结果、ULP 上限和确定性；
- 同一输入的 S_FP/V_FP 结果逐位相同。

oracle 必须明确设置舍入模式并关闭 fast-math。宿主语言默认浮点结果不能直接作为标准答案。

8.9 普通内存和 mixed addressing

8.9.1 地址模板

对每个 mem form，从 YAML 的地址模板自动生成：

```
uniform-base
lane-address
SV-mix
```

只测试该 form 声明的模板，未声明组合必须是译码或操作数负例。

每个模板至少覆盖：

- base、index、scale 和正/负 immediate；
- 数学结果为 0、allocation 最后一个合法起点；
- base+offset 下溢；
- 乘法或加法超过地址范围；

- 自然对齐合法;
- 范围内但不对齐;
- 对齐但越界;
- 同时不对齐和越界时按 docs/02-programming-model.md 权威表选择 fault;
- 跨两个相邻 allocation;
- guard 抑制唯一越界 lane。

uniform-base 必须证明 scalar form 只形成一个地址和一个事件。SMEMX 另外改变 SGPR64 base、SGPR32 byte index 和 simm16, 证明公式是 $\text{base} + \text{zero_extend}(\text{index}) + \text{signed}(\text{imm16})$, 且仍不读取 VGPR。SMEMX scale 固定为 1, 未定义 mods 位非零必须拒绝。

VMEM 字段必须按模板交叉验证:

- lane-address: vaddr 是每 lane 地址, sbase 必须为零, simm16 是有符号字节位移;
- SV-mix: sbase 是 SGPR64 base, vaddr 是每 lane VGPR32 无符号 index, 先乘 form 声明的 scale, 再加 simm16;
- 两类都使用 vdata 作为 load 目标/store 源, 未定义 x5 位必须为零;
- 让各 lane 的 vaddr 取 0x7fffffff/0x80000000/0xffffffff, 防止把 SV-mix index 错做符号扩展;
- 把 SGPR/VGPR 域互换、lane-address 非零 sbase、SV-mix 缺少任一项都必须被拒绝。

local 专项必须证明它只有 vector lane-address: 同一数值 offset 在不同 lane 指向不同 local allocation。任何 scalar local、uniform-base local、SV-mix local、带非零 sbase 的 local 机器字或汇编文本都必须拒绝。

mixed/extended 地址故障逐项覆盖保留 scale/modifier、非法寄存器域组合、数学下溢/溢出、allocation 越界和自然对齐失败。地址空间只由 opcode 决定: 必须有一个用例把某个 shared 窗口偏移搬进 SGPR64 再交给 global load, 期望结果是 MEMORY_BOUNDS 之类的地址故障, 而不是任何“指针类型”检查。普通 SV-mix 的每个合法 scale 都要有正例, 未声明 scale 要有负例; SMEMX 和 VATOMX 都要断言 scale 固定为 1。任一参与 lane 失败时事件数必须为零。

8.9.2 值、事件和回滚

load/store 检查小端字节顺序、窄 load 扩展、F16 高位清零、向量元素顺序和事件数。

- S_MEM 成功时恰好一个事件;
- V_MEM 每个参与 lane 一个事件;
- V lane 同址不能被测试工具误算成一个架构事件;
- 向量最后一个元素失败时整条指令无事件;
- store 的 S/U 位型重复形式不得同时存在。

8.9.3 内存顺序

强制 litmus 至少包括:

- 同 lane 同址写后读;
- release/acquire 消息传递;
- scope 不足时不建立跨代理同步;
- store buffering;

- shared barrier 只排序 shared;
- local 空间 lane 隔离;
- 窄字节写后宽读的小端拼装;
- 无凭空值;
- 普通/原子重叠的禁止分类。

允许结果必须由第 4 章模型产生。禁止结果一旦观察到即 FAIL；允许结果没有观察到不算失败。

8.10 S_ATOM/V_ATOM

8.10.1 space、order 和 scope

先断言所有规范名称都以 S_ATOM. 或 V_ATOM. 开头，交换操作只接受 XCHG。对每个原子 op/type/space，直接枚举 order 和 scope modifier:

op	legal order	global scope	shared scope
LOAD	RELAXED, ACQUIRE	CTA/DEVICE/SYSTEM	CTA
STORE	RELAXED, RELEASE	CTA/DEVICE/SYSTEM	CTA
ADD/MIN/MAX/			
AND/OR/XOR/XCHG	四种全部	CTA/DEVICE/SYSTEM	CTA
CAS	四种全部	CTA/DEVICE/SYSTEM	CTA

space 必须由 form/opcode 固定为 GLOBAL 或 SHARED，不能从地址寄存器猜测；order/scope 作为同一操作的编码 modifier 覆盖，测试清单不得把每个组合统计成不同语义 family。shared 非 CTA scope、param/const/local 原子和未知 space/scope 名称都要覆盖。每个合法组合检查自然对齐、空间、地址模板、S/V 事件数和寄存器域。

canonical 文本固定为 (S_ATOM|V_ATOM).<op>.<type>.<space>.<order>.<scope>; space 和两个 modifier 都不能省略。对每个合法组合执行:

```
T = disassemble(W, canonical=true)
assert T 的类型后依次紧跟合法 .<space>.<order>.<scope>
assert assemble(T) == W
assert 删除、交换或重复 space/order/scope 会被拒绝
```

至少 round-trip S_ATOM.LOAD.U32.GLOBAL.ACQUIRE.DEVICE、S_ATOM.XCHG.U32.SHARED.RELAXED.CTA 和 V_ATOM.CAS.U64.GLOBAL.ACQ_REL.SYSTEM。其他交换拼写必须被拒绝。

对 SATOM、VATOM、VATOMX 各取至少一个原始机器字，单独把 scope 两位改为 3：必须译码为 ILLEGAL_INSTRUCTION，不能反汇编出未知 scope 文本，也不能读取地址/数据源或产生事件。

已定义的 modifier 组成非法组合时才期望 ILLEGAL_OPERAND，例如 shared 配 DEVICE scope。测试报告必须把这两类负例分栏，不能合并成“任意非法 modifier”。

VATOMX 逐字段检查 vdst/sbase/vindex/vdata0/vdata1/order/scope/x，并证明它没有 immediate 容器。canonical 名称中的 space 固定为 GLOBAL。每 lane 地址必须等于 SGPR64_base + zero_extend(VGPR32_index[lane])，scale 固定为 1。任何额外缩放、非零保留 x、非法寄存器域、越界和未对齐都做负例，任一 lane 失败时所有 lane 零事件回滚。

8.10.2 atomic load/store

atomic load 必须:

- 返回 modification order 中可读的完整旧值;
- 不追加修改;

- 不接受 release-only order;
- U64 不撕裂。

atomic store 必须:

- 追加一个完整新值;
- 没有旧值目标;
- 不接受 acquire-only order;
- release 语义能参与消息传递。

测试不能用 RMW 加零代替 atomic load, 也不能用 XCHG 代替 atomic store 的 oracle。

8.10.3 RMW、CAS 和同址多 lane

ADD/MIN/MAX/AND/OR/XOR/XCHG/CAS 至少覆盖正常值、边界值、返回旧值和最终值。CAS 必须分别覆盖 expected 相等和不等: 两种情况都返回 old; 成功写 replacement, 失败写回 old; 两种情况都按第 4 章检查 modification-order 节点数。LOAD 必须没有写数据源, STORE 必须没有旧值目标, CAS 必须同时有 expected、replacement 和 old-value 目标。

V_ATOM 的多个 lane 命中同址时, oracle 枚举所有满足 lane 程序序的顺序, 不假定 lane 0 先执行。

S_ATOM 同一动态指令必须恰好出现一个原子事件; 需要修改的位置在 modification order 中恰好占一个节点。

8.11 W 控制流

8.11.1 BRA 与间接跳转

BRA、BRA.P、JUMP.IND 至少覆盖:

- 直接前跳、后跳和自环;
- BRA.P 全 taken、全 fall-through 和真正分歧;
- 间接 S 目标的最低、最高合法地址;
- 未对齐、文本外、指令中间和非法机器字目标;
- 尝试使用 V 目标寄存器的静态拒绝。

每步比较 PC、active mask 和隐藏状态摘要。测试接口可以暴露只读调试摘要, 但程序本身不能读取隐藏项。

JUMP.IND 必须另外跑完整 scalar-ready 矩阵。NS1..NS4 都产生 DIVERGENCE_FAULT, 并且不能读取目标 SGPR; 直接 BRA、BRA.P 不套用这项检查。JUMP.IND 成功和失败都不得改变调用栈。

8.11.2 CALL/RET

强制程序包括:

1. 一层直接 CALL/RET;
2. 多层嵌套调用;
3. CALL.IND 使用 S 目标;
4. 递归直到声明深度边界;
5. CALL 内部包含已闭合的 SSY/BRA.P/JOIN;

6. 每次 CALL 的帧只保存准确的 PC+8, RET 恢复该 PC;
7. 空状态 RET;
8. 非法 CALL 目标;
9. 返回前仍有未闭合 SSY 区域;
10. 调用深度超限。

CALL、CALL.IND 和 RET 虽然是 W/CONTROL 指令，也必须跑完整 scalar-ready 矩阵。NS1..NS4 的期望都是 DIVERGENCE_FAULT；状态检查发生在读取间接目标或调用状态之前。失败用例必须证明没有半压栈或半弹栈。

调用栈 oracle 必须独立维护每 warp LIFO，并逐次比较：

```
CALL 成功    -> push({return_pc=old_pc+8})
CALL.IND 成功 -> 同上，再把 PC 设为冻结的 SGPR64 目标
SSY 成功     -> 新 reconv frame.owner_call_depth = call_stack.depth
RET 成功     -> 不存在 owner_call_depth==call_stack.depth 的 reconv frame;
               pop 后 PC=return_pc
JUMP.IND     -> 栈内容和深度完全不变
```

调用帧调试摘要若暴露除 return_pc 以外的架构字段即 FAIL。分别用 descriptor call_stack_depth=0、1、实现最大值 测试：depth=0 时第一次 CALL 就满；其他值恰好允许对应数量，下一次 CALL 报故障。两层以上嵌套调用必须证明 RET 严格后进先出。

目标非法、descriptor 栈满、空栈 RET、当前 callee 仍有 owner_call_depth==call_stack.depth 的重汇聚帧和非 scalar-ready 分别注入，并按 docs/02-programming-model.md 的权威顺序组合成多故障用例；任一失败都不得改变 PC、调用栈或重汇聚栈。

8.11.3 隐藏重汇聚

SSY/BRA.P/JOIN/EXIT 套件至少执行：

- 无分歧路径；
- 单层分歧；
- 多层正确嵌套；
- 一条路径部分 EXIT；
- 一条路径全部 EXIT；
- 两条路径分别有 EXIT；
- 尾 warp；
- 错误 JOIN 目标；
- 区域交叉；
- 从区域内部跳到外部；
- 在 FIRST/SECOND 路径的 JOIN 前执行 CALL，必须先报 DIVERGENCE_FAULT；
- scalar-ready 时 CALL，callee 内部建立并闭合自己的嵌套区域；
- 在 FIRST/SECOND 路径执行 RET，必须先报 DIVERGENCE_FAULT；
- scalar-ready 但控制区域关系仍非法的跨区域 RET。

owner_call_depth 必须按 SSY 动态实例精确测试：

- 在调用深度 0、1 和至少两层嵌套调用中执行 SSY，新帧分别记录当时的 call_stack.depth；
- SSY 之后再 CALL，已有 caller 帧的 owner_call_depth 保持旧值，不能随调用深度一起增加；
- callee 内 SSY 建立的帧满足 owner_call_depth == 当前 call_stack.depth，在匹配 JOIN 前 RET 必须报 RECONVERGENCE_FAULT；
- callee 的帧 JOIN 闭合后 RET 成功；较小 owner_call_depth 的 caller ARMED 帧可以跨 CALL/RET 保留；
- JOIN 只弹出匹配的重汇聚帧，不改调用帧；RET 只弹 return_pc，不改写 caller 重汇聚帧；
- SSY 目标、栈容量或其他检查失败时，不得留下带 owner_call_depth 的半帧。

比较可见结果时必须检查：

- lane 不丢失、不重复、不复活；
- 每个非 EXIT lane 恰好执行它所属路径；
- JOIN 后仍 live 的 lane 恢复到同一控制点；
- 路径执行次序符合 YAML 固定规则；
- 程序不能读取或伪造隐藏重汇聚状态。

8.12 SYNC 与 X

先做静态清单和编码门禁：

```
bar-sync/cta == BAR.SYNC.CTA == (class=5, format=0, opcode=3)
(class=5, format=0, opcode=4) 未分配 -> ILLEGAL_INSTRUCTION
(class=5, format=0, opcode=5) 未分配 -> ILLEGAL_INSTRUCTION
```

family/form 总数必须保持 YAML 的运行时去重结果不变。汇编/反汇编只接受 BAR.SYNC.CTA id 作为 canonical 屏障文本；BAR.ARRIVE.CTA、BAR.WAIT.CTA、BARRIER 及一切旧拼写都必须按未知助记符拒绝，id 缺失或超出 0..7 也必须拒绝。逐位核对 YAML 示例机器字，并对 slot3 的最小值、最大值和中间值独立重算 64 位机器字；a/b/imm16/scope2/order2/x6 非零必须报 ILLEGAL_INSTRUCTION。

每个动态用例都比较 8 槽完整状态 arrived_set/waiters、CTA 的 live_owner_set，以及每 warp 的 blocked record。所有集合元素必须是 linear_tid=warp_id*32+lane_id。测试矩阵至少包含：

类别	必测情况	强制结果
启动	满 CTA、尾 warp、槽 0 和槽 7	每槽 idle (arrived_set/waiters 空)；live_owner_set 恰为真实 linear_tid
owner 身份	不同 warp 的相同 lane_id、同 warp 不同 lane_id、尾 lane	(warp_id, lane_id) 映射到唯一 linear_tid；所有集合只比较 linear_tid
正常同步	单/多 warp，不同到达顺序	每 warp 一次整体 release；每条 record 冻结 {warp_id, A, old_PC+8}；arrived_set == live_owner_set 时所有记录一起恢复，槽立即清回 idle
槽复用	同一槽连续两次以上屏障，中间夹 shared 读写	第二次屏障从空 arrived_set 开始；第一次的 waiter/arrival 不残留，也不需要区分代

类别	必测情况	强制结果
多槽	槽 0..7 交错使用	槽互不干扰；一个槽阻塞不影响另一个槽的完成判定
scalar-ready	在 FIRST/SECOND 路径上执行屏障	DIVERGENCE_FAULT；零 arrival、零 blocked record、PC 不动、槽状态不变
scalar-ready	SR1/SR2/SR3 三种就绪状态	正常到达；只有 ARMED 帧不妨碍屏障
EXIT 完成	部分 warp 已到达并阻塞，剩余 owner 全部 EXIT	live_owner_set 缩小后立即重新判定，waiter 被唤醒，槽清回 idle
EXIT 无 release	退出线程先写 shared 再 EXIT，唤醒的 waiter 读同一地址	EXIT 不建立 release/acquire 边，该读属于数据竞争，不得断言看到新值
blocked record	阻塞、恢复、尝试第二条 record	每 warp 至多一条；阻塞/恢复保持 active/live/reconv/call，挂起路径不能切入；恢复只写 resume PC、清记录、置 ready
DEADLOCK	一部分 warp 到达槽 N，其余 warp 既不到达也不退出	arrived_set 追不上 live_owner_set，按第 3 章第 12 节报告 DEADLOCK
内存	shared release/acquire litmus；同程序换 global/local/param/const/host	shared 禁止旧值结果；其他空间不得因屏障额外有序，global 需原子/FENCE
CTA 完成	所有 warp 完成，分别注入非空 arrived_set 或非空 waiters	仅 8 槽全 idle 才完成；任一非 idle 状态拒绝完成

阻塞/恢复专项必须证明：BarrierWaitRecord 只有 warp_id/owner_snapshot/resume_pc 三个字段，owner_snapshot=A 且 resume_pc=old_PC+8；arrival 只提交一次；挂起期间不重复 release；恢复只写记录指定 PC 和 ready，不改 active/live/reconv/call。没有 expected、成员 mask 或子集参数的正例；任何测试工具自行缩小 live_owner_set 都是 FAIL。

还必须断言这些概念在整个实现中不存在：屏障 token 及其寄存器影子标签、槽 generation 计数、SYNC/SPLIT 模式字段、consumed_set，以及 EXIT 上的任何屏障前置检查。调试接口暴露其中任何一项即 FAIL。

寄存器无影子状态必须做正面证明：对任意 VGPR/SGPR 写入序列，只要 32 位（或 64 位对）位型相同，后续所有指令的可观察行为就必须相同。测试通过“同值不同来路”生成对照组，例如同一个位型分别来自立即数 MOV、混合源 MOV、load 返回、ALU 结果和 MMA 输出，随后执行同一段代码，要求逐位一致。任何来路差异都是 FAIL。

跨 lane X 测试必须明确 C/M/P/R，并覆盖：

- 全 warp、单 lane、奇偶 lane、连续子集和尾 warp；
- 空贡献集；
- member mask 包含/排除 inactive lane 的规定行为；

- mode/width/member mask 一致性;
- 每 lane 不同 control;
- 源 lane 不存在或不在 P;
- 原地源/目标别名;
- 候选缺失和会合中 EXIT。

V_SHUFFLE.DOWN.B32 必须同时覆盖 opcode 11 的 VGPR delta form 和 opcode 13 的立即数 delta form。对 delta 0,1,2,4,8,16,31 及每个合法 width，两种 form 在所有 lane 获得相同 delta 时必须逐位等价；立即数 32..255、非法 width、非零 smask/x5 和错误 opcode 都必须拒绝。

协议错误产生 COLLECTIVE_FAULT 时，所有接收者目标保持。

8.13 MMA

8.13.1 片段 ABI

清单必须断言 M16N8K16 形状只有 MMA.M16N8K16.F16.F16.F32 一个 form；任何其他 M16N8K16 类型或 modifier 都必须被拒绝。YAML 若启用其他形状，仍须按各自结构化合同进入 all-form 门禁，但不得被算成第二个 M16N8K16 form。

测试生成器先把该 form 的 YAML matrix_contract 解析成坐标，并断言它与第 7 章公式逐项相同；不能只比较一段描述字符串。独立 oracle 直接实现同一映射。对 lane l:

```
A: m=l//2, k=8*(l%2)+2*r+h      (r=0..3,h=0..1)
B: k=l//2, n=4*(l%2)+2*r+h      (r=0..1,h=0..1)
C: m=l//2, n=4*(l%2)+r          (r=0..3)
D: m=l//2, n=4*(l%2)+r          (r=0..3)
```

A/B 的 h=0/1 必须分别命中 VGPR 低/高 16 位；C/D 每个 VGPR 恰好一个 F32。测试必须证明 A/B/C/D 全部 256/128/128/128 个逻辑元素恰好映射一次，没有重叠或洞。

片段组固定为 A/B/C/D 每 lane 4/2/4/4 个 VGPR，对齐 4/2/4/4；只允许 D=C 完整别名。测试必须直接按坐标填 A/B/C，不能让被测 pack helper 同时生成输入和标准答案。至少自动生成：

- 每个 A 元素单独置 1；
- 每个 B 元素单独置 1；
- 每个 C 元素使用唯一编码；
- packed 元素的低半/高半或各子字段基向量；
- 最低、最高合法片段基址；
- 允许的完整别名；
- 错误对齐、越界和部分重叠。

任何坐标或打包顺序错误都直接 FAIL。

8.13.2 数值

每个输出先令 $acc = C[m,n]$ ，再严格执行：

```
for k = 0,1,...,15:
  acc = FFMA.F32(
    V_CVT.F32.F16(A[m,k]),
    V_CVT.F32.F16(B[k,n]),
    acc)
```

```
D[m,n] = acc
```

每一步都在步末 RNE 舍入到 F32。oracle 禁止树形归约、跨 k 扩展精度或 FP64 累加。强制数据包括：

- 零矩阵和单位基向量；
- 精确小整数；
- 正负零和抵消；
- normal、subnormal、最大有限数、Inf 和 NaN；
- 能区分固定 k 顺序与树形归约的样本；
- 固定 seed 随机片段。

全部 D 元素逐位比较，不能使用相对误差替代位精确要求。

至少再做一个“手工坐标小样”：绕开通用 pack/unpack helper，直接给选定 lane、寄存器偏移和位片写值，手工计算一个 D 坐标。这个用例用来发现测试工具和实现共同误读同一份映射。

8.13.3 会合和整体提交

必须检查 `active_mask == live_mask == 0xffffffff`、固定 PT、所有 lane 同一动态 PC 和一致片段基址。缺 lane、不同动态 PC、片段参数不一致、会合时 EXIT 或非法 guard 必须产生规定故障。故障时任何 D 元素都不能写；即使最后一个输出才发现错误，前面的输出也必须回滚。

8.14 all-form 覆盖门禁

8.14.1 从 YAML 生成清单

构建开始时生成：

```
required_forms = unique(forms enabled by current YAML and feature set)
required_families = unique(family_id of required_forms)
required_modifier_instances =
    expand each form's legal order/scope/address-modifier matrix
```

order、scope 和 scale 赋值属于 modifier instance，不得复制成新的 family/form 来虚增统计。生成器必须拒绝“操作数和语义相同、只因 modifier 取值不同就复制 form”的 YAML 条目。forms_total 只数 form；modifier instance 另行报告，不能混进 form 总数。

每个 form 都必须有以下 coverage tag：

```
decode-positive
decode-negative
assemble
disassemble
fixed64
machine-class
reader-class-view
semantic-normal
semantic-boundary
guard_policy
required_state
register-domain
register-pair64
aliasing
fault-or-no-fault
generated-reference
```

适用时还必须有：

```
scalar-ready
cross-domain
```

```

vector-optional-guard
mixed-source-selector
mixed-source-pair64
wide-mul-mad
int-mad-minmax-abs-neg
ctz-rotate-pack
brev-bfe-bfi
target
call-ret
hidden-reconvergence
reconv-owner-call-depth
mixed-addressing
smemx-vatomx
memory-align-bounds
memory-order
atomic-load
atomic-store
atomic-rmw
atomic-cas
atomic-order-scope-modifier
atomic-space-order-scope
sync
cross-lane
f32-compare
v-f16
fp-bitexact
approx-ulp
mma-fragment
mma-structured-mapping

```

门禁算法：

```

for form in required_forms:
    assert generated_reference_contains(form.id)
    for tag in mandatory_tags(form):
        assert passing_test_count(form.id, tag) >= 1
    for instance in required_modifier_instances(form):
        assert encode_decode_round_trip(instance)
        assert semantic_modifier_oracle_passes(instance)

assert covered_form_ids == required_forms.ids
assert report.forms_total == len(required_forms)
assert report.families_total == len(required_families)
assert report.modifier_instances_total == len(required_modifier_instances)
assert no_test_references_unknown_form

```

同 family 的另一个 form 不能代替当前 form。一个测试可以覆盖多个 tag，但必须真的执行各 tag 的 oracle，不能只在元数据中挂名字。

8.14.2 S/V 对称性和 only-form 检查

生成器根据第 7 章和 YAML 自动建立：

```

dual_pairs
s_only_forms
v_only_forms
cross_domain_forms
scalar_ready_forms

```

scalar_ready_forms 直接选择全部 required_state: scalar_ready 的 form，并反向断言全部

execution_domain: scalar form 都在集合中。非 scalar 执行域的成员至少包括 CALL、CALL.IND、JUMP.IND、RET 和 BAR.SYNC.CTA。集合中每个 form 都必须带 scalar-ready coverage tag，并跑完整 SR/NS 矩阵。

mixed_source_forms 直接选择全部含 vsrc32 或 vsrc64 操作数的 form，并断言它们的 encoding_format 都属于 {V1,V2,V3,VCMP}、execution_domain 都是 vector。集合中每个 form 都必须带

mixed-source-selector tag，含 vsrc64 的还必须带 mixed-source-pair64，并按 8.6.1 覆盖该格式的每个合法 selector 码。

对 `dual_pairs`，要求 S/V 有共同的数学边界向量，并额外检查执行次数、寄存器域和事件数。`compare` 的每个关系/类型和 CVT 的每个 `dst-type.src-type` 必须进入 `dual_pairs`，缺任一侧都失败。对 `only-form`，要求另一域的拼写和编码不存在。对跨域 `form`，要求源/目标方向与规范完全一致。

8.14.3 新 form 门禁

任何新增 `form` 必须在同一变更中带上：

- 唯一编码和负例变异规则；
- 8 选 1 的机器 `class`，以及需要时的用户可读简称；
- 操作数域、`guard_policy` 与 `required_state`；
- 正常、边界和故障语义；
- 适用的 `scalar-ready`、跨域、控制、内存、FP、X 或 MMA 专项；
- 生成参考条目；
- `all-form coverage tags`。

只增加 YAML 条目但没有测试 `oracle`，或只增加测试名字但没有向量，构建必须失败。

8.15 差分、形式属性和发布报告

最低验证链路：

1. YAML schema 与编码静态检查；
2. 独立 `assembler/decoder` 闭环；
3. 指令级参考模型；
4. 随机定义良好的程序差分；
5. 内存 `litmus` 模型检查；
6. RTL/模拟器逐提交比较；
7. 设备结果和 `fault trace` 回读。

至少建立以下形式属性：

- 每个 64 位机器字至多选择一个 `form`；
- 每个 `form` 的所有位都属于字段或 `must-zero`；
- 机器 `class` 只能是 `SYS/SALU/VALU/MEMORY/CONTROL/SYNC/CROSSLANE/MATRIX` 之一；
- 用户可读简称不得覆盖或改变机器 `class`；
- S/V 域约束不会被绕过；
- 每个 S `form` 在非 `scalar-ready` 时都只产生 `DIVERGENCE_FAULT`；
- `CALL`、`CALL.IND`、`JUMP.IND`、`RET` 保持 `CONTROL class`，并执行 `scalar-ready` 检查；
- 故障指令不产生部分提交；
- `CALL/RET` 和隐藏重汇聚状态满足 LIFO/结构约束；
- `atomic load` 不改 `modification order`；

- MMA 的 D 片段整体提交。

发布报告必须列出：

- 从 YAML 生成的 family/form 数和摘要；
- 每个 form 的 coverage tags 与用例 ID；
- 固定 64 位编码正/负例结果；
- scalar-ready 与跨域矩阵；
- S/V 双版本和 only-form 检查；
- 普通内存、mixed addressing 和原子专项；
- 控制流、CALL/RET、隐藏重汇聚、SYNC/X 结果；
- FP 固定/随机向量与观察到的最大 ULP；
- MMA ABI、数值和会合结果；
- 所有失败、跳过和 NOT_APPLICABLE 项。

只要存在缺失 form、缺失 oracle、未决规范点或未解释差异，报告就不能标记 PASS。

附录A 指令形式参考（自动生成）

本章由 isa/vtx1/isa.yaml 自动生成，请勿手工编辑。

- ISA: VTX-1 ISA 1.0 Draft
- 版本: 1.0-draft
- 指令字宽: 64 位
- Family 数: 66
- Form 数: 379
- Descriptor contract: {call_stack_depth: {maximum: 16, minimum: 0}}
- Barrier contract: {idle_slot: {arrived_set_empty: true, waiters_empty: true}, live_owner_set: {exit_contributes_release: false,

initial: every real linear_tid launched in the CTA, shrinks_on: EXIT}, max_blocked_records_per_warp: 1, owner_identity: {equivalent_tuple: [warp_id, lane_id], formula: linear_tid = warp_id
32 + lane_id, name: linear_tid}, wait_record_fields: [warp_id, owner_snapshot, resume_pc]}

NOP

- Family ID: nop
- 语义组: system_control

No architectural effect except advancing PC.

NOP — base

- 执行域: system
- 编码格式: SYS
- 语义组: system_control
- (class, format, opcode): (SYS, 0, 0)
- Guard policy: required_pt
- Required state: none

语法: NOP

Operands

名称	类型	访问	字段	说明
pc	implicit_state	read_write	—	—

Semantics:

Advance PC by one 64-bit instruction.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: NOP

示例字段值: —

64 位机器字: 0x0000000000000000

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	是	—	Opcode-defined register or small encoded operand A.
b	34:27	—	是	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

TRAP

- Family ID: trap
- 语义组: system_control

Raise a software trap with a 32-bit reason.

TRAP — reason

- 执行域: system
- 编码格式: SYS
- 语义组: system_control
- (class, format, opcode): (SYS, 0, 1)
- Guard policy: required_pt
- Required state: none

语法: TRAP 0

Operands

名称	类型	访问	字段	说明
reason	uimm16	read	imm16	—

Semantics:

If any guarded lane participates, raise SOFTWARE_TRAP with reason; otherwise advance PC.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: TRAP 0

示例字段值: —

64 位机器字: 0x0000000000000080

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
a	26:19	—	是	—	Opcode-defined register or small encoded operand A.
b	34:27	—	是	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	否	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_GETREG

- Family ID: s-getreg
- 语义组: special_register

Read a uniform special register.

S_GETREG.U32 — u32

- 执行域: scalar
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_GETREG.U32 s0, USR_WARP_ID

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	a	—
sr	uniform_special	read	b	—

Semantics:

Snapshot the selected 32-bit uniform special register into dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_GETREG.U32 s0, USR_WARP_ID

示例字段值: —

64 位机器字: 0x0000000000000100

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_GETREG.U64 — u64

- 执行域: scalar
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_GETREG.U64 s0:s1, USR_CLOCK64

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	a	—
sr	uniform_special	read	b	—

Semantics:

Snapshot the selected 64-bit uniform special register into the even SGPR pair dst.

Constraints:

- The selected register must be 64-bit and dst must be an even SGPR pair.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_GETREG.U64 s0:s1, USR_CLOCK64

示例字段值: —

64 位机器字: 0x0000000000000180

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_GETREG

- Family ID: v-getreg
- 语义组: special_register

Read a lane special register.

V_GETREG.LANE.U32 — lane.u32

- 执行域: vector
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 4)
- Guard policy: optional
- Required state: none

语法： V_GETREG.LANE.U32 v0, LSR_LANE_ID

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	a	—
sr	lane_special	read	b	—

Semantics:

Each participating lane snapshots its selected 32-bit lane special register.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_GETREG.LANE.U32 v0, LSR_LANE_ID

示例字段值： —

64 位机器字： 0x00000000000000200

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.

字段	位段	固定值	必须为零	保留值	说明
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_GETREG.UNIFORM.U32 — uniform.u32

- 执行域: vector
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 5)
- Guard policy: optional
- Required state: none

语法: V_GETREG.UNIFORM.U32 v0, USR_WARP_ID

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	a	—
sr	uniform_special	read	b	—

Semantics:

Each participating lane snapshots its selected 32-bit lane special register.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_GETREG.UNIFORM.U32 v0, USR_WARP_ID

示例字段值: —

64 位机器字: 0x00000000000000280

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_GETREG.LANE.U64 — lane.u64

- 执行域: vector
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 6)
- Guard policy: optional
- Required state: none

语法: V_GETREG.LANE.U64 v0:v1, LSR_LOCAL_BASE

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	a	—
sr	lane_special	read	b	—

Semantics:

Each participating lane snapshots its selected 64-bit lane special register.

Constraints:

- The selected register must be 64-bit and dst must be an even VGPR pair.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_GETREG.LANE.U64 v0:v1, LSR_LOCAL_BASE

示例字段值: —

64 位机器字: 0x00000000000000300

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.

字段	位段	固定值	必须为零	保留值	说明
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_GETREG.UNIFORM.U64 — uniform.u64

- 执行域：vector
- 编码格式：SYS
- 语义组：special_register
- (class, format, opcode): (SYS, 0, 7)
- Guard policy: optional
- Required state: none

语法：V_GETREG.UNIFORM.U64 v0:v1, USR_CLOCK64

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	a	—
sr	uniform_special	read	b	—

Semantics:

Each participating lane snapshots its selected 64-bit lane special register.

Constraints:

- The selected register must be 64-bit and dst must be an even VGPR pair.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例：V_GETREG.UNIFORM.U64 v0:v1, USR_CLOCK64

示例字段值：—

64 位机器字：0x0000000000000380

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_SETREG

- Family ID: s-setreg
- 语义组: special_register

Write a writable uniform special register.

S_SETREG.U32 — u32

- 执行域: scalar
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 8)
- Guard policy: required_pt

- Required state: scalar_ready

语法: S_SETREG.U32 USR_STATUS, s0

Operands

名称	类型	访问	字段	说明
sr	uniform_special	write	a	—
src	sgpr32	read	b	—

Semantics:

Write src to the selected writable uniform special register.

Constraints:

- The selected register must have read_write access and be 32-bit.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SETREG.U32 USR_STATUS, s0

示例字段值: —

64 位机器字: 0x00000000000000400

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.

字段	位段	固定值	必须为零	保留值	说明
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SETREG

- Family ID: v-setreg
- 语义组: special_register

Write a writable lane special register.

V_SETREG.U32 — u32

- 执行域: vector
- 编码格式: SYS
- 语义组: special_register
- (class, format, opcode): (SYS, 0, 9)
- Guard policy: required_pt
- Required state: none

语法: V_SETREG.U32 LSR_DEBUG, v0

Operands

名称	类型	访问	字段	说明
sr	lane_special	write	a	—
src	vgpr32	read	b	—

Semantics:

Each participating lane writes src to its selected writable lane special register.

Constraints:

- The selected register must have read_write access and be 32-bit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_SETREG.U32 LSR_DEBUG, v0

示例字段值： —

64 位机器字： 0x00000000000000480

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	0	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	否	—	Opcode-defined register or small encoded operand A.
b	34:27	—	否	—	Opcode-defined register or small encoded operand B.
c	42:35	—	是	—	Opcode-defined register or small encoded operand C.
imm16	58:43	—	是	—	Opcode-defined 16-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_MOV

- Family ID: s-mov
- 语义组: move

Move scalar register or immediate data without conversion.

S_MOV.B32 — b32.reg

- 执行域: scalar
- 编码格式: S1
- 语义组: move
- (class, format, opcode): (SALU, 0, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MOV.B32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Copy all 32 source bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MOV.B32 s0, s0

示例字段值: —

64 位机器字: 0x0000000000000001

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_MOV.B32 — b32.imm

- 执行域: scalar
- 编码格式: SIMM
- 语义组: move
- (class, format, opcode): (SALU, 4, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MOV.B32 s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	simm24	read	imm24	—

Semantics:

Sign-extend imm24 to 32 bits and copy it to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MOV.B32 s0, 0

示例字段值: —

64 位机器字: 0x0000000000000041

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	是	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_MOV.B64 — b64.reg

- 执行域: scalar
- 编码格式: S1
- 语义组: move
- (class, format, opcode): (SALU, 0, 1)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MOV.B64 s0:s1, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
src	sgpr64	read	sa	—

Semantics:

Copy all 64 source bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MOV.B64 s0:s1, s0:s1

示例字段值: —

64 位机器字: 0x0000000000000081

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_MOV.B64 — b64.imm

- 执行域: scalar
- 编码格式: SIMM
- 语义组: move
- (class, format, opcode): (SALU, 4, 1)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_MOV.B64 s0:s1, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
src	simm24	read	imm24	—

Semantics:

Sign-extend imm24 to 64 bits and copy it to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_MOV.B64 s0:s1, 0

示例字段值： —

64 位机器字： 0x000000000000000C1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	是	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ADD

- Family ID: s-add
- 语义组: integer_arithmetic

Scalar add with register or immediate second source.

S_ADD.U32 — u32.reg

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ADD.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Add a and b, writing the low 32 bits to dst.

Constraints:

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_ADD.U32 s0, s0, s0

示例字段值: —

64 位机器字: 0x0000000000000011

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_ADD.U32 — u32.imm

- 执行域: scalar
- 编码格式: SIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 4, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ADD.U32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	simm24	read	imm24	—

Semantics:

Add a and sign-extended imm24, writing the low 32 bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_ADD.U32 s0, s0, 0

示例字段值: —

64 位机器字: 0x00000000000000141

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ADD.U64 — u64.reg

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 1)

- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ADD.U64 s0:s1, s0:s1, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr64	read	sa	—
b	sgpr64	read	sb	—

Semantics:

Add a and b, writing the low 64 bits to dst.

Constraints:

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_ADD.U64 s0:s1, s0:s1, s0:s1

示例字段值: —

64 位机器字: 0x0000000000000091

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.

字段	位段	固定值	必须为零	保留值	说明
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_ADD.U64 — u64.imm

- 执行域: scalar
- 编码格式: SIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 4, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ADD.U64 s0:s1, s0:s1, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr64	read	sa	—
b	simm24	read	imm24	—

Semantics:

Add a and sign-extended imm24, writing the low 64 bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_ADD.U64 s0:s1, s0:s1, 0

示例字段值: —

64 位机器字: 0x000000000000001C1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_SUB

- Family ID: s-sub
- 语义组: integer_arithmetic

Scalar subtract with register or immediate second source.

S_SUB.U32 — u32.reg

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_SUB.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Subtract b from a, writing the low 32 bits to dst.

Constraints:

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SUB.U32 s0, s0, s0

示例字段值: —

64 位机器字: 0x0000000000000111

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_SUB.U32 — u32.imm

- 执行域: scalar

- 编码格式: SIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 4, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_SUB.U32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	simm24	read	imm24	—

Semantics:

Subtract a and sign-extended imm24, writing the low 32 bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SUB.U32 s0, s0, 0

示例字段值: —

64 位机器字: 0x00000000000000241

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_SUB.U64 — u64.reg

- 执行域：scalar
- 编码格式：S2
- 语义组：integer_arithmetic
- (class, format, opcode): (SALU, 1, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_SUB.U64 s0:s1, s0:s1, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr64	read	sa	—
b	sgpr64	read	sb	—

Semantics:

Subtract b from a, writing the low 64 bits to dst.

Constraints:

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_SUB.U64 s0:s1, s0:s1, s0:s1

示例字段值： —

64 位机器字： 0x00000000000000191

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_SUB.U64 — u64.imm

- 执行域： scalar
- 编码格式： SIMM
- 语义组： integer_arithmetic
- (class, format, opcode): (SALU, 4, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_SUB.U64 s0:s1, s0:s1, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr64	read	sa	—
b	simm24	read	imm24	—

Semantics:

Subtract a and sign-extended imm24, writing the low 64 bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SUB.U64 s0:s1, s0:s1, 0

示例字段值: —

64 位机器字: 0x000000000000002C1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_MUL

- Family ID: s-mul
- 语义组: integer_arithmetic

Scalar multiply with register or immediate second source.

S_MUL.U32 — u32.reg

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MUL.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Multiply a by b, writing the low 32 bits to dst.

Constraints:

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MUL.U32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000211

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MUL.U32 — u32.imm

- 执行域: scalar
- 编码格式: SIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 4, 6)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MUL.U32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	simm24	read	imm24	—

Semantics:

Multiply a and sign-extended imm24, writing the low 32 bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MUL.U32 s0, s0, 0

示例字段值： —

64 位机器字： 0x00000000000000341

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_MUL.U64 — u64.reg

- 执行域： scalar
- 编码格式： S2
- 语义组： integer_arithmetic
- (class, format, opcode): (SALU, 1, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_MUL.U64 s0:s1, s0:s1, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—

名称	类型	访问	字段	说明
a	sgpr64	read	sa	—
b	sgpr64	read	sb	—

Semantics:

Multiply a by b, writing the low 64 bits to dst.

Constraints:

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_MUL.U64 s0:s1, s0:s1, s0:s1

示例字段值： —

64 位机器字： 0x00000000000000291

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MUL.U64 — u64.imm

- 执行域： scalar
- 编码格式： SIMM

- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 4, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MUL.U64 s0:s1, s0:s1, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr64	read	sa	—
b	simm24	read	imm24	—

Semantics:

Multiply a and sign-extended imm24, writing the low 64 bits to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MUL.U64 s0:s1, s0:s1, 0

示例字段值: —

64 位机器字: 0x000000000000003C1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.

字段	位段	固定值	必须为零	保留值	说明
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_MAD

- Family ID: s-mad
- 语义组: integer_arithmetic

Scalar integer multiply-add.

S_MAD.U32 — u32

- 执行域: scalar
- 编码格式: S3
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 2, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MAD.U32 s0, s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—
c	sgpr32	read	sc	—

Semantics:

Compute $(a*b+c)$ modulo 2^{32} .

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_MAD.U32 s0, s0, s0, s0

示例字段值： —

64 位机器字： 0x0000000000000021

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
sc	50:43	—	否	—	Scalar source C.
x13	63:51	—	是	—	Opcode-defined extension; unused bits are zero.

S_MUL.WIDE

- Family ID: s-mul-wide
- 语义组: wide_integer

Produce the full 64-bit product of two 32-bit operands.

S_MUL.WIDE.U32 — u32

- 执行域: scalar
- 编码格式: S2
- 语义组: wide_integer
- (class, format, opcode): (SALU, 1, 6)

- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MUL.WIDE.U32 s0:s1, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Write the unsigned 32-by-32 full product to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MUL.WIDE.U32 s0:s1, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000311

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.

字段	位段	固定值	必须为零	保留值	说明
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MUL.WIDE.S32 — s32

- 执行域: scalar
- 编码格式: S2
- 语义组: wide_integer
- (class, format, opcode): (SALU, 1, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MUL.WIDE.S32 s0:s1, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Write the signed two-complement 32-by-32 full product to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MUL.WIDE.S32 s0:s1, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000391

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MAD.WIDE

- Family ID: s-mad-wide
- 语义组: wide_integer

Multiply two 32-bit operands and add a 64-bit accumulator.

S_MAD.WIDE.U32 — u32

- 执行域: scalar
- 编码格式: S3
- 语义组: wide_integer
- (class, format, opcode): (SALU, 2, 1)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MAD.WIDE.U32 s0:s1, s0, s0, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

名称	类型	访问	字段	说明
c	sgpr64	read	sc	—

Semantics:

Write $(\text{unsigned}(a) * \text{unsigned}(b) + c) \bmod 2^{64}$ to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MAD.WIDE.U32 s0:s1, s0, s0, s0:s1

示例字段值: —

64 位机器字: 0x000000000000000A1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
sc	50:43	—	否	—	Scalar source C.
x13	63:51	—	是	—	Opcode-defined extension; unused bits are zero.

S_MAD.WIDE.S32 — s32

- 执行域: scalar

- 编码格式: S3
- 语义组: wide_integer
- (class, format, opcode): (SALU, 2, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MAD.WIDE.S32 s0:s1, s0, s0, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—
c	sgpr64	read	sc	—

Semantics:

Write (signed(a)*signed(b)+c) modulo 2^{64} to dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_MAD.WIDE.S32 s0:s1, s0, s0, s0:s1

示例字段值: —

64 位机器字: 0x00000000000000121

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
sc	50:43	—	否	—	Scalar source C.
x13	63:51	—	是	—	Opcode-defined extension; unused bits are zero.

S_INT_MISC

- Family ID: s-int-misc
- 语义组: integer_arithmetic

S integer min, max, abs, and neg.

S_MIN.S32 — min.s32

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 8)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MIN.S32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute S32 MIN of a and b.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_MIN.S32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000411

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MIN.U32 — min.u32

- 执行域： scalar
- 编码格式： S2
- 语义组： integer_arithmetic
- (class, format, opcode): (SALU, 1, 9)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_MIN.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—

名称	类型	访问	字段	说明
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute U32 MIN of a and b.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_MIN.U32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000491

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MAX.S32 — max.s32

- 执行域： scalar

- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 10)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MAX.S32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute S32 MAX of a and b.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_MAX.S32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000511

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.

字段	位段	固定值	必须为零	保留值	说明
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_MAX.U32 — max.u32

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 11)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_MAX.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute U32 MAX of a and b.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_MAX.U32 s0, s0, s0

示例字段值: —

64 位机器字: 0x0000000000000591

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_ABS.S32 — abs.s32

- 执行域: scalar
- 编码格式: S1
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 0, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ABS.S32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Compute two-complement signed 32-bit ABS.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_ABS.S32 s0, s0

示例字段值: —

64 位机器字: 0x0000000000000101

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_NEG.S32 — neg.s32

- 执行域: scalar
- 编码格式: S1
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 0, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_NEG.S32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—

名称	类型	访问	字段	说明
src	sgpr32	read	sa	—

Semantics:

Compute two-complement signed 32-bit NEG.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_NEG.S32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000181

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_DIV_REM

- Family ID: s-div-rem
- 语义组: integer_arithmetic

Scalar signed and unsigned division and remainder.

S_DIV.U32 — div.u

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 12)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_DIV.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute u 32-bit quotient of a by b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- INTEGER_FAULT on divisor zero or signed overflow; ILLEGAL_OPERAND on an invalid register.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_DIV.U32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000611

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_DIV.S32 — div.s

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 13)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_DIV.S32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute s 32-bit quotient of a by b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- INTEGER_FAULT on divisor zero or signed overflow; ILLEGAL_OPERAND on an invalid register.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_DIV.S32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000691

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_REM.U32 — rem.u

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 14)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_REM.U32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute u 32-bit remainder of a by b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- INTEGER_FAULT on divisor zero or signed overflow; ILLEGAL_OPERAND on an invalid register.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_REM.U32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000711

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_REM.S32 — rem.s

- 执行域: scalar
- 编码格式: S2
- 语义组: integer_arithmetic
- (class, format, opcode): (SALU, 1, 15)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_REM.S32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute s 32-bit remainder of a by b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- INTEGER_FAULT on divisor zero or signed overflow; ILLEGAL_OPERAND on an invalid register.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_REM.S32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000791

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_BITWISE

- Family ID: s-bitwise
- 语义组: bit_manipulation

Scalar Boolean bit operations.

S_AND.B32 — and

- 执行域: scalar
- 编码格式: S2
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 1, 16)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_AND.B32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Apply bitwise AND to the source bits and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_AND.B32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000811

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_OR.B32 — or

- 执行域: scalar
- 编码格式: S2
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 1, 17)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_OR.B32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Apply bitwise OR to the source bits and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_OR.B32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000891

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_XOR.B32 — xor

- 执行域： scalar
- 编码格式： S2
- 语义组： bit_manipulation
- (class, format, opcode): (SALU, 1, 18)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_XOR.B32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Apply bitwise XOR to the source bits and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_XOR.B32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000911

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_NOT.B32 — not

- 执行域: scalar
- 编码格式: S1
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 0, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_NOT.B32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—

Semantics:

Apply bitwise NOT to the source bits and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_NOT.B32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000201

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_BREV.B32 — brev.b32

- 执行域: scalar
- 编码格式: S1
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 0, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_BREV.B32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Reverse all 32 bits.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_BREV.B32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000281

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_BFE.B32 — bfe.b32

- 执行域: scalar
- 编码格式: S3
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 2, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_BFE.B32 s0, s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—
offset	sgpr32	read	sb	—
width	sgpr32	read	sc	—

Semantics:

Extract width bits at offset with zero fill.

Constraints:

- All unused extension and reserved bits are zero.

- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_BFE.B32 s0, s0, s0, s0

示例字段值： —

64 位机器字： 0x000000000000001A1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
sc	50:43	—	否	—	Scalar source C.
x13	63:51	—	是	—	Opcode-defined extension; unused bits are zero.

S_BFI.B32 — bfi.b32

- 执行域： scalar
- 编码格式： S3
- 语义组： bit_manipulation
- (class, format, opcode): (SALU, 2, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_BFI.B32 s0, s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
base	sgpr32	read	sa	—
insert	sgpr32	read	sb	—
mask	sgpr32	read	sc	—

Semantics:

Insert selected bits into base according to mask.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_BFI.B32 s0, s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000221

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
sc	50:43	—	否	—	Scalar source C.

字段	位段	固定值	必须为零	保留值	说明
x13	63:51	—	是	—	Opcode-defined extension; unused bits are zero.

S_SHIFT_ROTATE

- Family ID: s-shift-rotate
- 语义组: bit_manipulation

Scalar shifts and rotates by an immediate count.

S_SHL.B32 — shl

- 执行域: scalar
- 编码格式: SIMM
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 4, 8)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_SHL.B32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—
count	uimm24	read	imm24	—

Semantics:

Apply SHL to src by count modulo 32 and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SHL.B32 s0, s0, 0

示例字段值: —

64 位机器字： 0x0000000000000441

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_SHR.B32 — shr

- 执行域：scalar
- 编码格式：SIMM
- 语义组：bit_manipulation
- (class, format, opcode): (SALU, 4, 9)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_SHR.B32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

名称	类型	访问	字段	说明
count	uimm24	read	imm24	—

Semantics:

Apply SHR to src by count modulo 32 and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例：S_SHR.B32 s0, s0, 0

示例字段值：—

64 位机器字：0x000000000000004C1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_SAR.S32 — sar

- 执行域: scalar
- 编码格式: SIMM
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 4, 10)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_SAR.S32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—
count	uimm24	read	imm24	—

Semantics:

Apply SAR to src by count modulo 32 and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SAR.S32 s0, s0, 0

示例字段值: —

64 位机器字: 0x00000000000000541

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ROL.B32 — rotl

- 执行域: scalar
- 编码格式: SIMM
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 4, 11)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ROL.B32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—
count	uimm24	read	imm24	—

Semantics:

Apply ROL to src by count modulo 32 and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_ROL.B32 s0, s0, 0

示例字段值： —

64 位机器字： 0x000000000000005C1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ROR.B32 — rotr

- 执行域： scalar
- 编码格式： SIMM
- 语义组： bit_manipulation
- (class, format, opcode): (SALU, 4, 12)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_ROR.B32 s0, s0, 0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—
count	uimm24	read	imm24	—

Semantics:

Apply ROR to src by count modulo 32 and write dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_ROR.B32 s0, s0, 0

示例字段值： —

64 位机器字： 0x00000000000000641

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_BITCOUNT

- Family ID: s-bitcount
- 语义组: bit_manipulation

Scalar leading-zero, trailing-zero, and population counts.

S_CLZ.B32 — clz

- 执行域: scalar
- 编码格式: S1
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 0, 6)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CLZ.B32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Write the 32-bit CLZ result for src; zero input yields 32 for CLZ/CTZ.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CLZ.B32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000301

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_CTZ.B32 — ctz

- 执行域: scalar
- 编码格式: S1
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 0, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CTZ.B32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Write the 32-bit CTZ result for src; zero input yields 32 for CLZ/CTZ.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CTZ.B32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000381

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_POPC.B32 — popc

- 执行域: scalar
- 编码格式: S1
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 0, 8)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_POPC.B32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Write the 32-bit POPC result for src; zero input yields 32 for CLZ/CTZ.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_POPC.B32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000401

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_PACK

- Family ID: s-pack

- 语义组: bit_manipulation

Pack and unpack scalar subwords.

S_PACK.U16X2 — pack.u16x2

- 执行域: scalar
- 编码格式: S2
- 语义组: bit_manipulation
- (class, format, opcode): (SALU, 1, 19)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_PACK.U16X2 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
lo	sgpr32	read	sa	—
hi	sgpr32	read	sb	—

Semantics:

Pack lo[15:0] into dst[15:0] and hi[15:0] into dst[31:16].

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_PACK.U16X2 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000991

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	19	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_UNPACK.LO16 — unpack.lo16

- 执行域：scalar
- 编码格式：S1
- 语义组：bit_manipulation
- (class, format, opcode): (SALU, 0, 9)
- Guard policy: required_pt
- Required state: scalar_ready

语法：S_UNPACK.LO16 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Zero-extend src[15:0] into dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_UNPACK.LO16 s0, s0

示例字段值： —

64 位机器字： 0x00000000000000481

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_UNPACK.HI16 — unpack.hi16

- 执行域： scalar
- 编码格式： S1
- 语义组： bit_manipulation
- (class, format, opcode): (SALU, 0, 10)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_UNPACK.HI16 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Zero-extend src[31:16] into dst.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_UNPACK.HI16 s0, s0

示例字段值： —

64 位机器字： 0x00000000000000501

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_COMPARE

- Family ID: s-compare
- 语义组: compare

Compare scalar operands and update SCC.

S_CMP.EQ — equal

- 执行域: scalar

- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.EQ s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the equal comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.EQ s0, s0

示例字段值: —

64 位机器字: 0x0000000000000031

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.NE — not-equal

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 1)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.NE s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the not-equal comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.NE s0, s0

示例字段值: —

64 位机器字：0x00000000000000B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.LT.S32 — signed-less

- 执行域：scalar
- 编码格式：SCMP
- 语义组：compare
- (class, format, opcode): (SALU, 3, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法：S_CMP.LT.S32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the signed-less comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.LT.S32 s0, s0

示例字段值: —

64 位机器字: 0x0000000000000131

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.LE.S32 — signed-less-or-equal

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 3)

- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.LE.S32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the signed-less-or-equal comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.LE.S32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000001B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.

字段	位段	固定值	必须为零	保留值	说明
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.GT.S32 — signed-greater

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.GT.S32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the signed-greater comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.GT.S32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000231

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.GE.S32 — signed-greater-or-equal

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.GE.S32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the signed-greater-or-equal comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.GE.S32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000002B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.LT.U32 — unsigned-less

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 6)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.LT.U32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the unsigned-less comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.LT.U32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000331

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.LE.U32 — unsigned-less-or-equal

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.LE.U32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the unsigned-less-or-equal comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.LE.U32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000003B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.GT.U32 — unsigned-greater

- 执行域：scalar
- 编码格式：SCMP
- 语义组：compare
- (class, format, opcode): (SALU, 3, 8)
- Guard policy: required_pt
- Required state: scalar_ready

语法：S_CMP.GT.U32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the unsigned-greater comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.GT.U32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000431

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.GE.U32 — unsigned-greater-or-equal

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 9)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.GE.U32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—

名称	类型	访问	字段	说明
b	sgpr32	read	sb	—

Semantics:

Set SCC to the result of the unsigned-greater-or-equal comparison of a and b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CMP.GE.U32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000004B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.EQ.F32 — f32.eq

- 执行域: scalar

- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 10)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.EQ.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Ordered IEEE F32 EQ comparison writes SCC implicitly.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_CMP.EQ.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000531

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.NE.F32 — f32.ne

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 11)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.NE.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Ordered IEEE F32 NE comparison writes SCC implicitly.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_CMP.NE.F32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000005B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.LT.F32 — f32.lt

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 12)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.LT.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Ordered IEEE F32 LT comparison writes SCC implicitly.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_CMP.LT.F32 s0, s0

示例字段值： —

64 位机器字： 0x00000000000000631

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.LE.F32 — f32.le

- 执行域： scalar
- 编码格式： SCMP
- 语义组： compare
- (class, format, opcode): (SALU, 3, 13)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_CMP.LE.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Ordered IEEE F32 LE comparison writes SCC implicitly.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_CMP.LE.F32 s0, s0

示例字段值： —

64 位机器字： 0x000000000000006B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.GT.F32 — f32.gt

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 14)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.GT.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Ordered IEEE F32 GT comparison writes SCC implicitly.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_CMP.GT.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000731

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.GE.F32 — f32.ge

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 15)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.GE.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Ordered IEEE F32 GE comparison writes SCC implicitly.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_CMP.GE.F32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000007B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.ORD.F32 — f32.ord

- 执行域: scalar
- 编码格式: SCMP
- 语义组: compare
- (class, format, opcode): (SALU, 3, 16)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CMP.ORD.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Apply IEEE F32 ORD: NaN handling follows docs/05-numeric-environment.md.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例： S_CMP.ORD.F32 s0, s0

示例字段值： —

64 位机器字： 0x00000000000000831

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CMP.UNO.F32 — f32.uno

- 执行域： scalar
- 编码格式： SCMP
- 语义组： compare
- (class, format, opcode): (SALU, 3, 17)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_CMP.UNO.F32 s0, s0

Operands

名称	类型	访问	字段	说明
scc	scc	write	—	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Apply IEEE F32 UNO: NaN handling follows docs/05-numeric-environment.md.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: S_CMP.UNO.F32 s0, s0

示例字段值: —

64 位机器字: 0x000000000000008B1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
zero8	26:19	—	是	—	Must be zero; SCMP writes SCC implicitly.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_SELECT

- Family ID: s-select

- 语义组: select

Select one scalar source according to SCC.

S_SELECT.B32 — b32

- 执行域: scalar
- 编码格式: S2
- 语义组: select
- (class, format, opcode): (SALU, 1, 20)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_SELECT.B32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
on_true	sgpr32	read	sa	—
on_false	sgpr32	read	sb	—
condition	scc	read	—	—

Semantics:

Write on_true when SCC is one, otherwise on_false.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_SELECT.B32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000A11

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	20	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_SELECT.B64 — b64

- 执行域: scalar
- 编码格式: S2
- 语义组: select
- (class, format, opcode): (SALU, 1, 21)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_SELECT.B64 s0:s1, s0:s1, s0:s1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sd	—
on_true	sgpr64	read	sa	—
on_false	sgpr64	read	sb	—
condition	scc	read	—	—

Semantics:

Write the selected 64-bit source according to SCC.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_SELECT.B64 s0:s1, s0:s1, s0:s1

示例字段值： —

64 位机器字： 0x00000000000000A91

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	21	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_CVT

- Family ID: s-cvt
- 语义组: conversion

Convert scalar integer and F32 values.

S_CVT.S32.F32 — s32.f32

- 执行域: scalar
- 编码格式: S1
- 语义组: conversion
- (class, format, opcode): (SALU, 0, 11)
- Guard policy: required_pt

- Required state: scalar_ready

语法: S_CVT.S32.F32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Convert IEEE F32 src to signed 32-bit integer using round-to-zero.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CVT.S32.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000581

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_CVT.U32.F32 — u32.f32

- 执行域: scalar
- 编码格式: S1
- 语义组: conversion
- (class, format, opcode): (SALU, 0, 12)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CVT.U32.F32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Convert IEEE F32 src to unsigned 32-bit integer using round-to-zero.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CVT.U32.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000601

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_CVT.F32.S32 — f32.s32

- 执行域: scalar
- 编码格式: S1
- 语义组: conversion
- (class, format, opcode): (SALU, 0, 13)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CVT.F32.S32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Convert signed 32-bit src to IEEE F32 round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CVT.F32.S32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000681

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_CVT.F32.U32 — f32.u32

- 执行域: scalar
- 编码格式: S1
- 语义组: conversion
- (class, format, opcode): (SALU, 0, 14)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CVT.F32.U32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Convert unsigned 32-bit src to IEEE F32 round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_CVT.F32.U32 s0, s0

示例字段值： —

64 位机器字： 0x00000000000000701

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_CVT.U16.U32 — u16.u32

- 执行域： scalar
- 编码格式： S1
- 语义组： conversion
- (class, format, opcode): (SALU, 0, 15)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_CVT.U16.U32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—

名称	类型	访问	字段	说明
src	sgpr32	read	sa	—

Semantics:

Truncate src to unsigned 16 bits and zero-extend to 32 bits.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CVT.U16.U32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000781

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_CVT.S16.S32 — s16.s32

- 执行域: scalar
- 编码格式: S1
- 语义组: conversion

- (class, format, opcode): (SALU, 0, 16)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_CVT.S16.S32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

Truncate src to 16 bits and sign-extend to 32 bits.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_CVT.S16.S32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000801

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.

字段	位段	固定值	必须为零	保留值	说明
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_F32_ARITH

- Family ID: s-f32-arith
- 语义组: floating_point

Scalar IEEE-754 binary32 arithmetic.

S_FADD.F32 — add

- 执行域: scalar
- 编码格式: S2
- 语义组: floating_point
- (class, format, opcode): (SALU, 1, 22)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FADD.F32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute IEEE binary32 ADD with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_FADD.F32 s0, s0, s0

示例字段值: —

64 位机器字：0x00000000000000B11

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	22	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_FSUB.F32 — sub

- 执行域：scalar
- 编码格式：S2
- 语义组：floating_point
- (class, format, opcode): (SALU, 1, 23)
- Guard policy: required_pt
- Required state: scalar_ready

语法：S_FSUB.F32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute IEEE binary32 SUB with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_FSUB.F32 s0, s0, s0

示例字段值: —

64 位机器字: 0x00000000000000B91

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	23	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_FMUL.F32 — mul

- 执行域: scalar
- 编码格式: S2
- 语义组: floating_point
- (class, format, opcode): (SALU, 1, 24)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_FMUL.F32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute IEEE binary32 MUL with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_FMUL.F32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000C11

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	24	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.

字段	位段	固定值	必须为零	保留值	说明
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_FFMA.F32 — fma

- 执行域: scalar
- 编码格式: S3
- 语义组: floating_point
- (class, format, opcode): (SALU, 2, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FFMA.F32 s0, s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—
c	sgpr32	read	sc	—

Semantics:

Compute IEEE binary32 FMA with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_FFMA.F32 s0, s0, s0, s0

示例字段值: —

64 位机器字: 0x000000000000002A1

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
sc	50:43	—	否	—	Scalar source C.
x13	63:51	—	是	—	Opcode-defined extension; unused bits are zero.

S_FMIN.F32 — min

- 执行域: scalar
- 编码格式: S2
- 语义组: floating_point
- (class, format, opcode): (SALU, 1, 25)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FMIN.F32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute IEEE binary32 MIN with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_FMIN.F32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000C91

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	25	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_FMAX.F32 — max

- 执行域： scalar
- 编码格式： S2
- 语义组： floating_point
- (class, format, opcode): (SALU, 1, 26)
- Guard policy： required_pt
- Required state： scalar_ready

语法： S_FMAX.F32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute IEEE binary32 MAX with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_FMAX.F32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000D11

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	26	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_FSQRT.F32 — sqrt

- 执行域: scalar
- 编码格式: S1
- 语义组: floating_point
- (class, format, opcode): (SALU, 0, 17)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FSQRT.F32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—

Semantics:

Compute IEEE binary32 SQRT with round-to-nearest-even and canonical NaN propagation.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_FSQRT.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000881

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_FABS.F32 — abs.f32

- 执行域: scalar
- 编码格式: S1
- 语义组: floating_point
- (class, format, opcode): (SALU, 0, 18)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FABS.F32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

IEEE F32 ABS by sign-bit transform, preserving NaN payload.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例: S_FABS.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000901

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_FNEG.F32 — neg.f32

- 执行域: scalar
- 编码格式: S1
- 语义组: floating_point
- (class, format, opcode): (SALU, 0, 19)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FNEG.F32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
src	sgpr32	read	sa	—

Semantics:

IEEE F32 NEG by sign-bit transform, preserving NaN payload.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.
- DIVERGENCE_FAULT when not scalar-ready; no source is read and no effect commits.

示例： S_FNEG.F32 s0, s0

示例字段值： —

64 位机器字： 0x00000000000000981

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	19	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

S_F32_DIV

- Family ID: s-f32-div
- 语义组: floating_point

Scalar binary32 division and reciprocal.

S_FDIV.F32 — div

- 执行域: scalar
- 编码格式: S2
- 语义组: floating_point
- (class, format, opcode): (SALU, 1, 27)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_FDIV.F32 s0, s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—
b	sgpr32	read	sb	—

Semantics:

Compute IEEE binary32 a divided by b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_FDIV.F32 s0, s0, s0

示例字段值： —

64 位机器字： 0x00000000000000D91

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	27	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
sb	42:35	—	否	—	Scalar source B.
x21	63:43	—	是	—	Opcode-defined extension; unused bits are zero.

S_FRCP.F32 — rcp

- 执行域: scalar
- 编码格式: S1
- 语义组: floating_point
- (class, format, opcode): (SALU, 0, 20)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_FRCP.F32 s0, s0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sd	—
a	sgpr32	read	sa	—

Semantics:

Compute IEEE binary32 reciprocal of a.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_FRCP.F32 s0, s0

示例字段值: —

64 位机器字: 0x00000000000000A01

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	1	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	20	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
sd	26:19	—	否	—	Scalar destination.
sa	34:27	—	否	—	Scalar source A.
x29	63:35	—	是	—	Opcode-defined extension; unused bits are zero.

V_MOV

- Family ID: v-mov
- 语义组: move

Per-lane move of register or immediate bits.

V_MOV.B32 — b32.reg

- 执行域: vector
- 编码格式: V1
- 语义组: move
- (class, format, opcode): (VALU, 0, 0)
- Guard policy: optional
- Required state: none

语法: V_MOV.B32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Copy 32 source bits into each participating lane. The source is a VGPR when ssrc is 0 and the frozen uniform SGPR when ssrc is 1, so this form is also the SGPR-to-VGPR broadcast.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- ssrc selects the source register file; there is no separate broadcast opcode.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_MOV.B32 v0, v0

示例字段值： —

64 位机器字： 0x0000000000000002

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_MOV.B32 — b32.imm

- 执行域： vector
- 编码格式： VIMM
- 语义组： move
- (class, format, opcode): (VALU, 4, 0)

- Guard policy: optional
- Required state: none

语法： V_MOV.B32 v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	simm24	read	imm24	—

Semantics:

Sign-extend imm24 to 32 bits in each participating lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_MOV.B32 v0, 0

示例字段值： —

64 位机器字： 0x0000000000000042

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	是	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_MOV.B64 — b64.reg

- 执行域: vector
- 编码格式: V1
- 语义组: move
- (class, format, opcode): (VALU, 0, 1)
- Guard policy: optional
- Required state: none

语法: V_MOV.B64 v0:v1, v0:v1

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
src	vsrc64	read	va	—

Semantics:

Copy 64 source bits in each participating lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_MOV.B64 v0:v1, v0:v1

示例字段值: —

64 位机器字: 0x0000000000000082

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_MOV.B64 — b64.imm

- 执行域: vector
- 编码格式: VIMM
- 语义组: move
- (class, format, opcode): (VALU, 4, 1)
- Guard policy: optional
- Required state: none

语法: V_MOV.B64 v0:v1, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
src	simm24	read	imm24	—

Semantics:

Sign-extend imm24 to 64 bits in each participating lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_MOV.B64 v0:v1, 0

示例字段值： —

64 位机器字： 0x00000000000000C2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	是	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ADD

- Family ID: v-add
- 语义组: integer_arithmetic

Per-lane vector add with VGPR or immediate source.

V_ADD.U32 — u32.reg

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 0)
- Guard policy: optional
- Required state: none

语法: V_ADD.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane adds a and b, writing the low 32 bits to dst.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ADD.U32 v0, v0, v0

示例字段值: —

64 位机器字: 0x0000000000000012

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_ADD.U32 — u32.imm

- 执行域: vector
- 编码格式: VIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 4, 2)
- Guard policy: optional
- Required state: none

语法: V_ADD.U32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—

名称	类型	访问	字段	说明
a	vgpr32	read	va	—
b	simm24	read	imm24	—

Semantics:

Each participating lane computes add of a and sign-extended imm24.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ADD.U32 v0, v0, 0

示例字段值: —

64 位机器字: 0x0000000000000142

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ADD.U64 — u64.reg

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 1)
- Guard policy: optional
- Required state: none

语法: V_ADD.U64 v0:v1, v0:v1, v0:v1

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc64	read	va	—
b	vsrc64	read	vb	—

Semantics:

Each participating lane adds a and b, writing the low 64 bits to dst.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ADD.U64 v0:v1, v0:v1, v0:v1

示例字段值: —

64 位机器字: 0x0000000000000092

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_ADD.U64 — u64.imm

- 执行域: vector
- 编码格式: VIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 4, 3)
- Guard policy: optional
- Required state: none

语法: V_ADD.U64 v0:v1, v0:v1, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—

名称	类型	访问	字段	说明
a	vgpr64	read	va	—
b	simm24	read	imm24	—

Semantics:

Each participating lane computes 64-bit add with sign-extended imm24.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ADD.U64 v0:v1, v0:v1, 0

示例字段值: —

64 位机器字: 0x000000000000001C2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SUB

- Family ID: v-sub
- 语义组: integer_arithmetic

Per-lane vector subtract with VGPR or immediate source.

V_SUB.U32 — u32.reg

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 2)
- Guard policy: optional
- Required state: none

语法: V_SUB.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane subtracts b from a, writing the low 32 bits to dst.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SUB.U32 v0, v0, v0

示例字段值: —

64 位机器字：0x00000000000000112

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_SUB.U32 — u32.imm

- 执行域：vector
- 编码格式：VIMM
- 语义组：integer_arithmetic
- (class, format, opcode): (VALU, 4, 4)
- Guard policy: optional
- Required state: none

语法：V_SUB.U32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vgpr32	read	va	—
b	simm24	read	imm24	—

Semantics:

Each participating lane computes subtract of a and sign-extended imm24.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SUB.U32 v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000242

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SUB.U64 — u64.reg

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 3)
- Guard policy: optional
- Required state: none

语法: V_SUB.U64 v0:v1, v0:v1, v0:v1

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc64	read	va	—
b	vsrc64	read	vb	—

Semantics:

Each participating lane subtracts b from a, writing the low 64 bits to dst.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SUB.U64 v0:v1, v0:v1, v0:v1

示例字段值: —

64 位机器字：0x0000000000000192

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_SUB.U64 — u64.imm

- 执行域：vector
- 编码格式：VIMM
- 语义组：integer_arithmetic
- (class, format, opcode): (VALU, 4, 5)
- Guard policy: optional
- Required state: none

语法： V_SUB.U64 v0:v1, v0:v1, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vgpr64	read	va	—
b	simm24	read	imm24	—

Semantics:

Each participating lane computes 64-bit subtract with sign-extended imm24.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SUB.U64 v0:v1, v0:v1, 0

示例字段值: —

64 位机器字: 0x000000000000002C2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_MUL

- Family ID: v-mul
- 语义组: integer_arithmetic

Per-lane vector multiply with VGPR or immediate source.

V_MUL.U32 — u32.reg

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 4)
- Guard policy: optional
- Required state: none

语法: V_MUL.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane multiplies a by b, writing the low 32 bits to dst.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_MUL.U32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000212

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MUL.U32 — u32.imm

- 执行域: vector

- 编码格式: VIMM
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 4, 6)
- Guard policy: optional
- Required state: none

语法: V_MUL.U32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vgpr32	read	va	—
b	simm24	read	imm24	—

Semantics:

Each participating lane computes multiply of a and sign-extended imm24.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_MUL.U32 v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000342

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_MUL.U64 — u64.reg

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 5)
- Guard policy: optional
- Required state: none

语法: V_MUL.U64 v0:v1, v0:v1, v0:v1

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc64	read	va	—
b	vsrc64	read	vb	—

Semantics:

Each participating lane multiplies a by b, writing the low 64 bits to dst.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_MUL.U64 v0:v1, v0:v1, v0:v1

示例字段值： —

64 位机器字： 0x00000000000000292

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MUL.U64 — u64.imm

- 执行域： vector
- 编码格式： VIMM
- 语义组： integer_arithmetic

- (class, format, opcode): (VALU, 4, 7)
- Guard policy: optional
- Required state: none

语法: V_MUL.U64 v0:v1, v0:v1, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vgpr64	read	va	—
b	simm24	read	imm24	—

Semantics:

Each participating lane computes 64-bit multiply with sign-extended imm24.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_MUL.U64 v0:v1, v0:v1, 0

示例字段值: —

64 位机器字: 0x000000000000003C2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_MAD

- Family ID: v-mad
- 语义组: integer_arithmetic

Vector integer multiply-add.

V_MAD.U32 — u32

- 执行域: vector
- 编码格式: V3
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 2, 0)
- Guard policy: optional
- Required state: none

语法: V_MAD.U32 v0, v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	c	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—

名称	类型	访问	字段	说明
a	vsrc32	read	va	—
b	vsrc32	read	vb	—
c	vsrc32	read	vc	—

Semantics:

Each participating lane computes $(a*b+c)$ modulo 2^{32} .

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_MAD.U32 v0, v0, v0, v0

示例字段值: —

64 位机器字: 0x0000000000000022

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_MUL.WIDE

- Family ID: v-mul-wide
- 语义组: wide_integer

Per-lane full 32-by-32 product.

V_MUL.WIDE.U32 — u32

- 执行域: vector
- 编码格式: V2
- 语义组: wide_integer
- (class, format, opcode): (VALU, 1, 6)
- Guard policy: optional
- Required state: none

语法: V_MUL.WIDE.U32 v0:v1, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane writes the unsigned full product.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_MUL.WIDE.U32 v0:v1, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000312

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MUL.WIDE.S32 — s32

- 执行域: vector
- 编码格式: V2
- 语义组: wide_integer
- (class, format, opcode): (VALU, 1, 7)
- Guard policy: optional
- Required state: none

语法: V_MUL.WIDE.S32 v0:v1, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane writes the signed full product.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_MUL.WIDE.S32 v0:v1, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000392

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MAD.WIDE

- Family ID: v-mad-wide
- 语义组: wide_integer

Per-lane wide multiply-add.

V_MAD.WIDE.U32 — u32

- 执行域: vector
- 编码格式: V3
- 语义组: wide_integer
- (class, format, opcode): (VALU, 2, 1)
- Guard policy: optional
- Required state: none

语法: V_MAD.WIDE.U32 v0:v1, v0, v0, v0:v1

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	c	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—
c	vsrc64	read	vc	—

Semantics:

Each participating lane computes $\text{unsigned}(a) * \text{unsigned}(b) + c$ modulo 2^{64} .

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_MAD.WIDE.U32 v0:v1, v0, v0, v0:v1

示例字段值： —

64 位机器字： 0x00000000000000A2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_MAD.WIDE.S32 — s32

- 执行域：vector
- 编码格式：V3
- 语义组：wide_integer
- (class, format, opcode): (VALU, 2, 2)

- Guard policy: optional
- Required state: none

语法： V_MAD.WIDE.S32 v0:v1, v0, v0, v0:v1

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	c	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—
c	vsrc64	read	vc	—

Semantics:

Each participating lane computes $\text{signed}(a) * \text{signed}(b) + c$ modulo 2^{64} .

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_MAD.WIDE.S32 v0:v1, v0, v0, v0:v1

示例字段值： —

64 位机器字： 0x00000000000000122

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_INT_MISC

- Family ID: v-int-misc
- 语义组: integer_arithmetic

V integer min, max, abs, and neg.

V_MIN.S32 — min.s32

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 8)
- Guard policy: optional

- Required state: none

语法： V_MIN.S32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute S32 MIN of a and b.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例： V_MIN.S32 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000412

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MIN.U32 — min.u32

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 9)
- Guard policy: optional
- Required state: none

语法: V_MIN.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute U32 MIN of a and b.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_MIN.U32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000492

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MAX.S32 — max.s32

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 10)
- Guard policy: optional
- Required state: none

语法: V_MAX.S32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute S32 MAX of a and b.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_MAX.S32 v0, v0, v0

示例字段值: —

64 位机器字：0x00000000000000512

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_MAX.U32 — max.u32

- 执行域：vector
- 编码格式：V2
- 语义组：integer_arithmetic
- (class, format, opcode): (VALU, 1, 11)
- Guard policy: optional
- Required state: none

语法： V_MAX.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute U32 MAX of a and b.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_MAX.U32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000592

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_ABS.S32 — abs.s32

- 执行域: vector
- 编码格式: V1
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 0, 2)
- Guard policy: optional
- Required state: none

语法: V_ABS.S32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Compute two-complement signed 32-bit ABS.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_ABS.S32 v0, v0

示例字段值: —

64 位机器字: 0x0000000000000102

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_NEG.S32 — neg.s32

- 执行域: vector
- 编码格式: V1
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 0, 3)

- Guard policy: optional
- Required state: none

语法: V_NEG.S32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Compute two-complement signed 32-bit NEG.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_NEG.S32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000182

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_DIV_REM

- Family ID: v-div-rem
- 语义组: integer_arithmetic

Per-lane signed and unsigned division and remainder.

V_DIV.U32 — div.u

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 12)
- Guard policy: optional
- Required state: none

语法: V_DIV.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes $u \div v$.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- INTEGER_FAULT for participating lanes with zero divisor or signed overflow.

示例: V_DIV.U32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000612

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_DIV.S32 — div.s

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 13)
- Guard policy: optional
- Required state: none

语法: V_DIV.S32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes s div.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- INTEGER_FAULT for participating lanes with zero divisor or signed overflow.

示例: V_DIV.S32 v0, v0, v0

示例字段值: —

64 位机器字：0x00000000000000692

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_REM.U32 — rem.u

- 执行域：vector
- 编码格式：V2
- 语义组：integer_arithmetic
- (class, format, opcode): (VALU, 1, 14)
- Guard policy: optional
- Required state: none

语法： V_REM.U32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes $u \bmod v$.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- INTEGER_FAULT for participating lanes with zero divisor or signed overflow.

示例: `V_REM.U32 v0, v0, v0`

示例字段值: —

64 位机器字: 0x00000000000000712

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_REM.S32 — rem.s

- 执行域: vector
- 编码格式: V2
- 语义组: integer_arithmetic
- (class, format, opcode): (VALU, 1, 15)
- Guard policy: optional
- Required state: none

语法: V_REM.S32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes $s \text{ rem.}$

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- `INTEGER_FAULT` for participating lanes with zero divisor or signed overflow.

示例: `V_REM.S32 v0, v0, v0`

示例字段值: —

64 位机器字: `0x00000000000000792`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_BITWISE

- Family ID: v-bitwise
- 语义组: bit_manipulation

Per-lane Boolean operations.

V_AND.B32 — and

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 16)
- Guard policy: optional
- Required state: none

语法: V_AND.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane applies bitwise AND.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_AND.B32 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000812

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_OR.B32 — or

- 执行域： vector

- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 17)
- Guard policy: optional
- Required state: none

语法: V_OR.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane applies bitwise OR.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_OR.B32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000892

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_XOR.B32 — xor

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 18)
- Guard policy: optional
- Required state: none

语法: V_XOR.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值

ssrc_sel	标量源	说明
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane applies bitwise XOR.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_XOR.B32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000912

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_NOT.B32 — not

- 执行域: vector
- 编码格式: V1
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 0, 4)
- Guard policy: optional
- Required state: none

语法: V_NOT.B32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—

Semantics:

Each participating lane applies bitwise NOT.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_NOT.B32 v0, v0

示例字段值： —

64 位机器字： 0x0000000000000202

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_BREV.B32 — brev.b32

- 执行域： vector
- 编码格式： V1
- 语义组： bit_manipulation
- (class, format, opcode): (VALU, 0, 5)
- Guard policy: optional
- Required state: none

语法： V_BREV.B32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Reverse all 32 bits.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_BREV.B32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000282

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_BFE.B32 — bfe.b32

- 执行域: vector
- 编码格式: V3
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 2, 3)
- Guard policy: optional
- Required state: none

语法: V_BFE.B32 v0, v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值
2	offset	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	width	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—
offset	vsrc32	read	vb	—
width	vsrc32	read	vc	—

Semantics:

Extract width bits at offset with zero fill.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例： V_BFE.B32 v0, v0, v0, v0

示例字段值： —

64 位机器字： 0x000000000000001A2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_BFI.B32 — bfi.b32

- 执行域: vector
- 编码格式: V3
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 2, 4)
- Guard policy: optional
- Required state: none

语法: V_BFI.B32 v0, v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	base	va 改从 SGPR 文件读取, warp 内广播同一个值
2	insert	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	mask	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
base	vsrc32	read	va	—
insert	vsrc32	read	vb	—
mask	vsrc32	read	vc	—

Semantics:

Insert selected bits into base according to mask.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_BFI.B32 v0, v0, v0, v0

示例字段值: —

64 位机器字: 0x0000000000000222

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHIFT_ROTATE

- Family ID: v-shift-rotate
- 语义组: bit_manipulation

Per-lane shifts and rotates.

V_SHL.B32 — shl

- 执行域: vector
- 编码格式: VIMM
- 语义组: bit_manipulation

- (class, format, opcode): (VALU, 4, 8)
- Guard policy: optional
- Required state: none

语法: V_SHL.B32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
count	uimm24	read	imm24	—

Semantics:

Each participating lane applies SHL by count modulo 32.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SHL.B32 v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000442

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHR.B32 — shr

- 执行域: vector
- 编码格式: VIMM
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 4, 9)
- Guard policy: optional
- Required state: none

语法: V_SHR.B32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
count	uimm24	read	imm24	—

Semantics:

Each participating lane applies SHR by count modulo 32.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SHR.B32 v0, v0, 0

示例字段值: —

64 位机器字: 0x000000000000004C2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SAR.S32 — sar

- 执行域: vector
- 编码格式: VIMM
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 4, 10)
- Guard policy: optional
- Required state: none

语法: V_SAR.S32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
count	uimm24	read	imm24	—

Semantics:

Each participating lane applies SAR by count modulo 32.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_SAR.S32 v0, v0, 0

示例字段值： —

64 位机器字： 0x00000000000000542

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ROL.B32 — rotl

- 执行域： vector
- 编码格式： VIMM
- 语义组： bit_manipulation

- (class, format, opcode): (VALU, 4, 11)
- Guard policy: optional
- Required state: none

语法： V_ROL.B32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
count	uimm24	read	imm24	—

Semantics:

Each participating lane applies ROL by count modulo 32.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_ROL.B32 v0, v0, 0

示例字段值： —

64 位机器字： 0x000000000000005C2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ROR.B32 — rotr

- 执行域: vector
- 编码格式: VIMM
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 4, 12)
- Guard policy: optional
- Required state: none

语法: V_ROR.B32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
count	uimm24	read	imm24	—

Semantics:

Each participating lane applies ROR by count modulo 32.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ROR.B32 v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000642

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
imm24	58:35	—	否	—	Signed or unsigned opcode-defined 24-bit immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHL.B32 — shl.reg

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 35)
- Guard policy: optional
- Required state: none

语法: V_SHL.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

ssrc_sel	标量源	说明
2	count	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—
count	vsrc32	read	vb	—

Semantics:

Shift each lane left by the low 5 bits of the count source. Only the low 5 bits of the count are used.

Constraints:

- At most one of the two sources may name an SGPR, chosen by ssrc_sel.
- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SHL.B32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000001192

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	35	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHR.B32 — shr.reg

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 36)
- Guard policy: optional
- Required state: none

语法: V_SHR.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值
2	count	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—
count	vsrc32	read	vb	—

Semantics:

Shift each lane right logically by the low 5 bits of the count source. Only the low 5 bits of the count are used.

Constraints:

- At most one of the two sources may name an SGPR, chosen by `ssrc_sel`.
- All unused extension and reserved bits are zero.

Faults:

- `ILLEGAL_INSTRUCTION` on a reserved encoding; `ILLEGAL_OPERAND` on an invalid register or modifier.

示例: `V_SHR.B32 v0, v0, v0`

示例字段值: —

64 位机器字: `0x0000000000001212`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	36	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_SAR.S32 — sar.reg

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 37)
- Guard policy: optional
- Required state: none

语法: V_SAR.S32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值
2	count	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—
count	vsrc32	read	vb	—

Semantics:

Shift each lane right arithmetically by the low 5 bits of the count source. Only the low 5 bits of the count are used.

Constraints:

- At most one of the two sources may name an SGPR, chosen by ssrc_sel.
- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SAR.S32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000001292

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	37	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_ROL.B32 — rotl.reg

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 38)
- Guard policy: optional
- Required state: none

语法: V_ROL.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值
2	count	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—
count	vsrc32	read	vb	—

Semantics:

Rotate each lane left by the low 5 bits of the count source. Only the low 5 bits of the count are used.

Constraints:

- At most one of the two sources may name an SGPR, chosen by ssrc_sel.
- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ROL.B32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000001312

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	38	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_ROR.B32 — rotr.reg

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 1, 39)
- Guard policy: optional
- Required state: none

语法: V_ROR.B32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值
2	count	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

名称	类型	访问	字段	说明
count	vsrc32	read	vb	—

Semantics:

Rotate each lane right by the low 5 bits of the count source. Only the low 5 bits of the count are used.

Constraints:

- At most one of the two sources may name an SGPR, chosen by ssrc_sel.
- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_ROR.B32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000001392

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	39	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_BITCOUNT

- Family ID: v-bitcount
- 语义组: bit_manipulation

Per-lane bit counts including CTZ.

V_CLZ.B32 — clz

- 执行域: vector
- 编码格式: V1
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 0, 6)
- Guard policy: optional
- Required state: none

语法: V_CLZ.B32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Each participating lane writes its CLZ result; zero yields 32 for CLZ/CTZ.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_CLZ.B32 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000302

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CTZ.B32 — ctz

- 执行域： vector
- 编码格式： V1
- 语义组： bit_manipulation
- (class, format, opcode): (VALU, 0, 7)
- Guard policy: optional
- Required state: none

语法： V_CTZ.B32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Each participating lane writes its CTZ result; zero yields 32 for CLZ/CTZ.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CTZ.B32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000382

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_POPC.B32 — popc

- 执行域: vector
- 编码格式: V1
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 0, 8)
- Guard policy: optional
- Required state: none

语法: V_POPC.B32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Each participating lane writes its POPC result; zero yields 32 for CLZ/CTZ.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_POPC.B32 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000402

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_PACK

- Family ID: v-pack
- 语义组: bit_manipulation

Per-lane pack and unpack operations.

V_PACK.U16X2 — pack.u16x2

- 执行域: vector
- 编码格式: V2
- 语义组: bit_manipulation

- (class, format, opcode): (VALU, 1, 19)
- Guard policy: optional
- Required state: none

语法: V_PACK.U16X2 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	lo	va 改从 SGPR 文件读取, warp 内广播同一个值
2	hi	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
lo	vsrc32	read	va	—
hi	vsrc32	read	vb	—

Semantics:

Pack low 16-bit halves into one 32-bit value per lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_PACK.U16X2 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000992

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	19	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_UNPACK.LO16 — unpack.lo16

- 执行域: vector
- 编码格式: V1
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 0, 9)
- Guard policy: optional
- Required state: none

语法: V_UNPACK.LO16 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Zero-extend the low 16 bits per lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_UNPACK.LO16 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000482

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_UNPACK.HI16 — unpack.hi16

- 执行域: vector
- 编码格式: V1
- 语义组: bit_manipulation
- (class, format, opcode): (VALU, 0, 10)
- Guard policy: optional
- Required state: none

语法: V_UNPACK.HI16 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Zero-extend the high 16 bits per lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_UNPACK.HI16 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000502

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_COMPARE

- Family ID: v-compare
- 语义组: compare

Compare per-lane values and write a vector predicate.

V_CMP.EQ — equal

- 执行域: vector
- 编码格式: VCMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 0)
- Guard policy: optional
- Required state: none

语法: V_CMP.EQ vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件

ssrc_sel	标量源	说明
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Set each participating destination predicate bit to the equal comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.EQ vp0, v0, v0

示例字段值: —

64 位机器字: 0x0000000000000032

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.NE — not-equal

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 1)
- Guard policy: optional
- Required state: none

语法: V_CMP.NE vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsr32	read	vb	—

Semantics:

Set each participating destination predicate bit to the not-equal comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_CMP.NE vp0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.LT.S32 — signed-less

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 2)
- Guard policy: optional
- Required state: none

语法: V_CMP.LT.S32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Set each participating destination predicate bit to the signed-less comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.LT.S32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x0000000000000132

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.LE.S32 — signed-less-or-equal

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 3)
- Guard policy: optional
- Required state: none

语法: V_CMP.LE.S32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Set each participating destination predicate bit to the signed-less-or-equal comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.LE.S32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x000000000000001B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.LT.U32 — unsigned-less

- 执行域: vector
- 编码格式: VCMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 4)
- Guard policy: optional
- Required state: none

语法: V_CMP.LT.U32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Set each participating destination predicate bit to the unsigned-less comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.LT.U32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000232

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.LE.U32 — unsigned-less-or-equal

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 5)
- Guard policy: optional
- Required state: none

语法: V_CMP.LE.U32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsr32	read	vb	—

Semantics:

Set each participating destination predicate bit to the unsigned-less-or-equal comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.LE.U32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x000000000000002B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.EQ.F32 — ordered-equal_f32

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 6)
- Guard policy: optional
- Required state: none

语法: V_CMP.EQ.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Set each participating destination predicate bit to the ordered-equal F32 comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.EQ.F32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x0000000000000332

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.LT.F32 — ordered-less_f32

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 7)
- Guard policy: optional
- Required state: none

语法: V_CMP.LT.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Set each participating destination predicate bit to the ordered-less F32 comparison.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CMP.LT.F32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x000000000000003B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.NE.F32 — f32.ne

- 执行域: vector
- 编码格式: VCMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 8)
- Guard policy: optional
- Required state: none

语法: V_CMP.NE.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Ordered IEEE F32 NE updates only participating destination predicate bits.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.NE.F32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000432

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.LE.F32 — f32.le

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 9)
- Guard policy: optional
- Required state: none

语法: V_CMP.LE.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsrc32	read	vb	—

Semantics:

Ordered IEEE F32 LE updates only participating destination predicate bits.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例： V_CMP.LE.F32 vp0, v0, v0

示例字段值： —

64 位机器字： 0x000000000000004B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.GT.F32 — f32.gt

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 10)
- Guard policy: optional
- Required state: none

语法: V_CMP.GT.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Ordered IEEE F32 GT updates only participating destination predicate bits.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.GT.F32 vp0, v0, v0

示例字段值: —

64 位机器字：0x00000000000000532

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.GE.F32 — f32.ge

- 执行域：vector
- 编码格式：VCMP
- 语义组：compare
- (class, format, opcode): (VALU, 3, 11)
- Guard policy: optional

- Required state: none

语法： V_CMP.GE.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Ordered IEEE F32 GE updates only participating destination predicate bits.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例： V_CMP.GE.F32 vp0, v0, v0

示例字段值： —

64 位机器字： 0x000000000000005B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.GT.S32 — gt.s32

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 12)
- Guard policy: optional
- Required state: none

语法: V_CMP.GT.S32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Update participating predicate bits with S32 GT.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.GT.S32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000632

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.GE.S32 — ge.s32

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 13)
- Guard policy: optional
- Required state: none

语法: V_CMP.GE.S32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Update participating predicate bits with S32 GE.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.GE.S32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x000000000000006B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.GT.U32 — gt.u32

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 14)
- Guard policy: optional
- Required state: none

语法: V_CMP.GT.U32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Update participating predicate bits with U32 GT.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.GT.U32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000732

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.GE.U32 — ge.u32

- 执行域: vector
- 编码格式: VCMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 15)
- Guard policy: optional
- Required state: none

语法: V_CMP.GE.U32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Update participating predicate bits with U32 GE.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.GE.U32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x000000000000007B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.ORD.F32 — f32.ord

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 16)
- Guard policy: optional
- Required state: none

语法: V_CMP.ORD.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsr32	read	vb	—

Semantics:

Apply IEEE F32 ORD: NaN handling follows docs/05-numeric-environment.md.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.ORD.F32 vp0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000832

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_CMP.UNO.F32 — f32.uno

- 执行域: vector
- 编码格式: VCOMP
- 语义组: compare
- (class, format, opcode): (VALU, 3, 17)
- Guard policy: optional
- Required state: none

语法: V_CMP.UNO.F32 vp0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vpd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Apply IEEE F32 UNO: NaN handling follows docs/05-numeric-environment.md.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CMP.UNO.F32 vp0, v0, v0

示例字段值: —

64 位机器字： 0x000000000000008B2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vpd	22:19	—	否	—	Vector predicate destination vp0..vp15.
zero4	26:23	—	是	—	Must be zero.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_SELECT

- Family ID: v-select
- 语义组: select

Select per-lane sources according to a vector predicate.

V_SELECT.B32 — b32

- 执行域: vector
- 编码格式: V3
- 语义组: select
- (class, format, opcode): (VALU, 2, 5)
- Guard policy: optional
- Required state: none

语法: V_SELECT.B32 v0, v0, v0, vp0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	on_true	va 改从 SGPR 文件读取, warp 内广播同一个值
2	on_false	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	—	保留; 本形式没有绑定该字段

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
on_true	vsrc32	read	va	—
on_false	vsrc32	read	vb	—
condition	vpred	read	vc	—

Semantics:

Each participating lane selects on_true when its predicate bit is one.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SELECT.B32 v0, v0, v0, vp0

示例字段值: —

64 位机器字: 0x000000000000002A2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_SELECT.B64 — b64

- 执行域: vector
- 编码格式: V3
- 语义组: select
- (class, format, opcode): (VALU, 2, 6)
- Guard policy: optional
- Required state: none

语法: V_SELECT.B64 v0:v1, v0:v1, v0:v1, vp0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	on_true	va 改从 SGPR 文件读取, warp 内广播同一个值
2	on_false	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	—	保留; 本形式没有绑定该字段

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vd	—
on_true	vsrc64	read	va	—
on_false	vsrc64	read	vb	—
condition	vpred	read	vc	—

Semantics:

Each participating lane selects one 64-bit source.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_SELECT.B64 v0:v1, v0:v1, v0:v1, vp0

示例字段值: —

64 位机器字: 0x00000000000000322

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT

- Family ID: v-cvt
- 语义组: conversion

Per-lane integer and F32 conversion.

V_CVT.S32.F32 — s32.f32

- 执行域: vector
- 编码格式: V1
- 语义组: conversion
- (class, format, opcode): (VALU, 0, 11)
- Guard policy: optional
- Required state: none

语法: V_CVT.S32.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件

ssrc	标量源	说明
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Convert F32 to signed 32-bit integer per lane using round-to-zero.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CVT.S32.F32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000582

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT.U32.F32 — u32.f32

- 执行域: vector
- 编码格式: V1
- 语义组: conversion
- (class, format, opcode): (VALU, 0, 12)
- Guard policy: optional
- Required state: none

语法: V_CVT.U32.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Convert F32 to unsigned 32-bit integer per lane using round-to-zero.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_CVT.U32.F32 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000602

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT.F32.S32 — f32.s32

- 执行域： vector
- 编码格式： V1
- 语义组： conversion
- (class, format, opcode): (VALU, 0, 13)
- Guard policy: optional
- Required state: none

语法： V_CVT.F32.S32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Convert signed 32-bit integer to F32 per lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_CVT.F32.S32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000682

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT.F32.U32 — f32.u32

- 执行域: vector
- 编码格式: V1
- 语义组: conversion
- (class, format, opcode): (VALU, 0, 14)
- Guard policy: optional
- Required state: none

语法: V_CVT.F32.U32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Convert unsigned 32-bit integer to F32 per lane.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_CVT.F32.U32 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000702

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT.U16.U32 — u16.u32

- 执行域： vector
- 编码格式： V1
- 语义组： conversion
- (class, format, opcode): (VALU, 0, 15)
- Guard policy: optional
- Required state: none

语法： V_CVT.U16.U32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Truncate to low U16 then zero-extend to U32.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CVT.U16.U32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000782

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT.S16.S32 — s16.s32

- 执行域: vector
- 编码格式: V1
- 语义组: conversion
- (class, format, opcode): (VALU, 0, 16)
- Guard policy: optional
- Required state: none

语法: V_CVT.S16.S32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Truncate to low S16 then sign-extend to S32.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_CVT.S16.S32 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000802

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_F32_ARITH

- Family ID: v-f32-arith
- 语义组: floating_point

Per-lane IEEE binary32 arithmetic.

V_FADD.F32 — add

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 20)

- Guard policy: optional
- Required state: none

语法： V_FADD.F32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes IEEE binary32 ADD with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_FADD.F32 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000A12

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	20	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FSUB.F32 — sub

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 21)
- Guard policy: optional
- Required state: none

语法: V_FSUB.F32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes IEEE binary32 SUB with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FSUB.F32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000A92

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	21	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FMUL.F32 — mul

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 22)
- Guard policy: optional
- Required state: none

语法: V_FMUL.F32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes IEEE binary32 MUL with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_FMUL.F32 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000B12

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	22	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FFMA.F32 — fma

- 执行域: vector
- 编码格式: V3
- 语义组: floating_point
- (class, format, opcode): (VALU, 2, 7)
- Guard policy: optional
- Required state: none

语法: V_FFMA.F32 v0, v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	c	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—
c	vsrc32	read	vc	—

Semantics:

Each participating lane computes IEEE binary32 FMA with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FFMA.F32 v0, v0, v0, v0

示例字段值: —

64 位机器字: 0x000000000000003A2

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_FMIN.F32 — min

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 23)
- Guard policy: optional
- Required state: none

语法: V_FMIN.F32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes IEEE binary32 MIN with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FMIN.F32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000B92

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	23	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FMAX.F32 — max

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 24)
- Guard policy: optional
- Required state: none

语法: V_FMAX.F32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes IEEE binary32 MAX with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_FMAX.F32 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000C12

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	24	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FSQRT.F32 — sqrt

- 执行域: vector
- 编码格式: V1
- 语义组: floating_point
- (class, format, opcode): (VALU, 0, 17)
- Guard policy: optional
- Required state: none

语法: V_FSQRT.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—

Semantics:

Each participating lane computes IEEE binary32 SQRT with round-to-nearest-even.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FSQRT.F32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000882

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_FABS.F32 — abs.f32

- 执行域: vector
- 编码格式: V1
- 语义组: floating_point
- (class, format, opcode): (VALU, 0, 18)
- Guard policy: optional
- Required state: none

语法: V_FABS.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

IEEE F32 ABS by sign-bit transform, preserving NaN payload.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_FABS.F32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000902

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_FNEG.F32 — neg.f32

- 执行域: vector
- 编码格式: V1
- 语义组: floating_point
- (class, format, opcode): (VALU, 0, 19)
- Guard policy: optional
- Required state: none

语法: V_FNEG.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

IEEE F32 NEG by sign-bit transform, preserving NaN payload.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_FNEG.F32 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000982

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	19	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_F32_DIV

- Family ID: v-f32-div
- 语义组: floating_point

Per-lane binary32 division and reciprocal.

V_FDIV.F32 — div

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 25)
- Guard policy: optional
- Required state: none

语法: V_FDIV.F32 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件

ssrc_sel	标量源	说明
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Each participating lane computes IEEE binary32 a/b.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FDIV.F32 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000C92

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	25	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FRCP.F32 — rcp

- 执行域: vector
- 编码格式: V1
- 语义组: floating_point
- (class, format, opcode): (VALU, 0, 20)
- Guard policy: optional
- Required state: none

语法: V_FRCP.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—

Semantics:

Each participating lane computes IEEE binary32 reciprocal.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_FRCF.F32 v0, v0

示例字段值： —

64 位机器字： 0x0000000000000A02

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	20	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_F16_ARITH

- Family ID: v-f16-arith
- 语义组: floating_point

Per-lane IEEE binary16 arithmetic in packed low halves.

V_FADD.F16 — add

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 26)
- Guard policy: optional
- Required state: none

语法: V_FADD.F16 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute binary16 ADD from low 16-bit elements and zero the upper half.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FADD.F16 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000D12

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	26	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FSUB.F16 — sub

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 27)
- Guard policy: optional
- Required state: none

语法: V_FSUB.F16 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值

ssrc_sel	标量源	说明
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute binary16 SUB from low 16-bit elements and zero the upper half.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FSUB.F16 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000D92

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	27	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.

字段	位段	固定值	必须为零	保留值	说明
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FMUL.F16 — mul

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 28)
- Guard policy: optional
- Required state: none

语法: V_FMUL.F16 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute binary16 MUL from low 16-bit elements and zero the upper half.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_FMUL.F16 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000E12

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	28	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FFMA.F16 — fma

- 执行域: vector
- 编码格式: V3
- 语义组: floating_point
- (class, format, opcode): (VALU, 2, 8)
- Guard policy: optional
- Required state: none

语法: V_FFMA.F16 v0, v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值
3	c	vc 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—
c	vsrc32	read	vc	—

Semantics:

Compute binary16 FMA from low 16-bit elements and zero the upper half.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FFMA.F16 v0, v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000422

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
vc	50:43	—	否	—	Vector source C.
ssrc_sel	52:51	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 vc. At most one source may come from the SGPR file.
x11	63:53	—	是	—	Opcode-defined extension; unused bits are zero.

V_FMIN.F16 — min

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 29)
- Guard policy: optional
- Required state: none

语法: V_FMIN.F16 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Compute binary16 MIN from low 16-bit elements and zero the upper half.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_FMIN.F16 v0, v0, v0

示例字段值: —

64 位机器字: 0x00000000000000E92

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	29	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FMAX.F16 — max

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 30)
- Guard policy: optional
- Required state: none

语法: V_FMAX.F16 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—

名称	类型	访问	字段	说明
b	vsrc32	read	vb	—

Semantics:

Compute binary16 MAX from low 16-bit elements and zero the upper half.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_FMAX.F16 v0, v0, v0

示例字段值： —

64 位机器字： 0x00000000000000F12

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	30	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.

字段	位段	固定值	必须为零	保留值	说明
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FDIV.F16 — div

- 执行域: vector
- 编码格式: V2
- 语义组: floating_point
- (class, format, opcode): (VALU, 1, 31)
- Guard policy: optional
- Required state: none

语法: V_FDIV.F16 v0, v0, v0

Scalar source selector

ssrc_sel	标量源	说明
0	—	所有源都来自 VGPR 文件
1	a	va 改从 SGPR 文件读取, warp 内广播同一个值
2	b	vb 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
a	vsrc32	read	va	—
b	vsrc32	read	vb	—

Semantics:

Divide IEEE F16 low halves and zero the upper half.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_FDIV.F16 v0, v0, v0

示例字段值: —

64 位机器字：0x0000000000000F92

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	31	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	否	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_FSQRT.F16 — sqrt

- 执行域：vector
- 编码格式：V1
- 语义组：floating_point
- (class, format, opcode): (VALU, 0, 21)
- Guard policy: optional
- Required state: none

语法： V_FSQRT.F16 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Square root IEEE F16 low half and zero upper half.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_FSQRT.F16 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000A82

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	21	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_FABS.F16 — abs

- 执行域: vector
- 编码格式: V1
- 语义组: floating_point
- (class, format, opcode): (VALU, 0, 22)
- Guard policy: optional
- Required state: none

语法: V_FABS.F16 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Clear F16 sign and zero upper half.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_FABS.F16 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000B02

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	22	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_FNEG.F16 — neg

- 执行域： vector
- 编码格式： V1
- 语义组： floating_point
- (class, format, opcode): (VALU, 0, 23)
- Guard policy: optional
- Required state: none

语法： V_FNEG.F16 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Toggle F16 sign and zero upper half.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: V_FNEG.F16 v0, v0

示例字段值: —

64 位机器字: 0x00000000000000B82

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	23	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_F16_CVT

- Family ID: v-f16-cvt
- 语义组: conversion

Convert between IEEE binary16 and binary32.

V_CVT.F32.F16 — f32.f16

- 执行域: vector
- 编码格式: V1
- 语义组: conversion
- (class, format, opcode): (VALU, 0, 24)
- Guard policy: optional
- Required state: none

语法: V_CVT.F32.F16 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取, warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Convert src[15:0] binary16 to binary32.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_CVT.F32.F16 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000C02

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	24	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_CVT.F16.F32 — f16.f32

- 执行域： vector
- 编码格式： V1
- 语义组： conversion
- (class, format, opcode): (VALU, 0, 25)

- Guard policy: optional
- Required state: none

语法： V_CVT.F16.F32 v0, v0

Scalar source selector

ssrc	标量源	说明
0	—	所有源都来自 VGPR 文件
1	src	va 改从 SGPR 文件读取，warp 内广播同一个值

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vsrc32	read	va	—

Semantics:

Round binary32 to binary16 in dst[15:0] and zero dst[31:16].

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_CVT.F16.F32 v0, v0

示例字段值： —

64 位机器字： 0x00000000000000C82

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	25	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A.
ssrc	35:35	—	否	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

V_PRED_LOGIC

- Family ID: v-pred-logic
- 语义组: predicate

Boolean operations on vector predicates.

V_PAND — and

- 执行域: vector
- 编码格式: V2
- 语义组: predicate
- (class, format, opcode): (VALU, 1, 32)
- Guard policy: optional
- Required state: none

语法: V_PAND vp0, vp0, vp0

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vd	—
a	vpred	read	va	—
b	vpred	read	vb	—

Semantics:

Apply predicate AND independently to all lane bits. Only participating predicate bits are updated; non-participating bits retain their entry value.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_PAND vp0, vp0, vp0

示例字段值: —

64 位机器字: 0x00000000000001012

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	32	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	是	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_POR — or

- 执行域: vector
- 编码格式: V2

- 语义组: predicate
- (class, format, opcode): (VALU, 1, 33)
- Guard policy: optional
- Required state: none

语法: V_POR vp0, vp0, vp0

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vd	—
a	vpred	read	va	—
b	vpred	read	vb	—

Semantics:

Apply predicate OR independently to all lane bits. Only participating predicate bits are updated; non-participating bits retain their entry value.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: V_POR vp0, vp0, vp0

示例字段值: —

64 位机器字: 0x00000000000001092

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	33	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	是	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_PXOR — xor

- 执行域: vector
- 编码格式: V2
- 语义组: predicate
- (class, format, opcode): (VALU, 1, 34)
- Guard policy: optional
- Required state: none

语法: V_PXOR vp0, vp0, vp0

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vd	—
a	vpred	read	va	—
b	vpred	read	vb	—

Semantics:

Apply predicate XOR independently to all lane bits. Only participating predicate bits are updated; non-participating bits retain their entry value.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_PXOR vp0, vp0, vp0

示例字段值： —

64 位机器字： 0x0000000000001112

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	34	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.
vb	42:35	—	否	—	Vector source B.
ssrc_sel	44:43	—	是	—	Scalar-source selector: 0 none, 1 va, 2 vb, 3 reserved. At most one source may come from the SGPR file.
x19	63:45	—	是	—	Opcode-defined extension; unused bits are zero.

V_PNOT — not

- 执行域： vector
- 编码格式： V1
- 语义组： predicate
- (class, format, opcode): (VALU, 0, 26)
- Guard policy: optional

- Required state: none

语法： V_PNOT vp0, vp0

Operands

名称	类型	访问	字段	说明
dst	vpred	write	vd	—
a	vpred	read	va	—

Semantics:

Apply predicate NOT independently to all lane bits. Only participating predicate bits are updated; non-participating bits retain their entry value.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例： V_PNOT vp0, vp0

示例字段值： —

64 位机器字： 0x00000000000000D02

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	2	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	26	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A.

字段	位段	固定值	必须为零	保留值	说明
ssrc	35:35	—	是	—	Scalar-source selector: 0 reads va from the VGPR file, 1 reads it from the SGPR file.
x28	63:36	—	是	—	Opcode-defined extension; unused bits are zero.

S_LD

- Family ID: s-ld
- 语义组: memory_load

Uniform scalar loads across all architectural address spaces.

S_LD.GLOBAL.U32 — global.u32.base

- 执行域: scalar
- 编码格式: SMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 0, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_LD.GLOBAL.U32 s0, [s0:s1 + 0]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: sbase + sign_extend(simm24)
- 地址操作数: [sbase, simm24]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sdata	—

名称	类型	访问	字段	说明
base	address_global_uniform	read	sbase	—
offset	simm24	read	simm24	—

Semantics:

Load U32 at byte address sbase + sign_extend(simm24) in global space.

Constraints:

- Every byte the access covers must be mapped and permitted in the declared address space, and the address must satisfy natural alignment.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_LD.GLOBAL.U32 s0, [s0:s1 + 0]

示例字段值: —

64 位机器字: 0x0000000000000003

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_LD.GLOBAL.U64 — global.u64.base

- 执行域: scalar
- 编码格式: SMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 0, 1)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_LD.GLOBAL.U64 s0:s1, [s0:s1 + 0]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: sbase + sign_extend(simm24)
- 地址操作数: [sbase, simm24]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	sdata	—
base	address_global_uniform	read	sbase	—
offset	simm24	read	simm24	—

Semantics:

Load U64 at byte address sbase + sign_extend(simm24) in global space.

Constraints:

- Every byte the access covers must be mapped and permitted in the declared address space, and the address must satisfy natural alignment.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_LD.GLOBAL.U64 s0:s1, [s0:s1 + 0]

示例字段值： —

64 位机器字： 0x0000000000000083

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_LD.GLOBAL.U32 — global.u32.index

- 执行域： scalar
- 编码格式： SMEMX
- 语义组： memory_load
- (class, format, opcode): (MEMORY, 6, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_LD.GLOBAL.U32 s0, [s0:s1 + s0 + 0]

Address template

- 地址空间: global
- 地址模式: scalar_indexed
- 表达式: $\text{sbase} + \text{zero_extend}(\text{sindex}) + \text{sign_extend}(\text{imm16})$
- 地址操作数: [sbase, sindex, imm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sdata	—
base	address_global_uniform	read	sbase	—
index	sgpr_index	read	sindex	—
offset	simm16	read	imm16	—

Semantics:

Load U32 at byte address $\text{sbase} + \text{zero_extend}(\text{sindex}) + \text{sign_extend}(\text{imm16})$ in global space.

Constraints:

- Every byte the access covers must be mapped and permitted in the declared address space, and the address must satisfy natural alignment.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: `S_LD.GLOBAL.U32 s0, [s0:s1 + s0 + 0]`

示例字段值: —

64 位机器字: 0x0000000000000063

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	6	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar memory data.
sbase	34:27	—	否	—	Uniform base.
sindex	42:35	—	否	—	Scalar index.
imm16	58:43	—	否	—	Signed byte offset.
mods	63:59	—	是	—	Address modifiers.

S_LD.SHARED.U32 — shared.u32

- 执行域：scalar
- 编码格式：SMEM
- 语义组：memory_load
- (class, format, opcode): (MEMORY, 0, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_LD.SHARED.U32 s0, [s0 + 0]

Address template

- 地址空间：shared
- 地址模式：uniform_base
- 表达式：sbase + sign_extend(simm24)
- 地址操作数：[sbase, simm24]
- 偏移单位：bytes
- 缩放：1

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sdata	—
base	address_shared_uniform	read	sbase	—

名称	类型	访问	字段	说明
offset	simm24	read	simm24	—

Semantics:

Load U32 at byte address sbase + sign_extend(simm24) in shared space.

Constraints:

- Every byte the access covers must be mapped and permitted in the declared address space, and the address must satisfy natural alignment.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_LD.SHARED.U32 s0, [s0 + 0]

示例字段值： —

64 位机器字： 0x0000000000000103

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_LD.CONST.U32 — const.u32

- 执行域: scalar
- 编码格式: SMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 0, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_LD.CONST.U32 s0, [s0:s1 + 0]

Address template

- 地址空间: const
- 地址模式: uniform_base
- 表达式: sbase + sign_extend(simm24)
- 地址操作数: [sbase, simm24]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sdata	—
base	address_const_uniform	read	sbase	—
offset	simm24	read	simm24	—

Semantics:

Load U32 at byte address sbase + sign_extend(simm24) in const space.

Constraints:

- Every byte the access covers must be mapped and permitted in the declared address space, and the address must satisfy natural alignment.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_LD.CONST.U32 s0, [s0:s1 + 0]

示例字段值: —

64 位机器字：0x00000000000000183

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_LD.PARAM.U32 — param.u32

- 执行域：scalar
- 编码格式：SMEM
- 语义组：memory_load
- (class, format, opcode): (MEMORY, 0, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_LD.PARAM.U32 s0, [s0:s1 + 0]

Address template

- 地址空间：param
- 地址模式：uniform_base
- 表达式：sbase + sign_extend(simm24)
- 地址操作数：[sbase, simm24]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	sdata	—
base	address_param_uniform	read	sbase	—
offset	simm24	read	simm24	—

Semantics:

Load U32 at byte address $sbase + \text{sign_extend}(\text{simm24})$ in param space.

Constraints:

- Every byte the access covers must be mapped and permitted in the declared address space, and the address must satisfy natural alignment.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: `S_LD.PARAM.U32 s0, [s0:s1 + 0]`

示例字段值: —

64 位机器字: 0x0000000000000203

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.

字段	位段	固定值	必须为零	保留值	说明
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ST

- Family ID: s-st
- 语义组: memory_store

Uniform scalar stores with base, indexed, and mixed addressing.

S_ST.GLOBAL.U32 — global.u32.base

- 执行域: scalar
- 编码格式: SMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 0, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ST.GLOBAL.U32 [s0:s1 + 0], s0

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: sbase + sign_extend(simm24)
- 地址操作数: [sbase, simm24]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	sgpr32	read	sdata	—
base	address_global_uniform	read	sbase	—

名称	类型	访问	字段	说明
offset	simm24	read	simm24	—

Semantics:

Store U32 at byte address sbase + sign_extend(simm24) in global space.

Constraints:

- Const and param spaces are not valid store destinations; effective address must be naturally aligned.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_ST.GLOBAL.U32 [s0:s1 + 0], s0

示例字段值: —

64 位机器字: 0x00000000000000283

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ST.GLOBAL.U64 — global.u64.base

- 执行域: scalar
- 编码格式: SMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 0, 6)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ST.GLOBAL.U64 [s0:s1 + 0], s0:s1

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: sbase + sign_extend(simm24)
- 地址操作数: [sbase, simm24]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	sgpr64	read	sdata	—
base	address_global_uniform	read	sbase	—
offset	simm24	read	simm24	—

Semantics:

Store U64 at byte address sbase + sign_extend(simm24) in global space.

Constraints:

- Const and param spaces are not valid store destinations; effective address must be naturally aligned.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_ST.GLOBAL.U64 [s0:s1 + 0], s0:s1

示例字段值: —

64 位机器字：0x0000000000000303

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ST.GLOBAL.U32 — global.u32.index

- 执行域：scalar
- 编码格式：SMEMX
- 语义组：memory_store
- (class, format, opcode): (MEMORY, 6, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法：S_ST.GLOBAL.U32 [s0:s1 + s0 + 0], s0

Address template

- 地址空间：global
- 地址模式：scalar_indexed
- 表达式：sbase + zero_extend(sindex) + sign_extend(imm16)
- 地址操作数：[sbase, sindex, imm16]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	sgpr32	read	sdata	—
base	address_global_uni form	read	sbase	—
index	sgpr_index	read	sindex	—
offset	simm16	read	imm16	—

Semantics:

Store U32 at byte address $sbase + \text{zero_extend}(sindex) + \text{sign_extend}(imm16)$ in global space.

Constraints:

- Const and param spaces are not valid store destinations; effective address must be naturally aligned.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: `S_ST.GLOBAL.U32 [s0:s1 + s0 + 0], s0`

示例字段值: —

64 位机器字: 0x00000000000000163

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	6	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar memory data.

字段	位段	固定值	必须为零	保留值	说明
sbase	34:27	—	否	—	Uniform base.
sindex	42:35	—	否	—	Scalar index.
imm16	58:43	—	否	—	Signed byte offset.
mods	63:59	—	是	—	Address modifiers.

S_ST.SHARED.U32 — shared.u32

- 执行域: scalar
- 编码格式: SMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 0, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ST.SHARED.U32 [s0 + 0], s0

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: sbase + sign_extend(simm24)
- 地址操作数: [sbase, simm24]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	sgpr32	read	sdata	—
base	address_shared_uniform	read	sbase	—
offset	simm24	read	simm24	—

Semantics:

Store U32 at byte address sbase + sign_extend(simm24) in shared space.

Constraints:

- Const and param spaces are not valid store destinations; effective address must be naturally aligned.
- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- MISALIGNED_ACCESS on a participating address that is not naturally aligned; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on an address-space mismatch.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_ST.SHARED.U32 [s0 + 0], s0

示例字段值： —

64 位机器字： 0x0000000000000383

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdata	26:19	—	否	—	Scalar load destination or store source.
sbase	34:27	—	否	—	Uniform scalar address base.
simm24	58:35	—	否	—	Signed 24-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD

- Family ID: v-ld
- 语义组: memory_load

Per-lane vector loads including uniform-base plus lane-index addressing.

V_LD.GLOBAL.U32 — global.u32.uniform_base

- 执行域: vector

- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 0)
- Guard policy: optional
- Required state: none

语法: V_LD.GLOBAL.U32 v0, [s0:s1 + 0]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_global_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.GLOBAL.U32 v0, [s0:s1 + 0]

示例字段值: —

64 位机器字: 0x0000000000000013

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.GLOBAL.U32 — global.u32.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 1)
- Guard policy: optional
- Required state: none

语法: V_LD.GLOBAL.U32 v0, [v0:v1 + 0]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
address	address_global_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.GLOBAL.U32 v0, [v0:v1 + 0]

示例字段值: —

64 位机器字: 0x0000000000000093

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.GLOBAL.U32 — global.u32.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 2)
- Guard policy: optional
- Required state: none

语法: V_LD.GLOBAL.U32 v0, [s0:s1 + v0 + 0]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simmm16})$
- 地址操作数: [sbase, vaddr, simmm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simmm16	read	simmm16	—

Semantics:

Load U32 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simmm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_LD.GLOBAL.U32 v0, [s0:s1 + v0 + 0]

示例字段值： —

64 位机器字： 0x00000000000000113

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.GLOBAL.U64 — global.u64.uniform_base

- 执行域： vector
- 编码格式： VMEM
- 语义组： memory_load
- (class, format, opcode): (MEMORY, 1, 3)
- Guard policy： optional
- Required state： none

语法： V_LD.GLOBAL.U64 v0:v1, [s0:s1 + 0]

Address template

- 地址空间： global
- 地址模式： uniform_base
- 表达式： $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数： [sbase, simm16]
- 偏移单位： bytes
- 缩放： 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_global_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_LD.GLOBAL.U64 v0:v1, [s0:s1 + 0]

示例字段值： —

64 位机器字： 0x00000000000000193

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.GLOBAL.U64 — global.u64.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 4)
- Guard policy: optional
- Required state: none

语法: V_LD.GLOBAL.U64 v0:v1, [v0:v1 + 0]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
address	address_global_lane	read	vaddr	—

名称	类型	访问	字段	说明
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.GLOBAL.U64 v0:v1, [v0:v1 + 0]

示例字段值: —

64 位机器字: 0x00000000000000213

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.GLOBAL.U64 — global.u64.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 5)
- Guard policy: optional
- Required state: none

语法: V_LD.GLOBAL.U64 v0:v1, [s0:s1 + v0 + 0]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.GLOBAL.U64 v0:v1, [s0:s1 + v0 + 0]

示例字段值: —

64 位机器字: 0x0000000000000293

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.CONST.U32 — const.u32.uniform_base

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 12)
- Guard policy: optional
- Required state: none

语法: V_LD.CONST.U32 v0, [s0:s1 + 0]

Address template

- 地址空间: const
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(sbase) + \text{sign_extend}(simm16)$
- 地址操作数: [sbase, simm16]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_const_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: `V_LD.CONST.U32 v0, [s0:s1 + 0]`

示例字段值: —

64 位机器字: 0x00000000000000613

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.

字段	位段	固定值	必须为零	保留值	说明
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.CONST.U32 — const.u32.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 13)
- Guard policy: optional
- Required state: none

语法: V_LD.CONST.U32 v0, [v0:v1 + 0]

Address template

- 地址空间: const
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
address	address_const_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_LD.CONST.U32 v0, [v0:v1 + 0]

示例字段值： —

64 位机器字： 0x00000000000000693

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.CONST.U32 — const.u32.sv_mix

- 执行域： vector
- 编码格式： VMEM
- 语义组： memory_load
- (class, format, opcode): (MEMORY, 1, 14)
- Guard policy： optional
- Required state： none

语法： V_LD.CONST.U32 v0, [s0:s1 + v0 + 0]

Address template

- 地址空间: const
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_const_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.CONST.U32 v0, [s0:s1 + v0 + 0]

示例字段值: —

64 位机器字: 0x00000000000000713

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.CONST.U64 — const.u64.uniform_base

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 15)
- Guard policy: optional
- Required state: none

语法: V_LD.CONST.U64 v0:v1, [s0:s1 + 0]

Address template

- 地址空间: const
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_const_uniform	read	sbase	—

名称	类型	访问	字段	说明
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.CONST.U64 v0:v1, [s0:s1 + 0]

示例字段值: —

64 位机器字: 0x00000000000000793

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.CONST.U64 — const.u64.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 16)
- Guard policy: optional
- Required state: none

语法: V_LD.CONST.U64 v0:v1, [v0:v1 + 0]

Address template

- 地址空间: const
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
address	address_const_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.CONST.U64 v0:v1, [v0:v1 + 0]

示例字段值: —

64 位机器字: 0x00000000000000813

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.CONST.U64 — const.u64.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 17)
- Guard policy: optional
- Required state: none

语法: V_LD.CONST.U64 v0:v1, [s0:s1 + v0 + 0]

Address template

- 地址空间: const
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes

- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_const_unif orm	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using `sv_mix: unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16)`.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: `V_LD.CONST.U64 v0:v1, [s0:s1 + v0 + 0]`

示例字段值: —

64 位机器字: 0x00000000000000893

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.

字段	位段	固定值	必须为零	保留值	说明
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.PARAM.U32 — param.u32.uniform_base

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 18)
- Guard policy: optional
- Required state: none

语法: V_LD.PARAM.U32 v0, [s0:s1 + 0]

Address template

- 地址空间: param
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_param_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.PARAM.U32 v0, [s0:s1 + 0]

示例字段值: —

64 位机器字: 0x00000000000000913

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.PARAM.U32 — param.u32.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 19)
- Guard policy: optional

- Required state: none

语法： V_LD.PARAM.U32 v0, [v0:v1 + 0]

Address template

- 地址空间： param
- 地址模式： lane_address
- 表达式： unsigned(vaddr) + sign_extend(simm16)
- 地址操作数： [vaddr, simm16]
- 偏移单位： bytes
- 缩放： 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
address	address_param_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using lane_address: unsigned(vaddr) + sign_extend(simm16).

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_LD.PARAM.U32 v0, [v0:v1 + 0]

示例字段值： —

64 位机器字： 0x00000000000000993

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	19	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.PARAM.U32 — param.u32.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 20)
- Guard policy: optional
- Required state: none

语法: V_LD.PARAM.U32 v0, [s0:s1 + v0 + 0]

Address template

- 地址空间: param
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—

名称	类型	访问	字段	说明
base	address_param_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using `sv_mix: unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16)`.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: `V_LD.PARAM.U32 v0, [s0:s1 + v0 + 0]`

示例字段值: —

64 位机器字: `0x00000000000000A13`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	20	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.PARAM.U64 — param.u64.uniform_base

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 21)
- Guard policy: optional
- Required state: none

语法: V_LD.PARAM.U64 v0:v1, [s0:s1 + 0]

Address template

- 地址空间: param
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_param_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.PARAM.U64 v0:v1, [s0:s1 + 0]

示例字段值： —

64 位机器字： 0x00000000000000A93

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	21	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.PARAM.U64 — param.u64.lane_address

- 执行域： vector
- 编码格式： VMEM
- 语义组： memory_load
- (class, format, opcode): (MEMORY, 1, 22)
- Guard policy: optional
- Required state: none

语法： V_LD.PARAM.U64 v0:v1, [v0:v1 + 0]

Address template

- 地址空间： param

- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
address	address_param_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.PARAM.U64 v0:v1, [v0:v1 + 0]

示例字段值: —

64 位机器字: 0x00000000000000B13

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	22	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.PARAM.U64 — param.u64.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 1, 23)
- Guard policy: optional
- Required state: none

语法: V_LD.PARAM.U64 v0:v1, [s0:s1 + v0 + 0]

Address template

- 地址空间: param
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_param_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using `sv_mix: unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16)`.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: `V_LD.PARAM.U64 v0:v1, [s0:s1 + v0 + 0]`

示例字段值: —

64 位机器字: `0x00000000000000B93`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	23	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.SHARED.U32 — `shared.u32.uniform_base`

- 执行域: vector
- 编码格式: VSHMEM

- 语义组: memory_load
- (class, format, opcode): (MEMORY, 2, 0)
- Guard policy: optional
- Required state: none

语法: V_LD.SHARED.U32 v0, [s0 + 0]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_shared_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.SHARED.U32 v0, [s0 + 0]

示例字段值: —

64 位机器字: 0x0000000000000023

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	是	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.SHARED.U32 — shared.u32.lane_address

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 2, 1)
- Guard policy: optional
- Required state: none

语法: V_LD.SHARED.U32 v0, [v0 + 0]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
address	address_shared_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using lane_address: unsigned(vaddr) + sign_extend(simm16).

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.SHARED.U32 v0, [v0 + 0]

示例字段值: —

64 位机器字: 0x000000000000000A3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	是	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.SHARED.U32 — shared.u32.sv_mix

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 2, 2)
- Guard policy: optional
- Required state: none

语法: V_LD.SHARED.U32 v0, [s0 + v0 + 0]

Address template

- 地址空间: shared
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
base	address_shared_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.SHARED.U32 v0, [s0 + v0 + 0]

示例字段值: —

64 位机器字: 0x00000000000000123

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.SHARED.U64 — shared.u64.uniform_base

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 2, 3)
- Guard policy: optional
- Required state: none

语法： V_LD.SHARED.U64 v0:v1, [s0 + 0]

Address template

- 地址空间： shared
- 地址模式： uniform_base
- 表达式： unsigned(sbase) + sign_extend(simm16)
- 地址操作数： [sbase, simm16]
- 偏移单位： bytes
- 缩放： 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_shared_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using uniform_base: unsigned(sbase) + sign_extend(simm16).

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_LD.SHARED.U64 v0:v1, [s0 + 0]

示例字段值： —

64 位机器字： 0x000000000000001A3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	是	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.SHARED.U64 — shared.u64.lane_address

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 2, 4)
- Guard policy: optional
- Required state: none

语法: V_LD.SHARED.U64 v0:v1, [v0 + 0]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
address	address_shared_lane	read	vaddr	—

名称	类型	访问	字段	说明
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.SHARED.U64 v0:v1, [v0 + 0]

示例字段值: —

64 位机器字: 0x00000000000000223

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	是	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.SHARED.U64 — shared.u64.sv_mix

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 2, 5)
- Guard policy: optional
- Required state: none

语法: V_LD.SHARED.U64 v0:v1, [s0 + v0 + 0]

Address template

- 地址空间: shared
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
base	address_shared_unifrom	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_LD.SHARED.U64 v0:v1, [s0 + v0 + 0]

示例字段值: —

64 位机器字: 0x000000000000002A3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.LOCAL.U32 — local.u32.lane_address

- 执行域: vector
- 编码格式: VLMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 3, 0)
- Guard policy: optional
- Required state: none

语法: V_LD.LOCAL.U32 v0, [v0 + 0]

Address template

- 地址空间: local
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vdata	—
address	address_local_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U32 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$. Local space always addresses the current lane's private window.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: `V_LD.LOCAL.U32 v0, [v0 + 0]`

示例字段值: —

64 位机器字: 0x0000000000000033

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector local-memory data.
vaddr	34:27	—	否	—	Per-lane local-memory address.
sbase	42:35	—	是	—	Optional uniform base.

字段	位段	固定值	必须为零	保留值	说明
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_LD.LOCAL.U64 — local.u64.lane_address

- 执行域: vector
- 编码格式: VLMEM
- 语义组: memory_load
- (class, format, opcode): (MEMORY, 3, 1)
- Guard policy: optional
- Required state: none

语法: V_LD.LOCAL.U64 v0:v1, [v0 + 0]

Address template

- 地址空间: local
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
dst	vgpr64	write	vdata	—
address	address_local_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Load U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$. Local space always addresses the current lane's private window.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_LD.LOCAL.U64 v0:v1, [v0 + 0]

示例字段值： —

64 位机器字： 0x00000000000000B3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector local-memory data.
vaddr	34:27	—	否	—	Per-lane local-memory address.
sbase	42:35	—	是	—	Optional uniform base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST

- Family ID: v-st
- 语义组: memory_store

Per-lane vector stores including uniform-base plus lane-index addressing.

V_ST.GLOBAL.U32 — global.u32.uniform_base

- 执行域: vector
- 编码格式: VMEM

- 语义组: memory_store
- (class, format, opcode): (MEMORY, 1, 24)
- Guard policy: optional
- Required state: none

语法: V_ST.GLOBAL.U32 [s0:s1 + 0], v0

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
base	address_global_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Store U32 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.GLOBAL.U32 [s0:s1 + 0], v0

示例字段值: —

64 位机器字: 0x00000000000000C13

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	24	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.GLOBAL.U32 — global.u32.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 1, 25)
- Guard policy: optional
- Required state: none

语法: V_ST.GLOBAL.U32 [v0:v1 + 0], v0

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
address	address_global_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U32 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.GLOBAL.U32 [v0:v1 + 0], v0

示例字段值: —

64 位机器字: 0x00000000000000C93

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	25	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.GLOBAL.U32 — global.u32.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 1, 26)
- Guard policy: optional
- Required state: none

语法: V_ST.GLOBAL.U32 [s0:s1 + v0 + 0], v0

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simmm16})$
- 地址操作数: [sbase, vaddr, simmm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simmm16	read	simmm16	—

Semantics:

Store U32 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simmm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例：V_ST.GLOBAL.U32 [s0:s1 + v0 + 0], v0

示例字段值：—

64 位机器字：0x00000000000000D13

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	26	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.GLOBAL.U64 — global.u64.uniform_base

- 执行域：vector
- 编码格式：VMEM
- 语义组：memory_store
- (class, format, opcode): (MEMORY, 1, 27)
- Guard policy: optional
- Required state: none

语法： V_ST.GLOBAL.U64 [s0:s1 + 0], v0:v1

Address template

- 地址空间： global
- 地址模式： uniform_base
- 表达式： $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数： [sbase, simm16]
- 偏移单位： bytes
- 缩放： 1

Operands

名称	类型	访问	字段	说明
src	vgpr64	read	vdata	—
base	address_global_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Store U64 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_ST.GLOBAL.U64 [s0:s1 + 0], v0:v1

示例字段值： —

64 位机器字： 0x00000000000000D93

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	27	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	是	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.GLOBAL.U64 — global.u64.lane_address

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 1, 28)
- Guard policy: optional
- Required state: none

语法: V_ST.GLOBAL.U64 [v0:v1 + 0], v0:v1

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr64	read	vdata	—
address	address_global_lane	read	vaddr	—

名称	类型	访问	字段	说明
offset	simm16	read	simm16	—

Semantics:

Store U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.GLOBAL.U64 [v0:v1 + 0], v0:v1

示例字段值: —

64 位机器字: 0x00000000000000E13

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	28	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	是	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.GLOBAL.U64 — global.u64.sv_mix

- 执行域: vector
- 编码格式: VMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 1, 29)
- Guard policy: optional
- Required state: none

语法: V_ST.GLOBAL.U64 [s0:s1 + v0 + 0], v0:v1

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr64	read	vdata	—
base	address_global_uni form	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U64 per participating lane using sv_mix: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.GLOBAL.U64 [s0:s1 + v0 + 0], v0:v1

示例字段值: —

64 位机器字: 0x00000000000000E93

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	1	否	—	Class-local payload format.
opcode	12:7	29	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector load destination or store source.
vaddr	34:27	—	否	—	Per-lane byte address or index.
sbase	42:35	—	否	—	Optional uniform scalar base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.SHARED.U32 — shared.u32.uniform_base

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 2, 6)
- Guard policy: optional
- Required state: none

语法: V_ST.SHARED.U32 [s0 + 0], v0

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(sbase) + \text{sign_extend}(simm16)$
- 地址操作数: [sbase, simm16]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
base	address_shared_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Store U32 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.SHARED.U32 [s0 + 0], v0

示例字段值: —

64 位机器字: 0x00000000000000323

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	是	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.

字段	位段	固定值	必须为零	保留值	说明
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.SHARED.U32 — shared.u32.lane_address

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 2, 7)
- Guard policy: optional
- Required state: none

语法: V_ST.SHARED.U32 [v0 + 0], v0

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
address	address_shared_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U32 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_ST.SHARED.U32 [v0 + 0], v0

示例字段值： —

64 位机器字： 0x000000000000003A3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	是	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.SHARED.U32 — shared.u32.sv_mix

- 执行域： vector
- 编码格式： VSHMEM
- 语义组： memory_store
- (class, format, opcode): (MEMORY, 2, 8)
- Guard policy: optional
- Required state: none

语法： V_ST.SHARED.U32 [s0 + v0 + 0], v0

Address template

- 地址空间： shared
- 地址模式： sv_mix
- 表达式： unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16)
- 地址操作数： [sbase, vaddr, simm16]
- 偏移单位： bytes
- 缩放： 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
base	address_shared_un iform	read	sbase	—
index	vgpr_index	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U32 per participating lane using sv_mix: unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16).

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_ST.SHARED.U32 [s0 + v0 + 0], v0

示例字段值： —

64 位机器字： 0x00000000000000423

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.SHARED.U64 — shared.u64.uniform_base

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 2, 9)
- Guard policy: optional
- Required state: none

语法: V_ST.SHARED.U64 [s0 + 0], v0:v1

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [sbase, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr64	read	vdata	—

名称	类型	访问	字段	说明
base	address_shared_uniform	read	sbase	—
offset	simm16	read	simm16	—

Semantics:

Store U64 per participating lane using uniform_base: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.SHARED.U64 [s0 + 0], v0:v1

示例字段值: —

64 位机器字: 0x000000000000004A3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	是	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.SHARED.U64 — shared.u64.lane_address

- 执行域: vector
- 编码格式: VSHMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 2, 10)
- Guard policy: optional
- Required state: none

语法: V_ST.SHARED.U64 [v0 + 0], v0:v1

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr64	read	vdata	—
address	address_shared_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.SHARED.U64 [v0 + 0], v0:v1

示例字段值： —

64 位机器字： 0x0000000000000523

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	2	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector shared-memory data.
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	是	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.SHARED.U64 — shared.u64.sv_mix

- 执行域：vector
- 编码格式：VSHMEM
- 语义组：memory_store
- (class, format, opcode): (MEMORY, 2, 11)
- Guard policy: optional
- Required state: none

语法： V_ST.SHARED.U64 [s0 + v0 + 0], v0:v1

Address template

- 地址空间：shared

- 地址模式: `sv_mix`
- 表达式: `unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16)`
- 地址操作数: `[sbase, vaddr, simm16]`
- 偏移单位: `bytes`
- 缩放: `1`

Operands

名称	类型	访问	字段	说明
<code>src</code>	<code>vgpr64</code>	<code>read</code>	<code>vdata</code>	—
<code>base</code>	<code>address_shared_uniform</code>	<code>read</code>	<code>sbase</code>	—
<code>index</code>	<code>vgpr_index</code>	<code>read</code>	<code>vaddr</code>	—
<code>offset</code>	<code>simm16</code>	<code>read</code>	<code>simm16</code>	—

Semantics:

Store U64 per participating lane using `sv_mix`: `unsigned(sbase) + zero_extend(vaddr) + sign_extend(simm16)`.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- `MISALIGNED_ACCESS` on a participating misaligned address; `MEMORY_BOUNDS` on overflow or on an unmapped or permission-denied byte; `ILLEGAL_OPERAND` on address-space/type mismatch.

示例: `V_ST.SHARED.U64 [s0 + v0 + 0], v0:v1`

示例字段值: —

64 位机器字: `0x000000000000005A3`

编码字段

字段	位段	固定值	必须为零	保留值	说明
<code>class</code>	<code>3:0</code>	<code>3</code>	否	—	Execution class.
<code>format</code>	<code>6:4</code>	<code>2</code>	否	—	Class-local payload format.
<code>opcode</code>	<code>12:7</code>	<code>11</code>	否	—	Opcode local to class and format.
<code>guard</code>	<code>18:13</code>	—	否	—	Header lane guard; zero is PT.
<code>vdata</code>	<code>26:19</code>	—	否	—	Vector shared-memory data.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane shared-memory address or index.
sbase	42:35	—	否	—	Optional uniform shared-memory base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.LOCAL.U32 — local.u32.lane_address

- 执行域: vector
- 编码格式: VLMEM
- 语义组: memory_store
- (class, format, opcode): (MEMORY, 3, 2)
- Guard policy: optional
- Required state: none

语法: V_ST.LOCAL.U32 [v0 + 0], v0

Address template

- 地址空间: local
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr32	read	vdata	—
address	address_local_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U32 per participating lane using lane_address: unsigned(vaddr) + sign_extend(simm16). Local space always addresses the current lane's private window.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例： V_ST.LOCAL.U32 [v0 + 0], v0

示例字段值： —

64 位机器字： 0x00000000000000133

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector local-memory data.
vaddr	34:27	—	否	—	Per-lane local-memory addresses.
sbase	42:35	—	是	—	Optional uniform base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_ST.LOCAL.U64 — local.u64.lane_address

- 执行域： vector
- 编码格式： VLMEM

- 语义组: memory_store
- (class, format, opcode): (MEMORY, 3, 3)
- Guard policy: optional
- Required state: none

语法: V_ST.LOCAL.U64 [v0 + 0], v0:v1

Address template

- 地址空间: local
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$
- 地址操作数: [vaddr, simm16]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
src	vgpr64	read	vdata	—
address	address_local_lane	read	vaddr	—
offset	simm16	read	simm16	—

Semantics:

Store U64 per participating lane using lane_address: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm16})$. Local space always addresses the current lane's private window.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- MISALIGNED_ACCESS on a participating misaligned address; MEMORY_BOUNDS on overflow or on an unmapped or permission-denied byte; ILLEGAL_OPERAND on address-space/type mismatch.

示例: V_ST.LOCAL.U64 [v0 + 0], v0:v1

示例字段值: —

64 位机器字: 0x000000000000001B3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	3	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdata	26:19	—	否	—	Vector local-memory data.
vaddr	34:27	—	否	—	Per-lane local-memory address.
sbase	42:35	—	是	—	Optional uniform base.
simm16	58:43	—	否	—	Signed 16-bit byte offset.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_ATOM

- Family ID: s-atom
- 语义组: atomic

Scalar atomic load, store, RMW, and CAS.

S_ATOM.LOAD.U32.GLOBAL — load.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 0)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.LOAD.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.LOAD.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0]

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x2000000000000043

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	是	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.STORE.U32.GLOBAL — store.u32.global.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 1)

- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.STORE.U32.GLOBAL.{order}.{scope} [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.STORE.U32.GLOBAL.RELAXED.DEVICE [s0:s1 + 0], s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000000C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	是	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.ADD.U32.GLOBAL — add.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 2)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.ADD.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.
- **DIVERGENCE_FAULT** before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.ADD.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0`

示例字段值: `{order: 0, scope: 1}`

64 位机器字: `0x20000000000000143`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.

字段	位段	固定值	必须为零	保留值	说明
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XCHG.U32.GLOBAL — xchg.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.XCHG.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—

名称	类型	访问	字段	说明
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XCHG.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000001C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.

字段	位段	固定值	必须为零	保留值	说明
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.AND.U32.GLOBAL — and.u32.global.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_ATOM.AND.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间：global
- 地址模式：uniform_base
- 表达式：unsigned(sbase) + sign_extend(simm8)
- 地址操作数：[sbase, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uni form	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.AND.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000243

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.OR.U32.GLOBAL — or.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.OR.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.OR.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0`

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000002C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XOR.U32.GLOBAL — xor.u32.global.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic

- (class, format, opcode): (MEMORY, 4, 6)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.XOR.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XOR.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000343

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MIN.U32.GLOBAL — min.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.MIN.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.MIN.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000003C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcodedefine d extension; unused bit is zero.

S_ATOM.MAX.U32.GLOBAL — max.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 8)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.MAX.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—

名称	类型	访问	字段	说明
address	address_global_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.MAX.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000443

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.CAS.U32.GLOBAL — cas.u32.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 9)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.CAS.U32.GLOBAL.{order}.{scope} s0, [s0:s1 + 0], s0, s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global

- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [sbase, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_global_uniform	read	sbase	—
compare	sgpr32	read	sdata0	—
replacement	sgpr32	read	sdata1	—
offset	simmm8	read	simmm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.CAS.U32.GLOBAL.RELAXED.DEVICE s0, [s0:s1 + 0], s0, s0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000004C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	否	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.LOAD.U64.GLOBAL — load.u64.global.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 10)

- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.LOAD.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.LOAD.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0]

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000543

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	是	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.STORE.U64.GLOBAL — store.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 11)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.STORE.U64.GLOBAL.{order}.{scope} [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.

- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.STORE.U64.GLOBAL.RELAXED.DEVICE [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000005C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	是	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.ADD.U64.GLOBAL — add.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 12)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.ADD.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [sbase, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simmm8	read	simmm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.ADD.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000643

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XCHG.U64.GLOBAL — xchg.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 13)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.XCHG.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—

名称	类型	访问	字段	说明
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XCHG.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000006C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.AND.U64.GLOBAL — and.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 14)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.AND.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global

- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [sbase, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simmm8	read	simmm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.AND.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000743

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.OR.U64.GLOBAL — or.u64.global.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 15)
- Guard policy: required_pt

- Required state: scalar_ready

语法: S_ATOM.OR.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.OR.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000007C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XOR.U64.GLOBAL — xor.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 16)
- Guard policy: required_pt
- Required state: scalar_ready

语法: `S_ATOM.XOR.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.
- **DIVERGENCE_FAULT** before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.XOR.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1`

示例字段值: `{order: 0, scope: 1}`

64 位机器字: `0x20000000000000843`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.

字段	位段	固定值	必须为零	保留值	说明
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MIN.U64.GLOBAL — min.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 17)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.MIN.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—

名称	类型	访问	字段	说明
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.MIN.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000008C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.

字段	位段	固定值	必须为零	保留值	说明
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MAX.U64.GLOBAL — max.u64.global.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 18)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_ATOM.MAX.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间：global
- 地址模式：uniform_base
- 表达式：unsigned(sbase) + sign_extend(simm8)
- 地址操作数：[sbase, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uni form	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.MAX.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000943

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.CAS.U64.GLOBAL — cas.u64.global.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 19)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.CAS.U64.GLOBAL.{order}.{scope} s0:s1, [s0:s1 + 0], s0:s1, s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_global_uniform	read	sbase	—
compare	sgpr64	read	sdata0	—
replacement	sgpr64	read	sdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.CAS.U64.GLOBAL.RELAXED.DEVICE s0:s1, [s0:s1 + 0], s0:s1, s0:s1`

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000009C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	19	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	否	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.LOAD.U32.SHARED — load.u32.shared.satom

- 执行域：scalar
- 编码格式：SATOM

- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 20)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.LOAD.U32.SHARED.{order}.{scope} s0, [s0 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.LOAD.U32.SHARED.RELAXED.CTA s0, [s0 + 0]

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000A43

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	20	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	是	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.STORE.U32.SHARED — store.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 21)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.STORE.U32.SHARED.{order}.{scope} [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_shared_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.STORE.U32.SHARED.RELAXED.CTA [s0 + 0], s0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000AC3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	21	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	是	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.ADD.U32.SHARED — add.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 22)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.ADD.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [sbase, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—

名称	类型	访问	字段	说明
address	address_shared_unifform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.ADD.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000B43

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	22	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XCHG.U32.SHARED — xchg.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 23)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.XCHG.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared

- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XCHG.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000BC3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	23	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.AND.U32.SHARED — and.u32.shared.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 24)
- Guard policy: required_pt

- Required state: scalar_ready

语法: S_ATOM.AND.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.AND.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000C43

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	24	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.OR.U32.SHARED — or.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 25)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.OR.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.
- **DIVERGENCE_FAULT** before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.OR.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0`

示例字段值: `{order: 0, scope: 0}`

64 位机器字: `0x00000000000000CC3`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	25	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.

字段	位段	固定值	必须为零	保留值	说明
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XOR.U32.SHARED — xor.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 26)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.XOR.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—

名称	类型	访问	字段	说明
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XOR.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000D43

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	26	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.

字段	位段	固定值	必须为零	保留值	说明
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MIN.U32.SHARED — min.u32.shared.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 27)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_ATOM.MIN.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间：shared
- 地址模式：uniform_base
- 表达式：unsigned(sbase) + sign_extend(simm8)
- 地址操作数：[sbase, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_unifform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.MIN.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000DC3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	27	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MAX.U32.SHARED — max.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 28)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.MAX.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr32	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.MAX.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0`

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000E43

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	28	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.CAS.U32.SHARED — cas.u32.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic

- (class, format, opcode): (MEMORY, 4, 29)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.CAS.U32.SHARED.{order}.{scope} s0, [s0 + 0], s0, s0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr32	write	sdst	—
address	address_shared_uniform	read	sbase	—
compare	sgpr32	read	sdata0	—
replacement	sgpr32	read	sdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.

- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例： S_ATOM.CAS.U32.SHARED.RELAXED.CTA s0, [s0 + 0], s0, s0

示例字段值： {order: 0, scope: 0}

64 位机器字： 0x00000000000000EC3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	29	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	否	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.LOAD.U64.SHARED — load.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 30)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.LOAD.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.LOAD.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0]

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000F43

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	30	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	是	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.STORE.U64.SHARED — store.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 31)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.STORE.U64.SHARED.{order}.{scope} [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_shared_uniform	read	sbase	—

名称	类型	访问	字段	说明
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.STORE.U64.SHARED.RELAXED.CTA [s0 + 0], s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000FC3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	31	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	是	—	Scalar atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.ADD.U64.SHARED — add.u64.shared.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 32)
- Guard policy: required_pt
- Required state: scalar_ready

语法：S_ATOM.ADD.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间：shared

- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.ADD.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000001043

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	32	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XCHG.U64.SHARED — xchg.u64.shared.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 33)
- Guard policy: required_pt

- Required state: scalar_ready

语法: S_ATOM.XCHG.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XCHG.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x000000000000010C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	33	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.AND.U64.SHARED — and.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 34)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.AND.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.
- **DIVERGENCE_FAULT** before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.AND.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1`

示例字段值: `{order: 0, scope: 0}`

64 位机器字: `0x0000000000001143`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	34	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.

字段	位段	固定值	必须为零	保留值	说明
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.OR.U64.SHARED — or.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 35)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.OR.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—

名称	类型	访问	字段	说明
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.OR.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000011C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	35	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.

字段	位段	固定值	必须为零	保留值	说明
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.XOR.U64.SHARED — xor.u64.shared.satom

- 执行域：scalar
- 编码格式：SATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 4, 36)
- Guard policy: required_pt
- Required state: scalar_ready

语法： S_ATOM.XOR.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间：shared
- 地址模式：uniform_base
- 表达式：unsigned(sbase) + sign_extend(simm8)
- 地址操作数：[sbase, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_unifform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.XOR.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000001243

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	36	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MIN.U64.SHARED — min.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 37)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.MIN.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: `S_ATOM.MIN.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1`

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000012C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	37	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.MAX.U64.SHARED — max.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic

- (class, format, opcode): (MEMORY, 4, 38)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.MAX.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: $\text{unsigned}(\text{sbase}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
value	sgpr64	read	sdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.MAX.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000001343

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	38	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	是	—	Second scalar atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

S_ATOM.CAS.U64.SHARED — cas.u64.shared.satom

- 执行域: scalar
- 编码格式: SATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 4, 39)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_ATOM.CAS.U64.SHARED.{order}.{scope} s0:s1, [s0 + 0], s0:s1, s0:s1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: uniform_base
- 表达式: unsigned(sbase) + sign_extend(simm8)
- 地址操作数: [sbase, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	sgpr64	write	sdst	—
address	address_shared_uniform	read	sbase	—
compare	sgpr64	read	sdata0	—
replacement	sgpr64	read	sdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.
- DIVERGENCE_FAULT before reading address, order, scope, or data when the warp is not scalar-ready.

示例: S_ATOM.CAS.U64.SHARED.RELAXED.CTA s0:s1, [s0 + 0], s0:s1, s0:s1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000013C3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	4	否	—	Class-local payload format.
opcode	12:7	39	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
sdst	26:19	—	否	—	Scalar atomic old-value destination.
sbase	34:27	—	否	—	Uniform scalar atomic address base.
sdata0	42:35	—	否	—	First scalar atomic data source.
sdata1	50:43	—	否	—	Second scalar atomic data source for CAS.

字段	位段	固定值	必须为零	保留值	说明
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM

- Family ID: v-atom
- 语义组: atomic

Vector atomic operations including mixed addressing.

V_ATOM.LOAD.U32.GLOBAL — load.u32.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 0)
- Guard policy: optional
- Required state: none

语法: V_ATOM.LOAD.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.LOAD.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0]

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x2000000000000053

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	是	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.STORE.U32.GLOBAL — store.u32.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 1)
- Guard policy: optional
- Required state: none

语法: V_ATOM.STORE.U32.GLOBAL.{order}.{scope} [v0:v1 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.STORE.U32.GLOBAL.RELAXED.DEVICE [v0:v1 + 0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	是	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.ADD.U32.GLOBAL — add.u32.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 2)
- Guard policy: optional

- Required state: none

语法: `V_ATOM.ADD.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: `unsigned(vaddr) + sign_extend(simm8)`
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: `V_ATOM.ADD.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0`

示例字段值: {order: 0, scope: 1}

64 位机器字：0x2000000000000153

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XCHG.U32.GLOBAL — xchg.u32.global.vatom

- 执行域：vector

- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 3)
- Guard policy: optional
- Required state: none

语法: `V_ATOM.XCHG.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: `unsigned(vaddr) + sign_extend(simm8)`
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.

- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.XCHG.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x200000000000001D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.AND.U32.GLOBAL — and.u32.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 4)
- Guard policy: optional
- Required state: none

语法: V_ATOM.AND.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.AND.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x2000000000000253

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.OR.U32.GLOBAL — or.u32.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 5)
- Guard policy: optional
- Required state: none

语法: V_ATOM.OR.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [vaddr, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—

名称	类型	访问	字段	说明
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.OR.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x200000000000002D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XOR.U32.GLOBAL — xor.u32.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 6)
- Guard policy: optional
- Required state: none

语法： V_ATOM.XOR.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间：global

- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XOR.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000353

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MIN.U32.GLOBAL — min.u32.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 7)
- Guard policy: optional

- Required state: none

语法： V_ATOM.MIN.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.MIN.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0

示例字段值： {order: 0, scope: 1}

64 位机器字：0x200000000000003D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MAX.U32.GLOBAL — max.u32.global.vatom

- 执行域：vector

- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 8)
- Guard policy: optional
- Required state: none

语法: `V_ATOM.MAX.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: `unsigned(vaddr) + sign_extend(simm8)`
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.

- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.MAX.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000453

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.CAS.U32.GLOBAL — cas.u32.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 9)
- Guard policy: optional
- Required state: none

语法: V_ATOM.CAS.U32.GLOBAL.{order}.{scope} v0, [v0:v1 + 0], v0, v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_global_lane	read	vaddr	—
compare	vgpr32	read	vdata0	—
replacement	vgpr32	read	vdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.CAS.U32.GLOBAL.RELAXED.DEVICE v0, [v0:v1 + 0], v0, v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000004D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	否	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.LOAD.U64.GLOBAL — load.u64.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 10)
- Guard policy: optional
- Required state: none

语法: V_ATOM.LOAD.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—

名称	类型	访问	字段	说明
address	address_global_lane	read	vaddr	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.LOAD.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0]

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000553

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.

字段	位段	固定值	必须为零	保留值	说明
vdata0	42:35	—	是	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.STORE.U64.GLOBAL — store.u64.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 11)
- Guard policy: optional
- Required state: none

语法： V_ATOM.STORE.U64.GLOBAL.{order}.{scope} [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间：global
- 地址模式：lane_address
- 表达式：unsigned(vaddr) + sign_extend(simm8)
- 地址操作数：[vaddr, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.STORE.U64.GLOBAL.RELAXED.DEVICE [v0:v1 + 0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000005D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdst	26:19	—	是	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.ADD.U64.GLOBAL — add.u64.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 12)
- Guard policy: optional
- Required state: none

语法: V_ATOM.ADD.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.ADD.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000653

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XCHG.U64.GLOBAL — xchg.u64.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 13)

- Guard policy: optional
- Required state: none

语法: V_ATOM.XCHG.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XCHG.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000006D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.AND.U64.GLOBAL — and.u64.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 14)
- Guard policy: optional
- Required state: none

语法: V_ATOM.AND.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.

示例: `V_ATOM.AND.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1`

示例字段值: `{order: 0, scope: 1}`

64 位机器字: `0x20000000000000753`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.OR.U64.GLOBAL — or.u64.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 15)
- Guard policy: optional
- Required state: none

语法: V_ATOM.OR.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.OR.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000007D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XOR.U64.GLOBAL — xor.u64.global.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 16)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XOR.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [vaddr, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—

名称	类型	访问	字段	说明
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XOR.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000853

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MIN.U64.GLOBAL — min.u64.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 17)
- Guard policy: optional
- Required state: none

语法：V_ATOM.MIN.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间：global

- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.MIN.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000008D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MAX.U64.GLOBAL — max.u64.global.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 18)
- Guard policy: optional

- Required state: none

语法: `V_ATOM.MAX.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: `unsigned(vaddr) + sign_extend(simm8)`
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: `V_ATOM.MAX.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1`

示例字段值: {order: 0, scope: 1}

64 位机器字：0x20000000000000953

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.CAS.U64.GLOBAL — cas.u64.global.vatom

- 执行域：vector

- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 19)
- Guard policy: optional
- Required state: none

语法: `V_ATOM.CAS.U64.GLOBAL.{order}.{scope} v0:v1, [v0:v1 + 0], v0:v1, v0:v1`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: lane_address
- 表达式: `unsigned(vaddr) + sign_extend(simm8)`
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_global_lane	read	vaddr	—
compare	vgpr64	read	vdata0	—
replacement	vgpr64	read	vdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.

示例： `V_ATOM.CAS.U64.GLOBAL.RELAXED.DEVICE v0:v1, [v0:v1 + 0], v0:v1, v0:v1`

示例字段值： `{order: 0, scope: 1}`

64 位机器字： `0x200000000000009D3`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	19	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	否	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.LOAD.U32.SHARED — load.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 20)
- Guard policy: optional
- Required state: none

语法: V_ATOM.LOAD.U32.SHARED.{order}.{scope} v0, [v0 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.LOAD.U32.SHARED.RELAXED.CTA v0, [v0 + 0]

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000A53

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	20	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	是	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.STORE.U32.SHARED — store.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 21)
- Guard policy: optional
- Required state: none

语法: V_ATOM.STORE.U32.SHARED.{order}.{scope} [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_shared_lane	read	vaddr	—

名称	类型	访问	字段	说明
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.STORE.U32.SHARED.RELAXED.CTA [v0 + 0], v0

示例字段值： {order: 0, scope: 0}

64 位机器字： 0x00000000000000AD3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	21	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	是	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.

字段	位段	固定值	必须为零	保留值	说明
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.ADD.U32.SHARED — add.u32.shared.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 22)
- Guard policy: optional
- Required state: none

语法： V_ATOM.ADD.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间：shared
- 地址模式：lane_address
- 表达式：unsigned(vaddr) + sign_extend(simm8)
- 地址操作数：[vaddr, simm8]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.ADD.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000B53

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	22	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XCHG.U32.SHARED — xchg.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 23)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XCHG.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XCHG.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x0000000000000BD3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	23	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.AND.U32.SHARED — and.u32.shared.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 24)

- Guard policy: optional
- Required state: none

语法: V_ATOM.AND.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.AND.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000C53

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	24	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.OR.U32.SHARED — or.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 25)
- Guard policy: optional
- Required state: none

语法: V_ATOM.OR.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.

- **ILLEGAL_OPERAND** when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- **MISALIGNED_ACCESS** on any participating address that violates natural alignment.
- **MEMORY_BOUNDS** on overflow, or on an unmapped or permission-denied byte.

示例： `V_ATOM.OR.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0`

示例字段值： `{order: 0, scope: 0}`

64 位机器字： `0x00000000000000CD3`

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	25	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XOR.U32.SHARED — xor.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 26)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XOR.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XOR.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000D53

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	26	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MIN.U32.SHARED — min.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 27)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MIN.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [vaddr, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—

名称	类型	访问	字段	说明
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.MIN.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000DD3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	27	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MAX.U32.SHARED — max.u32.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 28)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MAX.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared

- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr32	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.MAX.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000E53

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	28	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.CAS.U32.SHARED — cas.u32.shared.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 29)
- Guard policy: optional

- Required state: none

语法: `V_ATOM.CAS.U32.SHARED.{order}.{scope} v0, [v0 + 0], v0, v0`

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: `unsigned(vaddr) + sign_extend(simm8)`
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
address	address_shared_lane	read	vaddr	—
compare	vgpr32	read	vdata0	—
replacement	vgpr32	read	vdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U32 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- `ILLEGAL_INSTRUCTION` when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- `ILLEGAL_OPERAND` when a defined, non-reserved order/scope value forms a combination outside `legal_orders/legal_scopes`, or when an address/data operand is invalid.
- `MISALIGNED_ACCESS` on any participating address that violates natural alignment.
- `MEMORY_BOUNDS` on overflow, or on an unmapped or permission-denied byte.

示例: `V_ATOM.CAS.U32.SHARED.RELAXED.CTA v0, [v0 + 0], v0, v0`

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000000ED3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	29	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	否	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.LOAD.U64.SHARED — load.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 30)
- Guard policy: optional
- Required state: none

语法: V_ATOM.LOAD.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.

- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.LOAD.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0]

示例字段值： {order: 0, scope: 0}

64 位机器字： 0x00000000000000F53

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	30	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	是	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.STORE.U64.SHARED — store.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 31)
- Guard policy: optional
- Required state: none

语法: V_ATOM.STORE.U64.SHARED.{order}.{scope} [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.STORE.U64.SHARED.RELAXED.CTA [v0 + 0], v0:v1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x0000000000000FD3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	31	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	是	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.ADD.U64.SHARED — add.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 32)
- Guard policy: optional
- Required state: none

语法: V_ATOM.ADD.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—

名称	类型	访问	字段	说明
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.ADD.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000001053

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	32	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XCHG.U64.SHARED — xchg.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 33)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XCHG.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared

- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XCHG.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x000000000000010D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	33	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.AND.U64.SHARED — and.u64.shared.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 34)
- Guard policy: optional

- Required state: none

语法： V_ATOM.AND.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.AND.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值： {order: 0, scope: 0}

64 位机器字：0x0000000000001153

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	34	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.OR.U64.SHARED — or.u64.shared.vatom

- 执行域：vector

- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 35)
- Guard policy: optional
- Required state: none

语法: V_ATOM.OR.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simmm8})$
- 地址操作数: [vaddr, simmm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simmm8	read	simmm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.

- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.OR.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值： {order: 0, scope: 0}

64 位机器字： 0x000000000000011D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	35	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.

字段	位段	固定值	必须为零	保留值	说明
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.XOR.U64.SHARED — xor.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 36)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XOR.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XOR.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x0000000000001253

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	36	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.

字段	位段	固定值	必须为零	保留值	说明
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MIN.U64.SHARED — min.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 37)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MIN.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—

名称	类型	访问	字段	说明
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.MIN.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值： {order: 0, scope: 0}

64 位机器字： 0x000000000000012D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	37	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.

字段	位段	固定值	必须为零	保留值	说明
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.MAX.U64.SHARED — max.u64.shared.vatom

- 执行域: vector
- 编码格式: VATOM
- 语义组: atomic
- (class, format, opcode): (MEMORY, 5, 38)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MAX.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared

- 地址模式: lane_address
- 表达式: $\text{unsigned}(\text{vaddr}) + \text{sign_extend}(\text{simm8})$
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
value	vgpr64	read	vdata0	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.MAX.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x00000000000001353

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	38	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	是	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.CAS.U64.SHARED — cas.u64.shared.vatom

- 执行域：vector
- 编码格式：VATOM
- 语义组：atomic
- (class, format, opcode): (MEMORY, 5, 39)
- Guard policy: optional

- Required state: none

语法： V_ATOM.CAS.U64.SHARED.{order}.{scope} v0:v1, [v0 + 0], v0:v1, v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA]

Address template

- 地址空间: shared
- 地址模式: lane_address
- 表达式: unsigned(vaddr) + sign_extend(simm8)
- 地址操作数: [vaddr, simm8]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
address	address_shared_lane	read	vaddr	—
compare	vgpr64	read	vdata0	—
replacement	vgpr64	read	vdata1	—
offset	simm8	read	simm8	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U64 atomic event in shared space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.CAS.U64.SHARED.RELAXED.CTA v0:v1, [v0 + 0], v0:v1, v0:v1

示例字段值: {order: 0, scope: 0}

64 位机器字: 0x000000000000013D3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	5	否	—	Class-local payload format.
opcode	12:7	39	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Vector atomic old-value destination.
vaddr	34:27	—	否	—	Per-lane atomic address or index.
vdata0	42:35	—	否	—	First vector atomic data source.
vdata1	50:43	—	否	—	Second vector atomic data source for CAS.
simm8	58:51	—	否	—	Signed 8-bit byte offset.
order	60:59	—	否	—	0 RELAXED, 1 ACQUIRE, 2 RELEASE, 3 ACQ_REL.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

V_ATOM.LOAD.U32.GLOBAL — load.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 0)
- Guard policy: optional
- Required state: none

语法: V_ATOM.LOAD.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.

- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.LOAD.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0]

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000073

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	是	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.STORE.U32.GLOBAL — store.u32.global.vatomx

- 执行域： vector
- 编码格式： VATOMX
- 语义组： atomic
- (class, format, opcode): (MEMORY, 7, 1)

- Guard policy: optional
- Required state: none

语法: `V_ATOM.STORE.U32.GLOBAL.{order}.{scope} [s0:s1 + v0], v0`

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: `V_ATOM.STORE.U32.GLOBAL.RELAXED.DEVICE [s0:s1 + v0], v0`

示例字段值: {order: 0, scope: 1}

64 位机器字： 0x20000000000000F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	是	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.ADD.U32.GLOBAL — add.u32.global.vatomx

- 执行域：vector
- 编码格式：VATOMX
- 语义组：atomic
- (class, format, opcode): (MEMORY, 7, 2)
- Guard policy: optional
- Required state: none

语法： V_ATOM.ADD.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.ADD.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x2000000000000173

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.XCHG.U32.GLOBAL — xchg.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 3)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XCHG.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XCHG.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000001F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.AND.U32.GLOBAL — and.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 4)
- Guard policy: optional
- Required state: none

语法: V_ATOM.AND.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.AND.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000273

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.OR.U32.GLOBAL — or.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 5)
- Guard policy: optional
- Required state: none

语法: V_ATOM.OR.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix

- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: $[\text{sbase}, \text{vindex}]$
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: `V_ATOM.OR.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0`

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000002F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.XOR.U32.GLOBAL — xor.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 6)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XOR.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]

- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uni form	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.XOR.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000373

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.MIN.U32.GLOBAL — min.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 7)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MIN.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.MIN.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000003F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.

字段	位段	固定值	必须为零	保留值	说明
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.MAX.U32.GLOBAL — max.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 8)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MAX.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—

名称	类型	访问	字段	说明
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
value	vgpr32	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.MAX.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x20000000000000473

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.

字段	位段	固定值	必须为零	保留值	说明
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.CAS.U32.GLOBAL — cas.u32.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 9)
- Guard policy: optional
- Required state: none

语法: V_ATOM.CAS.U32.GLOBAL.{order}.{scope} v0, [s0:s1 + v0], v0, v0

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr32	write	vdst	—

名称	类型	访问	字段	说明
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
compare	vgpr32	read	vdata0	—
replacement	vgpr32	read	vdata1	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U32 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.CAS.U32.GLOBAL.RELAXED.DEVICE v0, [s0:s1 + v0], v0, v0

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000004F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.

字段	位段	固定值	必须为零	保留值	说明
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.
vdata1	58:51	—	否	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.LOAD.U64.GLOBAL — load.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 10)
- Guard policy: optional
- Required state: none

语法: V_ATOM.LOAD.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0]

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—

名称	类型	访问	字段	说明
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one LOAD U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.LOAD.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0]

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000573

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.

字段	位段	固定值	必须为零	保留值	说明
vdata0	50:43	—	是	—	Atomic data zero.
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.STORE.U64.GLOBAL — store.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 11)
- Guard policy: optional
- Required state: none

语法: V_ATOM.STORE.U64.GLOBAL.{order}.{scope} [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, RELEASE]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
base	address_global_uniform	read	sbase	—

名称	类型	访问	字段	说明
index	vgpr_index	read	vindex	—
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one STORE U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.STORE.U64.GLOBAL.RELAXED.DEVICE [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x200000000000005F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	是	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.ADD.U64.GLOBAL — add.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 12)
- Guard policy: optional
- Required state: none

语法: V_ATOM.ADD.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one ADD U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.ADD.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000673

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.XCHG.U64.GLOBAL — xchg.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 13)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XCHG.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XCHG U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.XCHG.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x200000000000006F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.AND.U64.GLOBAL — and.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 14)
- Guard policy: optional
- Required state: none

语法: V_ATOM.AND.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one AND U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.AND.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000773

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	14	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.OR.U64.GLOBAL — or.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 15)
- Guard policy: optional
- Required state: none

语法: V_ATOM.OR.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one OR U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.OR.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000007F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	15	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.XOR.U64.GLOBAL — xor.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 16)
- Guard policy: optional
- Required state: none

语法: V_ATOM.XOR.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one XOR U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.XOR.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000873

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	16	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.MIN.U64.GLOBAL — min.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 17)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MIN.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MIN U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.MIN.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x200000000000008F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	17	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.MAX.U64.GLOBAL — max.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 18)
- Guard policy: optional
- Required state: none

语法: V_ATOM.MAX.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
value	vgpr64	read	vdata0	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one MAX U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例： V_ATOM.MAX.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1

示例字段值： {order: 0, scope: 1}

64 位机器字： 0x20000000000000973

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	18	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	是	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

V_ATOM.CAS.U64.GLOBAL — cas.u64.global.vatomx

- 执行域: vector
- 编码格式: VATOMX
- 语义组: atomic
- (class, format, opcode): (MEMORY, 7, 19)
- Guard policy: optional
- Required state: none

语法: V_ATOM.CAS.U64.GLOBAL.{order}.{scope} v0:v1, [s0:s1 + v0], v0:v1, v0:v1

Atomic modifiers

- Legal orders: [RELAXED, ACQUIRE, RELEASE, ACQ_REL]
- Legal scopes: [CTA, DEVICE, SYSTEM]

Address template

- 地址空间: global
- 地址模式: sv_mix
- 表达式: $\text{unsigned}(\text{sbase}) + \text{zero_extend}(\text{vindex}) * \text{scale}$
- 地址操作数: [sbase, vindex]
- 偏移单位: bytes
- 缩放: 1

Operands

名称	类型	访问	字段	说明
old	vgpr64	write	vdst	—
base	address_global_uniform	read	sbase	—
index	vgpr_index	read	vindex	—

名称	类型	访问	字段	说明
compare	vgpr64	read	vdata0	—
replacement	vgpr64	read	vdata1	—
order	atomic_order	control	order	—
scope	memory_scope	control	scope	—

Semantics:

Perform one CAS U64 atomic event in global space using runtime-selected legal order and scope.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION when the encoded scope field is reserved value 3; reserved decoding precedes legal-matrix validation.
- ILLEGAL_OPERAND when a defined, non-reserved order/scope value forms a combination outside legal_orders/legal_scopes, or when an address/data operand is invalid.
- MISALIGNED_ACCESS on any participating address that violates natural alignment.
- MEMORY_BOUNDS on overflow, or on an unmapped or permission-denied byte.

示例: V_ATOM.CAS.U64.GLOBAL.RELAXED.DEVICE v0:v1, [s0:s1 + v0], v0:v1, v0:v1

示例字段值: {order: 0, scope: 1}

64 位机器字: 0x200000000000009F3

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	3	否	—	Execution class.
format	6:4	7	否	—	Class-local payload format.
opcode	12:7	19	否	—	Opcode local to class and format.
guard	18:13	—	否	—	Header lane guard; zero is PT.
vdst	26:19	—	否	—	Old-value destination.
sbase	34:27	—	否	—	Uniform base.
vindex	42:35	—	否	—	Per-lane index.
vdata0	50:43	—	否	—	Atomic data zero.

字段	位段	固定值	必须为零	保留值	说明
vdata1	58:51	—	否	—	CAS replacement.
order	60:59	—	否	—	Atomic order.
scope	62:61	—	否	[3]	0 CTA, 1 DEVICE, 2 SYSTEM; 3 reserved.
x	63:63	—	是	—	Must be zero unless defined.

SSY

- Family ID: ssy
- 语义组: structured_control

Push a structured reconvergence token.

SSY — direct

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: structured_control
- (class, format, opcode): (CONTROL, 0, 0)
- Guard policy: required_pt
- Required state: none

语法: SSY 0

Operands

名称	类型	访问	字段	说明
target	disp30	control	disp30	—
reconvergence_stack	implicit_state	read_write	—	—
call_stack	call_stack	read	—	—

Semantics:

Validate the JOIN target and stack capacity, then push a reconvergence frame whose owner_call_depth is exactly call_stack.depth at SSY execution; also record target, current active mask, empty pending/arrived masks, and phase=ARMED.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

- disp30 is a signed instruction-word displacement: $\text{target_pc} = \text{next_pc} + (\text{sign_extend_30}(\text{disp30}) < < 3)$.
- The new frame sets $\text{owner_call_depth} = \text{call_stack.depth}$; SSY reads but never modifies call_stack .

Faults:

- ILLEGAL_OPERAND on an unaligned or out-of-text target; RECONVERGENCE_FAULT on a control-stack protocol violation.

示例: SSY 0

示例字段值: —

64 位机器字: 0x0000000000000004

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	否	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

BRA

- Family ID: bra

- 语义组: unstructured_control

Direct PC-relative branch.

BRA — direct

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: unstructured_control
- (class, format, opcode): (CONTROL, 0, 1)
- Guard policy: required_pt
- Required state: none

语法: BRA 0

Operands

名称	类型	访问	字段	说明
target	disp30	control	disp30	—

Semantics:

Jump to next_pc + (sign_extend_30(disp30) << 3) without scalar-ready or call-stack effects.

Constraints:

- All unused extension and reserved bits are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid operand.

示例: BRA 0

示例字段值: —

64 位机器字: 0x00000000000000084

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
disp30	48:19	—	否	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

BRA.P

- Family ID: bra-p
- 语义组: structured_control

Conditionally branch the guarded lane subset.

BRA.P — direct

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: structured_control
- (class, format, opcode): (CONTROL, 0, 2)
- Guard policy: explicit_condition
- Required state: none

语法: BRA.P PT, 0

Operands

名称	类型	访问	字段	说明
target	disp30	control	disp30	—
active_mask	implicit_state	read_write	—	—
condition	pred_cond	control	cond6	—

Semantics:

Branch guarded lanes to target and retain the complement for structured reconvergence.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.
- disp30 is a signed instruction-word displacement: $\text{target_pc} = \text{next_pc} + (\text{sign_extend_30}(\text{disp30}) \ll 3)$.

Faults:

- ILLEGAL_OPERAND on an unaligned or out-of-text target; RECONVERGENCE_FAULT on a control-stack protocol violation.

示例: BRA.P PT, 0

示例字段值: —

64 位机器字: 0x00000000000000104

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	否	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	否	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

JOIN

- Family ID: join
- 语义组: structured_control

Reconverge at the top SSY target.

JOIN — base

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: structured_control
- (class, format, opcode): (CONTROL, 0, 3)
- Guard policy: required_pt
- Required state: none

语法: JOIN

Operands

名称	类型	访问	字段	说明
reconvergence_stack	implicit_state	read_write	—	—

Semantics:

Require PC == top.reconv_pc. ARMED: require active_mask==entry_mask, pop, advance. FIRST: save active_mask to arrived_mask, set SECOND, restore pending_mask&live_mask, clear pending_mask, jump pending_pc. SECOND: require arrived_mask|active_mask==entry_mask, pop, restore the union masked by live_mask, advance. Any empty-stack, PC, phase, or mask mismatch raises RECONVERGENCE_FAULT atomically.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_OPERAND on an unaligned or out-of-text target; RECONVERGENCE_FAULT on a control-stack protocol violation.

示例: JOIN

示例字段值: —

64 位机器字: 0x00000000000000184

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.

字段	位段	固定值	必须为零	保留值	说明
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	是	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

EXIT

- Family ID: exit
- 语义组: structured_control

Permanently retire guarded lanes.

EXIT — base

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: structured_control
- (class, format, opcode): (CONTROL, 0, 4)
- Guard policy: explicit_condition
- Required state: none

语法: EXIT PT

Operands

名称	类型	访问	字段	说明
live_mask	implicit_state	read_write	—	—
condition	pred_cond	control	cond6	—

Semantics:

Retire every participating lane. Each retired lane is atomically removed from the CTA's live_owner_set, which can complete a barrier that the remaining owners are waiting on. EXIT itself performs no shared release.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- ILLEGAL_OPERAND on an unaligned or out-of-text target; RECONVERGENCE_FAULT on a control-stack protocol violation.

示例：EXIT PT

示例字段值：—

64 位机器字：0x00000000000000204

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	是	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	否	—	Data condition encoded as PT, !PT, vpN, or !vpN.

字段	位段	固定值	必须为零	保留值	说明
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

CALL

- Family ID: call
- 语义组: call_return

Call a direct or indirect subroutine.

CALL — direct

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: call_return
- (class, format, opcode): (CONTROL, 0, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: CALL 0

Operands

名称	类型	访问	字段	说明
target	disp30	control	disp30	—
call_stack	call_stack	read_write	—	—

Semantics:

Require scalar_ready and $\text{call_stack.depth} < \text{descriptor.call_stack_depth} \leq \text{architectural_limits.call_stack_depth}$; push only $\text{return_pc} = \text{PC} + 8$, preserving any caller ARMED reconvergence frames unchanged; then transfer to the disp30 PC-relative target. No control context is stored in the call frame.

Constraints:

- $\text{descriptor.call_stack_depth}$ is in 0..16 and call_stack.depth is strictly below it before push.
- Caller reconvergence frames, including ARMED frames with lower owner_call_depth , are allowed and remain untouched.

Faults:

- DIVERGENCE_FAULT before reading target or stack state when not scalar-ready.
- RECONVERGENCE_FAULT when descriptor.call_stack_depth is exceeded; ILLEGAL_OPERAND on an invalid target.

示例：CALL 0

示例字段值：—

64 位机器字：0x0000000000000284

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	否	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

CALL.IND — indirect

- 执行域：warp_control
- 编码格式：CTRL
- 语义组：call_return
- (class, format, opcode): (CONTROL, 0, 6)

- Guard policy: required_pt
- Required state: scalar_ready

语法: CALL.IND s0:s1

Operands

名称	类型	访问	字段	说明
target	sgpr64	control	aux8	—
call_stack	call_stack	read_write	—	—

Semantics:

Require scalar_ready and $\text{call_stack.depth} < \text{descriptor.call_stack_depth} \leq \text{architectural_limits.call_stack_depth}$; push only $\text{return_pc} = \text{PC} + 8$, preserving any caller ARMED reconvergence frames unchanged; then transfer to the aligned in-text SGPR64 target. No control context is stored in the call frame.

Constraints:

- $\text{descriptor.call_stack_depth}$ is in 0..16 and call_stack.depth is strictly below it before push.
- Caller reconvergence frames, including ARMED frames with lower owner_call_depth , are allowed and remain untouched.

Faults:

- DIVERGENCE_FAULT before reading target or stack state when not scalar-ready.
- RECONVERGENCE_FAULT when $\text{descriptor.call_stack_depth}$ is exceeded; ILLEGAL_OPERAND on an invalid target.

示例: CALL.IND s0:s1

示例字段值: —

64 位机器字: 0x00000000000000304

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
disp30	48:19	—	是	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	否	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

RET

- Family ID: ret
- 语义组: call_return

Return from a subroutine.

RET — base

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: call_return
- (class, format, opcode): (CONTROL, 0, 7)
- Guard policy: required_pt
- Required state: scalar_ready

语法: RET

Operands

名称	类型	访问	字段	说明
call_stack	call_stack	read_write	—	—

Semantics:

Require scalar_ready and a nonempty call stack. Let depth=call_stack.depth. Reject only if a reconvergence frame exists with owner_call_depth == depth; caller frames with smaller owner_call_depth, including ARMED frames, are allowed. Then read top.return_pc, pop exactly that

return_pc entry, and set PC to it.

Constraints:

- RET does not reject or pop caller frames with owner_call_depth < call_stack.depth.
- The call frame contains only return_pc.

Faults:

- DIVERGENCE_FAULT before reading stack state when not scalar-ready.
- RECONVERGENCE_FAULT on empty call stack or any unclosed callee frame whose owner_call_depth equals current call_stack.depth.

示例: RET

示例字段值: —

64 位机器字: 0x00000000000000384

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	是	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	是	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

JUMP.IND

- Family ID: jump-ind
- 语义组: unstructured_control

Indirect scalar-ready jump.

JUMP.IND — indirect

- 执行域: warp_control
- 编码格式: CTRL
- 语义组: unstructured_control
- (class, format, opcode): (CONTROL, 0, 8)
- Guard policy: required_pt
- Required state: scalar_ready

语法: JUMP.IND s0:s1

Operands

名称	类型	访问	字段	说明
target	sgpr64	control	aux8	—

Semantics:

After scalar-ready validation, jump to the aligned in-text SGPR target without changing the call stack.

Constraints:

- All unused extension and reserved bits are zero.
- The warp must be scalar-ready before reading any dynamic source or hidden state.

Faults:

- DIVERGENCE_FAULT before reading target; ILLEGAL_OPERAND on an invalid target.

示例: JUMP.IND s0:s1

示例字段值: —

64 位机器字: 0x00000000000000404

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	4	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
disp30	48:19	—	是	—	Signed instruction-word displacement relative to next_pc.
cond6	54:49	—	是	—	Data condition encoded as PT, !PT, vpN, or !vpN.
aux8	62:55	—	否	—	Opcode-defined auxiliary register or control value.
x1	63:63	—	是	—	Opcode-defined extension; unused bit is zero.

FENCE

- Family ID: fence
- 语义组: memory_ordering

Order memory accesses at a selected scope.

FENCE.CTA — cta

- 执行域: cta_sync
- 编码格式: SYNC
- 语义组: memory_ordering
- (class, format, opcode): (SYNC, 0, 0)
- Guard policy: required_pt
- Required state: none

语法: FENCE.CTA

Operands

名称	类型	访问	字段	说明
memory_state	implicit_state	read_write	—	—

Semantics:

Complete prior accesses and order later accesses at CTA scope.

Constraints:

- All active lanes execute the same scope; control and register fields are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: FENCE.CTA

示例字段值: —

64 位机器字: 0x0300000000000005

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	5	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	是	—	Opcode-defined register A.
b	34:27	—	是	—	Opcode-defined register B.
imm16	50:35	—	是	—	Opcode-defined 16-bit immediate.
slot3	53:51	—	是	—	Barrier slot 0..7.
scope2	55:54	0	否	—	Memory scope.
order2	57:56	3	否	—	Memory order.
x6	63:58	—	是	—	Opcode-defined extension; unused bits are zero.

FENCE.DEVICE — device

- 执行域: cta_sync
- 编码格式: SYNC

- 语义组: memory_ordering
- (class, format, opcode): (SYNC, 0, 1)
- Guard policy: required_pt
- Required state: none

语法: FENCE.DEVICE

Operands

名称	类型	访问	字段	说明
memory_state	implicit_state	read_write	—	—

Semantics:

Complete prior accesses and order later accesses at DEVICE scope.

Constraints:

- All active lanes execute the same scope; control and register fields are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: FENCE.DEVICE

示例字段值: —

64 位机器字: 0x0340000000000085

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	5	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	是	—	Opcode-defined register A.
b	34:27	—	是	—	Opcode-defined register B.
imm16	50:35	—	是	—	Opcode-defined 16-bit immediate.

字段	位段	固定值	必须为零	保留值	说明
slot3	53:51	—	是	—	Barrier slot 0..7.
scope2	55:54	1	否	—	Memory scope.
order2	57:56	3	否	—	Memory order.
x6	63:58	—	是	—	Opcode-defined extension; unused bits are zero.

FENCE.SYSTEM — system

- 执行域: cta_sync
- 编码格式: SYNC
- 语义组: memory_ordering
- (class, format, opcode): (SYNC, 0, 2)
- Guard policy: required_pt
- Required state: none

语法: FENCE.SYSTEM

Operands

名称	类型	访问	字段	说明
memory_state	implicit_state	read_write	—	—

Semantics:

Complete prior accesses and order later accesses at SYSTEM scope.

Constraints:

- All active lanes execute the same scope; control and register fields are zero.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register or modifier.

示例: FENCE.SYSTEM

示例字段值: —

64 位机器字: 0x0380000000000105

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	5	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.

字段	位段	固定值	必须为零	保留值	说明
opcode	12:7	2	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	是	—	Opcode-defined register A.
b	34:27	—	是	—	Opcode-defined register B.
imm16	50:35	—	是	—	Opcode-defined 16-bit immediate.
slot3	53:51	—	是	—	Barrier slot 0..7.
scope2	55:54	2	否	—	Memory scope.
order2	57:56	3	否	—	Memory order.
x6	63:58	—	是	—	Opcode-defined extension; unused bits are zero.

BAR.SYNC

- Family ID: bar-sync
- 语义组: barrier

Whole-CTA barrier with release/acquire ordering.

BAR.SYNC.CTA — cta

- 执行域: cta_sync
- 编码格式: SYNC
- 语义组: barrier
- (class, format, opcode): (SYNC, 0, 3)
- Guard policy: required_pt
- Required state: scalar_ready

语法: BAR.SYNC.CTA 3

Operands

名称	类型	访问	字段	说明
barrier	barrier_id	control	slot3	—

Semantics:

Convert every entry-active lane to its CTA linear_tid, atomically add those owners to the slot's arrived_set, make each arrival a shared CTA release, and block the whole warp with one BarrierWaitRecord {warp_id, owner_snapshot, resume_pc=old_PC+8}. The barrier completes as soon as arrived_set equals the CTA's live_owner_set; every waiter then takes a shared CTA acquire and resumes by writing only PC=resume_pc and ready, and the slot is atomically cleared to idle.

Constraints:

- slot3 is the explicit barrier id 0..7; each CTA has eight slots, an idle slot has an empty arrived_set and no waiters, and every barrier owner identity is CTA linear_tid = warp_id*32+lane_id.
- BAR.SYNC.CTA requires scalar_ready, so the warp must be fully reconverged; a divergent warp faults before any arrival is recorded.
- EXIT removes the exiting linear_tid from live_owner_set and may therefore complete a pending barrier, but EXIT itself contributes no shared release.
- All unused extension and reserved bits are zero.

Faults:

- DIVERGENCE_FAULT when the warp is not scalar-ready; ILLEGAL_OPERAND on an invalid slot id; ILLEGAL_INSTRUCTION on reserved bits.

示例： BAR.SYNC.CTA 3

示例字段值： —

64 位机器字： 0x0018000000000185

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	5	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	3	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
a	26:19	—	是	—	Opcode-defined register A.
b	34:27	—	是	—	Opcode-defined register B.

字段	位段	固定值	必须为零	保留值	说明
imm16	50:35	—	是	—	Opcode-defined 16-bit immediate.
slot3	53:51	—	否	—	Barrier slot 0..7.
scope2	55:54	—	是	—	Memory scope.
order2	57:56	—	是	—	Memory order.
x6	63:58	—	是	—	Opcode-defined extension; unused bits are zero.

X_BROADCAST

- Family ID: x-broadcast
- 语义组: crosslane

Broadcast a selected active lane value.

X_BROADCAST.B32 — reg_lane

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: crosslane
- (class, format, opcode): (CROSSLANE, 0, 0)
- Guard policy: required_pt
- Required state: none

语法: X_BROADCAST.B32 v0, v0, s0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane	sgpr32	read	vb	—

Semantics:

Copy src from the selected lane to dst in every active lane.

Constraints:

- The lane selector is warp-uniform and names an active lane.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例： X_BROADCAST.B32 v0, v0, s0

示例字段值： —

64 位机器字： 0x0000000000000006

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	否	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	是	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

X_BROADCAST.B32 — imm_lane

- 执行域：warp_collective
- 编码格式：COLL
- 语义组：crosslane

- (class, format, opcode): (CROSSLANE, 0, 1)
- Guard policy: required_pt
- Required state: none

语法: X_BROADCAST.B32 v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane	uimm8	read	imm8	—

Semantics:

Broadcast src from lane imm8[4:0] to every active lane; imm8[7:5] is zero.

Constraints:

- imm8 is in 0..31 and names an active lane.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: X_BROADCAST.B32 v0, v0, 0

示例字段值: —

64 位机器字: 0x0000000000000086

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	1	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.

字段	位段	固定值	必须为零	保留值	说明
vb	42:35	—	是	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	否	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_READFIRST

- Family ID: s-readfirst
- 语义组: crosslane

Read a VGPR value from the first active lane into an SGPR.

S_READFIRST.B32 — b32

- 执行域: scalar
- 编码格式: COLL
- 语义组: crosslane
- (class, format, opcode): (CROSSLANE, 0, 4)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_READFIRST.B32 s0, v0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	smask	—
src	vgpr32	read	va	—

Semantics:

Find the least-numbered active lane and copy its src value to scalar dst.

Constraints:

- The active mask is nonempty.

- The warp must be scalar-ready before source snapshot, validation, and commit.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register pair or statically invalid operand.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例： S_READFIRST.B32 s0, v0

示例字段值： —

64 位机器字： 0x0000000000000206

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	4	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	是	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	是	—	Vector source B or lane selector.
smask	50:43	—	否	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	是	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

S_READFIRST.B64 — b64

- 执行域: scalar
- 编码格式: COLL
- 语义组: crosslane
- (class, format, opcode): (CROSSLANE, 0, 5)
- Guard policy: required_pt
- Required state: scalar_ready

语法: S_READFIRST.B64 s0:s1, v0:v1

Operands

名称	类型	访问	字段	说明
dst	sgpr64	write	smask	—
src	vgpr64	read	va	—

Semantics:

After scalar-ready succeeds, select the least-numbered live lane and copy its complete VGPR pair to the SGPR pair.

Constraints:

- The warp must be scalar-ready before selecting the lane or reading either half of the source pair.
- Both encoded pair bases are even and the complete low/high register pair is in range.

Faults:

- ILLEGAL_INSTRUCTION on a reserved encoding; ILLEGAL_OPERAND on an invalid register pair or statically invalid operand.
- DIVERGENCE_FAULT if the warp is not scalar-ready; the instruction commits no architectural effect.

示例: S_READFIRST.B64 s0:s1, v0:v1

示例字段值: —

64 位机器字: 0x00000000000000286

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	5	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	是	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	是	—	Vector source B or lane selector.
smask	50:43	—	否	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	是	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_VOTE

- Family ID: v-vote
- 语义组: vote

Reduce a vector predicate across active lanes.

V_VOTE.ANY — any

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: vote
- (class, format, opcode): (CROSSLANE, 0, 6)
- Guard policy: required_pt
- Required state: none

语法: V_VOTE.ANY vp0

Operands

名称	类型	访问	字段	说明
dst	scc	write	—	—
predicate	vpred	read	va	—

Semantics:

Reduce participating predicate bits with ANY and write SCC implicitly.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例：V_VOTE.ANY vp0

示例字段值：—

64 位机器字：0x00000000000000306

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	6	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	是	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	是	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	是	—	Opcode-defined collective immediate.

字段	位段	固定值	必须为零	保留值	说明
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_VOTE.ALL — all

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: vote
- (class, format, opcode): (CROSSLANE, 0, 7)
- Guard policy: required_pt
- Required state: none

语法: V_VOTE.ALL vp0

Operands

名称	类型	访问	字段	说明
dst	scc	write	—	—
predicate	vpred	read	va	—

Semantics:

Reduce participating predicate bits with ALL and write SCC implicitly.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: V_VOTE.ALL vp0

示例字段值: —

64 位机器字: 0x00000000000000386

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	7	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	是	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	是	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	是	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_VOTE.BALLOT — ballot

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: vote
- (class, format, opcode): (CROSSLANE, 0, 8)
- Guard policy: required_pt
- Required state: none

语法: V_VOTE.BALLOT s0, vp0

Operands

名称	类型	访问	字段	说明
dst	sgpr32	write	smask	—
predicate	vpred	read	va	—

Semantics:

BALLOT the predicate bits over active lanes and write the scalar result.

Constraints:

- Encoded registers must belong to the declared register files; every unused payload field is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: V_VOTE.BALLOT s0, vp0

示例字段值: —

64 位机器字: 0x0000000000000406

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	8	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	是	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	是	—	Vector source B or lane selector.
smask	50:43	—	否	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	是	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHUFFLE

- Family ID: v-shuffle
- 语义组: shuffle

Exchange values between active lanes.

V_SHUFFLE.IDX.B32 — idx

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: shuffle
- (class, format, opcode): (CROSSLANE, 0, 9)
- Guard policy: required_pt
- Required state: none

语法: V_SHUFFLE.IDX.B32 v0, v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane_or_delta	vgpr32	read	vb	—
width	uimm8	read	imm8	—

Semantics:

Each active lane reads src from the lane selected by IDX within the encoded power-of-two width.

Constraints:

- Width is one of 2, 4, 8, 16, or 32; source lane is active or the result is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: V_SHUFFLE.IDX.B32 v0, v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000486

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	9	否	—	Opcode local to class and format.

字段	位段	固定值	必须为零	保留值	说明
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	否	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	否	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHUFFLE.UP.B32 — up

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: shuffle
- (class, format, opcode): (CROSSLANE, 0, 10)
- Guard policy: required_pt
- Required state: none

语法: V_SHUFFLE.UP.B32 v0, v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane_or_delta	vgpr32	read	vb	—
width	uimm8	read	imm8	—

Semantics:

Each active lane reads src from the lane selected by UP within the encoded power-of-two width.

Constraints:

- Width is one of 2, 4, 8, 16, or 32; source lane is active or the result is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: V_SHUFFLE.UP.B32 v0, v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000506

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	10	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	否	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	否	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHUFFLE.DOWN.B32 — down

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: shuffle
- (class, format, opcode): (CROSSLANE, 0, 11)
- Guard policy: required_pt
- Required state: none

语法: V_SHUFFLE.DOWN.B32 v0, v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane_or_delta	vgpr32	read	vb	—
width	uimm8	read	imm8	—

Semantics:

Each active lane reads src from the lane selected by DOWN within the encoded power-of-two width.

Constraints:

- Width is one of 2, 4, 8, 16, or 32; source lane is active or the result is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: V_SHUFFLE.DOWN.B32 v0, v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000586

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	11	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.

字段	位段	固定值	必须为零	保留值	说明
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	否	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	否	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHUFFLE.DOWN.B32 — down_imm

- 执行域: warp_collective
- 编码格式: COLL
- 语义组: shuffle
- (class, format, opcode): (CROSSLANE, 0, 13)
- Guard policy: required_pt
- Required state: none

语法: V_SHUFFLE.DOWN.B32 v0, v0, 0, 32

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane_or_delta	uimm8	read	vb	—
width	uimm8	read	imm8	—

Semantics:

Each active lane reads src from lane_id plus the encoded immediate delta within the encoded power-of-two width.

Constraints:

- Delta is an integer from 0 through 31; width is one of 2, 4, 8, 16, or 32; source lane is active or the result is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例： V_SHUFFLE.DOWN.B32 v0, v0, 0, 32

示例字段值： —

64 位机器字： 0x0100000000000686

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	13	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.
va	34:27	—	否	—	Vector source.
vb	42:35	—	否	—	Immediate DOWN lane delta.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	否	—	Encoded power-of-two shuffle width.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

V_SHUFFLE.XOR.B32 — xor

- 执行域：warp_collective

- 编码格式: COLL
- 语义组: shuffle
- (class, format, opcode): (CROSSLANE, 0, 12)
- Guard policy: required_pt
- Required state: none

语法: V_SHUFFLE.XOR.B32 v0, v0, v0, 0

Operands

名称	类型	访问	字段	说明
dst	vgpr32	write	vd	—
src	vgpr32	read	va	—
lane_or_delta	vgpr32	read	vb	—
width	uimm8	read	imm8	—

Semantics:

Each active lane reads src from the lane selected by XOR within the encoded power-of-two width.

Constraints:

- Width is one of 2, 4, 8, 16, or 32; source lane is active or the result is zero.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例: V_SHUFFLE.XOR.B32 v0, v0, v0, 0

示例字段值: —

64 位机器字: 0x00000000000000606

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	6	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	12	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd	26:19	—	否	—	Vector destination.

字段	位段	固定值	必须为零	保留值	说明
va	34:27	—	否	—	Vector source A or predicate encoding.
vb	42:35	—	否	—	Vector source B or lane selector.
smask	50:43	—	是	—	SGPR lane-mask source or scalar destination.
imm8	58:51	—	否	—	Opcode-defined collective immediate.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.

MMA

- Family ID: mma
- 语义组: matrix_multiply

Warp-cooperative matrix multiply-accumulate.

MMA.M16N8K16.F16.F16.F32 — m16n8k16.f16xf16.f32

- 执行域: warp_matrix
- 编码格式: MMA
- 语义组: matrix_multiply
- (class, format, opcode): (MATRIX, 0, 0)
- Guard policy: required_pt
- Required state: none

语法: MMA.M16N8K16.F16.F16.F32 v0, v0, v0, v0

Matrix contract

合同项	值
shape	{k: 16, m: 16, n: 8}
element_types	{A: F16, B: F16, C: F32, D: F32}

合同项	值
fragments	{A: {base_alignment: 4, mapping: 'For lane l: row= $l > 1$, $k_0 = (l \& 1) * 8$. Register $r = 0..3$ low16 is $A[\text{row}, k_0 + 2 * r]$, high16 is $A[\text{row}, k_0 + 2 * r + 1]$.'}, registers_per_lane: 4}, B: {base_alignment: 2, mapping: 'For lane l: $k = l > 1$, $n_0 = (l \& 1) * 4$. Register $r = 0..1$ low16 is $B[k, n_0 + 2 * r]$, high16 is $B[k, n_0 + 2 * r + 1]$.'}, registers_per_lane: 2}, C: { base_alignment: 4, mapping: 'For lane l: row= $l > 1$, $n_0 = (l \& 1) * 4$. Register $r = 0..3$ is F32 $C[\text{row}, n_0 + r]$.'}, registers_per_lane: 4}, D: {base_alignment: 4, mapping: 'For lane l: row= $l > 1$, $n_0 = (l \& 1) * 4$. Register $r = 0..3$ is F32 $D[\text{row}, n_0 + r]$.'}, registers_per_lane: 4}}
f16_packing	Each A/B register contains two F16 values: low16 is the lower indexed element and high16 is the next element.
aliasing	{allowed: [D=C], forbidden: [D overlaps A, D overlaps B, partial D/C overlap, A/B/C overlap each other]}
numeric	{formula: 'acc_0=C[row,n]; for k=0..15: acc_(k+1)=RN32(FFMA_F32(exact_F16(A[row,k]), exact_F16(B[k,n]), acc_k)); D[row,n]=acc_16', k_order: strictly increasing 0..15, rounding: RNE after every F32 FFMA step; no tree reassociation, special_values: 'NaN, Inf, subnormal, and signed-zero behavior is exactly docs/05-numeric-environment.md, including canonical qNaN for numerical NaN results.'}
participation	{required_exec_equals_live: true, required_live_lanes: 32}

Operands

名称	类型	访问	字段	说明
dst	mma_fragment	write	vd_base	—
a	mma_fragment	read	va_base	—
b	mma_fragment	read	vb_base	—
c	mma_fragment	read	vc_base	—

Semantics:

For every row and column, initialize $\text{acc} = C[\text{row}, n]$, then for $k = 0..15$ in increasing order set $\text{acc} = \text{RN32}(\text{FFMA_F32}(\text{exact_F16}(A[\text{row}, k]), \text{exact_F16}(B[k, n]), \text{acc}))$; write D only after all 32 lanes complete.

Constraints:

- All active lanes execute with identical aligned fragment bases and complete fragment register ranges.

Faults:

- COLLECTIVE_FAULT if participating lanes disagree on uniform control or required source availability.

示例： MMA.M16N8K16.F16.F16.F32 v0, v0, v0, v0

示例字段值： —

64 位机器字： 0x0000000000000007

编码字段

字段	位段	固定值	必须为零	保留值	说明
class	3:0	7	否	—	Execution class.
format	6:4	0	否	—	Class-local payload format.
opcode	12:7	0	否	—	Opcode local to class and format.
guard	18:13	0	否	—	Header lane guard; zero is PT.
vd_base	26:19	—	否	—	Destination fragment base.
va_base	34:27	—	否	—	A fragment base.
vb_base	42:35	—	否	—	B fragment base.
vc_base	50:43	—	否	—	C fragment base.
attr8	58:51	—	是	—	Opcode-defined matrix attributes.
x5	63:59	—	是	—	Opcode-defined extension; unused bits are zero.